

Lemma Supplement: An Elementary Comparison for Four-Arc Regions in a Planar Strip Density

Proof built, Lean-verified, and semantically accepted by
Agentic Lean Math Assistant · github.com/RosaRojacr/agentic-lean-math-assistant

Technical supplement and declaration guide

This supplement gives the declaration-by-declaration mathematical account of the publication closure. The main article contains the coordinate constructions, the quantitative comparison, and its complete elementary proof. The entries below distinguish the mathematical statement and its explanation from the separately displayed source location and Lean code. Declarations assigned to the main argument include all four published roots and the substantive geometry and calculus they use; auxiliary coordinate decompositions, measure transport, and repetitive arithmetic are assigned to the supplement. A classification is a placement decision, not a claim that an infrastructure lemma is a separate geometric theorem.

The notation of the article is retained: for fixed $\lambda > 1$, $B, D, S, J, H, A_3, P_3, A_4, P_4$ suppress their density argument, and j denotes the explicit continuous extension of J' . For a general one-sided cap, write $L > 0$ for its chord, $0 < \theta < \pi$ for its half-angle, (m, b) for its chord midpoint, and $R = L/(2\sin\theta)$ for its radius. The normalized arc functions are

$$\ell(\theta) = \frac{\theta}{\sin\theta}, \quad a(\theta) = \frac{\theta - \sin\theta\cos\theta}{4\sin^2\theta}.$$

The latter is named `area` in the source and must not be confused with the weighted-area functional. For a strip core, write $L > 0$, $0 < k \leq 1$, $R = 1/k$, and $\beta = \arcsink$. For a three-arc assembly, write $0 < h < 1$, $q = 2h - 1$, $R = 1/h$, $\beta = \arccosq$, $\alpha = \arccos(q/\lambda)$, $\phi = \pi - \beta$, and $b = R(\sin\alpha - \sin\beta)$. Every reference to an admissible cap, core, candidate or assembly includes the inequalities in its defining entry. An unconstrained real density parameter is used only where the declaration itself permits it.

Real functions in the source are total: division by zero is defined to be zero, real square root of a nonpositive argument is zero, and the inverse trigonometric functions use their total principal extensions. Geometric and differential conclusions are asserted only under their stated domain hypotheses. A volume or Hausdorff-measure formula in the nonnegative extended reals uses the embedding $x \mapsto \max\{x, 0\}$; all geometric lengths and areas embedded in that way are nonnegative. The coordinate plane is transported to its Euclidean realization before forming one-dimensional Hausdorff measure. This distinction prevents accidental use of the maximum product norm for arc length. Auxiliary parametrized perimeter sums are explicitly identified with canonical complete-frontier integrals before being used in the comparison.

The six reused lower-level modules provide normalized calculus, circular-segment geometry, carrier integration and Euclidean frontier measure. Their entries are restricted to the dependency inventory for the published roots; no earlier final exclusion theorem is reused. The independent retained receipt and exact inhabited root checks cover the geometric comparison, scalar comparison, elementary route and geometric realization; the recorded axiom lists contain only the standard propositional extensionality, classical choice and quotient soundness principles. Elaborated dependency checks retain the lower-third tangent argument, greatest-root and support-comparison steps, and actual carrier/integral bridges. Compilation and these checks are evidence about the specified statements, not a substitute for reading their mathematical meaning.

There are no configured generated declaration families in this frozen package, and none of its inventory entries is marked generated. Consequently there is no generated-family ledger or representative selection to add. The external classification and the bounded-variation transfer discussed in the article remain informal cited mathematics, not additional formal hypotheses. The unchanged original note is credited as the source; its two erroneous area-minus-perimeter theorem displays are not certified.

Conceptual declaration ledger

CMVElementary.A3

Classification: main

Source: CMVElementary.lean:41-44

Declaration SHA-256: cb2c3a767813dd6743a002f60c9affd33c258de379854d6c91d5deadc49363b5

Mathematical statement. For real λ, t , with $q = 2t - 1$, define

$$A_3(t) = \frac{B(q) + \pi/2 + (2t + 1)D(q)}{t^2}.$$

For $\lambda > 1$ and $0 < t < 1$, the area bridge identifies this scalar with the actual weighted area of E_t . Its value at one is used only for the scalar endpoint.

```
def A3 (lam t : ℝ) : ℝ :=
  let q := 2 * t - 1
  (B lam q + Real.pi / 2 + (2 * t + 1) * D lam q) / t ^ 2
```

CMVElementary.A3_continuousAt

Classification: main

Source: CMVElementary.lean:636-639

Declaration SHA-256: 2aa80bdfce7c015108ca71987762c2d6eec97c1a5537e77e8a4add60919387ba

Mathematical statement. For every real λ and every $t \neq 0$, A_3 is continuous at t .

Only its denominator can obstruct continuity of the total formula; this in particular includes the scalar right endpoint one.

```
theorem A3_continuousAt {lam t : ℝ} (ht : t ≠ 0) : ContinuousAt (A3 lam) t := by
  unfold A3 B D
  fun_prop (disch := positivity)
```

CMVElementary.A3_tendsto_atRight_zero

Classification: main

Source: CMVElementary.lean:667-678

Declaration SHA-256: bbfeb34d411d80b4748ecb5317b26152556ee012a4c270e7d6f639c2f722c14a

Mathematical statement. For $\lambda > 1$,

$$A_3(t) \rightarrow +\infty$$

as $t \downarrow 0$.

The normalized frozen area agrees eventually on the right of zero and has positive limiting numerator divided by t^2 .

```
theorem A3_tendsto_atRight_zero {lam : ℝ} (hlam : 1 < lam) :
  Tendsto (A3 lam) (ℕ[>] (0 : ℝ)) atTop := by
  have hid := scalar_identification_claims hlam
  have hsmall : ∀ t in ℕ[>] (0 : ℝ), t < 1 :=
    (eventually_lt_nhds zero_lt_one).filter_mono inf_le_left
  have heq : A3 lam =f[ℕ[>] (0 : ℝ)] LeanSuffixAnalytic.typeThreeArea lam := by
    filter_upwards [self_mem_nhdsWithin, hsmall] with t ht ht1
    exact (hid.area_three t ⟨ht, ht1.le⟩).symm
  exact (LeanSuffixAnalytic.typeThreeArea_tendsto_atRight_zero hlam).congr' heq.symm

/-- All calculus claims from the frozen variational identity and zero-end limit.
Continuity includes one; differentiability is asserted only on the open interval. -/
```

CMVElementary.A4

Classification: main

Source: CMVElementary.lean:49-50

Declaration SHA-256: 4a8eaff3edf0f9c1b5bdc4eebb83b5e656403bb3492c00db3ddccd5bd349b4df

Mathematical statement. For real λ, t , define

$$A_4(t) = \frac{2(B(t) + tD(t))}{t^2}.$$

For $\lambda > 1$ and $0 < t \leq 1$, this is identified with the integral over the four-arc carrier.

```
def A4 (lam t : ℝ) : ℝ := 2 * (B lam t + t * D lam t) / t ^ 2
```

CMVElementary.AuxiliaryClaims

Classification: main

Source: CMVElementary.lean:74-102

Declaration SHA-256: d28b69e2724973e27afe7c069b33cc5b996e8abf00ea05832fa32bd7ac02f146

Mathematical statement. The auxiliary assertion consists of the following claims. The function D is continuous, even and nonnegative on $[-1, 1]$, strictly increasing on $[0, 1]$, and differentiable on $(-1, 1)$. On the latter interval

$$S' = -2D,$$

and on the closed interval

$$S(-z) = \lambda\pi - S(z).$$

One has

$$\gamma_\lambda > 0;$$

J, j are continuous on $[0, 1]$;

$$J(0) = J(1) = \gamma_\lambda;$$

$$J' = j$$

on $(0, 1)$; and

$$J, H \geq \gamma_\lambda$$

on $(0, 1]$. Furthermore

$$j > 0$$

on $(0, 1/3)$,

$$j(1/3) = 0,$$

$$j < 0$$

on $(1/3, 1)$, and

$$H \geq J$$

on $[1/3, 1]$. On $(0, 1/3)$,

$$j' = 8D'(2u - 1) - 4D'(u) < 0$$

and

$$H' = -u[8D'(2u - 1) - 4D'(u)] > 0.$$

Finally

$$H(u) \rightarrow \gamma_\lambda$$

as $u \downarrow 0$.

This is a conjunction of the listed mathematical properties, not a theorem assuming them without proof. The corresponding construction or claims theorem supplies every component from the stated density and parameter hypotheses; the detailed identities and domains in the statement distinguish endpoint continuity from interior differentiability and actual geometry from scalar formulas.

```
structure AuxiliaryClaims (lam : ℝ) : Prop where
  d_continuous : ContinuousOn (D lam) (Icc (-1) 1)
  d_even : ∀ z ∈ Icc (-1) 1, D lam (-z) = D lam z
  d_nonneg : ∀ z ∈ Icc (-1) 1, 0 ≤ D lam z
  d_strictMono : StrictMonoOn (D lam) (Icc 0 1)
  s_deriv : ∀ z ∈ Ioo (-1) 1, HasDerivAt (S lam) (-2 * D lam z) z
  s_reflection : ∀ z ∈ Icc (-1) 1, S lam (-z) = lam * Real.pi - S lam z
  gamma_pos : 0 < gamma lam
  j_continuous : ContinuousOn (J lam) (Icc 0 1)
  jprime_continuous : ContinuousOn (jprime lam) (Icc 0 1)
  j_zero : J lam 0 = gamma lam
  j_one : J lam 1 = gamma lam
  j_deriv : ∀ u ∈ Ioo 0 1, HasDerivAt (J lam) (jprime lam u) u
  j_lower : ∀ u ∈ Ioc 0 1, gamma lam ≤ J lam u
  h_lower : ∀ u ∈ Ioc 0 1, gamma lam ≤ H lam u
  jprime_pos : ∀ u ∈ Ioo 0 (1 / 3), 0 < jprime lam u
  jprime_third : jprime lam (1 / 3) = 0
  jprime_neg : ∀ u ∈ Ioo (1 / 3) 1, jprime lam u < 0
  h_ge_j : ∀ u ∈ Icc (1 / 3) 1, J lam u ≤ H lam u
  d_differentiable : DifferentiableOn ℝ (D lam) (Ioo (-1) 1)
  jprime_deriv : ∀ u ∈ Ioo 0 (1 / 3), HasDerivAt (jprime lam)
    (8 * deriv (D lam) (2 * u - 1) - 4 * deriv (D lam) u) u
  jprime_deriv_neg : ∀ u ∈ Ioo 0 (1 / 3),
    8 * deriv (D lam) (2 * u - 1) - 4 * deriv (D lam) u < 0
  h_deriv : ∀ u ∈ Ioo 0 (1 / 3), HasDerivAt (H lam)
    (-u * (8 * deriv (D lam) (2 * u - 1) - 4 * deriv (D lam) u)) u
  h_deriv_pos : ∀ u ∈ Ioo 0 (1 / 3), 0 < deriv (H lam) u
  h_limit_zero : Tendsto (H lam) (ℕ[>] (0 : ℝ)) (ℕ (gamma lam))
```

CMVElementary.B

Classification: main

Source: CMVElementary.lean:30-32

Declaration SHA-256: 5258c909f795e5bbd4f5ef3c79d0f8a12d32481ff10a7612a85a63a34f2572c3

Mathematical statement. For real λ, z , define

$$B_\lambda(z) = \lambda \arccos(z/\lambda) + \arcsin z.$$

This is the angle combination in both area and perimeter formulas; the main proof fixes

$$\lambda > 1$$

and suppresses that subscript.

```
def B (lam z : ℝ) : ℝ :=
  lam * Real.arccos (z / lam) + Real.arcsin z
```

CMVElementary.C

Classification: main

Source: CMVElementaryGeometry.lean:39-41

Declaration SHA-256: dac24abe77e8fbb5c839aa4c94d56e4b1ad024651bcedbea970677ac66fb909e

Mathematical statement. Define

$$C_h = K_h \cup H_h \cup \sigma(H_h).$$

Both attaching chords are included in this closed union; their strict interiors are proved to be interior points of the union.

```
def C (lam h : ℝ) : Set PlanePoint :=
  K lam h ∪ upperCap4 lam h ∪ sigma '' upperCap4 lam h
```

CMVElementary.D

Classification: main

Source: CMVElementary.lean:33-35

Declaration SHA-256: 03723a810a1a63abdd42a17f0c33c9ae90ceffff8b93ca60d7527ed4f150d055

Mathematical statement. For real λ, z , define

$$D_\lambda(z) = \sqrt{1 - z^2/\lambda^2} - \sqrt{1 - z^2}.$$

This radical difference is the nonnegative width correction on the geometric domain, with the total real conventions described in the introduction to the supplement.

```
def D (lam z : ℝ) : ℝ :=
  Real.sqrt (1 - z ^ 2 / lam ^ 2) - Real.sqrt (1 - z ^ 2)
```

CMVElementary.D_even

Classification: main

Source: CMVElementary.lean:391-393

Declaration SHA-256: 7b4f3c46b671c85fa9365ac233355bdd565a25b839e1a28c1a0f55338b48441a

Mathematical statement. For all real λ, z ,

$$D_\lambda(-z) = D_\lambda(z).$$

Both radicands depend only on the square of the argument.

```
theorem D_even (lam z : ℝ) : D lam (-z) = D lam z := by
  simp [D]
```

CMVElementary.D_nonneg

Classification: main

Source: CMVElementary.lean:394-403

Declaration SHA-256: 8ceb0e3455b90fa64499eee81dc63f1d0e17f21da82cc4b48255006f6cae2e2a

Mathematical statement. For $\lambda > 1$ and every real z , $D(z) \geq 0$.

Since

$$z^2/\lambda^2 \leq z^2,$$

monotonicity of total real square root compares the two radicands.

```
theorem D_nonneg {lam : ℝ} (hlam : 1 < lam) (z : ℝ) : 0 ≤ D lam z := by
  unfold D
  apply sub_nonneg.mpr
  apply Real.sqrt_le_sqrt
  have hsq : 1 ≤ lam ^ 2 := by nlinarith
  have hd : z ^ 2 / lam ^ 2 ≤ z ^ 2 := div_le_self (sq_nonneg z) hsq
  linarith

/-- Interior strict positivity plus endpoint continuity gives closed-interval
strict increase. No square-root derivative at one is used. -/
```

CMElementary.D_strictMono

Classification: main

Source: CMElementary.lean:404-409

Declaration SHA-256: 6591c7dbdbd88b546f03ea069e9476a8557083684fc0bb6944048ffcd4f3f44c

Mathematical statement. For $\lambda > 1$, D is strictly increasing on $[0, 1]$.

Positive interior derivative and continuity at both endpoints give strict increase on the entire closed interval.

```
theorem D_strictMono {lam : ℝ} (hlam : 1 < lam) :
  StrictMonoOn (D lam) (Icc 0 1) := by
  apply strictMonoOn_of_deriv_pos (convex_Icc _ _) (DerivativePilot.D_continuous lam).continuousOn
  intro z hz
  exact DerivativePilot.deriv_D_pos hlam (by simp only [interior_Icc] using hz)
```

CMElementary.DerivativePilot.D_continuous

Classification: main

Source: CMElementary.lean:283-286

Declaration SHA-256: c1e2565224862417f88cc26d21ac89a1eb14395b552065dfde479c911555d2da

Mathematical statement. For every real λ , D_λ is continuous on

$$\mathbb{R}.$$

This uses total continuous real square root and algebraic operations, not differentiability of square root at zero.

```
theorem D_continuous (lam : ℝ) : Continuous (D lam) := by
  unfold D
  fun_prop
```

CMElementary.DerivativePilot.D_differentiableOn

Classification: main

Source: CMElementary.lean:234-239

Declaration SHA-256: ea05176def925de59cfb082583ec3ac8045ebc1e63e39e642102b8aba520a3fd

Mathematical statement. For $\lambda > 1$, D is differentiable on $(-1, 1)$.

The local derivative formula is available at every point of this interval, with no assertion at either radical endpoint.

```
theorem D_differentiableOn {lam : ℝ} (hlam : 1 < lam) :
  DifferentiableOn ℝ (D lam) (Ioo (-1) 1) := by
  intro z hz
  exact (hasDerivAt_D hlam hz).differentiableAt.differentiableWithinAt

/-- On the lower third the two arguments have opposite strict signs. -/
```

CMElementary.DerivativePilot.D_normalization

Classification: main

Source: CMElementary.lean:171-176

Declaration SHA-256: 042c0b88a8b4aad471bbb7cd59d649270eaf82feb3afdd1eee7df1acb25b044d

Mathematical statement. For $\lambda > 0$, the functions $D(z)$ and

$$\sqrt{\lambda^2 - z^2} / \lambda - \sqrt{1 - z^2}$$

agree at every real z .

Apply the radical normalization pointwise; this is the exact identification with the frozen scalar formulas.

```
theorem D_normalization {lam : ℝ} (hlam : 0 < lam) :
  D lam = fun z => Real.sqrt (lam ^ 2 - z ^ 2) / lam - Real.sqrt (1 - z ^ 2) := by
  funext z
  rw [D, sqrt_normalization hlam]

/-- Every radical and denominator used in the interior derivative is positive. -/
```

CMElementary.DerivativePilot.H_continuous

Classification: main

Source: CMElementary.lean:291-294

Declaration SHA-256: 1e9b7d4f89a28deb3482239f3babe2259c9d36603e39a28f27e59937b9c67228

Mathematical statement. For every real λ , H is continuous on

$$\mathbb{R}.$$

Expand H, J, S, B, D, j ; total inverse trigonometric functions and square roots are continuous.

```
theorem H_continuous (lam : ℝ) : Continuous (H lam) := by
  unfold H J S B jprime D
  fun_prop
```

CMElementary.DerivativePilot.H_limit_zero

Classification: main

Source: CMElementary.lean:300-312

Declaration SHA-256: 3fa895803ef1aea4d3d273718ce23287763c81a52827fb5d35f0d8e9245453cf

Mathematical statement. For every real λ ,

$$\lim_{u \downarrow 0} H(u) = \gamma_\lambda.$$

Since

$$H(0) = J(0),$$

continuity gives the one-sided limit without differentiating at zero.

```
theorem H_limit_zero (lam : ℝ) :
  Tendsto (H lam) (ℕ[>] (0 : ℝ)) (ℕ (gamma lam)) := by
  have heq : H lam 0 = gamma lam := by simp [H] using J_zero lam
  rw [← heq]
  exact (H_continuous lam).continuousAt.continuousWithinAt

end DerivativePilot
```

```
namespace MonotoneTangent
```

```
-- A decreasing specified derivative places the left endpoint below the tangent
at the right endpoint. Only continuity, not differentiability, is needed at zero.
The specified derivative need not itself be continuous or differentiable. -/
```

CMElementary.DerivativePilot.J_zero

Classification: main

Source: CMElementary.lean:295-299

Declaration SHA-256: 89ec878363f839a318ed701d106d998cde8ae845aaa0b897db75b6d68f34219d

Mathematical statement. For every real λ ,

$$J(0) = \gamma_\lambda.$$

Evaluate at zero, use the inverse trigonometric reflection identities, and simplify the radical normalization.

```
theorem J_zero (lam : ℝ) : J lam 0 = gamma lam := by
  norm_num [J, S, B, D, gamma, zpow_neg, neg_div, Real.arccos_neg]
  ring

-- This is a continuity limit, not a derivative at zero. -/
```

CMElementary.DerivativePilot.deriv_D

Classification: main

Source: CMElementary.lean:204-209

Declaration SHA-256: 30a180dad9754ac8b27ff3cc498a89ca981d4205c49ae9ee5cd3a6be7fbe5a18

Mathematical statement. If $\lambda > 1$ and $-1 < z < 1$, the derivative value is

$$D'(z) = z[1/\sqrt{1-z^2} - 1/(\lambda\sqrt{\lambda^2-z^2})].$$

This extracts the ordinary derivative from the preceding differentiability assertion without making an endpoint claim.

```
theorem deriv_D {lam z : ℝ} (hlam : 1 < lam) (hz : z ∈ Ioo (-1) 1) :
  deriv (D lam) z = z * (1 / Real.sqrt (1 - z ^ 2) -
    1 / (lam * Real.sqrt (lam ^ 2 - z ^ 2))) :=
  (hasDerivAt_D hlam hz).deriv

-- Strict positivity of the coefficient does not require z to be nonzero. -/
```

CMElementary.DerivativePilot.deriv_D_neg

Classification: main

Source: CMElementary.lean:222-227

Declaration SHA-256: fd9ec3f8436fd5a4cbfa39412991f7fc77c1e1d9d9d6557a16b9878e3343ac1c

Mathematical statement. For $\lambda > 1$ and $-1 < z < 0$,

$$D'(z) < 0.$$

Multiply the positive reciprocal difference by the negative argument.

```
theorem deriv_D_neg {lam z : ℝ} (hlam : 1 < lam)
  (hz : z ∈ Ioo (-1) 0) : deriv (D lam) z < 0 := by
  have hdomain : z ∈ Ioo (-1) 1 := ⟨hz.1, by linarith [hz.2]⟩
  rw [deriv_D hlam hdomain]
  exact mul_neg_of_neg_of_pos hz.2 (derivative_coefficient_pos hlam hdomain)
```

CMElementary.DerivativePilot.deriv_D_pos

Classification: main

Source: CMElementary.lean:228-233

Declaration SHA-256: 2ecd442f21def60181fa9ad69af687a0bd5040c9880fdb5da7b9948bb3ba3df7

Mathematical statement. For $\lambda > 1$ and $0 < z < 1$,

$$D'(z) > 0.$$

Multiply the positive reciprocal difference by the positive argument; continuity subsequently extends strict increase to the closed interval.

```
theorem deriv_D_pos {lam z : ℝ} (hlam : 1 < lam)
  (hz : z ∈ Ioo 0 1) : 0 < deriv (D lam) z := by
  have hdomain : z ∈ Ioo (-1) 1 := ⟨by linarith [hz.1], hz.2⟩
  rw [deriv_D hlam hdomain]
  exact mul_pos hz.1 (derivative_coefficient_pos hlam hdomain)
```

CMElementary.DerivativePilot.deriv_H_pos_of_J

Classification: main

Source: CMElementary.lean:276-282

Declaration SHA-256: 303bde784351003abd78c85956bcd7a5a548511d46fc0a1e12378795be4dcb69

Mathematical statement. If $\lambda > 1$,

$$0 < u < 1/3,$$

and

$$J'(u) = j(u)$$

in the differentiability sense, then

$$H'(u) > 0.$$

Use the preceding product-rule formula and the strict negativity of j' on the lower third.

```
theorem deriv_H_pos_of_J {lam u : ℝ} (hlam : 1 < lam)
  (hu : u ∈ Ioo 0 (1 / 3))
  (hJ : HasDerivAt (J lam) (jprime lam u) u) : 0 < deriv (H lam) u := by
  rw [(hasDerivAt_H_of_J hlam hu hJ).deriv]
  exact mul_pos_of_neg_of_neg (neg_neg_of_pos hu.1) (jprime_deriv_neg hlam hu)

/-- Continuity, unlike differentiability, is valid at the radical endpoints. -/
```

CMElementary.DerivativePilot.derivative_coefficient_pos

Classification: main

Source: CMElementary.lean:210-221

Declaration SHA-256: 3b1dd20767a51d60e6a3184ceb9440736e6ef81eb3d05b79b085bff9fa38ccf9

Mathematical statement. For $\lambda > 1$ and $-1 < z < 1$,

$$1/\sqrt{1-z^2} - 1/(\lambda\sqrt{\lambda^2-z^2}) > 0.$$

The second positive denominator is strictly larger than the first; reversing positive reciprocals gives the claimed sign, including at $z = 0$.

```

theorem derivative_coefficient_pos {lam z : ℝ}
  (hlam : 1 < lam) (hz : z ∈ Ioo (-1) 1) :
  0 < 1 / Real.sqrt (1 - z ^ 2) -
  1 / (lam * Real.sqrt (lam ^ 2 - z ^ 2)) := by
  obtain ⟨hi, ho, hsi, hso⟩ := radical_denominators hlam hz
  have hs : Real.sqrt (1 - z ^ 2) < Real.sqrt (lam ^ 2 - z ^ 2) :=
    Real.sqrt_lt_sqrt hi.le (by nlinarith)
  have hscale : Real.sqrt (lam ^ 2 - z ^ 2) <
    lam * Real.sqrt (lam ^ 2 - z ^ 2) := by
    nlinarith [Real.sqrt_pos.2 ho]
  exact sub_pos.mpr (one_div_lt_one_div_of_lt hsi (hs.trans hscale))

```

CMElementary.DerivativePilot.hasDerivAt_D

Classification: main

Source: CMElementary.lean:187-203

Declaration SHA-256: 25ed9e8fe57ff5b6872ee31ca5e64c1daca4f7b0b2c2e6d3d202561278ec1daa

Mathematical statement. If $\lambda > 1$ and $-1 < z < 1$, D is differentiable at z , with

$$D'(z) = z[1/\sqrt{1-z^2} - 1/(\lambda\sqrt{\lambda^2-z^2})].$$

Differentiate the normalized radicals; the positive denominators make the chain rule legitimate for negative, zero and positive interior arguments.

```

theorem hasDerivAt_D {lam z : ℝ} (hlam : 1 < lam) (hz : z ∈ Ioo (-1) 1) :
  HasDerivAt (D lam)
  (z * (1 / Real.sqrt (1 - z ^ 2) -
    1 / (lam * Real.sqrt (lam ^ 2 - z ^ 2)))) z := by
  have hl : 0 < lam := by linarith
  obtain ⟨hi, ho, hsi, hso⟩ := radical_denominators hlam hz
  have hsone : Real.sqrt (lam ^ 2 - z ^ 2) ≠ 0 := ne_of_gt (Real.sqrt_pos.2 ho)
  rw [D_normalization hl]
  have houter := (((hasDerivAt_const z (lam ^ 2)).sub
    ((hasDerivAt_id z).pow 2)).sqrt (ne_of_gt ho)).div_const lam)
  have hinner := (((hasDerivAt_const z 1).sub
    ((hasDerivAt_id z).pow 2)).sqrt (ne_of_gt hi))
  simpa only [id_eq] using! (houter.sub hinner).congr_deriv (by
    norm_num [id_eq, Pi.sub_apply, Pi.pow_apply]
    field_simp [ne_of_gt hl, ne_of_gt hsi, hsone]
    ring)

```

CMElementary.DerivativePilot.hasDerivAt_H_of_J

Classification: main

Source: CMElementary.lean:266-275

Declaration SHA-256: 7aab8b2b2fb04f8525ea011add719fb8e7540c817e8f26fb8a7543159fe02613

Mathematical statement. If $\lambda > 1$,

$$0 < u < 1/3,$$

and J has derivative $j(u)$ at u , then H has derivative

$$-u[8D'(2u-1) - 4D'(u)]$$

there.

The product rule in $J - uj$ cancels the first derivative of J . The premise is supplied independently by the derivative theorem for J .

```

theorem hasDerivAt_H_of_J {lam u : ℝ} (hlam : 1 < lam)
  (hu : u ∈ Ioo 0 (1 / 3))

```

```

(hJ : HasDerivAt (J lam) (jprime lam u) u) :
HasDerivAt (H lam)
  (-u * (8 * deriv (D lam) (2 * u - 1) - 4 * deriv (D lam) u)) u := by
have hdu := hasDerivAt_jprime hlam (lower_third_signed_arguments hu).2
simp only [H] using! (hJ.sub ((hasDerivAt_id u).mul hdu)).congr_deriv (by
  simp only [id_eq]
  ring)

```

CMElementary.DerivativePilot.hasDerivAt_jprime

Classification: main

Source: CMElementary.lean:253-265

Declaration SHA-256: 94ad44e520ab302a768e15871418b7bd98703e11200ad83d0d006b7c265ad1c9

Mathematical statement. For $\lambda > 1$ and $0 < u < 1$, j is differentiable with

$$j'(u) = 8D'(2u - 1) - 4D'(u).$$

Apply the chain rule to $4[D(2u - 1) - D(u)]$; both arguments are strictly between minus one and one.

```

theorem hasDerivAt_jprime {lam u : ℝ} (hlam : 1 < lam)
  (hu : u ∈ Ioo 0 1) :
  HasDerivAt (jprime lam)
    (8 * deriv (D lam) (2 * u - 1) - 4 * deriv (D lam) u) u := by
have hq : 2 * u - 1 ∈ Ioo (-1) 1 := ⟨by linarith [hu.1], by linarith [hu.2]⟩
have hdq := (hasDerivAt_D hlam hq).differentiableAt.hasDerivAt
have hdu := (hasDerivAt_D hlam ⟨by linarith [hu.1], hu.2⟩).differentiableAt.hasDerivAt
have ha : HasDerivAt (fun x : ℝ => 2 * x - 1) 2 u := by
  simp using ((hasDerivAt_id u).const_mul 2).sub_const 1
  simp only [jprime, Function.comp_def] using!
  (((hdq.comp u ha).sub hdu).const_mul 4).congr_deriv (by ring)

/-- Product-rule interface; the record constructor supplies `hasDerivAt_J`. -/

```

CMElementary.DerivativePilot.jprime_continuous

Classification: main

Source: CMElementary.lean:287-290

Declaration SHA-256: 793ddb37fb86b542cee82ad8c3620a2966360336486c29f8d5e41082b4022fff

Mathematical statement. For every real λ , j is continuous on

$$\mathbb{R}.$$

It is a linear combination of the continuous function D and its affine composition.

```

theorem jprime_continuous (lam : ℝ) : Continuous (jprime lam) := by
  unfold jprime
  exact ((D_continuous lam).comp (by fun_prop) |>.sub (D_continuous lam)).const_mul 4

```

CMElementary.DerivativePilot.jprime_deriv_neg

Classification: main

Source: CMElementary.lean:244-252

Declaration SHA-256: 58ff7311ad822a152063bce0332ab5d88a9004d4d249e43b945f5276cfd42ce7

Mathematical statement. If $\lambda > 1$ and $0 < u < 1/3$, then

$$8D'(2u - 1) - 4D'(u) < 0.$$

The first derivative is negative and the second positive, so both displayed contributions are negative.

```

theorem jprime_deriv_neg {lam u : ℝ} (hlam : 1 < lam)
  (hu : u ∈ Ioo 0 (1 / 3)) :
  8 * deriv (D lam) (2 * u - 1) - 4 * deriv (D lam) u < 0 := by
  obtain ⟨hq, hpos⟩ := lower_third_signed_arguments hu
  have hn := deriv_D_neg hlam hq
  have hp := deriv_D_pos hlam hpos
  linarith

/-- Chain rule; no derivative at u=0 or u=1 is asserted. -/

```

CMVElementary.DerivativePilot.sqrt_normalization

Classification: main

Source: CMVElementary.lean:165-170

Declaration SHA-256: 4eb056d0891d93b9a10938ae816157b2fdb95e7a6c30ea9942c36548da144840

Mathematical statement. For $\lambda > 0$ and every real z ,

$$\sqrt{\lambda^2 - z^2} / \lambda = \sqrt{1 - z^2 / \lambda^2}.$$

Factor the radicand by λ^2 , use

$$\sqrt{\lambda^2} = \lambda,$$

and preserve the total square-root convention even outside the interval used geometrically.

```

theorem sqrt_normalization {lam : ℝ} (hlam : 0 < lam) (z : ℝ) :
  Real.sqrt (lam ^ 2 - z ^ 2) / lam = Real.sqrt (1 - z ^ 2 / lam ^ 2) := by
  have heq : 1 - z ^ 2 / lam ^ 2 = (lam ^ 2 - z ^ 2) / lam ^ 2 := by
    field_simp
  rw [heq, Real.sqrt_div' _ (sq_nonneg lam), Real.sqrt_sq_eq_abs, abs_of_pos hlam]

```

CMVElementary.E

Classification: main

Source: CMVElementaryGeometry.lean:54-57

Declaration SHA-256: f1d34fbfd5ea8d77eb2e384dd1e6f5674c85b88684e3f0510bf703608445bde2

Mathematical statement. Define

$$E_r = K_r^{(3)} \cup H_r^{(3)}.$$

The construction includes the bottom segment as an exposed boundary component and the upper chord as an internal attachment.

```

def E (lam r : ℝ) : Set PlanePoint := K3 lam r ∪ upperCap3 lam r

/-- Exact contract type of `CMVElementary.geometric_comparison`.
Only the four-arc carrier can be instantiated at one; its competitor has `r < h`. -/

```

CMVElementary.ElementaryRoute

Classification: main

Source: CMVElementary.lean:155-164

Declaration SHA-256: 4b48fa17e2a280d7cb3e3056c212a5e018edffc49f5b8e8a39a8fc2035f440f6

Mathematical statement. The elementary-route assertion is the conjunction of the auxiliary, same-curvature, three-arc calculus and scalar-identification assertions, together with existence, for every

$$0 < h \leq 1,$$

of ε, r satisfying the greatest-root assertion.

This is a conjunction of the listed mathematical properties, not a theorem assuming them without proof. The corresponding construction or claims theorem supplies every component from the stated density and parameter hypotheses; the detailed identities and domains in the statement distinguish endpoint continuity from interior differentiability and actual geometry from scalar formulas.

```
structure ElementaryRoute (lam : ℝ) : Prop where
  auxiliary : AuxiliaryClaims lam
  same_curvature : SameCurvatureClaims lam
  three_arc_calculus : ThreeArcCalculusClaims lam
  scalar_identification : ScalarIdentificationClaims lam
  greatest_root : ∀ h ∈ Ioc 0 1, ∃ epsilon r : ℝ, GreatestRootClaims lam h epsilon r

namespace DerivativePilot

/-- Positive-lambda normalization holds even outside the geometric domain. -/
```

CMElementary.FourArcRealization

Classification: main

Source: CMElementaryGeometry.lean:80-96

Declaration SHA-256: b4e3701b509f6d802066e8cb6919211e45a8c1e2c32ad1ab317a1b5e812217d1

Mathematical statement. There is a four-arc candidate with curvature h , angle

$$\arccos(h/\lambda),$$

density jump

$$\lambda > 1,$$

and incidence

$$\lambda \cos \alpha = h,$$

whose assembled coordinate set equals C_h . The region satisfies all stated regularity properties and

$$A_\lambda(C_h) = A_4(h),$$

$$P_\lambda(C_h) = P_4(h).$$

For

$$y = \pm 1,$$

points with

$$|x| < \sin(\arccos(h/\lambda))/h$$

are interior to C_h . The reflected cap is contained in

$$y \leq -1.$$

This is a conjunction of the listed mathematical properties, not a theorem assuming them without proof. The corresponding construction or claims theorem supplies every component from the stated density and parameter hypotheses; the detailed identities and domains in the statement distinguish endpoint continuity from interior differentiability and actual geometry from scalar formulas.

```
structure FourArcRealization (lam h : ℝ) : Prop where
  candidate_bridge : ∃ candidate : FourArcCandidate lam,
    candidate.h = h ∧ candidate.alpha = Real.arccos (h / lam) ∧
```

```

candidate.SatisfiesCMVTypeIVHypotheses ^ C lam h = candidate.assembly.carrier
regularity : RegionRegularity lam (C lam h)
area_formula : WeightedArea lam (C lam h) = A4 lam h
perimeter_formula : WeightedPerimeter lam (FrontierMeasure (C lam h)) = P4 lam h
upper_chord_interior :
  {p : PlanePoint | p.2 = 1 ^ |p.1| < Real.sin (Real.arccos (h / lam)) / h} ⊆
    interior (C lam h)
lower_chord_interior :
  {p : PlanePoint | p.2 = -1 ^ |p.1| < Real.sin (Real.arccos (h / lam)) / h} ⊆
    interior (C lam h)
lower_cap_location : sigma '' upperCap4 lam h ⊆ {p : PlanePoint | p.2 ≤ -1}

/-- Branch-complete carrier identification. The full exposed bottom segment is
retained, including its zero-length case; it is not an internal attaching chord. -/

```

CMElementary.GeometricRealizationStatement

Classification: main

Source: CMElementaryGeometry.lean:117-127

Declaration SHA-256: bf9d5e6f83903980043feb1686756fd6224652d95175a5fa7761171f2e85ae57

Mathematical statement. For every

$$\lambda > 1,$$

the four-arc realization holds for all

$$0 < h \leq 1,$$

and the three-arc realization holds for all

$$0 < r < 1.$$

This defines the unconditional realization proposition on the two different curvature domains. The four-arc endpoint is included, while every geometric three-arc parameter is strictly less than one.

```

def GeometricRealizationStatement : Prop :=
  ∀ lam : ℝ, 1 < lam →
    (∀ h : ℝ, 0 < h → h ≤ 1 → FourArcRealization lam h) ^
    (∀ r : ℝ, 0 < r → r < 1 → ThreeArcRealization lam r)

open scoped Topology

namespace GeometryBridge

-- The corrected target is P(C)-P(E), not the printed A(C)-P(E).

```

CMElementary.GeometricStatement

Classification: main

Source: CMElementaryGeometry.lean:58-69

Declaration SHA-256: 1f7b139bbebd695d7241107d88e880fde926dba2dd2619d65eb42a12b0005fa8

Mathematical statement. For every real $\lambda > 1$,

$$\gamma_\lambda > 0.$$

For every real h with $0 < h \leq 1$, there is a real r such that

$$\begin{aligned}
0 &< r < h, \\
A_\lambda(E_r) &= A_\lambda(C_h), \\
P_\lambda(C_h) - P_\lambda(E_r) &> \frac{\gamma_\lambda}{h} > 0.
\end{aligned}$$

Here area is Lebesgue weighted area and perimeter is Euclidean complete-frontier weighted perimeter.

This defines the primary geometric proposition with actual Lebesgue area and canonical Euclidean complete-frontier perimeter. Both area and perimeter identities must be derived for the coordinate carriers before the scalar witness can satisfy this statement.

```
def GeometricStatement : Prop :=
  ∀ lam : ℝ, 1 < lam →
    0 < gamma lam ∧
    ∀ h : ℝ, 0 < h → h ≤ 1 →
      ∃ r : ℝ, 0 < r ∧ r < h ∧
        WeightedArea lam (E lam r) = WeightedArea lam (C lam h) ∧
        gamma lam / h <
          WeightedPerimeter lam (FrontierMeasure (C lam h)) -
            WeightedPerimeter lam (FrontierMeasure (E lam r)) ∧
        0 < gamma lam / h

/-- Geometric finiteness is to be derived, not supplied by a caller. -/
```

CMElementary.GeometryBridge.C_area

Classification: main

Source: CMElementaryGeometry.lean:719-737

Declaration SHA-256: 917e6af80fc905a99360aafda85c3da61226b43930c862e06a404c527ac58964

Mathematical statement. For $\lambda > 1$ and $0 < h \leq 1$,

$$A_\lambda(C_h) = A_4(h).$$

Apply the exact carrier bridge and the candidate area theorem with the proved incidence relation; inverse trigonometric and radical identities give the displayed scalar formula.

```
theorem C_area {lam : ℝ} (hlam : 1 < lam) (h : ℝ) (hh : 0 < h) (h1 : h ≤ 1) :
  WeightedArea lam (C lam h) = A4 lam h := by
  rw [C_assembly hlam h hh h1]
  let a := fourCandidate_hlam h hh h1
  change a.WeightedArea = _
  rw [a.candidate_area_formula (fourCandidate_snell hlam h hh h1)]
  have hc : Real.cos a.beta = h := by
    change Real.cos (Real.pi/2-Real.arcsin h) = h
    rw [Real.cos_pi_div_two_sub, Real.sin_arcsin (by linarith) h1]
  rw [hc]
  change 4 * Real.sin (Real.arccos (h/lam))/h +
    2*(lam*Real.arccos (h/lam)-Real.sin (Real.arccos (h/lam))*h)/h^2 +
    2*(Real.arcsin h-h*Real.sqrt (1-h^2))/h^2 = _
  rw [Real.sin_arccos]
  unfold A4 B D
  rw [div_pow]
  field_simp
  ring
```

CMElementary.GeometryBridge.C_assembly

Classification: main

Source: CMElementaryGeometry.lean:639-643

Declaration SHA-256: 36111eea65048699c328fdfad74c7928894d6d24d25d7d1aeac3df275ad3b185

Mathematical statement. For $\lambda > 1$ and $0 < h \leq 1$, C_h equals the carrier of the constructed four-arc assembly.

Combine the three exact component set equalities. This is the coordinate bridge required before using any library integral identity.

```

theorem C_assembly {lam : ℝ} (hlam : 1 < lam) (h : ℝ) (hh : 0 < h) (h1 : h ≤ 1) :
  C lam h = (fourCandidate hlam h hh h1).assembly.carrier := by
  rw [C, K_core hlam h hh h1, cap4_lower hlam h hh h1, cap4_upper hlam h hh h1]
  rfl

```

CMElementary.GeometryBridge.C_perimeter

Classification: main

Source: CMElementaryGeometry.lean:738-747

Declaration SHA-256: 8c97261adbc31a2bd30879f7c2a5ab32572dd45c6907559d577bd3fdcf08d3b2

Mathematical statement. For $\lambda > 1$ and $0 < h \leq 1$,

$$P_\lambda(C_h) = P_4(h).$$

After identifying carriers, the candidate complete-frontier perimeter is

$$4\lambda\alpha/h + 4\arcsin(h)/h,$$

which is

$$4B(h)/h.$$

```

theorem C_perimeter {lam : ℝ} (hlam : 1 < lam) (h : ℝ) (hh : 0 < h) (h1 : h ≤ 1) :
  WeightedPerimeter lam (FrontierMeasure (C lam h)) = P4 lam h := by
  rw [C_assembly hlam h hh h1]
  let a := fourCandidate hlam h hh h1
  change a.WeightedPerimeter = _
  rw [a.candidate_perimeter_formula (fourCandidate_snell hlam h hh h1)]
  change 4*lam*Real.arccos (h/lam)/h + 4*Real.arcsin h/h = _
  unfold P4 B
  ring

```

CMElementary.GeometryBridge.E_area

Classification: main

Source: CMElementaryGeometry.lean:345-357

Declaration SHA-256: 26911f7fec1c4fdf0821ee4fddb03a1b3f0fb250f807d52386163b0c1a116545

Mathematical statement. For every admissible three-arc assembly of curvature h ,

$$A_\lambda(E_h) = A_3(h).$$

Identify the actual carriers, apply the assembly weighted-area theorem, and simplify sine differences and inverse angles into B, D .

```

theorem E_area {lam : ℝ} (a : TypeThreeAssembly lam) :
  WeightedArea lam (E lam a.h) = A3 lam a.h := by
  rw [E_assembly a]
  change a.WeightedArea = _
  rw [a.weightedArea_formula]
  unfold TypeThreeAssembly.scalarWeightedArea TypeThreeAssembly.angleTerm
  TypeThreeAssembly.sineGap TypeThreeAssembly.outerAngle
  TypeThreeAssembly.outerArgument TypeThreeAssembly.innerAngle
  TypeThreeAssembly.shapeParameter
  rw [sine_gap, Real.arccos_eq_pi_div_two_sub_arcsin (2 * a.h - 1)]
  unfold A3 B
  ring

```

CMElementary.GeometryBridge.E_assembly

Classification: main

Source: CMElementaryGeometry.lean:224-228

Declaration SHA-256: c1f0d7af54ce95a3387b069cc3722450d390ba0144860c4a288a74d575a7dd2c

Mathematical statement. For every admissible three-arc assembly of curvature h , its carrier equals E_h .

Combine the core and upper-cap set equalities; this supplies the exact carrier identification used in the integral bridges.

```
theorem E_assembly {lam : ℝ} (a : TypeThreeAssembly lam) :  
  E lam a.h = a.carrier := by  
  rw [E, K3_core a, cap3_outer a]  
  rfl
```

CMElementary.GeometryBridge.E_bottom_frontier

Classification: main

Source: CMElementaryGeometry.lean:288-304

Declaration SHA-256: ed0fb53bddfd325b31955a995791b4c29602ba5684e3bd105c2d1aa514d54853

Mathematical statement. For every admissible three-arc assembly of curvature h ,

$$\partial E_h \cap \{y = -1\} = \{(x, -1) : |x| \leq D(2h - 1)/h\}.$$

Closedness puts the frontier in the carrier; conversely the assembly proves its entire bottom segment is frontier. Combine with the bottom-section equality.

```
theorem E_bottom_frontier {lam : ℝ} (a : TypeThreeAssembly lam) :  
  frontier (E lam a.h) ∩ {p : PlanePoint | p.2 = -1} =  
    {p : PlanePoint | p.2 = -1 ∧ |p.1| ≤ D lam (2 * a.h - 1) / a.h} := by  
  apply Set.Subset.antisymm  
  · intro p hp  
    have hclosed : IsClosed (E lam a.h) := by  
      rw [E_assembly a]  
      exact a.isClosed_carrier  
    rw [← E_bottom_section]  
    exact ⟨hclosed.frontier_subset hp.1, hp.2⟩  
  · intro p hp  
    rw [E_assembly a]  
    refine ⟨a.bottomSegmentCarrier_subset_frontier ?_, hp.1⟩  
    change p.2 = -1 ∧ |p.1| ≤ a.sideHalfWidth  
    rw [side_width]  
    exact hp
```

CMElementary.GeometryBridge.E_bottom_section

Classification: main

Source: CMElementaryGeometry.lean:262-287

Declaration SHA-256: cb328b960a0430a84de9eaba267689fedbc4342a75a8851f06d00f004a71af25

Mathematical statement. For all real λ, r ,

$$E_r \cap \{y = -1\} = \{(x, -1) : |x| \leq D(2r - 1)/r\}.$$

The upper cap cannot meet the bottom interface; the core radicand is identically zero there, even with the total division convention.

```
theorem E_bottom_section (lam r : ℝ) :  
  E lam r ∩ {p : PlanePoint | p.2 = -1} =  
    {p : PlanePoint | p.2 = -1 ∧ |p.1| ≤ D lam (2 * r - 1) / r} := by  
  ext p
```

```

constructor
· rintro ⟨(hk | hc), hy⟩
· refine ⟨hy, ?_⟩
  have hx := hk.2.2
  change |p.1| ≤ D lam (2 * r - 1) / r +
    Real.sqrt ((1 / r) ^ 2 - (p.2 - (1 / r - 1)) ^ 2) at hx
  rw [hy, show (1 / r) ^ 2 - ((-1 : ℝ) - (1 / r - 1)) ^ 2 = 0 by ring,
    Real.sqrt_zero, add_zero] at hx
  exact hx
· have hh : 1 ≤ p.2 := hc.1
  change p.2 = -1 at hy
  linarith
· rintro ⟨hy, hx⟩
  refine ⟨Or.inl ?_, hy⟩
  change -1 ≤ p.2 ∧ p.2 ≤ 1 ∧ _
  rw [hy]
  refine ⟨le_rfl, by norm_num, ?_⟩
  change |p.1| ≤ D lam (2 * r - 1) / r +
    Real.sqrt ((1 / r) ^ 2 - ((-1 : ℝ) - (1 / r - 1)) ^ 2)
  simpa only [show (1 / r) ^ 2 - ((-1 : ℝ) - (1 / r - 1)) ^ 2 = 0 by ring,
    Real.sqrt_zero, add_zero] using hx

```

CMElementary.GeometryBridge.E_half_disk

Classification: main

Source: CMElementaryGeometry.lean:229-248

Declaration SHA-256: 33bc8668cf575aec788fdfae9213ec87b05ab1165d2368372477c264572d4074

Mathematical statement. For every real λ ,

$$E_{1/2} = \{(x, y) : x^2 + (y - 1)^2 \leq 4\}.$$

At this parameter the sine gap vanishes and both pieces have the same disk center. Split the disk at

$$y = 1$$

and simplify the core radicand to recover the whole disk.

```

theorem E_half_disk (lam : ℝ) :
  E lam (1 / 2) = {p : PlanePoint | p.1 ^ 2 + (p.2 - 1) ^ 2 ≤ 4} := by
  ext p
  simp only [E, K3, upperCap3, D, Set.mem_union, Set.mem_ofPred_eq]
  norm_num
  constructor
· rintro ⟨(hlo, hhi, hx) | ⟨hy, hd⟩⟩
  · have hs : 0 ≤ 4 - (p.2 - 1) ^ 2 := by nlinarith
    have := (abs_le_sqrt_iff p.1 _ hs).1 hx
    nlinarith
  · exact hd
· intro hd
  by_cases hy : p.2 ≤ 1
  · left
    have hlo : -1 ≤ p.2 := by nlinarith [sq_nonneg p.1]
    refine ⟨hlo, hy, ?_⟩
    apply (abs_le_sqrt_iff p.1 _ (by nlinarith [sq_nonneg p.1])).2
    nlinarith
  · exact Or.inr ⟨le_of_lt (lt_of_not_ge hy), hd⟩

```

CMElementary.GeometryBridge.E_perimeter

Classification: main

Source: CMElementaryGeometry.lean:358-370

Declaration SHA-256: a49183f07902e3ede19ad8a842f68990ab55b78592388d3d979e3541ccf7970b

Mathematical statement. For every admissible three-arc assembly of curvature h ,

$$P_\lambda(E_h) = P_3(h).$$

The assembly theorem already integrates density over the complete Euclidean frontier. Rewrite its angle and sine terms using the explicit scalar notation.

```
theorem E_perimeter {lam : ℝ} (a : TypeThreeAssembly lam) :
  WeightedPerimeter lam (FrontierMeasure (E lam a.h)) = P3 lam a.h := by
  rw [E_assembly a]
  change a.WeightedPerimeter = _
  rw [a.weightedPerimeter_formula]
  unfold TypeThreeAssembly.scalarWeightedPerimeter TypeThreeAssembly.angleTerm
  TypeThreeAssembly.sineGap TypeThreeAssembly.outerAngle
  TypeThreeAssembly.outerArgument TypeThreeAssembly.innerAngle
  TypeThreeAssembly.shapeParameter
  rw [sine_gap, Real.arccos_eq_pi_div_two_sub_arcsin (2 * a.h - 1)]
  unfold P3 B
  ring
```

CMElementary.GeometryBridge.K3_core

Classification: main

Source: CMElementaryGeometry.lean:176-212

Declaration SHA-256: 2864cc2e686087b540a455347a5834437db804c051a3d2801e42dcb5bd3e614c

Mathematical statement. For every admissible three-arc assembly of curvature h , its core carrier equals $K_h^{(3)}$.

The side width identity and horizontal-section lemma identify the rectangle plus two clipped disks with the single width inequality; the strip radicand is nonnegative because the radius exceeds one.

```
theorem K3_core {lam : ℝ} (a : TypeThreeAssembly lam) :
  K3 lam a.h = a.coreCarrier := by
  ext p
  by_cases hy : -1 ≤ p.2 ∧ p.2 ≤ 1
  · have hs : 0 ≤ a.radius ^ 2 - (p.2 - (a.radius - 1)) ^ 2 := by
      have hab : |p.2 - (a.radius - 1)| ≤ |a.radius| := by
        rw [abs_of_pos a.radius_pos]
        exact abs_le.mpr ⟨by linarith [hy.1], by linarith [hy.2, a.one_lt_radius]⟩
      have := sq_le_sq.mpr hab
      linarith
    have hsec := horizontal_section p.1 a.sideHalfWidth
    (a.radius ^ 2 - (p.2 - (a.radius - 1)) ^ 2) a.sideHalfWidth_nonneg hs
  simp only [K3, Set.mem_ofPred_eq, ← side_width a,
    hy.1, hy.2, true_and]
  change |p.1| ≤ a.sideHalfWidth +
    Real.sqrt (a.radius ^ 2 - (p.2 - (a.radius - 1)) ^ 2) ↔ _
  rw [hsec]
  simp only [TypeThreeAssembly.coreCarrier, TypeThreeAssembly.rectangleCarrier,
    TypeThreeAssembly.leftSegmentCarrier, TypeThreeAssembly.rightSegmentCarrier,
    TypeThreeAssembly.leftCenter, TypeThreeAssembly.rightCenter,
    Set.mem_union, Set.mem_inter_iff, Set.mem_ofPred_eq, hy.1, hy.2,
    and_true, sub_neg_eq_add]
  constructor
  · rintro (hp | ⟨hd, hx⟩ | ⟨hd, hx⟩)
    · exact Or.inl (Or.inl hp)
    · exact Or.inl (Or.inr ⟨by linarith, hx⟩)
    · exact Or.inr ⟨by linarith, hx⟩
  · rintro ((hp | ⟨hd, hx⟩) | ⟨hd, hx⟩)
    · exact Or.inl hp
    · exact Or.inr (Or.inl ⟨by linarith, hx⟩)
    · exact Or.inr (Or.inr ⟨by linarith, hx⟩)
  · constructor
  · intro hp
    exact (hy ⟨hp.1, hp.2.1⟩).elim
  · intro hp
    exact (hy (abs_le.mp (a.coreCarrier_y_bounds hp))).elim
```

CMElementary.GeometryBridge.K_core

Classification: main

Source: CMElementaryGeometry.lean:583-597

Declaration SHA-256: a3ce44dbe7cbd338db626634d2aa7957319833c03c7387021e4daeb815f196b9

Mathematical statement. For $\lambda > 1$ and $0 < h \leq 1$, K_h equals the strip-core carrier of the constructed four-arc candidate.

Insert its chord and side angle into the general core-width formula; the constant width offset simplifies to

$$D(h)/h.$$

```
theorem K_core {lam : ℝ} (hlam : 1 < lam) (h : ℝ) (hh : 0 < h) (h1 : h ≤ 1) :
  K lam h = (fourCandidate hlam h hh h1).stripCore.carrier := by
  rw [core_width]
  ext p
  simp only [K, Set.mem_ofPred_eq, FourArcCandidate.stripCore, FourArcCandidate.capChord,
    StripCore.radius, StripCore.sideAngle, fourCandidate, Real.cos_arcsin, Real.sin_arccos]
  have hw : 2 * Real.sqrt (1-(h/lam)^2)/h/2 -
    (1/h)*Real.sqrt (1-h^2) = D lam h / h := by
    unfold D
    rw [div_pow]
    ring
  rw [hw]
  simp only [abs_le]
  tauto
```

CMElementary.GeometryBridge.cap3_outer

Classification: main

Source: CMElementaryGeometry.lean:213-223

Declaration SHA-256: 2140a5194c99cf5b4856225e98d69256f9c79b99ca0647ae5ad4c1adf63bf5da

Mathematical statement. For every admissible three-arc assembly of curvature h , its upper cap carrier equals $H_h^{(3)}$.

Expand the center, radius and upper-half-plane condition. The result holds on both sides of the semicircular parameter.

```
theorem cap3_outer {lam : ℝ} (a : TypeThreeAssembly lam) :
  upperCap3 lam a.h = a.outerCap.carrier := by
  ext p
  simp only [OneSidedCircularCap.carrier, OneSidedCircularCap.radiusSquaredAt,
    OneSidedCircularCap.center, TypeThreeAssembly.outerCap_radius]
  change (1 ≤ p.2 ^ _) ↔
    ((p.1 - 0) ^ 2 + (p.2 - (1 - a.radius * Real.cos a.outerAngle)) ^ 2 ≤
      a.radius ^ 2 ^ 1 ≤ p.2)
  simp only [sub_zero]
  exact and_comm
```

CMElementary.GeometryBridge.cap4_lower

Classification: main

Source: CMElementaryGeometry.lean:611-638

Declaration SHA-256: 72a1ca1b6ff442b2e011b6c32059f321f81f287d8c74261f2c7a82c1ad2b72a7

Mathematical statement. For $\lambda > 1$ and $0 < h \leq 1$, $\sigma(H_h)$ equals the lower-cap carrier of the constructed four-arc assembly.

Reflection changes the sign of the vertical center and changes the upper half-plane to the lower half-plane while preserving the squared disk distance.

```
theorem cap4_lower {lam : ℝ} (hlam : 1 < lam) (h : ℝ) (hh : 0 < h) (h1 : h ≤ 1) :
  sigma '' upperCap4 lam h = (fourCandidate hlam h hh h1).assembly.lowerCap.carrier := by
```

```

let a := fourCandidate hlam h hh h1
have hr : a.assembly.lowerCap.radius = 1/h := a.four_arcs_common_radius.2
dsimp [a] at hr
ext p
simp only [OneSidedCircularCap.carrier, OneSidedCircularCap.radiusSquaredAt,
  OneSidedCircularCap.center, hr]
change (∃ q ∈ upperCap4 lam h, sigma q = p) ↔
  ((p.1-0)^2 + (p.2-(-1+(1/h)*Real.cos (Real.arccos (h/lam))))^2 ≤ (1/h)^2 ∧ p.2 ≤ -1)
constructor
· rintro (q,hq,rfl)
  change _ ≤ _ ^ -q.2 ≤ -1
  have hd := hq.2
  change q.1^2+(q.2-(1-(1/h)*Real.cos (Real.arccos (h/lam))))^2 ≤ (1/h)^2 at hd
  dsimp [sigma]
  constructor
  · nlinarith [hd]
  · linarith [hq.1]
· rintro (hd,hy)
  refine (sigma p, ?_, ?_)
  · change 1 ≤ -p.2 ^ _
    constructor
    · linarith
    · change p.1^2+(-p.2-(1-(1/h)*Real.cos (Real.arccos (h/lam))))^2 ≤ (1/h)^2
      nlinarith [hd]
  · simp [sigma]

```

CMVElementary.GeometryBridge.cap4_upper

Classification: main

Source: CMVElementaryGeometry.lean:598-610

Declaration SHA-256: f441242c0d9f6318efed4cd2ba6e4ad6a81fcdc8e3d7771eacd9f798d79f5bc2

Mathematical statement. For $\lambda > 1$ and $0 < h \leq 1$, H_h equals the upper-cap carrier of the constructed four-arc assembly.

Use the proved common radius h^{-1} and expand the cap center and disk inequality.

```

theorem cap4_upper {lam : ℝ} (h1 : 1 < lam) (h : ℝ) (hh : 0 < h) (h1 : h ≤ 1) :
  upperCap4 lam h = (fourCandidate hlam h hh h1).assembly.upperCap.carrier := by
let a := fourCandidate hlam h hh h1
have hr : a.assembly.upperCap.radius = 1/h := a.four_arcs_common_radius.1
dsimp [a] at hr
ext p
simp only [OneSidedCircularCap.carrier, OneSidedCircularCap.radiusSquaredAt,
  OneSidedCircularCap.center, hr]
change (1 ≤ p.2 ^ _) ↔ ((p.1-0)^2 +
  (p.2-(1-(1/h)*Real.cos (Real.arccos (h/lam))))^2 ≤ (1/h)^2 ∧ 1 ≤ p.2)
simp only [sub_zero]
exact and_comm

```

CMVElementary.GeometryBridge.core_width

Classification: main

Source: CMVElementaryGeometry.lean:539-582

Declaration SHA-256: af8d9f8e6e0bf124954fc510fa0ca2b2c3e4d56c02f7c034bd8fff1b46ccb71f

Mathematical statement. For a strip core with chord L , radius R and side angle β , its carrier is

$$\{(x, y) : |y| \leq 1, |x| \leq L/2 - R \cos \beta + \sqrt{R^2 - y^2}\}.$$

The identities

$$R \sin \beta = 1$$

and

$$(R\cos\beta)^2 = R^2 - 1$$

verify the offset-section hypotheses at every height in the strip.

```

theorem core_width (c : StripCore) :
  c.carrier = {p : PlanePoint | |p.2| ≤ 1 ∧
    |p.1| ≤ c.chord/2 - c.radius * Real.cos c.sideAngle +
      Real.sqrt (c.radius^2 - p.2^2)} := by
  have hc : 0 ≤ c.radius * Real.cos c.sideAngle := by
    rw [c.cos_sideAngle]
    exact mul_nonneg c.radius_pos.le (Real.sqrt_nonneg _)
  have hr : c.radius * Real.sin c.sideAngle = 1 := by
    rw [c.sin_sideAngle, StripCore.radius]
    field_simp [ne_of_gt c.curvature_pos]
  have ht := congrArg (fun z : ℝ => c.radius^2*z) (Real.sin_sq_add_cos_sq c.sideAngle)
  have hb : (c.radius * Real.cos c.sideAngle)^2 = c.radius^2 - 1 := by
    nlinarith [sq_nonneg (c.radius * Real.cos c.sideAngle)]
  ext p
  by_cases hy : |p.2| ≤ 1
  · have hy2 : p.2^2 ≤ 1 := by nlinarith [(sq_le_sq0 (abs_nonneg p.2) (by norm_num : (0:ℝ) ≤ 1)).mpr hy,
    sq_abs p.2]
    have hs : 0 ≤ c.radius^2 - p.2^2 := by nlinarith [sq_nonneg (c.radius * Real.cos c.sideAngle)]
    have hcs : c.radius * Real.cos c.sideAngle ≤ Real.sqrt (c.radius^2 - p.2^2) := by
      apply (sq_le_sq0 hc (Real.sqrt_nonneg _)).mp
      rw [Real.sq_sqrt hs, hb]
      linarith
    have he := offset_section p.1 (c.chord/2 - c.radius * Real.cos c.sideAngle)
      (c.chord/2) (c.radius^2 - p.2^2) hs (by linarith)
      (by linarith [c.chord_pos]) (by linarith)
    simp only [Set.mem_ofPred_eq, hy, true_and]
    rw [he]
    simp only [StripCore.carrier, StripCore.rectangleCarrier, StripCore.leftCapCarrier,
      StripCore.rightCapCarrier, StripCore.leftCenterX, StripCore.rightCenterX,
      Set.mem_union, Set.mem_ofPred_eq, hy, and_true, neg_div]
  constructor
  · rintro ((hp | ⟨hd, hx⟩) | ⟨hd, hx⟩)
    · exact Or.inl hp
    · exact Or.inr (Or.inl ⟨by nlinarith, hx⟩)
    · exact Or.inr (Or.inr ⟨by nlinarith, hx⟩)
  · rintro (hp | ⟨hd, hx⟩ | ⟨hd, hx⟩)
    · exact Or.inl (Or.inl hp)
    · exact Or.inl (Or.inr ⟨by nlinarith, hx⟩)
    · exact Or.inr ⟨by nlinarith, hx⟩
  · constructor
    · intro hp
      exact (hy (c.carrier_y_bounds hp)).elim
    · intro hp
      exact (hy hp.1).elim

```

CMVElementary.GeometryBridge.fourCandidate

Classification: main

Source: CMVElementaryGeometry.lean:371-379

Declaration SHA-256: 9592404e2a8e9db2de9f06d6cd4a116fe2bca9a8fee984888822274385f8cfb9

Mathematical statement. Given $\lambda > 1$ and $0 < h \leq 1$, construct the four-arc candidate with curvature h and angle

$$\arccos(h/\lambda).$$

The quotient lies strictly between zero and one, so the angle lies strictly between zero and

$$\pi/2,$$

as required by the candidate constructor.

```

def fourCandidate {lam : ℝ} (hlam : 1 < lam) (h : ℝ) (hh : 0 < h) (h1 : h ≤ 1) :
  FourArcCandidate lam where
  h := h

```

```

alpha := Real.arccos (h / lam)
h_pos := hh
h_le_one := h1
alpha_pos := Real.arccos_pos.mpr ((div_lt_one (by linarith)).2 (by linarith))
alpha_lt_pi_div_two := Real.arccos_lt_pi_div_two.mpr (div_pos hh (by linarith))

```

CMElementary.GeometryBridge.fourCandidate_snell

Classification: main

Source: CMElementaryGeometry.lean:380-389

Declaration SHA-256: d7b2da8df52c4eb799f1fd88604ac20c38b7b681ea3f936138fce2ab8263e2ba

Mathematical statement. For $\lambda > 1$ and $0 < h \leq 1$, the constructed candidate satisfies the density jump and incidence relation

$$\lambda \cos \alpha = h.$$

The cosine of the inverse cosine is valid on its argument interval; cancellation of positive λ gives the required curvature.

```

theorem fourCandidate_snell {lam : ℝ} (hlam : 1 < lam)
  (h : ℝ) (hh : 0 < h) (h1 : h ≤ 1) :
  (fourCandidate hlam h hh h1).SatisfiesCMVTypeIVHypotheses := by
  refine ⟨hlam, ?_⟩
  change lam * Real.cos (Real.arccos (h / lam)) = h
  rw [Real.cos_arccos (by have := div_pos hh (show 0 < lam by linarith); linarith)
    ((div_le_one (by linarith)).2 (by linarith))]]
  field_simp

```

CMElementary.GeometryBridge.four_euclidean_frontier_finite

Classification: main

Source: CMElementaryGeometry.lean:661-674

Declaration SHA-256: 7255bf8d8aff02f4551a691fb699b8b854db45bd02f73e2dbbb52c481c2b0c14

Mathematical statement. For every four-arc assembly, its frontier in the Euclidean realization has finite one-dimensional Hausdorff measure.

The exact frontier is a union of four arc traces whose individually proved Hausdorff measures are finite.

```

theorem four_euclidean_frontier_finite (a : FourArcAssembly) :
  (Measure.hausdorffMeasure 1 : Measure EuclideanPlane)
    (frontier (planeEuclideanHomeomorph '' a.carrier)) ≠ ⊤ := by
  rw [fourArc_euclidean_frontier]
  apply measure_union_ne_top
  · rw [exact_euclidean_leftArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top
  apply measure_union_ne_top
  · rw [exact_euclidean_rightArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top
  apply measure_union_ne_top <.>
  rw [exact_euclidean_capTrace_hausdorffMeasure] <.>
  exact ENNReal.ofReal_ne_top

```

CMElementary.GeometryBridge.four_lower_chord

Classification: main

Source: CMElementaryGeometry.lean:704-718

Declaration SHA-256: ea53d14df83b28b6ff1d37f67f32312ef47517db52c766fac334806d2ca49d27

Mathematical statement. For a four-arc assembly with chord length L , every $(x, -1)$ with

$$|x| < L/2$$

lies in the interior of its carrier.

Use membership in the lower cap chord and exclude each component of the frontier; this proves the lower attaching segment is not exposed perimeter.

```
theorem four_lower_chord (a : FourArcAssembly) {p : PlanePoint}
  (hy : p.2 = -1) (hx : |p.1| < a.core.chord/2) : p ∈ interior a.carrier := by
  have hnot : p ∉ frontier a.carrier := by
    rw [a.frontier_carrier]
    rintro (hl | hr | hu | hb)
    · linarith [(abs_lt.mp hx).1, hl.2.1]
    · linarith [(abs_lt.mp hx).2, hr.2.1]
    · have hh : 1 ≤ p.2 := hu.2
      linarith
    · exact strict_chord_not_arc a.lowerCap hy (by simp [FourArcAssembly.lowerCap] using hx) hb
  have hp : p ∈ a.carrier := Or.inr (a.lowerCap.chordCarrier_subset_carrier (by
    simp [FourArcAssembly.lowerCap, OneSidedCircularCap.chordCarrier] using And.intro hy hx.le))
  by_contra hn
  exact hnot ((mem_frontier_iff_notMem_interior hp).mpr hn)
```

CMElementary.GeometryBridge.four_realization

Classification: main

Source: CMElementaryGeometry.lean:748-774

Declaration SHA-256: 1b8185e0cc3f160a04acc0e5e04323f2a403c0ab76c247907611a7bdaa609ec8

Mathematical statement. For $\lambda > 1$ and $0 < h \leq 1$, the complete four-arc realization assertion holds.

Combine the explicit candidate, its incidence proof, all regularity and integral bridges, both strict chord-interior inclusions, and the reflected-cap location.

```
theorem four_realization {lam h : ℝ} (hlam : 1 < lam) (hh : 0 < h) (h1 : h ≤ 1) :
  FourArcRealization lam h := by
  let a := fourCandidate hlam h hh h1
  refine ⟨⟨a, rfl, rfl, fourCandidate_snell hlam h hh h1, C_assembly hlam h hh h1⟩,
    ?, C_area hlam h hh h1, C_perimeter hlam h hh h1, ?, ?, reflected_cap_below lam h⟩
  · rw [C_assembly hlam h hh h1]
    exact four_regularity lam a.assembly
  · intro p hp
    rw [C_assembly hlam h hh h1]
    apply four_upper_chord a.assembly hp.1
    change |p.1| < (2*Real.sin (Real.arccos (h/lam)))/h)/2
    have hx := hp.2
    convert hx using 1
    ring
  · intro p hp
    rw [C_assembly hlam h hh h1]
    apply four_lower_chord a.assembly hp.1
    change |p.1| < (2*Real.sin (Real.arccos (h/lam)))/h)/2
    have hx := hp.2
    convert hx using 1
    ring

end GeometryBridge

/-- Unconditional, branch-complete realization of the literal coordinate regions.
The area and perimeter fields descend from the actual integral/frontier theorems. -/
```

CMElementary.GeometryBridge.four_regularity

Classification: main

Source: CMElementaryGeometry.lean:675-688

Declaration SHA-256: 6063139ac078a2209fbe7fc6efd1db58eb52039ae2c5d24774d181000e68c1d8

Mathematical statement. For every real λ and every four-arc assembly, its carrier has the full region-regularity properties.

Closedness and boundedness are geometric; the bounded density is integrable on the finite-area carrier and on its finite-length frontier. No density-jump hypothesis is needed for this finiteness assertion.

```
theorem four_regularity (lam : ℝ) (a : FourArcAssembly) : RegionRegularity lam a.carrier := by
  refine ⟨a.isClosed_carrier,
    ((core_bounded a.core).union (cap_bounded a.upperCap)).union (cap_bounded a.lowerCap),
    a.measurableSet_carrier, ?_, ?_, ?_⟩
  · exact ((core_integrableOn lam a.core).union (cap_integrableOn lam a.upperCap)).union
    (cap_integrableOn lam a.lowerCap)
  · unfold FrontierMeasure
    rw [integrable_map_measure (measurable_stripDensity lam).aestronglyMeasurable
      planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable]
    exact euclideanStripDensity_integrableOn_of_measure_ne_top lam (four_euclidean_frontier_finite a)
  · unfold FrontierMeasure
    rw [Measure.map_apply planeEuclideanHomeomorph.symm.continuous.measurable MeasurableSet.univ]
    simpa using lt_top_iff_ne_top.mpr (four_euclidean_frontier_finite a)
```

CMElementary.GeometryBridge.four_upper_chord

Classification: main

Source: CMElementaryGeometry.lean:689-703

Declaration SHA-256: a1a7708ba2cae5d73698e1a96285a00a773db00178be03734f2731b4ee4f2e11

Mathematical statement. For a four-arc assembly with chord length L , every $(x, 1)$ with

$$|x| < L/2$$

lies in the interior of its carrier.

Such a point is in the attaching chord but excluded from the exact four-arc frontier, by strict chord inequalities and the location of the lower cap.

```
theorem four_upper_chord (a : FourArcAssembly) {p : PlanePoint}
  (hy : p.2 = 1) (hx : |p.1| < a.core.chord/2) : p ∈ interior a.carrier := by
  have hnot : p ∉ frontier a.carrier := by
    rw [a.frontier_carrier]
    rintro (hl | hr | hu | hb)
    · linarith [(abs_lt.mp hx).1, hl.2.1]
    · linarith [(abs_lt.mp hx).2, hr.2.1]
    · exact strict_chord_not_arc a.upperCap hy (by simpa [FourArcAssembly.upperCap] using hx) hu
    · have hh : p.2 ≤ -1 := hb.2
      linarith
  have hp : p ∈ a.carrier := Or.inl (Or.inr (a.upperCap.chordCarrier_subset_carrier (by
    simpa [FourArcAssembly.upperCap, OneSidedCircularCap.chordCarrier] using And.intro hy hx.le)))
  by_contra hn
  exact hnot ((mem_frontier_iff_notMem_interior hp).mpr hn)
```

CMElementary.GeometryBridge.reflected_cap_below

Classification: main

Source: CMElementaryGeometry.lean:317-324

Declaration SHA-256: 294e02a2d2dbbc8a935f73af2e7892771244bc90c2d4e857fd32d9d4484129b9

Mathematical statement. For all real λ, h ,

$$\sigma(H_h) \subseteq \{(x, y) : y \leq -1\}.$$

Reflection negates the upper-cap vertical coordinate, whose defining inequality is

$$y \geq 1.$$

```
theorem reflected_cap_below (lam h : ℝ) :
  sigma '' upperCap4 lam h ⊆ {p : PlanePoint | p.2 ≤ -1} := by
  rintro p ⟨q, hq, rfl⟩
  change -q.2 ≤ -1
  exact neg_le_neg hq.1

-- This mass bound uses the public complete-frontier decomposition and exact
-- trace measures; no modeled perimeter sum is used to infer finiteness.
```

CMElementary.GeometryBridge.side_width

Classification: main

Source: CMElementaryGeometry.lean:132-139

Declaration SHA-256: a690f71ca11be015e8cbefc4396612cc139bee2f73d2713555ec3c9c25c58675

Mathematical statement. For every three-arc assembly with $\lambda > 1$ and $0 < h < 1$, its side half-width equals

$$D(2h - 1)/h.$$

The assembly definition is the reciprocal curvature times a sine difference; the preceding radical identity identifies it with the coordinate width.

```
theorem side_width {lam : ℝ} (a : TypeThreeAssembly lam) :
  a.sideHalfWidth = D lam (2 * a.h - 1) / a.h := by
  rw [TypeThreeAssembly.sideHalfWidth]
  change (1 / a.h) * (Real.sin (Real.arccos ((2 * a.h - 1) / lam)) -
    Real.sin (Real.arccos (2 * a.h - 1))) = _
  rw [sine_gap]
  ring
```

CMElementary.GeometryBridge.sine_gap

Classification: main

Source: CMElementaryGeometry.lean:128-131

Declaration SHA-256: 972dfd8e12f3ba528b4bc8054ce9f622e3d59c0d76ad543257c792688bb4e54e

Mathematical statement. For all real λ, z ,

$$\sin(\arccos(z/\lambda)) - \sin(\arccos z) = D_\lambda(z).$$

Expand both sine-of-arccosine terms into total real square roots and use

$$(z/\lambda)^2 = z^2/\lambda^2.$$

```
theorem sine_gap (lam z : ℝ) :
  Real.sin (Real.arccos (z / lam)) - Real.sin (Real.arccos z) = D lam z := by
  simp only [Real.sin_arccos, D, div_pow]
```

CMElementary.GeometryBridge.three_frontier_finite

Classification: main

Source: CMElementaryGeometry.lean:325-344

Declaration SHA-256: 4ff646f8685352a1506b99597ef8d06abcc7865827dbb535b135a57304677708

Mathematical statement. The canonical complete-frontier measure of every admissible three-arc assembly has finite total mass.

Decompose its Euclidean frontier into two strip arcs, one exterior arc and a segment. Each exact Hausdorff measure is finite; transporting the restricted measure back preserves total mass.

```
theorem three_frontier_finite {lam : ℝ} (a : TypeThreeAssembly lam) :
  FrontierMeasure a.carrier Set.univ < T := by
  have hm : (MeasureTheory.Measure.hausdorffMeasure 1 : Measure EuclideanPlane)
    (frontier (planeEuclideanHomeomorph '' a.carrier)) ≠ T := by
    rw [a.euclidean_frontier]
    apply measure_union_ne_top
    · rw [a.exact_euclidean_leftArcTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top
    apply measure_union_ne_top
    · rw [a.exact_euclidean_rightArcTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top
    apply measure_union_ne_top
    · rw [a.exact_euclidean_capTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top
    · rw [a.exact_euclidean_bottomSegmentCarrier_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top
  unfold FrontierMeasure
  rw [Measure.map_apply planeEuclideanHomeomorph.symm.continuous.measurable MeasurableSet.univ]
  simpa using lt_top_iff_ne_top.mpr hm
```

CMElementary.GeometryBridge.three_realization

Classification: main

Source: CMElementaryGeometry.lean:494-514

Declaration SHA-256: db0addcbf73e2d7c5cb5db55b0994f694f8583f8e05e0428f700aa9cac29e2be

Mathematical statement. For $\lambda > 1$ and $0 < r < 1$, the complete three-arc realization assertion holds.

Construct the assembly at r , apply the carrier and integral bridges, prove its major-angle implication by inverse-cosine order, and use the exact disk identity at one half.

```
theorem three_realization {lam r : ℝ} (hlam : 1 < lam) (hr : 0 < r) (hr1 : r < 1) :
  ThreeArcRealization lam r := by
  let a : TypeThreeAssembly lam := ⟨r, hlam, hr, hr1⟩
  refine ⟨⟨a, rfl, E_assembly a⟩, three_regularity a, E_area a, E_perimeter a,
    ⟨a.outerAngle_pos, a.outerAngle_lt_pi⟩, ?_, ?_, E_bottom_frontier a, ?_⟩
  · intro hm
    have hn : (2*r-1)/lam < 0 := div_neg_of_neg_of_pos (by linarith) (by linarith)
    have h := Real.arccos_lt_arccos (show -1 ≤ (2*r-1)/lam from a.outerArgument_mem.1.le)
      hn (show (0:ℝ) ≤ 1 by norm_num)
    simpa using h
  · intro p hp
    rw [E_assembly a]
    apply three_strict_chord a hp.1
    simpa [TypeThreeAssembly.radius, TypeThreeAssembly.outerAngle,
      TypeThreeAssembly.outerArgument, TypeThreeAssembly.shapeParameter, a, div_eq_mul_inv,
      mul_comm] using hp.2
  · intro heq
    rw [heq]
    exact E_half_disk lam
```

CMElementary.GeometryBridge.three_regularity

Classification: main

Source: CMElementaryGeometry.lean:422-427

Declaration SHA-256: 73f3ce51b28ce02d6cc63d5fb204d846cba402ac345eddf320f580aac4f46265

Mathematical statement. Every admissible three-arc assembly of curvature h gives a region E_h satisfying the full region-regularity assertion.

Transport the carrier equality and combine closedness, boundedness, measurability, actual area and frontier integrability, and finite frontier mass.

```
theorem three_regularity {lam : ℝ} (a : TypeThreeAssembly lam) : RegionRegularity lam (E lam a.h) := by
  rw [E_assembly a]
  exact ⟨a.isClosed_carrier, three_bounded a, a.measurableSet_carrier,
    a.integrableOn_carrier, a.integrable_frontierMeasure, three_frontier_finite a⟩

-- Local geometric lemma, using only public cap definitions and identities.
```

CMElementary.GeometryBridge.three_strict_chord

Classification: main

Source: CMElementaryGeometry.lean:450-493

Declaration SHA-256: 4464866837d47e9a8c1cbf531c0302898770abce4a983a8a096cdbf3a49d1a2b

Mathematical statement. For an admissible three-arc assembly, every point $(x, 1)$ with

$$|x| < R \sin \alpha$$

lies in the interior of its carrier.

The point is on the attaching chord and hence in the union, but the exact frontier decomposition excludes it from every arc and from the bottom segment. Membership outside the frontier implies interior membership.

```
theorem three_strict_chord {lam : ℝ} (a : TypeThreeAssembly lam) {p : PlanePoint}
  (hy : p.2 = 1) (hx : |p.1| < a.radius * Real.sin a.outerAngle) :
  p ∈ interior a.carrier := by
  have hnot : p ∉ frontier a.carrier := by
    rw [a.frontier_carrier]
    rintro (hl | hr | hu | hb)
    · have hc := hl.1
      have hs := hl.2.1
      change p.1 ≤ -a.sideHalfWidth at hs
      simp only [TypeThreeAssembly.leftCenter, hy] at hc
      have ht := three_top_circle a
      have hw := a.sideHalfWidth
      have hwidth : a.sideHalfWidth + a.radius * Real.sin a.innerAngle =
        a.radius * Real.sin a.outerAngle := by rw [TypeThreeAssembly.sideHalfWidth]; ring
      have heq : -(p.1+a.sideHalfWidth) = a.radius * Real.sin a.innerAngle := by
        apply (sq_eq_sq0 (by linarith) (mul_nonneg a.radius_pos.le a.innerSin_pos.le)).mp
        nlinarith
      linarith [(abs_lt.mp hx).1]
    · have hc := hr.1
      have hs := hr.2.1
      change a.sideHalfWidth ≤ p.1 at hs
      simp only [TypeThreeAssembly.rightCenter, hy] at hc
      have ht := three_top_circle a
      have hwidth : a.sideHalfWidth + a.radius * Real.sin a.innerAngle =
        a.radius * Real.sin a.outerAngle := by rw [TypeThreeAssembly.sideHalfWidth]; ring
      have heq : p.1-a.sideHalfWidth = a.radius * Real.sin a.innerAngle := by
        apply (sq_eq_sq0 (by linarith) (mul_nonneg a.radius_pos.le a.innerSin_pos.le)).mp
        nlinarith
      linarith [(abs_lt.mp hx).2]
    · exact strict_chord_not_arc a.outerCap hy (by
      change |p.1-0| < (2*a.radius*Real.sin a.outerAngle)/2
```

```

      simp only [sub_zero]
      linarith) hu
    · have hh : p.2 = -1 := hb.1
      linarith
have hp : p ∈ a.carrier := 0r.inr (a.outerCap.chordCarrier_subset_carrier (by
  change p.2 = 1 ^ |p.1-0| ≤ (2*a.radius*Real.sin a.outerAngle)/2
  constructor
  · exact hy
  · simp only [sub_zero]
    linarith [hx.le]))
by_contra hn
exact hnot ((mem_frontier_iff_notMem_interior hp).mpr hn)

```

CMElementary.GreatestRootClaims

Classification: main

Source: CMElementary.lean:125-140

Declaration SHA-256: 2882e2312daec98183445737a88569349612dfd73820523f6968960d175c540c

Mathematical statement. One has

$$0 < \varepsilon < r < h$$

and

$$A_3(\varepsilon) > A_4(h).$$

The level set $Z_{\lambda, h, \varepsilon}$ is nonempty and compact, with greatest element r and

$$A_3(r) = A_4(h).$$

For $r < t \leq h$,

$$A_3(t) < A_4(h).$$

The function $Q_{\lambda, h}$ is continuous on $[r, h]$, has derivative

$$A_4(h) - A_3(t) > 0$$

on (r, h) , and satisfies

$$Q_{\lambda, h}(r) < Q_{\lambda, h}(h).$$

This is a conjunction of the listed mathematical properties, not a theorem assuming them without proof. The corresponding construction or claims theorem supplies every component from the stated density and parameter hypotheses; the detailed identities and domains in the statement distinguish endpoint continuity from interior differentiability and actual geometry from scalar formulas.

```

structure GreatestRootClaims (lam h epsilon r : ℝ) : Prop where
  epsilon_pos : 0 < epsilon
  epsilon_lt_r : epsilon < r
  r_lt_h : r < h
  epsilon_above : A4 lam h < A3 lam epsilon
  level_nonempty : (Z lam h epsilon).Nonempty
  level_compact : IsCompact (Z lam h epsilon)
  greatest : IsGreatest (Z lam h epsilon) r
  equal_area : A3 lam r = A4 lam h
  barrier : ∀ t ∈ Ioc r h, A3 lam t < A4 lam h
  q_continuous : ContinuousOn (Q lam h) (Icc r h)
  q_deriv : ∀ t ∈ Ioo r h, HasDerivAt (Q lam h) (A4 lam h - A3 lam t) t
  q_deriv_pos : ∀ t ∈ Ioo r h, 0 < A4 lam h - A3 lam t
  q_strict : Q lam h r < Q lam h h

/-- Required identification when reusing the frozen scalar library. -/

```

CMElementary.H

Classification: main

Source: CMElementary.lean:57-58

Declaration SHA-256: 7854edb36a536e3f5422a2368eda17806c34bc8a4785388c6c3b36ab5259bb52

Mathematical statement. Define

$$H(u) = J(u) - u j(u).$$

The area identity gives

$$H(h) = h^2[A_4(h) - A_3(h)]$$

when $h \neq 0$.

```
def H (lam u : ℝ) : ℝ := J lam u - u * jprime lam u
```

CMElementary.H_ge_J

Classification: main

Source: CMElementary.lean:502-512

Declaration SHA-256: c17279ea5358a898bb176a8ea0bfb9b69861b60846ef9ece0ade666383e410f5

Mathematical statement. For $\lambda > 1$ and $1/3 \leq u \leq 1$,

$$H(u) \geq J(u).$$

On this interval

$$|2u - 1| \leq u,$$

so $j(u) \leq 0$ and $-u j(u) \geq 0$, including both endpoints.

```
theorem H_ge_J {lam u : ℝ} (h1 : 1 < lam) (hu : u ∈ Icc (1 / 3) 1) :
  J lam u ≤ H lam u := by
  have hu0 : 0 ≤ u := by linarith [hu.1]
  have ha : |2 * u - 1| ≤ u := abs_le.mpr ⟨by linarith [hu.1], by linarith [hu.2]⟩
  have hd := (D_strictMono hlam).monotoneOn ⟨abs_nonneg _, ha.trans hu.2⟩ ⟨hu0, hu.2⟩ ha
  rw [D_abs] at hd
  have hj : jprime lam u ≤ 0 := by unfold jprime; linarith
  unfold H
  nlinarith [mul_nonpos_of_nonneg_of_nonpos hu0 hj]

/-- J rises to one third and falls thereafter, with both endpoint values gamma. -/
```

CMElementary.H_lower

Classification: main

Source: CMElementary.lean:535-544

Declaration SHA-256: 720b73434d49824a156db89a9d0ba747d35c36753fbd55b86da3e68c83bdd227

Mathematical statement. For $\lambda > 1$ and $0 \leq u \leq 1$,

$$H(u) \geq \gamma_\lambda.$$

On the closed lower third use the decreasing-derivative tangent lemma; on the rest use

$$H \geq J \geq \gamma_\lambda.$$

The proof does not use the separately established positivity of H' .

```

theorem H_lower {lam u : ℝ} (hlam : 1 < lam) (hu : u ∈ Icc 0 1) :
  gamma lam ≤ H lam u := by
  by_cases hut : u ≤ 1 / 3
  · exact MonotoneTangent.h_lower_third_of_monotone_tangent
    (fun z _ => D_even lam z) (D_strictMono hlam) (J_continuous lam).continuousOn
    (DerivativePilot.J_zero lam) (fun t ht => hasDerivAt_J hlam ht) ⟨hu.1, hut⟩
  · exact (J_lower hlam hu).trans (H_ge_J hlam ⟨(not_le.mp hut).le, hu.2⟩)

/-- Every auxiliary field follows from `1 < lam`, including the separate
lower-third derivative identities and the continuous limit at zero. -/

```

CMVElementary.J

Classification: main

Source: CMVElementary.lean:53-54

Declaration SHA-256: 25d540a43a2aa9c7be06cd85d61e6c7b622dc32f471c48ed20eb065861a68717

Mathematical statement. Define

$$J(u) = 2S(u) - S(2u - 1) - \pi/2.$$

At positive equal curvature, this is curvature times the difference of the two support quantities

$$P - hA.$$

```

def J (lam u : ℝ) : ℝ := 2 * S lam u - S lam (2 * u - 1) - Real.pi / 2

```

CMVElementary.J_continuous

Classification: main

Source: CMVElementary.lean:456-459

Declaration SHA-256: 84b0b93cd0d9eb84e88353584df2d2027343d5022858b007f7419abca4bc0d41

Mathematical statement. For every real λ , J is continuous on

$$\mathbb{R}.$$

Continuity follows from the total inverse trigonometric and radical definitions and affine composition.

```

theorem J_continuous (lam : ℝ) : Continuous (J lam) := by
  unfold J S B D
  fun_prop

```

CMVElementary.J_lower

Classification: main

Source: CMVElementary.lean:513-534

Declaration SHA-256: 7c17f318bc98780bcbcb8604e45210707c09ca380628497b97638e7a07fb4d393

Mathematical statement. For $\lambda > 1$ and $0 \leq u \leq 1$,

$$J(u) \geq \gamma_\lambda.$$

The derivative is positive below one third and negative above it; the two endpoint values are equal to the same constant.

```

theorem J_lower {lam u : ℝ} (hlam : 1 < lam) (hu : u ∈ Icc 0 1) :
  gamma lam ≤ J lam u := by
  by_cases hut : u ≤ 1 / 3
  · have hm : StrictMonoOn (J lam) (Icc 0 (1 / 3)) := by
    apply strictMonoOn_of_deriv_pos (convex_Icc _ _) (J_continuous lam).continuousOn
    intro t ht
    rw [interior_Icc] at ht

```

```

    rw [(hasDerivAt_J hlam ⟨ht.1, by linarith [ht.2]⟩).deriv]
    exact jprime_pos hlam ht
  simp only [DerivativePilot.J_zero] using
    hm.monotoneOn (show (0 : ℝ) ∈ Icc 0 (1 / 3) by norm_num) ⟨hu.1, hut⟩ hu.1
  · have hm : StrictAntiOn (J lam) (Icc (1 / 3) 1) := by
    apply strictAntiOn_of_deriv_neg (convex_Icc _ _) (J_continuous lam).continuousOn
    intro t ht
    rw [interior_Icc] at ht
    rw [(hasDerivAt_J hlam ⟨by linarith [ht.1], ht.2⟩).deriv]
    exact jprime_neg hlam ht
  simp only [J_one] using
    hm.antitoneOn ⟨(not_le.mp hut).le, hu.2⟩ (show (1 : ℝ) ∈ Icc (1 / 3) 1 by norm_num) hu.2

/-- The closed lower third uses the selected decreasing-derivative tangent
argument. The upper interval uses `H ≥ J ≥ gamma`, including one. -/

```

CMElementary.J_one

Classification: main

Source: CMElementary.lean:460-463

Declaration SHA-256: b4fb08f5f0734c8c331c867147a539407df5ce79eda91cd380170130cdf58655

Mathematical statement. For every real λ ,

$$J(1) = \gamma_\lambda.$$

At one both occurrences of S combine to

$$S(1) - \pi/2,$$

which is the displayed constant.

```

theorem J_one (lam : ℝ) : J lam 1 = gamma lam := by
  norm_num [J, S, B, D, gamma, zpow_neg]
  ring

```

CMElementary.K

Classification: main

Source: CMElementaryGeometry.lean:30-33

Declaration SHA-256: a10d4858e4823efb7ef74e910633b656d8f41250c171a018f71c6c9e83eb6efa

Mathematical statement. For real λ, h , define

$$K_h = \{(x, y) : -1 \leq y \leq 1, |x| \leq D(h)/h + \sqrt{h^{-2} - y^2}\}.$$

The four-arc strip core is given by its actual horizontal sections, not by an area formula.

```

def K (lam h : ℝ) : Set PlanePoint :=
  {p | -1 ≤ p.2 ∧ p.2 ≤ 1 ∧
    |p.1| ≤ D lam h / h + Real.sqrt ((1 / h) ^ 2 - p.2 ^ 2)}

```

CMElementary.K3

Classification: main

Source: CMElementaryGeometry.lean:42-47

Declaration SHA-256: 2a77d29e87870894cbc97b678ab985c697c0b3bb4aa43807c4d681411e9c085b

Mathematical statement. For real λ, r , put

$$q = 2r - 1$$

and define

$$K_r^{(3)} = \{(x, y) : -1 \leq y \leq 1, |x| \leq D(q)/r + \sqrt{r^{-2} - (y - (r^{-1} - 1))^2}\}.$$

The shifted circle center gives tangency to the bottom interface. Its geometric realization is asserted for $\lambda > 1$,

$$0 < r < 1.$$

```
def K3 (lam r : ℝ) : Set PlanePoint :=
  let q := 2 * r - 1
  {p | -1 ≤ p.2 ^ p.2 ≤ 1 ^
    |p.1| ≤ D lam q / r + Real.sqrt ((1 / r) ^ 2 - (p.2 - (1 / r - 1)) ^ 2)}

/-- Disk intersected with `y ≥ 1`: also the major segment when `r < 1/2`. -/
```

CMElementary.MonotoneTangent.endpoint_tangent

Classification: main

Source: CMElementary.lean:313-330

Declaration SHA-256: 88e52ddc8ea9dcc69a9b432eccd375de5c9f6debf8af47520109cf01915bce90

Mathematical statement. Let $u \geq 0$, let F be continuous on $[0, u]$, and suppose

$$F'(t) = g(t)$$

for $0 < t < u$, with g nonincreasing on $[0, u]$. Then

$$F(0) \leq F(u) - ug(u).$$

For $u = 0$ this is equality. Otherwise the mean value theorem gives

$$F(u) - F(0) = ug(c),$$

where $0 < c < u$, and

$$g(c) \geq g(u).$$

Neither differentiability nor continuity of g is required.

```
theorem endpoint_tangent
  {F g : ℝ → ℝ} {u : ℝ} (hu : 0 ≤ u)
  (hcont : ContinuousOn F (Icc 0 u))
  (hderiv : ∀ t ∈ Ioo 0 u, HasDerivAt F (g t) t)
  (hanti : AntitoneOn g (Icc 0 u)) :
  F 0 ≤ F u - u * g u := by
  rcases eq_or_lt_of_le hu with rfl | hu
  · simp
  obtain ⟨c, hc, hslope⟩ := exists_hasDerivAt_eq_slope F g hu hcont hderiv
  have hgc : g u ≤ g c :=
    hanti ⟨hc.1.le, hc.2.le⟩ ⟨hu.le, le_rfl⟩ hc.2.le
  have hbound : g u * u ≤ F u - F 0 := by
    apply (le_div_iff₀ hu).mp
    simpa only [sub_zero] using hgc.trans_eq hslope
  nlinarith

/-- The reflected expression is antitone by order alone: its first argument
falls and its second argument rises. Strict increase is more than sufficient. -/
```

CMVElementary.MonotoneTangent.even_difference_antitone

Classification: main

Source: CMVElementary.lean:345-360

Declaration SHA-256: 97bae7778a3a2dbe35cd6cd7377c12ce6215839e12460f95936b813a45c4b542

Mathematical statement. If

$$d(-z) = d(z)$$

for $-1 \leq z \leq 1$, and d is strictly increasing on $[0, 1]$, then $t \mapsto 4[d(2t - 1) - d(t)]$ is nonincreasing on $[0, 1/3]$.

Evenness changes the first argument to $1 - 2t$, reducing the claim to the reflected-difference order argument.

```
theorem even_difference_antitone
  {d : ℝ → ℝ}
  (heven : ∀ z ∈ Icc (-1 : ℝ) 1, d (-z) = d z)
  (hmono : StrictMonoOn d (Icc 0 1)) :
  AntitoneOn (fun t : ℝ => 4 * (d (2 * t - 1) - d t)) (Icc 0 (1 / 3)) := by
  have hreflect (t : ℝ) (ht : t ∈ Icc (0 : ℝ) (1 / 3)) :
    d (2 * t - 1) = d (1 - 2 * t) := by
    have hz : 1 - 2 * t ∈ Icc (-1 : ℝ) 1 := ⟨by linarith [ht.2], by linarith [ht.1]⟩
    rw [show 2 * t - 1 = -(1 - 2 * t) by ring]
    exact heven (1 - 2 * t) hz
  intro x hx y hy hxy
  change 4 * (d (2 * y - 1) - d y) ≤ 4 * (d (2 * x - 1) - d x)
  rw [hreflect x hx, hreflect y hy]
  exact reflected_difference_antitone hmono hx hy hxy

/-- Conditional CMV interface using only the two specified D properties. -/
```

CMVElementary.MonotoneTangent.h_lower_third_of_monotone_tangent

Classification: main

Source: CMVElementary.lean:370-390

Declaration SHA-256: 0e2a91072c4780dbfb00df264aa74c10fd7f6ee762c1b356a804608b807e7ace

Mathematical statement. Assume D is even on $[-1, 1]$ and strictly increasing on $[0, 1]$, J is continuous on $[0, 1]$,

$$J(0) = \gamma_\lambda,$$

and

$$J' = j$$

on $(0, 1)$. Then

$$H(u) \geq \gamma_\lambda$$

for $0 \leq u \leq 1/3$.

Restrict to $[0, u]$, use the nonincreasing specified derivative, and apply the endpoint tangent inequality. These five premises are proved independently, so the argument does not assume its own conclusion.

```
theorem h_lower_third_of_monotone_tangent {lam : ℝ}
  (heven : ∀ z ∈ Icc (-1 : ℝ) 1, D lam (-z) = D lam z)
  (hmono : StrictMonoOn (D lam) (Icc 0 1))
  (hcont : ContinuousOn (J lam) (Icc 0 1))
  (hzero : J lam 0 = gamma lam)
  (hderiv : ∀ t ∈ Ioo (0 : ℝ) 1, HasDerivAt (J lam) (jprime lam t) t)
  {u : ℝ} (hu : u ∈ Icc (0 : ℝ) (1 / 3)) :
  gamma lam ≤ H lam u := by
  have hsub : Icc (0 : ℝ) u ⊆ Icc 0 1 := by
  intro t ht
```

```

    exact ⟨ht.1, by linarith [ht.2, hu.2]⟩
  have hsubthird : Icc (0 : ℝ) u ⊆ Icc 0 (1 / 3) := by
    intro t ht
    exact ⟨ht.1, ht.2.trans hu.2⟩
  have htangent := endpoint_tangent hu.1 (hcont.mono hsub)
    (fun t ht => hderiv t ⟨ht.1, by linarith [ht.2, hu.2]⟩)
    ((jprime_antitone_lower_third heven hmono).mono hsubthird)
  simpa only [hzero, H] using htangent

end MonotoneTangent

```

CMElementary.MonotoneTangent.jprime_antitone_lower_third

Classification: main

Source: CMElementary.lean:361-369

Declaration SHA-256: 4a69d4e44ae00cd18b8420c38e3a7a4932e9ddfeacaf9bb7f9417aec67061e0

Mathematical statement. If D is even on $[-1, 1]$ and strictly increasing on $[0, 1]$, then j is nonincreasing on $[0, 1/3]$.

This conditional lemma applies the preceding order argument to the actual radical difference; it does not assume any bound for H .

```

theorem jprime_antitone_lower_third {lam : ℝ}
  (heven : ∀ z ∈ Icc (-1 : ℝ) 1, D lam (-z) = D lam z)
  (hmono : StrictMonoOn (D lam) (Icc 0 1)) :
  AntitoneOn (jprime lam) (Icc 0 (1 / 3)) :=
  even_difference_antitone heven hmono

/-- The actual CMV H bound on the closed lower third, conditional on five
independently required auxiliary properties, not on AuxiliaryClaims as a whole.
In particular this cannot use that record's h_lower field circularly. -/

```

CMElementary.MonotoneTangent.reflected_difference_antitone

Classification: main

Source: CMElementary.lean:331-344

Declaration SHA-256: 94ffae68ac5742cce7dc826290e898045b7e92e6507e0ceebc649344ec938685

Mathematical statement. If

$$d: \mathbb{R} \rightarrow \mathbb{R}$$

is strictly increasing on $[0, 1]$, then $t \mapsto 4[d(1 - 2t) - d(t)]$ is nonincreasing on $[0, 1/3]$.

For increasing t , the first argument decreases while the second increases; both remain in $[0, 1]$.

```

theorem reflected_difference_antitone
  {d : ℝ → ℝ} (hmono : StrictMonoOn d (Icc 0 1)) :
  AntitoneOn (fun t : ℝ => 4 * (d (1 - 2 * t) - d t)) (Icc 0 (1 / 3)) := by
  intro x hx y hy hxy
  have hx01 : x ∈ Icc (0 : ℝ) 1 := ⟨hx.1, by linarith [hx.2]⟩
  have hy01 : y ∈ Icc (0 : ℝ) 1 := ⟨hy.1, by linarith [hy.2]⟩
  have hxr : 1 - 2 * x ∈ Icc (0 : ℝ) 1 := ⟨by linarith [hx.2], by linarith [hx.1]⟩
  have hyr : 1 - 2 * y ∈ Icc (0 : ℝ) 1 := ⟨by linarith [hy.2], by linarith [hy.1]⟩
  have hfirst := hmono.monotoneOn hyr hxr (by linarith : 1 - 2 * y ≤ 1 - 2 * x)
  have hsecond := hmono.monotoneOn hx01 hy01 hxy
  linarith

/-- Evenness identifies the negative-side argument with the reflected one.
No assertion about d' or d'' is involved, including at the endpoint -1. -/

```

CMVElementary.P3

Classification: main

Source: CMVElementary.lean:45-48

Declaration SHA-256: cb5785ad3e43cf3da1bc0fe1a18a37ce203b8840e222dbdc84f4f9f0806b00b4

Mathematical statement. For real λ, t , with $q = 2t - 1$, define

$$P_3(t) = \frac{2(B(q) + \pi/2 + D(q))}{t}.$$

The last radical term includes the density-one exposed bottom segment; the geometric identity is proved separately for $0 < t < 1$.

```
def P3 (lam t : ℝ) : ℝ :=
  let q := 2 * t - 1
  2 * (B lam q + Real.pi / 2 + D lam q) / t
```

CMVElementary.P3_continuousAt

Classification: main

Source: CMVElementary.lean:640-644

Declaration SHA-256: 7e674cd3499110482a7c44788eb37aa27ecb4b4c5616c3ed50d965f7e0f157c3

Mathematical statement. For every real λ and every $t \neq 0$, P_3 is continuous at t .

Continuity of the numerator and the nonzero denominator gives the local result, including at one.

```
theorem P3_continuousAt {lam t : ℝ} (ht : t ≠ 0) : ContinuousAt (P3 lam) t := by
  unfold P3 B D
  fun_prop (disch := positivity)

/-- Local equality on an open neighborhood is what transports an ordinary derivative. -/
```

CMVElementary.P4

Classification: main

Source: CMVElementary.lean:51-52

Declaration SHA-256: fa98d4e58766dded99b172ec727c96faed5ce0f690c12b06cd4d7e96d0228209

Mathematical statement. For real λ, t , define

$$P_4(t) = \frac{4B(t)}{t}.$$

On the geometric domain this sums two strip arcs and two exterior arcs with their respective density weights.

```
def P4 (lam t : ℝ) : ℝ := 4 * B lam t / t
```

CMVElementary.Q

Classification: main

Source: CMVElementary.lean:62-64

Declaration SHA-256: d3e3e508995be11614f5392de567ba9897b0da4cbe3fbd07c5418d5edee16afe

Mathematical statement. Define

$$Q_{\lambda, h}(t) = P_3(t) + t[A_4(h) - A_3(t)].$$

Differentiating this support expression cancels the area derivative through

$$P_3' = tA_3'.$$

```
def Q (lam h t : ℝ) : ℝ := P3 lam t + t * (A4 lam h - A3 lam t)

/-- Exact contract type of `CMElementary.scalar_comparison`. -/
```

CMElementary.RegionRegularity

Classification: main

Source: CMElementaryGeometry.lean:70-79

Declaration SHA-256: f00a35a7ca4c7b622746574abc71bc8314649524dfbea7af49e098990a986e81

Mathematical statement. For a real λ and planar set F , this is the conjunction that F is closed, bounded and measurable, f_λ is integrable over F and its canonical frontier measure, and

$$\mathcal{H}^1(\partial F) < \infty.$$

These are properties to be proved of the coordinate sets. This record is not a general bounded-variation or reduced-boundary theorem.

```
structure RegionRegularity (lam : ℝ) (F : Set PlanePoint) : Prop where
  closed : IsClosed F
  bounded : Bornology.IsBounded F
  measurable : MeasurableSet F
  area_integrable : IntegrableOn (StripDensity lam) F
  perimeter_integrable : Integrable (StripDensity lam) (FrontierMeasure F)
  frontier_finite : FrontierMeasure F univ < T

/-- Carrier identification and actual integral identities on the whole
four-arc domain, including `h = 1`. -/
```

CMElementary.S

Classification: main

Source: CMElementary.lean:36-37

Declaration SHA-256: eca01a5287b23fda232f56d07e08db5438afa4ad2b1cdad6f24ae23ee6cbb5f0

Mathematical statement. Define

$$S(z) = B(z) - zD(z).$$

The cancellation

$$S' = -2D$$

makes this combination useful for comparing $P - hA$.

```
def S (lam z : ℝ) : ℝ := B lam z - z * D lam z
```

CMElementary.S_reflection

Classification: main

Source: CMElementary.lean:410-413

Declaration SHA-256: 26f33ed7c33bedaeef432b4c32e750889ffa3f094e21ef2c9ac3dfd7f1097086c

Mathematical statement. For every real λ, z ,

$$S(-z) = \lambda\pi - S(z).$$

Use evenness of D , oddness of arcsine, and

$$\arccos(-x) = \pi - \arccos x.$$

```
theorem S_reflection (lam z : ℝ) : S lam (-z) = lam * Real.pi - S lam z := by
  simp only [S, B, neg_div, Real.arccos_neg, Real.arcsin_neg, D_even]
  ring
```

CMElementary.SameCurvatureClaims

Classification: main

Source: CMElementary.lean:103-113

Declaration SHA-256: 92c4827d671dfeb577f54bd3e2cd8d72eee9b5c2fe740b8108100be082a5abfd

Mathematical statement. For every

$$0 < h \leq 1,$$

the support difference multiplied by h equals $J(h)$, and

$$h^2[A_4(h) - A_3(h)] = H(h).$$

Moreover

$$A_4(h) - A_3(h) \geq \gamma_\lambda / h^2 > 0$$

and

$$[P_4(h) - hA_4(h)] - [P_3(h) - hA_3(h)] \geq \gamma_\lambda / h > 0.$$

This is a conjunction of the listed mathematical properties, not a theorem assuming them without proof. The corresponding construction or claims theorem supplies every component from the stated density and parameter hypotheses; the detailed identities and domains in the statement distinguish endpoint continuity from interior differentiability and actual geometry from scalar formulas.

```
structure SameCurvatureClaims (lam : ℝ) : Prop where
  support_identity : ∀ h ∈ Ioc 0 1,
    h * ((P4 lam h - h * A4 lam h) - (P3 lam h - h * A3 lam h)) = J lam h
  area_identity : ∀ h ∈ Ioc 0 1,
    h ^ 2 * (A4 lam h - A3 lam h) = J lam h - h * jprime lam h
  area_gap : ∀ h ∈ Ioc 0 1,
    gamma lam / h ^ 2 ≤ A4 lam h - A3 lam h ∧ 0 < gamma lam / h ^ 2
  support_gap : ∀ h ∈ Ioc 0 1,
    gamma lam / h ≤ (P4 lam h - h * A4 lam h) - (P3 lam h - h * A3 lam h) ∧
    0 < gamma lam / h
```

CMElementary.ScalarIdentificationClaims

Classification: main

Source: CMElementary.lean:141-154

Declaration SHA-256: 223a2325121ec324ac1308ae10d552d69116f31febe80ced7adf5e478138df58

Mathematical statement. For $-1 \leq z \leq 1$,

$$\sqrt{\lambda^2 - z^2} / \lambda = \sqrt{1 - z^2 / \lambda^2}.$$

For $0 < t \leq 1$, the frozen three-arc and four-arc radical differences equal $D(2t - 1)$ and $D(t)$, respectively, and their area and perimeter functions equal $A_3(t), P_3(t), A_4(t), P_4(t)$, respectively.

This is a conjunction of the listed mathematical properties, not a theorem assuming them without proof. The corresponding construction or claims theorem supplies every component from the stated density and parameter hypotheses; the detailed identities and domains in the statement distinguish endpoint continuity from interior differentiability and actual geometry from scalar formulas.

```
structure ScalarIdentificationClaims (lam : ℝ) : Prop where
  sqrt_normalization : ∀ z ∈ Icc (-1) 1,
    Real.sqrt (lam ^ 2 - z ^ 2) / lam = Real.sqrt (1 - z ^ 2 / lam ^ 2)
  delta_three : ∀ t ∈ Ioc 0 1,
    LeanSuffixAnalytic.typeThreeDelta lam t = D lam (2 * t - 1)
  delta_four : ∀ t ∈ Ioc 0 1, LeanSuffixAnalytic.typeFourDelta lam t = D lam t
  area_three : ∀ t ∈ Ioc 0 1, LeanSuffixAnalytic.typeThreeArea lam t = A3 lam t
  perimeter_three : ∀ t ∈ Ioc 0 1,
    LeanSuffixAnalytic.typeThreePerimeter lam t = P3 lam t
  area_four : ∀ t ∈ Ioc 0 1, LeanSuffixAnalytic.typeFourArea lam t = A4 lam t
  perimeter_four : ∀ t ∈ Ioc 0 1, LeanSuffixAnalytic.typeFourPerimeter lam t = P4 lam t

/-- Proof-route obligations, not an assumption allowing either root to bypass
its elementary argument. The retained dependency audit inspects proof terms. -/
```

CMElementary.ScalarStatement

Classification: main

Source: CMElementary.lean:65-73

Declaration SHA-256: c257cd1dfa40523e9c40ad9df9512627dd550fab8ba104a9be0654c1e976617

Mathematical statement. For every real $\lambda > 1$,

$$\gamma_\lambda > 0.$$

For every real h with $0 < h \leq 1$, there is a real r such that

$$\begin{aligned} 0 &< r < h, \\ A_3(r) &= A_4(h), \\ P_4(h) - P_3(r) &> \frac{\gamma_\lambda}{h} > 0. \end{aligned}$$

This defines the separately published scalar proposition. The existence of an equal-area parameter and the strict quantitative perimeter margin are proved together; neither is an input assumption.

```
def ScalarStatement : Prop :=
  ∀ lam : ℝ, 1 < lam →
    0 < gamma lam ∧
    ∀ h : ℝ, 0 < h → h ≤ 1 →
      ∃ r : ℝ, 0 < r ∧ r < h ∧ A3 lam r = A4 lam h ∧
        gamma lam / h < P4 lam h - P3 lam r ∧ 0 < gamma lam / h

/-- Mandatory auxiliary-function claims. These are conclusions to prove from
`1 < lam`, never extra hypotheses of either comparison theorem. -/
```

CMElementary.ThreeArcCalculusClaims

Classification: main

Source: CMElementary.lean:114-124

Declaration SHA-256: 9f403aa6eca1f9679fb0baa7f3fae671458d71319127b51fe69683462617d8e3

Mathematical statement. The functions A_3, P_3 are continuous on $(0, 1]$, differentiable on $(0, 1)$, satisfy

$$A_3(t) \rightarrow +\infty$$

as $t \downarrow 0$, and

$$P_3(t) = tA_3(t)$$

on $(0, 1)$.

This is a conjunction of the listed mathematical properties, not a theorem assuming them without proof. The corresponding construction or claims theorem supplies every component from the stated density and parameter hypotheses; the detailed identities and domains in the statement distinguish endpoint continuity from interior differentiability and actual geometry from scalar formulas.

```
structure ThreeArcCalculusClaims (lam : ℝ) : Prop where
  area_continuous : ContinuousOn (A3 lam) (Ioc 0 1)
  perimeter_continuous : ContinuousOn (P3 lam) (Ioc 0 1)
  area_differentiable : DifferentiableOn ℝ (A3 lam) (Ioo 0 1)
  perimeter_differentiable : DifferentiableOn ℝ (P3 lam) (Ioo 0 1)
  area_limit_zero : Tendsto (A3 lam) (N[>] (0 : ℝ)) atTop
  variational_identity : ∀ t ∈ Ioo 0 1,
    HasDerivAt (P3 lam) (t * deriv (A3 lam) t) t

/-- The selected root and its strict barrier; no uniqueness or global area
monotonicity is required. Differentiation stays inside `(r,h)`, even for `h=1`. -/
```

CMElementary.ThreeArcRealization

Classification: main

Source: CMElementaryGeometry.lean:97-116

Declaration SHA-256: dabdc6347c8e954b7cae2391d1c86b0e87c40ca9fe83f8a5de7e2c23a0179ca2

Mathematical statement. There is a three-arc assembly of curvature r whose carrier equals E_r . It has the stated regularity and

$$A_\lambda(E_r) = A_3(r),$$

$$P_\lambda(E_r) = P_3(r).$$

Its exterior half-angle

$$\alpha = \arccos((2r - 1)/\lambda)$$

lies in $(0, \pi)$, and exceeds

$$\pi/2$$

if $r < 1/2$. The chord points $(x, 1)$ with

$$|x| < \sin \alpha / r$$

are interior. Its bottom frontier is exactly

$$\{(x, -1) : |x| \leq D(2r - 1)/r\}.$$

If $r = 1/2$, then

$$E_r = \{(x, y) : x^2 + (y - 1)^2 \leq 4\}.$$

This is a conjunction of the listed mathematical properties, not a theorem assuming them without proof. The corresponding construction or claims theorem supplies every component from the stated density and parameter hypotheses; the detailed identities and domains in the statement distinguish endpoint continuity from interior differentiability and actual geometry from scalar formulas.

```
structure ThreeArcRealization (lam r : ℝ) : Prop where
  assembly_bridge : ∃ assembly : TypeThreeAssembly lam,
    assembly.h = r ∧ E lam r = assembly.carrier
  regularity : RegionRegularity lam (E lam r)
```

```

area_formula : WeightedArea lam (E lam r) = A3 lam r
perimeter_formula : WeightedPerimeter lam (FrontierMeasure (E lam r)) = P3 lam r
angle_bounds : 0 < Real.arccos ((2 * r - 1) / lam) ^
  Real.arccos ((2 * r - 1) / lam) < Real.pi
major_angle : r < 1 / 2 → Real.pi / 2 < Real.arccos ((2 * r - 1) / lam)
upper_chord_interior :
  {p : PlanePoint | p.2 = 1 ^
    |p.1| < Real.sin (Real.arccos ((2 * r - 1) / lam)) / r} ⊆ interior (E lam r)
bottom_frontier : frontier (E lam r) ∩ {p : PlanePoint | p.2 = -1} =
  {p : PlanePoint | p.2 = -1 ^ |p.1| ≤ D lam (2 * r - 1) / r}
semicircle_disk : r = 1 / 2 →
  E lam r = {p : PlanePoint | p.1 ^ 2 + (p.2 - 1) ^ 2 ≤ 4}

/-- Exact contract type of `CMVElementary.geometric_realization`.
These are unconditional geometric conclusions on the specified domains; the
comparison theorem must not accept these records as extra assumptions. -/

```

CMVElementary.Z

Classification: main

Source: CMVElementary.lean:59-61

Declaration SHA-256: c84d33665cc18e69dec374d2734337ae96f82e06e26b17b735e6b9f2251c24d9

Mathematical statement. Define

$$Z_{\lambda,h,\varepsilon} = \{t \in [\varepsilon, h] : A_3(t) = A_4(h)\}.$$

A compact level set is obtained after a small positive left endpoint is chosen from the area divergence.

```

def Z (lam h epsilon : ℝ) : Set ℝ :=
  {t ∈ Icc epsilon h | A3 lam t = A4 lam h}

```

CMVElementary.area_identity

Classification: main

Source: CMVElementary.lean:580-586

Declaration SHA-256: b69223db97a18767cba5904b4c3b135f6c23755f91e40636295e797774659382

Mathematical statement. For real λ and $h \neq 0$,

$$h^2[A_4(h) - A_3(h)] = J(h) - h j(h).$$

The identity is algebraic after denominators are cleared; it supplies the independent area gap rather than presupposing it.

```

theorem area_identity {lam h : ℝ} (hh : h ≠ 0) :
  h ^ 2 * (A4 lam h - A3 lam h) = J lam h - h * jprime lam h := by
  unfold A4 A3 J S jprime
  field_simp [hh]
  ring

/-- Exact scalar identities and both positive quantitative gaps, including h=1. -/

```

CMVElementary.auxiliary_claims

Classification: main

Source: CMVElementary.lean:545-573

Declaration SHA-256: 40439108bc1575c7407f75b4eb147fcee6626bb19b9ed5061fbd985ae03037d5

Mathematical statement. For every

$$\lambda > 1,$$

The auxiliary assertion consists of the following claims. The function D is continuous, even and nonnegative on $[-1, 1]$, strictly increasing on $[0, 1]$, and differentiable on $(-1, 1)$. On the latter interval

$$S' = -2D,$$

and on the closed interval

$$S(-z) = \lambda\pi - S(z).$$

One has

$$\gamma_\lambda > 0;$$

J, j are continuous on $[0, 1]$;

$$J(0) = J(1) = \gamma_\lambda;$$

$$J' = j$$

on $(0, 1)$; and

$$J, H \geq \gamma_\lambda$$

on $(0, 1]$. Furthermore

$$j > 0$$

on $(0, 1/3)$,

$$j(1/3) = 0,$$

$$j < 0$$

on $(1/3, 1)$, and

$$H \geq J$$

on $[1/3, 1]$. On $(0, 1/3)$,

$$j' = 8D'(2u - 1) - 4D'(u) < 0$$

and

$$H' = -u[8D'(2u - 1) - 4D'(u)] > 0.$$

Finally

$$H(u) \rightarrow \gamma_\lambda$$

as $u \downarrow 0$.

Combines the independently proved inequalities, derivatives and limits; none is an assumption of the comparison.

```
theorem auxiliary_claims {lam : ℝ} (hlam : 1 < lam) : AuxiliaryClaims lam where
  d_continuous := (DerivativePilot.D_continuous lam).continuousOn
  d_even := fun z _ => D_even lam z
  d_nonneg := fun z _ => D_nonneg hlam z
  d_strictMono := D_strictMono hlam
  s_deriv := fun _ hz => hasDerivAt_S hlam hz
  s_reflection := fun z _ => S_reflection lam z
  gamma_pos := gamma_pos hlam
  j_continuous := (J_continuous lam).continuousOn
  jprime_continuous := (DerivativePilot.jprime_continuous lam).continuousOn
  j_zero := DerivativePilot.J_zero lam
  j_one := J_one lam
  j_deriv := fun _ hu => hasDerivAt_J hlam hu
  j_lower := fun _ hu => J_lower hlam ⟨hu.1.le, hu.2⟩
```

```

h_lower := fun _ hu => H_lower hlam (hu.1.le, hu.2)
jprime_pos := fun _ hu => jprime_pos hlam hu
jprime_third := jprime_third lam
jprime_neg := fun _ hu => jprime_neg hlam hu
h_ge_j := fun _ hu => H_ge_J hlam hu
d_differentiable := DerivativePilot.D_differentiableOn hlam
jprime_deriv := fun _ hu => DerivativePilot.hasDerivAt_jprime hlam
  (DerivativePilot.lower_third_signed_arguments hu).2
jprime_deriv_neg := fun _ hu => DerivativePilot.jprime_deriv_neg hlam hu
h_deriv := fun _ hu => DerivativePilot.hasDerivAt_H_of_J hlam hu
  (hasDerivAt_J hlam (DerivativePilot.lower_third_signed_arguments hu).2)
h_deriv_pos := fun _ hu => DerivativePilot.deriv_H_pos_of_J hlam hu
  (hasDerivAt_J hlam (DerivativePilot.lower_third_signed_arguments hu).2)
h_limit_zero := DerivativePilot.H_limit_zero lam

```

CMElementary.elementary_route

Classification: main

Source: CMElementary.lean:755-765

Declaration SHA-256: a1b39bd2222e649a050d9ab1986edb3f443ebd0f728e84997914f57699a72387

Mathematical statement. For every

$$\lambda > 1,$$

The elementary-route assertion is the conjunction of the auxiliary, same-curvature, three-arc calculus and scalar-identification assertions, together with existence, for every

$$0 < h \leq 1,$$

of ε, r satisfying the greatest-root assertion.

Assembles all ingredients and the greatest-root construction from the density hypothesis alone.

```

theorem elementary_route : ∀ lam : ℝ, 1 < lam → ElementaryRoute lam := by
  intro lam hlam
  exact {
    auxiliary := auxiliary_claims hlam
    same_curvature := same_curvature_claims hlam
    three_arc_calculus := three_arc_calculus_claims hlam
    scalar_identification := scalar_identification_claims hlam
    greatest_root := fun _ hh => greatest_root_claims hlam hh }

/-- Corrected perimeter-minus-perimeter comparison. The Q improvement is strict
and the same-curvature support bound supplies the explicit gamma/h margin. -/

```

CMElementary.exists_greatest_level_root

Classification: main

Source: GreatestLevelRoot.lean:12-48

Declaration SHA-256: 0789ece1798caf97991b6506267400db996fa20c928696c85876071a12271aa2

Mathematical statement. Let $a < b$, let

$$F: \mathbb{R} \rightarrow \mathbb{R}$$

be continuous on $[a, b]$, and suppose

$$F(a) > V > F(b).$$

There exists $r \in (a, b)$ with

$$F(r) = V,$$

greatest in $[a, b] \cap F^{-1}(\{V\})$, and

$$F(t) < V$$

for $r < t \leq b$.

The intermediate value theorem supplies a level point; the closed level set in a compact interval has a greatest point. A later value at or above the level would give a later root, contradicting maximality.

```

theorem exists_greatest_level_root
  {F : ℝ → ℝ} {a b V : ℝ}
  (hab : a < b) (hF : ContinuousOn F (Icc a b))
  (ha : V < F a) (hb : F b < V) :
  ∃ r ∈ Ioo a b, F r = V ∧
    IsGreatest (Icc a b ∩ F ⁻¹' {V}) r ∧
    ∀ t ∈ Ioc r b, F t < V := by
  obtain ⟨x, hx, hxV⟩ := intermediate_value_Icc' hab.le hF ⟨hb.le, ha.le⟩
  have hcompact : IsCompact (Icc a b ∩ F ⁻¹' {V}) :=
    isCompact_Icc.of_isClosed_subset
      (hF.preimage_isClosed_of_isClosed isClosed_Icc isClosed_singleton)
    inter_subset_left
  obtain ⟨r, hr, hmax⟩ := hcompact.exists_isGreatest ⟨x, hx, hxV⟩
  have hrV : F r = V := hr.2
  have hne_a : r ≠ a := by
    intro h
    subst r
    exact (ne_of_gt ha) hrV
  have hne_b : r ≠ b := by
    intro h
    subst r
    exact (ne_of_lt hb) hrV
  have har : a < r := lt_of_le_of_ne hr.1.1 hne_a.symm
  have hrb : r < b := lt_of_le_of_ne hr.1.2 hne_b
  refine ⟨r, ⟨har, hrb⟩, hrV, ⟨hr, hmax⟩, ?_⟩
  intro t ht
  by_contra h
  have hVt : V ≤ F t := le_of_not_gt h
  have hFt : ContinuousOn F (Icc t b) := hF.mono (by
    intro z hz
    exact ⟨hr.1.1.trans (ht.1.le.trans hz.1), hz.2⟩)
  obtain ⟨z, hz, hzV⟩ := intermediate_value_Icc' ht.2 hFt ⟨hb.le, hVt⟩
  have hzr : z ≤ r := hmax ⟨⟨hr.1.1.trans (ht.1.le.trans hz.1), hz.2⟩, hzV⟩
  exact (not_le_of_gt (ht.1.trans_le hz.1)) hzr

/-- A variational identity converts a strict area barrier into a strict
support comparison. Differentiability at the endpoints is not required. -/

```

CMVElementary.gamma

Classification: main

Source: CMVElementary.lean:38-40

Declaration SHA-256: 89a6e7b6d34e63c790ca99225f414245b7afb3c33a52b21c3baa8e107a3e1c53

Mathematical statement. Define

$$\gamma_\lambda = \lambda \arccos(1/\lambda) - \sqrt{1 - \lambda^{-2}}.$$

It is the common endpoint value of J ; positivity is a theorem under

$$\lambda > 1,$$

not part of this definition.

```

def gamma (lam : ℝ) : ℝ :=
  lam * Real.arccos (1 / lam) - Real.sqrt (1 - lam ^ (-2 : ℤ))

```

CMVElementary.gamma_pos

Classification: main

Source: CMVElementary.lean:446-455

Declaration SHA-256: 62668904d84e7cbf785e00bbc726c316b4b51445dcf41d3119a83f33d568f2ff

Mathematical statement. For every

$$\lambda > 1,$$
$$\gamma_\lambda > 0.$$

With

$$\theta = \arccos(1/\lambda) > 0,$$

use

$$\sin\theta \leq \theta < \lambda\theta$$

and

$$\sin\theta = \sqrt{1 - \lambda^{-2}}.$$

```
theorem gamma_pos {lam : ℝ} (hlam : 1 < lam) : 0 < gamma lam := by
  have h1 : 0 < lam := by linarith
  have ha : 0 < Real.arccos (1 / lam) := Real.arccos_pos.mpr ((div_lt_one h1).mpr hlam)
  have hs := Real.sin_le ha.le
  rw [Real.sin_arccos] at hs
  have heq : lam ^ (-2 : ℤ) = (1 / lam) ^ 2 := by simp [zpow_neg]
  unfold gamma
  rw [heq]
  nlinarith
```

CMVElementary.geometric_comparison

Classification: main

Source: CMVElementaryGeometry.lean:783-798

Declaration SHA-256: 6a426578c4a240700c25141374531b26d559de5c1a0468aaffc359064d84ae40

Mathematical statement. For every real $\lambda > 1$,

$$\gamma_\lambda > 0.$$

For every real h with $0 < h \leq 1$, there is a real r such that

$$\begin{aligned} 0 &< r < h, \\ A_\lambda(E_r) &= A_\lambda(C_h), \\ P_\lambda(C_h) - P_\lambda(E_r) &> \frac{\gamma_\lambda}{h} > 0. \end{aligned}$$

Here area is Lebesgue weighted area and perimeter is Euclidean complete-frontier weighted perimeter.

Apply the scalar comparison and use the four actual integral identities supplied by geometric realization. The chosen competitor has parameter strictly below one even at the four-arc endpoint.

```
theorem geometric_comparison : GeometricStatement := by
  intro lam hlam
  obtain ⟨hgamma, hscalar⟩ := scalar_comparison lam hlam
  refine ⟨hgamma, ?_⟩
  intro h hh h1
  obtain ⟨r, hr, hrh, harea, hperimeter, hmargin⟩ := hscalar h hh h1
  have hr1 : r < 1 := hrh.trans_le h1
  have hfour := (geometric_realization lam hlam).1 h hh h1
```

```

have hthre := (geometric_realization lam hlam).2 r hr hr1
refine {r, hr, hrh, ?_, ?_, hmargin}
· rw [hthre.area_formula, hfour.area_formula]
  exact harea
· rw [hfour.perimeter_formula, hthre.perimeter_formula]
  exact hperimeter

end CMElementary

```

CMElementary.geometric_realization

Classification: main

Source: CMElementaryGeometry.lean:775-782

Declaration SHA-256: f642078096f16ab8666c13613957258016cb00512ac7e7875b5d3006f18b1646

Mathematical statement. For every

$$\lambda > 1,$$

the four-arc realization holds for all

$$0 < h \leq 1,$$

and the three-arc realization holds for all

$$0 < r < 1.$$

There is a four-arc candidate with curvature h , angle

$$\arccos(h/\lambda),$$

density jump

$$\lambda > 1,$$

and incidence

$$\lambda \cos \alpha = h,$$

whose assembled coordinate set equals C_h . The region satisfies all stated regularity properties and

$$A_\lambda(C_h) = A_4(h),$$

$$P_\lambda(C_h) = P_4(h).$$

For

$$y = \pm 1,$$

points with

$$|x| < \sin(\arccos(h/\lambda))/h$$

are interior to C_h . The reflected cap is contained in

$$y \leq -1.$$

There is a three-arc assembly of curvature r whose carrier equals E_r . It has the stated regularity and

$$A_\lambda(E_r) = A_3(r),$$

$$P_\lambda(E_r) = P_3(r).$$

Its exterior half-angle

$$\alpha = \arccos((2r - 1)/\lambda)$$

lies in $(0, \pi)$, and exceeds

$$\pi/2$$

if $r < 1/2$. The chord points $(x, 1)$ with

$$|x| < \sin \alpha / r$$

are interior. Its bottom frontier is exactly

$$\{(x, -1) : |x| \leq D(2r - 1)/r\}.$$

If $r = 1/2$, then

$$E_r = \{(x, y) : x^2 + (y - 1)^2 \leq 4\}.$$

These are unconditional conclusions on the stated domains, not additional hypotheses of the comparison.

```
theorem geometric_realization : GeometricRealizationStatement := by
  intro lam hlam
  exact ⟨fun _ hh h1 => GeometryBridge.four_realization hlam hh h1,
    fun _ hr hr1 => GeometryBridge.three_realization hlam hr hr1⟩

/-- Corrected quantitative comparison for the literal closed carriers.
The scalar witness lies strictly below one, even when the four-arc parameter is
one; both measure identities come from unconditional geometric realizations. -/
```

CMElementary.greatest_root_claims

Classification: main

Source: CMElementary.lean:692-754

Declaration SHA-256: 53fc167a9298d1af24874c5394cc36a6b816243904575d5d9ae34c2802ef1d73

Mathematical statement. For $\lambda > 1$ and $0 < h \leq 1$, there exist real ε, r satisfying the greatest-root assertion.

One has

$$0 < \varepsilon < r < h$$

and

$$A_3(\varepsilon) > A_4(h).$$

The level set $Z_{\lambda, h, \varepsilon}$ is nonempty and compact, with greatest element r and

$$A_3(r) = A_4(h).$$

For $r < t \leq h$,

$$A_3(t) < A_4(h).$$

The function $Q_{\lambda, h}$ is continuous on $[r, h]$, has derivative

$$A_4(h) - A_3(t) > 0$$

on (r, h) , and satisfies

$$Q_{\lambda, h}(r) < Q_{\lambda, h}(h).$$

Divergence chooses the left endpoint; compactness chooses the greatest root; the variational identity gives the strict support comparison.

```
theorem greatest_root_claims {lam h : ℝ} (hlam : 1 < lam) (hh : h ∈ Ioc 0 1) :
  ∃ epsilon r : ℝ, GreatestRootClaims lam h epsilon r := by
  have hc := three_arc_calculus_claims hlam
```

```

have hg := same_curvature_claims hlam
have hlarge := hc.area_limit_zero.eventually_gt_atTop (A4 lam h)
have hsmall :  $\forall^f t \text{ in } \mathcal{N}[>] (0 : \mathbb{R}), t < h :=$ 
  (eventually_lt_nhds hh.1).filter_mono inf_le_left
have hpositive :  $\forall^f t \text{ in } \mathcal{N}[>] (0 : \mathbb{R}), 0 < t :=$  self_mem_nhdsWithin
obtain (epsilon, heAbove, heh, hepos) :=
  (hlarge.and (hsmall.and hpositive)).exists
have hesub :  $\text{Icc } \epsilon h \subseteq \text{Ioc } (0 : \mathbb{R}) 1 :=$  by
  intro t ht
  exact (hepos.trans_le ht.1, ht.2.trans hh.2)
have hA := hc.area_continuous.mono hesub
have hhbelow :  $A3 \text{ lam } h < A4 \text{ lam } h :=$  by
  obtain (hgap, hpos) := hg.area_gap h hh
  linarith
obtain (r, hr, hequal, hgreatest, hbarrier) :=
  exists_greatest_level_root heh hA heAbove hhbelow
have hrpos :  $0 < r :=$  hepos.trans hr.1
have hrsub :  $\text{Icc } r h \subseteq \text{Ioc } (0 : \mathbb{R}) 1 :=$  by
  intro t ht
  exact (hrpos.trans_le ht.1, ht.2.trans hh.2)
have hAr := hc.area_continuous.mono hrsub
have hPr := hc.perimeter_continuous.mono hrsub
have hvar :  $\forall t \in \text{Ioo } r h, \exists d : \mathbb{R},$ 
  HasDerivAt (A3 lam) d t  $\wedge$  HasDerivAt (P3 lam) (t * d) t := by
  intro t ht
  have ht01 :  $t \in \text{Ioo } (0 : \mathbb{R}) 1 :=$  (hrpos.trans ht.1, ht.2.trans_le hh.2)
  exact (deriv (A3 lam) t,
    (hc.area_differentiable t ht01).differentiableAt
      (isOpen_Ioo.mem_nhds ht01) |>.hasDerivAt,
    hc.variational_identity t ht01)
refine (epsilon, r, {
  epsilon_pos := hepos
  epsilon_lt_r := hr.1
  r_lt_h := hr.2
  epsilon_above := heAbove
  level_nonempty := (r, hgreatest.1)
  level_compact := ?_
  greatest := hgreatest
  equal_area := hequal
  barrier := hbarrier
  q_continuous := hPr.add (continuousOn_id.mul (continuousOn_const.sub hAr))
  q_deriv := ?_
  q_deriv_pos := fun t ht => sub_pos.mpr (hbarrier t (ht.1, ht.2.le))
  q_strict := ?_ })
· exact isCompact_Icc.of_isClosed_subset
  (hA.preimage_isClosed_of_isClosed isClosed_Icc isClosed_singleton)
  inter_subset_left
· intro t ht
  obtain (d, hAd, hPd) := hvar t ht
  simp only [Q, id_eq] using!
  (hPd.add ((hasDerivAt_id t).mul
    ((hasDerivAt_const t (A4 lam h)).sub hAd))).congr_deriv (by
    simp only [id_eq, Pi.sub_apply]
    ring)
· have hstrict := strict_support_comparison hr.2 hAr hPr hequal
  (fun t ht => hbarrier t (ht.1, ht.2.le)) hvar
  simp only [Q, hequal, sub_self, mul_zero, add_zero] using hstrict

/-- The complete elementary route, including the greatest root and strict Q
comparison, follows from the density hypothesis alone. -/

```

CMVElementary.hasDerivAt_B

Classification: main

Source: CMVElementary.lean:414-431

Declaration SHA-256: e1d93dd53807589db2db47f2a40f0b76ab73445942336bd1513b60f62d0be20a

Mathematical statement. For $\lambda > 1$ and $-1 < z < 1$, B is differentiable and

$$B'(z) = 1/\sqrt{1-z^2} - \lambda/\sqrt{\lambda^2-z^2}.$$

The arguments of inverse trigonometric derivatives are interior to their domains. Normalize the outer radical to obtain the displayed derivative.

```
theorem hasDerivAt_B {lam z : ℝ} (hlam : 1 < lam) (hz : z ∈ Ioo (-1) 1) :
  HasDerivAt (B lam)
    (1 / Real.sqrt (1 - z ^ 2) - lam / Real.sqrt (lam ^ 2 - z ^ 2)) z := by
  have hl : 0 < lam := by linarith
  have hrlo : -1 < z / lam := (lt_div_iff₀ hl).mpr (by linarith [hz.1])
  have hrhi : z / lam < 1 := (div_lt_one hl).mpr (by linarith [hz.2])
  have hs : Real.sqrt (1 - (z / lam) ^ 2) = Real.sqrt (lam ^ 2 - z ^ 2) / lam := by
    simp only [div_pow] using (DerivativePilot.sqrt_normalization hl z).symm
  have ho := (DerivativePilot.radical_denominators hlam hz).2.1
  have hc := (Real.hasDerivAt_arccos (ne_of_gt hrlo) (ne_of_lt hrhi)).comp z
    ((hasDerivAt_id z).div_const lam)
  have ha := Real.hasDerivAt_arcsin (ne_of_gt hz.1) (ne_of_lt hz.2)
  simp only [B, Function.comp_def, id_eq] using!
  ((hc.const_mul lam).add ha).congr_deriv (by
    rw [hs]
    field_simp [ne_of_gt hl, ne_of_gt (Real.sqrt_pos.mpr ho)]
    ring)
```

CMElementary.hasDerivAt_J

Classification: main

Source: CMElementary.lean:464-473

Declaration SHA-256: 6ea60c2da7853b72c8a574b92d5a4f302da93a6ae6fbe80e9bc57d4bda8e18c6

Mathematical statement. For $\lambda > 1$ and $0 < u < 1$, J has derivative $j(u)$.

Apply

$$S' = -2D$$

to the two arguments and use the factor two from differentiating $2u - 1$.

```
theorem hasDerivAt_J {lam u : ℝ} (hlam : 1 < lam) (hu : u ∈ Ioo 0 1) :
  HasDerivAt (J lam) (jprime lam u) u := by
  have hq : 2 * u - 1 ∈ Ioo (-1) 1 := ⟨by linarith [hu.1], by linarith [hu.2]⟩
  have hu' : u ∈ Ioo (-1) 1 := ⟨by linarith [hu.1], hu.2⟩
  have ha : HasDerivAt (fun x : ℝ => 2 * x - 1) 2 u := by
    simp using ((hasDerivAt_id u).const_mul 2).sub_const 1
  simp only [J, Function.comp_def] using!
  (((hasDerivAt_S hlam hu').const_mul 2).sub ((hasDerivAt_S hlam hq).comp u ha)).sub_const
    (Real.pi / 2)).congr_deriv (by unfold jprime; ring)
```

CMElementary.hasDerivAt_S

Classification: main

Source: CMElementary.lean:432-445

Declaration SHA-256: 6a00be6e98e39b229cb8434cdcde52aad95e9edad573a64b0f990aa6ffc1e07b

Mathematical statement. For $\lambda > 1$ and $-1 < z < 1$, S has derivative

$$-2D(z)$$

at z .

Differentiate $B - zD$; the radical identities simplify

$$B' + D = zD',$$

giving the cancellation.

```
theorem hasDerivAt_S {lam z : ℝ} (hlam : 1 < lam) (hz : z ∈ Ioo (-1) 1) :
  HasDerivAt (S lam) (-2 * D lam z) z := by
  have h1 : 0 < lam := by linarith
  obtain ⟨hi, ho, hsi, hso⟩ := DerivativePilot.radical_denominators hlam hz
  have hsout : Real.sqrt (lam ^ 2 - z ^ 2) ≠ 0 := ne_of_gt (Real.sqrt_pos.mpr ho)
  have hraw := (hasDerivAt_B hlam hz).sub ((hasDerivAt_id z).mul (DerivativePilot.hasDerivAt_D hlam hz))
  simpa only [S] using! hraw.congr_deriv (by
    simp only [id_eq, one_mul]
    rw [DerivativePilot.D_normalization h1]
    dsimp
    field_simp [ne_of_gt h1, ne_of_gt hsi, hsout]
    nlinarith [Real.sq_sqrt hi.le, Real.sq_sqrt ho.le])

/-- With `alpha = arccos (1/lam) > 0`, use `sin alpha ≤ alpha < lam*alpha`. -/
```

CMElementary.jprime

Classification: main

Source: CMElementary.lean:55-56

Declaration SHA-256: 7436fde598ecb0a0029bf5d63b6fb219ce3a88490ddc40a38e83b449bcad9edd

Mathematical statement. Define

$$j(u) = 4[D(2u - 1) - D(u)]$$

for real parameters.

The theorem

$$J' = j$$

holds in $(0, 1)$. The explicit function j is continuous also at the endpoints and is not defined by taking an endpoint derivative.

```
def jprime (lam u : ℝ) : ℝ := 4 * (D lam (2 * u - 1) - D lam u)
```

CMElementary.jprime_neg

Classification: main

Source: CMElementary.lean:493-501

Declaration SHA-256: ef17824b8194ad2eb67c2e74af7d4d9917476e38bb12159acac0cb8f06705777

Mathematical statement. For $\lambda > 1$ and $1/3 < u < 1$,

$$j(u) < 0.$$

The inequality

$$|2u - 1| < u$$

reduces the sign to strict increase of D on its nonnegative interval.

```
theorem jprime_neg {lam u : ℝ} (hlam : 1 < lam) (hu : u ∈ Ioo (1 / 3) 1) :
  jprime lam u < 0 := by
  have hu0 : 0 ≤ u := by linarith [hu.1]
  have ha : |2 * u - 1| < u := abs_lt.mpr ⟨by linarith [hu.1], by linarith [hu.2]⟩
  have hd := D_strictMono hlam ⟨abs_nonneg _, ha.le.trans hu.2.le⟩ ⟨hu0, hu.2.le⟩ ha
  rw [D_abs] at hd
  unfold jprime
  linarith
```

CMVElementary.jprime_pos

Classification: main

Source: CMVElementary.lean:479-487

Declaration SHA-256: 518f854f89ae00c586e7e8fb6fde430c62ba1dd0993821abffa9e6d56c20870f

Mathematical statement. For $\lambda > 1$ and $0 < u < 1/3$,

$$j(u) > 0.$$

Here

$$1 - 2u > u;$$

evenness and strict increase of D give the sign of the difference.

```
theorem jprime_pos {lam u : ℝ} (hlam : 1 < lam) (hu : u ∈ Ioo 0 (1 / 3)) :  
  0 < jprime lam u := by  
  have hdu : u ∈ Icc (0 : ℝ) 1 := ⟨hu.1.le, by linarith [hu.2]⟩  
  have hdr : 1 - 2 * u ∈ Icc (0 : ℝ) 1 := ⟨by linarith [hu.2], by linarith [hu.1]⟩  
  have hd := D_strictMono hlam hdu hdr (by linarith [hu.2] : u < 1 - 2 * u)  
  unfold jprime  
  rw [show 2 * u - 1 = -(1 - 2 * u) by ring, D_even]  
  linarith
```

CMVElementary.jprime_third

Classification: main

Source: CMVElementary.lean:488-492

Declaration SHA-256: d0c7dbb0422c10f59db756962a83f292b34b4533d9c653ccc1f9b40c626313c7

Mathematical statement. For every real λ ,

$$j(1/3) = 0.$$

The two arguments are negatives of each other and D is even.

```
theorem jprime_third (lam : ℝ) : jprime lam (1 / 3) = 0 := by  
  unfold jprime  
  rw [show (2 * (1 / 3) - 1 : ℝ) = -(1 / 3) by norm_num, D_even]  
  ring
```

CMVElementary.same_curvature_claims

Classification: main

Source: CMVElementary.lean:587-605

Declaration SHA-256: 3f33553e4d18e3fbda9a031f7ee7ae0a6105fe8e92768737441091a565b9f2b8

Mathematical statement. For every

$$\lambda > 1,$$

For every

$$0 < h \leq 1,$$

the support difference multiplied by h equals $J(h)$, and

$$h^2[A_4(h) - A_3(h)] = H(h).$$

Moreover

$$A_4(h) - A_3(h) \geq \gamma_\lambda/h^2 > 0$$

and

$$[P_4(h) - hA_4(h)] - [P_3(h) - hA_3(h)] \geq \gamma_\lambda/h > 0.$$

Uses the two algebraic identities and the proved lower bounds for the auxiliary functions.

```
theorem same_curvature_claims {lam : ℝ} (hlam : 1 < lam) : SameCurvatureClaims lam where
support_identity := fun _ hh => support_identity (ne_of_gt hh.1)
area_identity := fun _ hh => area_identity (ne_of_gt hh.1)
area_gap := by
  intro h hh
  constructor
  · apply (div_le_iff₀ (sq_pos_of_pos hh.1)).mpr
    rw [mul_comm, area_identity (ne_of_gt hh.1)]
    exact H_lower hl原因 (hh.1.le, hh.2)
  · exact div_pos (gamma_pos hl原因) (sq_pos_of_pos hh.1)
support_gap := by
  intro h hh
  constructor
  · apply (div_le_iff₀ hh.1).mpr
    rw [mul_comm, support_identity (ne_of_gt hh.1)]
    exact J_lower hl原因 (hh.1.le, hh.2)
  · exact div_pos (gamma_pos hl原因) hh.1

/-- Identification with the frozen normalization, including the scalar endpoint one. -/
```

CMVElementary.scalar_comparison

Classification: main

Source: CMVElementary.lean:766-779

Declaration SHA-256: a640af5855e5df41a3c4e3a32f7162ed7950996079c4770f3205ede3aa37114d

Mathematical statement. For every real $\lambda > 1$,

$$\gamma_\lambda > 0.$$

For every real h with $0 < h \leq 1$, there is a real r such that

$$\begin{aligned} 0 &< r < h, \\ A_3(r) &= A_4(h), \\ P_4(h) - P_3(r) &> \frac{\gamma_\lambda}{h} > 0. \end{aligned}$$

Choose the greatest root in the elementary route. Its strict support improvement and the quantitative same-curvature support gap give the strict perimeter inequality.

```
theorem scalar_comparison : ScalarStatement := by
  intro lam hl原因
  have route := elementary_route lam hl原因
  refine ⟨route.auxiliary.gamma_pos, ?_⟩
  intro h hhpos hh1
  obtain ⟨epsilon, r, root⟩ := route.greatest_root h ⟨hhpos, hh1⟩
  obtain ⟨hgap, hmargin⟩ := route.same_curvature.support_gap h ⟨hhpos, hh1⟩
  refine ⟨r, root.epsilon_pos.trans root.epsilon_lt_r, root.r_lt_h,
    root.equal_area, ?_, hmargin⟩
  have hstrict := root.q_strict
  simp only [Q, root.equal_area, sub_self, mul_zero, add_zero] at hstrict
  linarith

end CMVElementary
```

CMVElementary.scalar_identification_claims

Classification: main

Source: CMVElementary.lean:606-635

Declaration SHA-256: dee7445c85542ef1b0ae33e2658325f6df23a74fbf06639fe17dbbbad41e7d8d

Mathematical statement. For every

$$\lambda > 1,$$

For $-1 \leq z \leq 1$,

$$\sqrt{\lambda^2 - z^2} / \lambda = \sqrt{1 - z^2 / \lambda^2}.$$

For $0 < t \leq 1$, the frozen three-arc and four-arc radical differences equal $D(2t - 1)$ and $D(t)$, respectively, and their area and perimeter functions equal $A_3(t)$, $P_3(t)$, $A_4(t)$, $P_4(t)$, respectively.

Normalizes the radicals with positive density, then simplifies the four scalar formulas.

```
theorem scalar_identification_claims {lam : ℝ} (hlam : 1 < lam) :
  ScalarIdentificationClaims lam := by
  have h1 : 0 < lam := lt_trans zero_lt_one hlam
  have hthree (t : ℝ) :
    LeanSuffixAnalytic.typeThreeDelta lam t = D lam (2 * t - 1) := by
      unfold LeanSuffixAnalytic.typeThreeDelta LeanSuffixAnalytic.typeThreeShape D
      rw [DerivativePilot.sqrt_normalization h1]
  have hfour (t : ℝ) : LeanSuffixAnalytic.typeFourDelta lam t = D lam t := by
    unfold LeanSuffixAnalytic.typeFourDelta D
    rw [DerivativePilot.sqrt_normalization h1]
  refine ⟨fun z _ => DerivativePilot.sqrt_normalization h1 z,
    fun t _ => hthree t, fun t _ => hfour t, ?_, ?_, ?_, ?_⟩
  · intro t _
    unfold LeanSuffixAnalytic.typeThreeArea A3
    rw [hthree]
    unfold LeanSuffixAnalytic.typeThreeAngle LeanSuffixAnalytic.typeThreeShape B
    ring
  · intro t _
    unfold LeanSuffixAnalytic.typeThreePerimeter P3
    rw [hthree]
    rfl
  · intro t _
    unfold LeanSuffixAnalytic.typeFourArea A4
    rw [hfour]
    rfl
  · intro t _
    rfl

/-- The closed right endpoint uses continuity of radicals and inverse trig functions,
not differentiability there. In fact continuity holds at every nonzero parameter. -/
```

CMVElementary.sigma

Classification: main

Source: CMVElementaryGeometry.lean:28-29

Declaration SHA-256: 98806701b3f8eb5d60ae9b027ae38c5a4e8f3e80159c19646f8418f1e33424f4

Mathematical statement. For

$$p = (x, y) \in \mathbb{R}^2,$$

define

$$\sigma(p) = (x, -y).$$

This horizontal-axis reflection produces the lower cap, distinct from the vertical reflection symmetry of the families.

```
def sigma (p : PlanePoint) : PlanePoint := (p.1, -p.2)
```

CMElementary.strict_support_comparison

Classification: main

Source: GreatestLevelRoot.lean:49-79

Declaration SHA-256: 84d7d46ea81553126d7454da55589ffbab4f6aa5e17b969665fa757ca0a1b655

Mathematical statement. Let $a < b$. Suppose A, P are continuous on $[a, b]$,

$$A(a) = V,$$

and

$$A(t) < V$$

on (a, b) . Suppose also that for every interior t , there is a real d with

$$A'(t) = d$$

and

$$P'(t) = td,$$

both derivatives existing. Then

$$P(a) < P(b) + b[V - A(b)].$$

The continuous function

$$Q(t) = P(t) + t[V - A(t)]$$

has derivative

$$V - A(t) > 0$$

in the interior and is strictly increasing on the closed interval. No endpoint differentiability is needed.

```
theorem strict_support_comparison
  {A P : ℝ → ℝ} {a b V : ℝ}
  (hab : a < b)
  (hA : ContinuousOn A (Icc a b))
  (hP : ContinuousOn P (Icc a b))
  (ha : A a = V)
  (hbelow : ∀ t ∈ Ioo a b, A t < V)
  (hvar : ∀ t ∈ Ioo a b,
    ∃ d : ℝ, HasDerivAt A d t ∧ HasDerivAt P (t * d) t) :
  P a < P b + b * (V - A b) := by
let Q : ℝ → ℝ := fun t => P t + t * (V - A t)
have hQ : ContinuousOn Q (Icc a b) :=
  hP.add (continuousOn_id.mul (continuousOn_const.sub hA))
have hQderiv : ∀ t ∈ Ioo a b, HasDerivAt Q (V - A t) t := by
  intro t ht
  obtain ⟨d, hAd, hPd⟩ := hvar t ht
  have hcalc : HasDerivAt Q (t * d + (1 * (V - A t) + t * (0 - d))) t :=
    hPd.add ((hasDerivAt_id t).mul ((hasDerivAt_const t V).sub hAd))
  convert hcalc using 1
  ring
have hmono : StrictMonoOn Q (Icc a b) :=
  strictMonoOn_of_deriv_pos (convex_Icc a b) hQ (by
    intro t ht
    have ht' : t ∈ Ioo a b := by simpa only [interior_Icc] using ht
    rw [(hQderiv t ht').deriv]
    exact sub_pos.mpr (hbelow t ht'))
```

```
have hcompare := hmono (left_mem_Icc.mpr hab.le) (right_mem_Icc.mpr hab.le) hab
simp only [Q, ha, sub_self, mul_zero, add_zero] using hcompare
```

```
/-- The greatest-root argument followed by the variational comparison.
Neither uniqueness of the level nor global monotonicity of area is assumed. -/
```

CMElementary.support_identity

Classification: main

Source: CMElementary.lean:574-579

Declaration SHA-256: 4e41e8ef0dec9764529d7bbbd6d571c686395fbef0c4920da247dc563a9bed

Mathematical statement. For real λ and $h \neq 0$,

$$h[P_4(h) - hA_4(h)] - [P_3(h) - hA_3(h)] = J(h).$$

Substitute the scalar definitions, clear nonzero powers of h , and collect the terms defining S and J .

```
theorem support_identity {lam h : ℝ} (hh : h ≠ 0) :
  h * ((P4 lam h - h * A4 lam h) - (P3 lam h - h * A3 lam h)) = J lam h := by
  unfold P4 A4 P3 A3 J S
  field_simp [hh]
  ring
```

CMElementary.three_arc_calculus_claims

Classification: main

Source: CMElementary.lean:679-691

Declaration SHA-256: 66d348c34396a46da7b0e5c806d3140bd9f3e5f226bcb53a0dc55b40173d5d26

Mathematical statement. For every

$$\lambda > 1,$$

The functions A_3, P_3 are continuous on $(0, 1]$, differentiable on $(0, 1)$, satisfy

$$A_3(t) \rightarrow +\infty$$

as $t \downarrow 0$, and

$$P_3'(t) = tA_3'(t)$$

on $(0, 1)$.

Combines continuity with the transported variational identity and divergence.

```
theorem three_arc_calculus_claims {lam : ℝ} (hlam : 1 < lam) :
  ThreeArcCalculusClaims lam where
  area_continuous := fun _ ht => (A3_continuousAt (ne_of_gt ht.1)).continuousWithinAt
  perimeter_continuous := fun _ ht => (P3_continuousAt (ne_of_gt ht.1)).continuousWithinAt
  area_differentiable := fun _ ht =>
    (three_arc_variational_hasDerivAt hlam ht).1.differentiableAt.differentiableWithinAt
  perimeter_differentiable := fun _ ht =>
    (three_arc_variational_hasDerivAt hlam ht).2.differentiableAt.differentiableWithinAt
  area_limit_zero := A3_tendsto_atRight_zero hlam
  variational_identity := fun _ ht => (three_arc_variational_hasDerivAt hlam ht).2

/-- Divergence supplies the left endpoint; the retained greatest-level lemma
selects the final crossing. The same witness is used for every Q claim. -/
```

CMElementary.three_arc_variational_hasDerivAt

Classification: main

Source: CMElementary.lean:657-666

Declaration SHA-256: 3fff307dbffe97d952d3fb947b083b8ae73f0b09e0a5cf42f65cc1bf735c4d5f

Mathematical statement. For $\lambda > 1$ and $0 < t < 1$, A_3 is differentiable at t , and P_3 has derivative

$$tA_3'(t)$$

there.

Use local equality with the normalized frozen formulas and their proved variational identity; no monotonicity of the area is used.

```

theorem three_arc_variational_hasDerivAt {lam t : ℝ}
  (hlam : 1 < lam) (ht : t ∈ Ioo 0 1) :
  HasDerivAt (A3 lam) (deriv (A3 lam) t) t ∧
  HasDerivAt (P3 lam) (t * deriv (A3 lam) t) t := by
  obtain ⟨ha, hp⟩ := three_arc_eventuallyEq hlam ht
  obtain ⟨hda, hdp⟩ := LeanSuffixAnalytic.typeThree_variational_hasDerivAt hlam ht.1 ht.2
  rw [ha.deriv_eq]
  exact ⟨hda.congr_of_eventuallyEq ha, hdp.congr_of_eventuallyEq hp⟩

/-- Equality eventually to the right of zero suffices for the divergence transport. -/

```

CMElementary.upperCap3

Classification: main

Source: CMElementaryGeometry.lean:48-53

Declaration SHA-256: a9e700433ec6f55d3afb3184a3b903dc0222e7e85c81667191ac80eb26edc559

Mathematical statement. Put

$$q = 2r - 1$$

and define

$$H_r^{(3)} = \{(x, y) : y \geq 1, x^2 + [y - (1 - r^{-1}\cos(\arccos(q/\lambda)))]^2 \leq r^{-2}\}.$$

Intersecting the disk with the half-plane retains the major segment when $r < 1/2$.

```

def upperCap3 (lam r : ℝ) : Set PlanePoint :=
  let q := 2 * r - 1
  {p | 1 ≤ p.2 ∧
    p.1 ^ 2 + (p.2 - (1 - (1 / r) * Real.cos (Real.arccos (q / lam)))) ^ 2 ≤
    (1 / r) ^ 2}

```

CMElementary.upperCap4

Classification: main

Source: CMElementaryGeometry.lean:34-38

Declaration SHA-256: de3d1bb9a28a3f39df18922596753333140775f67fd4b89756b1e878de59e811

Mathematical statement. Define

$$H_h = \{(x, y) : y \geq 1, x^2 + [y - (1 - h^{-1}\cos(\arccos(h/\lambda)))]^2 \leq h^{-2}\}.$$

This is the upper disk segment with center specified from curvature and exterior angle. The geometric theorem imposes

$$\lambda > 1,$$

$$0 < h \leq 1.$$

```
def upperCap4 (lam h : ℝ) : Set PlanePoint :=
  {p | 1 ≤ p.2 ^
    p.1 ^ 2 + (p.2 - (1 - (1 / h) * Real.cos (Real.arccos (h / lam)))) ^ 2 ≤
    (1 / h) ^ 2}
```

FourArcAssembly.frontier_carrier

Classification: main

Source: CMVGeometry.lean:3252-3259

Declaration SHA-256: 7da383d241cb3cc30fc9da784ef7200d9bb87d82e25006745acf6f7f7d86a78f

Mathematical statement. The complete topological frontier of a four-arc assembly equals its declared four-arc trace.

Combine both inclusions; this identity is the topological basis for the canonical perimeter formula.

```
theorem frontier_carrier (a : FourArcAssembly) :
  frontier a.carrier = a.boundaryTrace :=
  Set.Subset.antisymm a.frontier_carrier_subset_boundaryTrace
    a.boundaryTrace_subset_frontier_carrier

/-- All four boundary arcs have the same radius when the exterior chord has the
source incidence length `2 * sin alpha / h`. -/
```

FourArcAssembly.weightedArea_formula

Classification: main

Source: CMVGeometry.lean:2554-2602

Declaration SHA-256: 839e3c1d7a83b5801ac118ef77feb57b415f9e5a71dfa154b8987ba3b91118a0

Mathematical statement. For any real λ , the assembly weighted area equals the core Euclidean area plus λ times the sum of its upper and lower cap Euclidean areas.

Actual integrals add because component overlaps lie on area-null interface lines; the density is one in the core and λ almost everywhere in each cap.

```
theorem weightedArea_formula (lam : ℝ) (a : FourArcAssembly) :
  a.weightedArea lam =
    a.core.euclideanArea + lam *
      (a.upperCap.euclideanArea + a.lowerCap.euclideanArea) := by
  have hcoreUpper : AEDisjoint volume a.core.carrier a.upperCap.carrier := by
    refine measure_mono_null ?_ (volume_horizontalline 1)
    rintro p ⟨hcore, hupper⟩
    have hyCore := a.core.carrier_y_bounds hcore
    have hyUpper : (1 : ℝ) ≤ p.2 := by
      simpa [upperCap] using hupper.2
    exact le_antisymm (le_trans (le_abs_self p.2) hyCore) hyUpper
  have hfirstLower : AEDisjoint volume
    (a.core.carrier ∪ a.upperCap.carrier) a.lowerCap.carrier := by
    refine measure_mono_null ?_ (volume_horizontalline (-1))
    rintro p ⟨hfirst, hlower⟩
    rcases hfirst with hcore | hupper
    · have hyCore := a.core.carrier_y_bounds hcore
      have hyLower : p.2 ≤ (-1 : ℝ) := by
        simpa [lowerCap] using hlower.2
      have hneg : -p.2 ≤ 1 := le_trans (neg_le_abs p.2) hyCore
      exact le_antisymm hyLower (by linarith)
    · have hyUpper : (1 : ℝ) ≤ p.2 := by
        simpa [upperCap] using hupper.2
      have hyLower : p.2 ≤ (-1 : ℝ) := by
        simpa [lowerCap] using hlower.2
      linarith
```

```

have hcore := core_integrableOn lam a.core
have hupper := cap_integrableOn lam a.upperCap
have hlower := cap_integrableOn lam a.lowerCap
rw [weightedArea, WeightedArea, carrier,
  setIntegral_union, hfirstLower
  a.lowerCap.measurableSet_carrier.nullMeasurableSet
  (hcore.union hupper) hlower,
  setIntegral_union, hcoreUpper
  a.upperCap.measurableSet_carrier.nullMeasurableSet hcore hupper]
rw [show (f p in a.core.carrier, StripDensity lam p) =
  a.core.euclideanArea by
  simp [WeightedArea] using a.core.weightedArea_formula lam,
  show (f p in a.upperCap.carrier, StripDensity lam p) =
  lam * a.upperCap.euclideanArea by
  simp [WeightedArea] using
  cap_upper_weightedArea lam a.upperCap rfl rfl,
  show (f p in a.lowerCap.carrier, StripDensity lam p) =
  lam * a.lowerCap.euclideanArea by
  simp [WeightedArea] using
  cap_lower_weightedArea lam a.lowerCap rfl rfl]
ring

/-- The boundary integral equals the four-arc component formula. -/

```

FourArcAssembly.weightedPerimeter_formula

Classification: main

Source: CMVGeometry.lean:2603-2624

Declaration SHA-256: 43ff4cdbd6774032399c0e7d7f5767a18500986a522e0ace1e470e2035290acd

Mathematical statement. For any real λ , the auxiliary assembly perimeter equals the core exposed-arc length plus λ times the sum of the two exterior arc lengths.

Integrate the sum of four parametrized measures, using their integrability and their branchwise constant densities.

```

theorem weightedPerimeter_formula (lam : ℝ) (a : FourArcAssembly) :
  a.weightedPerimeter lam =
    a.core.boundaryArcLength + lam *
      (a.upperCap.arclength + a.lowerCap.arclength) := by
  rw [weightedPerimeter, WeightedPerimeter, boundaryMeasure]
  rw [integral_add_measure
    (((leftCoreBoundaryMeasure_integrable lam a.core).add_measure
      (rightCoreBoundaryMeasure_integrable lam a.core)).add_measure
      (capBoundaryMeasure_integrable lam a.upperCap))
    (capBoundaryMeasure_integrable lam a.lowerCap)]
  rw [integral_add_measure
    ((leftCoreBoundaryMeasure_integrable lam a.core).add_measure
      (rightCoreBoundaryMeasure_integrable lam a.core))
    (capBoundaryMeasure_integrable lam a.upperCap)]
  rw [integral_add_measure
    (leftCoreBoundaryMeasure_integrable lam a.core)
    (rightCoreBoundaryMeasure_integrable lam a.core)]
  rw [leftCore_weightedIntegral, rightCore_weightedIntegral,
    cap_upper_weightedIntegral lam a.upperCap rfl rfl,
    cap_lower_weightedIntegral lam a.lowerCap rfl rfl,
    StripCore.boundaryArcLength]
  ring

```

FourArcCandidate.candidate_area_formula

Classification: main

Source: FourArcCandidate.lean:410-430

Declaration SHA-256: b6d944ecc5d4bab77cadf263520f194142a9d6855088a9b98e5873be144ab8cc

Mathematical statement. For a candidate satisfying

$$\lambda > 1$$

and

$$\lambda \cos \alpha = h,$$

its area is

$$4 \sin \alpha / h + 2[\lambda \alpha - \sin \alpha \cos \beta] / h^2 + 2[\arcsin h - h \sqrt{1 - h^2}] / h^2,$$

where

$$\beta = \pi/2 - \arcsin h.$$

Expand the actual component integrals, use the Snell incidence to replace the weighted cap cosine term, and simplify the core side-cap areas.

```
theorem candidate_area_formula
  (h : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.WeightedArea =
    4 * sin candidate.alpha / candidate.h +
    2 * (lam * candidate.alpha -
      sin candidate.alpha * cos candidate.beta) / candidate.h ^ 2 +
    2 * (arcsin candidate.h -
      candidate.h * sqrt(1 - candidate.h ^ 2)) / candidate.h ^ 2 := by
  rw [candidate.weightedArea_components, StripCore.euclideanArea,
    candidate.side_caps_area_formula]
  simp only [stripCore, capChord]
  have hcosBeta : cos candidate.beta = candidate.h := by
    rw [beta, Real.cos_pi_div_two_sub]
    exact Real.sin_arcsin (by linarith [candidate.h_pos]) candidate.h_le_one
  rw [hcosBeta, capArea]
  field_simp [ne_of_gt candidate.h_pos]
  have hsnell_mul := congrArg (fun z : ℝ => sin candidate.alpha * z)
    h.snell_incidence
  nlinarith [hsnell_mul]

end FourArcCandidate
```

FourArcCandidate.candidate_perimeter_formula

Classification: main

Source: FourArcCandidate.lean:391-409

Declaration SHA-256: b4949bdc7019f429151f2faf734db8466461943a33be006f4c104caebb493fe0

Mathematical statement. For a candidate satisfying

$$\lambda > 1$$

and

$$\lambda \cos \alpha = h,$$

its perimeter is

$$4\lambda\alpha/h + 4\arcsin(h)/h.$$

Use the component length identity and the common radius. The stated contract retains the physical incidence hypothesis although the length simplification itself needs only the candidate bounds.

```
theorem candidate_perimeter_formula
  (_h : candidate.SatisfiesCMVTypeIVHypotheses) :
  candidate.WeightedPerimeter =
    4 * lam * candidate.alpha / candidate.h +
    4 * arcsin candidate.h / candidate.h := by
  rw [candidate.weightedPerimeter_components,
```

```

StripCore.boundaryArcLength]
change 4 * (arcsin candidate.h / sin (arcsin candidate.h)) +
  2 * lam * (2 * sin candidate.alpha / candidate.h) *
    (candidate.alpha / sin candidate.alpha) =
4 * lam * candidate.alpha / candidate.h +
  4 * arcsin candidate.h / candidate.h
rw [Real.sin_arcsin (by linarith [candidate.h_pos])
  candidate.h_le_one]
field_simp [ne_of_gt candidate.h_pos, ne_of_gt candidate.sin_alpha_pos]
ring

/-- CMV's area display following equation (27), derived from the rectangle and
three cap pieces. -/

```

FourArcCandidate.four_arcs_common_radius

Classification: main

Source: FourArcCandidate.lean:167-181

Declaration SHA-256: cb52732d26d7e21d171f55b06767618effe21e04fd18d38de3625ce3799422be

Mathematical statement. For every candidate, both exterior cap radii equal the strip-core radius

$$1/h.$$

The chord formula fixes the common radius independently of any density or incidence assumption.

```

theorem four_arcs_common_radius :
  candidate.assembly.upperCap.radius = candidate.stripCore.radius ^
  candidate.assembly.lowerCap.radius = candidate.stripCore.radius := by
have hinc : candidate.stripCore.chord * candidate.stripCore.curvature =
  2 * sin candidate.assembly.outerAngle := by
  simp only [stripCore, assembly, capChord]
  field_simp [ne_of_gt candidate.h_pos]
constructor
· exact candidate.assembly.outer_radius_eq_core_radius hinc
· simpa [FourArcAssembly.lowerCap, FourArcAssembly.upperCap,
  OneSidedCircularCap.radius] using
  candidate.assembly.outer_radius_eq_core_radius hinc

/-- Reflection in the horizontal axis exchanges the upper and lower cap arcs,
including the `h = 1` endpoint. -/

```

FrontierMeasure

Classification: main

Source: CMVGeometry.lean:61-66

Declaration SHA-256: 7323bb80c0a565daf410d27d199e2143ee33a0ecca0460c52504f2e7e50b48e2

Mathematical statement. For a planar carrier F , define its canonical boundary measure by pulling Euclidean

$$\mathcal{H}^1$$

restricted to the frontier of its Euclidean image back to the coordinate plane.

The coordinate homeomorphism makes this precisely the complete topological frontier with Euclidean length; the measure is determined by the carrier.

```

def FrontierMeasure (carrier : Set PlanePoint) : Measure PlanePoint :=
  Measure.map planeEuclideanHomeomorph.symm
    ((μH[1] : Measure EuclideanPlane).restrict
      (frontier (planeEuclideanHomeomorph '' carrier)))

/-- Weighted perimeter is the density integral against a boundary measure. -/

```

LeanSuffixAnalytic.typeThreeArea_tendsto_atRight_zero

Classification: main

Source: LeanSuffixAnalytic.lean:828-871

Declaration SHA-256: e82ffa97f750ea7c64d3d22eebba58e03fe2fc0f731297b9787ae711ded5a49a

Mathematical statement. For $\lambda > 1$, the normalized three-arc area tends to

$$+\infty$$

as $h \downarrow 0$.

The numerator tends to

$$\lambda \arccos(-1/\lambda) + \sqrt{\lambda^2 - 1} / \lambda > 0.$$

A positive lower numerator bound and a diverging reciprocal imply the result.

```
theorem typeThreeArea_tendsto_atRight_zero {lam : ℝ} (hlam : 1 < lam) :
  Tendsto (typeThreeArea lam) (ℳ[>] 0) atTop := by
  let numerator : ℝ → ℝ := fun h =>
    typeThreeAngle lam h + π / 2 +
    (typeThreeShape h + 2) * typeThreeDelta lam h
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hrad : 0 < lam ^ 2 - 1 := by nlinarith
  have hnum_cont : ContinuousAt numerator 0 := by
    dsimp [numerator, typeThreeAngle, typeThreeDelta, typeThreeShape]
    fun_prop
  have hnum_zero : numerator 0 =
    lam * arccos (-1 / lam) + sqrt (lam ^ 2 - 1) / lam := by
    simp [numerator, typeThreeAngle, typeThreeDelta, typeThreeShape]
    ring
  have hC : 0 < numerator 0 := by
    rw [hnum_zero]
    exact add_pos_of_nonneg_of_pos
      (mul_nonneg hlam_pos.le (Real.arccos_nonneg _))
      (div_pos (Real.sqrt_pos.2 hrad) hlam_pos)
  have hnum_tendsto : Tendsto numerator (ℳ[>] (0 : ℝ)) (ℳ (numerator 0)) :=
    hnum_cont.tendsto.mono_left inf_le_left
  have hnum_lb : ∀ h in ℳ[>] (0 : ℝ), numerator 0 / 2 < numerator h :=
    hnum_tendsto.eventually_const_lt (by linarith)
  have hsmall : ∀ h in ℳ[>] (0 : ℝ), h < 1 :=
    (eventually_lt_nhds (show (0 : ℝ) < 1 by norm_num)).filter_mono inf_le_left
  have hlower : Tendsto (fun h : ℝ => numerator 0 / 2 * h⁻¹)
    (ℳ[>] 0) atTop :=
    tendsto_inv_nhdsGT_zero.const_mul_atTop (by linarith)
  apply tendsto_atTop_mono' (ℳ[>] (0 : ℝ)) _ hlower
  filter_upwards [hnum_lb, self_mem_nhdsWithin, hsmall] with h hnum hh hlt
  have hhpos : 0 < h := hh
  have hsq_pos : 0 < h ^ 2 := sq_pos_of_pos hhpos
  have hsquare_le : h ^ 2 ≤ h := by nlinarith
  calc
    numerator 0 / 2 * h⁻¹ = numerator 0 / 2 / h := by
      simp only [div_eq_mul_inv]
    _ ≤ numerator 0 / 2 / h ^ 2 :=
      div_le_div_of_nonneg_left (by linarith) hsq_pos hsquare_le
    _ ≤ numerator h / h ^ 2 :=
      div_le_div₀ (by linarith) hnum.le hsq_pos le_rfl
    _ = typeThreeArea lam h := by rfl

/-- For `lam > 1`, the regular type-(iii) fold is strictly increasing
throughout the admissible height interval. -/
```

LeanSuffixAnalytic.typeThree_variational_hasDerivAt

Classification: main

Source: LeanSuffixAnalytic.lean:554-639

Declaration SHA-256: 727f5f60431aed129e76617edde1e3ccdf8d18a1ae71fbf80ca33147d09b9cfa

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, the normalized three-arc area is differentiable at h , and its perimeter has derivative h times that area derivative.

The proof differentiates the two explicit quotients and cancels their difference by the radical identity. It is a local variational result, not a global monotonicity theorem.

```
theorem typeThree_variational_hasDerivAt {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeArea lam) (deriv (typeThreeArea lam) h) h ^
  HasDerivAt (typeThreePerimeter lam)
    (h * deriv (typeThreeArea lam) h) h := by
  let q := typeThreeShape h
  let D := typeThreeDelta lam h
  let p := 2 * (1 / sqrt (1 - q ^ 2) -
    lam / sqrt (lam ^ 2 - q ^ 2))
  let dp := 2 * q * (1 / sqrt (1 - q ^ 2) -
    1 / (lam * sqrt (lam ^ 2 - q ^ 2)))
  have hh_ne : h ≠ 0 := ne_of_gt hh
  have hshape := hasDerivAt_typeThreeShape h
  have hangle : HasDerivAt (typeThreeAngle lam) p h := by
    simpa only [p, q] using hasDerivAt_typeThreeAngle hlam hh hh1
  have hdelta : HasDerivAt (typeThreeDelta lam) dp h := by
    simpa only [dp, q] using hasDerivAt_typeThreeDelta hlam hh hh1
  have hq_arg : 0 < 1 - q ^ 2 := by
    dsimp only [q, typeThreeShape]
    nlinarith
  have hd_arg : 0 < lam ^ 2 - q ^ 2 := by
    have hlam_sq : 1 < lam ^ 2 := by nlinarith
    have hq_sq : q ^ 2 < 1 := by
      dsimp only [q, typeThreeShape]
      nlinarith
    linarith
  have hlam_ne : lam ≠ 0 := ne_of_gt (lt_trans (by norm_num) hlam)
  have hq_ne : sqrt (1 - q ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hq_arg)
  have hd_ne : sqrt (lam ^ 2 - q ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  have hkey : p - q * dp + 2 * D = 0 := by
    dsimp only [p, dp, D, typeThreeDelta]
    change
      2 * (1 / sqrt (1 - q ^ 2) - lam / sqrt (lam ^ 2 - q ^ 2)) -
      q * (2 * q * (1 / sqrt (1 - q ^ 2) -
        1 / (lam * sqrt (lam ^ 2 - q ^ 2)))) +
      2 * (sqrt (lam ^ 2 - q ^ 2) / lam - sqrt (1 - q ^ 2)) = 0
    field_simp [hlam_ne, hq_ne, hd_ne]
    ring_nf
    rw [Real.sq_sqrt hq_arg.le, Real.sq_sqrt hd_arg.le]
    ring
  have hB := hangle.add_const (π / 2)
  have hareaRaw :=
    (hB.add ((hshape.add_const 2).mul hdelta)).div
      ((hasDerivAt_id h).pow 2) (pow_ne_zero 2 hh_ne)
  have harea : HasDerivAt (typeThreeArea lam)
    (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
      (typeThreeAngle lam h + π / 2 + (q + 2) * D) * (2 * h)) /
      (h ^ 2) ^ 2) h := by
    simpa only [typeThreeArea, q, D, id_eq] using!
      hareaRaw.congr_deriv (by
        norm_num [id_eq, Pi.add_apply, Pi.mul_apply, Pi.pow_apply])
  have hperimeterRaw :=
    ((hB.add hdelta).const_mul (2 : ℝ)).div (hasDerivAt_id h) hh_ne
  have hperimeter : HasDerivAt (typeThreePerimeter lam)
    (((2 * (p + dp)) * h -
```

```

2 * (typeThreeAngle lam h + π / 2 + D)) / h ^ 2) h := by
simp only [typeThreePerimeter, D, id_eq] using!
hperimeterRaw.congr_deriv (by
  norm_num [id_eq, Pi.add_apply, Pi.mul_apply, Pi.pow_apply])
refine (harea.congr_deriv harea.deriv.symm, hperimeter.congr_deriv ?_)
rw [harea.deriv]
have halgebra : ∀ (q D p dp B h : ℝ), h ≠ 0 → q = 2 * h - 1 →
  p - q * dp + 2 * D = 0 →
  ((2 * (p + dp)) * h - 2 * (B + D)) / h ^ 2 =
  h * (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
    (B + (q + 2) * D) * (2 * h)) / (h ^ 2) ^ 2) := by
intro q D p dp B h hh_ne hq hkey
calc
  ((2 * (p + dp)) * h - 2 * (B + D)) / h ^ 2 =
  h * (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
    (B + (q + 2) * D) * (2 * h)) / (h ^ 2) ^ 2) +
  (p - q * dp + 2 * D) / h := by
  rw [hq]
  field_simp [hh_ne]
  ring
  _ = h * (((p + (2 * D + (q + 2) * dp)) * h ^ 2 -
    (B + (q + 2) * D) * (2 * h)) / (h ^ 2) ^ 2) := by
  rw [hkey]
  ring
exact halgebra
q D p dp (typeThreeAngle lam h + π / 2) h hh_ne (by
  simp only [q, typeThreeShape]) hkey

/-- Exact type-(iii) variational identity on the regular domain. -/

```

OneSidedCircularCap.arcPoint_in_open_side

Classification: main

Source: CMVGeometry.lean:253-266

Declaration SHA-256: 3dbf6a865d1d6331f9c3406c05af249acf00d7c487349d982b588394a814d697

Mathematical statement. If

$$|t| < \theta,$$

the selected upper arc point has

$$y > b,$$

and the selected lower arc point has

$$y < b.$$

Strict cosine comparison puts all non-endpoint arc parameters strictly on the declared side, including for major caps.

```

theorem arcPoint_in_open_side (c : OneSidedCircularCap) {t : ℝ}
  (ht : |t| < c.theta) :
  match c.side with
  | .upper => c.baseY < (c.arcPoint t).2
  | .lower => (c.arcPoint t).2 < c.baseY := by
have hc : cos c.theta < cos t := by
  rw [← Real.cos_abs t]
  exact Real.cos_lt_cos_of_nonneg_of_le_pi (abs_nonneg t)
  (le_of_lt c.theta_lt_pi) ht
have hr := c.radius_pos
cases h : c.side <|> simp [arcPoint, h] <|> nlinarith

/-- The selected circular arc has no self-intersections, including on the
major branch. -/

```

OneSidedCircularCap.carrier

Classification: main

Source: CMVGeometry.lean:146-152

Declaration SHA-256: e0644dd0b5f330958b79efbb3810eb1cc2f4356e5db251a95736b532b75bfcf1

Mathematical statement. The cap carrier is its closed radius- R disk intersected with

$$y \geq b$$

for an upper cap, or

$$y \leq b$$

for a lower cap.

This single coordinate definition covers the full half-angle interval and retains the attaching chord.

```
def carrier (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | c.radiusSquaredAt p ≤ c.radius ^ 2 ∧
    match c.side with
    | .upper => c.baseY ≤ p.2
    | .lower => p.2 ≤ c.baseY}

/-- The open cap interior. Endpoints and the chord are deliberately absent. -/
```

OneSidedCircularCap.volume_carrier

Classification: main

Source: CMVGeometry.lean:1996-2028

Declaration SHA-256: 0d51f9d06b45af9eac50b2062d616aee51a260200ad5a2e96d2ecbe8e98be85c

Mathematical statement. The Lebesgue volume of a cap carrier equals its scalar Euclidean area

$$L^2a(\theta),$$

embedded in the nonnegative extended reals.

Apply the upper or lower circular-segment volume formula with the cap center and radius, then use the chord-radius identity.

```
theorem volume_carrier (c : OneSidedCircularCap) :
  volume c.carrier = ENNReal.ofReal c.euclideanArea := by
  have hreal :
    c.radius ^ 2 * (c.theta - sin c.theta * cos c.theta) =
      c.euclideanArea := by
    rw [euclideanArea, area, radius]
    field_simp [ne_of_gt c.sin_theta_pos]
    ring
  cases hside : c.side with
  | upper =>
    calc
      volume c.carrier =
        ENNReal.ofReal
          (c.radius ^ 2 *
            (c.theta - sin c.theta * cos c.theta)) := by
      simpa [carrier, radiusSquaredAt, center, hside] using
        volume_upper_circular_segment c.midpointX
          (c.baseY - c.radius * cos c.theta) c.radius c.theta
          c.radius_pos c.theta_pos.le c.theta_lt_pi.le
      _ = ENNReal.ofReal c.euclideanArea := congrArg ENNReal.ofReal hreal
  | lower =>
    calc
      volume c.carrier =
        ENNReal.ofReal
```

```

      (c.radius ^ 2 *
       (c.theta - sin c.theta * cos c.theta)) := by
    simp [carrier, radiusSquaredAt, center, hside] using
      volume_lower_circular_segment c.midpointX
      (c.baseY + c.radius * cos c.theta) c.radius c.theta
      c.radius_pos c.theta_pos.le c.theta_lt_pi.le
    _ = ENNReal.ofReal c.euclideanArea := congrArg ENNReal.ofReal hreal
end OneSidedCircularCap

```

StripCore.volume_carrier

Classification: main

Source: CMVGeometry.lean:2202-2242

Declaration SHA-256: a426f0908aec7b73455f4a0093170232e841cf46ab8f580ee2fe442ff66a7997

Mathematical statement. The planar volume of the strip core equals its scalar area

$$2L + 8a(\beta),$$

embedded in the nonnegative extended reals.

The rectangle and two caps overlap only along planar-null lines, so their actual volumes add.

```

lemma volume_carrier (c : StripCore) :
  volume c.carrier = ENNReal.ofReal c.euclideanArea := by
  have hrectLeft : AEDisjoint volume c.rectangleCarrier c.leftCapCarrier := by
    refine measure_mono_null ?_ (volume_verticalline (-c.chord / 2))
    rintro p ⟨hrect, hleft⟩
    exact le_antisymm hleft.2.1 hrect.1
  have hfirstRight : AEDisjoint volume
    (c.rectangleCarrier ∪ c.leftCapCarrier) c.rightCapCarrier := by
    refine measure_mono_null ?_ (volume_verticalline (c.chord / 2))
    rintro p ⟨hfirst, hright⟩
    rcases hfirst with hrect | hleft
    · exact le_antisymm hrect.2.1 hright.2.1
    · ex falso
      nlinarith [hleft.2.1, hright.2.1, c.chord_pos]
  have hleftMeas : MeasurableSet c.leftCapCarrier := by
    rw [leftCapCarrier]
    exact ((isClosed_le
      (((continuous_fst.sub continuous_const).pow 2).add
       (continuous_snd.pow 2)) continuous_const).inter
      ((isClosed_le continuous_fst continuous_const).inter
       (isClosed_le continuous_snd.abs continuous_const)))
      .measurableSet
  have hrightMeas : MeasurableSet c.rightCapCarrier := by
    rw [rightCapCarrier]
    exact ((isClosed_le
      (((continuous_fst.sub continuous_const).pow 2).add
       (continuous_snd.pow 2)) continuous_const).inter
      ((isClosed_le continuous_const continuous_fst).inter
       (isClosed_le continuous_snd.abs continuous_const)))
      .measurableSet
  rw [carrier, measure_union, hrightMeas.nullMeasurableSet hfirstRight,
    measure_union, hleftMeas.nullMeasurableSet hrectLeft,
    volume_rectangleCarrier, volume_leftCapCarrier, volume_rightCapCarrier]
  have hq : 0 ≤ 2 * c.chord := mul_nonneg (by norm_num) c.chord_pos.le
  have ha : 0 ≤ 4 * area c.sideAngle := by
    exact mul_nonneg (by norm_num) (area_pos c.sideAngle_pos c.sideAngle_lt_pi).le
  rw [← ENNReal.ofReal_add hq ha, ← ENNReal.ofReal_add (add_nonneg hq ha) ha]
  congr 1
  rw [euclideanArea]
  ring

```

end StripCore

StripCore.weightedArea_formula

Classification: main

Source: CMVGeometry.lean:2315-2333

Declaration SHA-256: 22039501f3884648956188fd011ab629e4d948f733db9c19194b8b310134e20a

Mathematical statement. For every real λ , the weighted area of a strip core equals its Euclidean scalar area.

Its carrier lies entirely in the closed density-one strip; the carrier-volume theorem evaluates the integral of one.

```
lemma StripCore.weightedArea_formula (lam : ℝ) (c : StripCore) :  
  WeightedArea lam c.carrier = c.euclideanArea := by  
  rw [WeightedArea]  
  calc  
    (∫ p in c.carrier, StripDensity lam p) =  
      ∫ _p in c.carrier, (1 : ℝ) := by  
        apply setIntegral_congr_fun c.measurableSet_carrier  
        intro p hp  
        rw [StripDensity, if_pos (c.carrier_y_bounds hp)]  
    _ = c.euclideanArea := by  
      have ha : 0 ≤ c.euclideanArea := by  
        rw [StripCore.euclideanArea]  
        exact add_nonneg (mul_nonneg (by norm_num) c.chord_pos.le)  
          (mul_nonneg (by norm_num)  
            (area_pos c.sideAngle_pos c.sideAngle_lt_pi).le)  
      rw [integral_const, measureReal_restrict_apply_univ,  
        Measure.real, c.volume_carrier, ENNReal.toReal_ofReal ha]  
      simp
```

StripDensity

Classification: main

Source: CMVGeometry.lean:43-54

Declaration SHA-256: 51e629a8115629e4652f417b9dcfc46beea6a584d8ebcb351e746df181f97ab1

Mathematical statement. For all real λ , define

$$f_\lambda(x, y) = 1$$

when $|y| \leq 1$, and

$$f_\lambda(x, y) = \lambda$$

otherwise.

The closed strip includes both interfaces. Positivity is supplied by the comparison hypothesis

$$\lambda > 1,$$

not by the total definition.

```
def StripDensity (lam : ℝ) (p : PlanePoint) : ℝ :=  
  if |p.2| ≤ 1 then 1 else lam  
  
@[simp] theorem stripDensity_upper_interface (lam x : ℝ) :  
  StripDensity lam (x, 1) = 1 := by  
  simp [StripDensity]  
  
@[simp] theorem stripDensity_lower_interface (lam x : ℝ) :  
  StripDensity lam (x, -1) = 1 := by  
  simp [StripDensity]  
  
/-- Lebesgue weighted area for a general measurable planar carrier. -/
```

TypeThreeAssembly.bottomChord_eq_zero_iff

Classification: main

Source: TypeThreeAssembly.lean:355-358

Declaration SHA-256: 1b82dffbe772075bcb5095c9dc075644068e9232dc927ae0cd23919b863062dd

Mathematical statement. The bottom chord has zero length if and only if $h = 1/2$.

Its length is twice the side half-width, whose exact zero locus was proved.

```
theorem bottomChord_eq_zero_iff : a.bottomChord = 0 ↔ a.h = 1 / 2 := by
  rw [bottomChord, mul_eq_zero]
  norm_num
  exact a.sideHalfWidth_eq_zero_iff
```

TypeThreeAssembly.core_weightedArea_formula

Classification: main

Source: TypeThreeAssembly.lean:1530-1550

Declaration SHA-256: 5a80fac760bbad4e64452fa4d575b54c924b85ffbad6442cb8801547dabcbba7

Mathematical statement. The weighted area of the three-arc core equals its rectangle area plus auxiliary side-cap area.

The core lies in the density-one strip, so its actual volume computation also evaluates its density integral.

```
theorem core_weightedArea_formula :
  _root_.WeightedArea lam a.coreCarrier =
    a.rectangleArea + a.sideCap.euclideanArea := by
  rw [_root_.WeightedArea]
  calc
    (f p in a.coreCarrier, StripDensity lam p) =
      f _p in a.coreCarrier, (1 : ℝ) := by
        apply setIntegral_congr_fun a.isClosed_coreCarrier.measurableSet
        intro p hp
        rw [StripDensity, if_pos (a.coreCarrier_y_bounds hp)]
    _ = a.rectangleArea + a.sideCap.euclideanArea := by
      have hnonneg :
        0 ≤ a.rectangleArea + a.sideCap.euclideanArea :=
          add_nonneg
          (mul_nonneg (by norm_num) a.sideHalfWidth_nonneg)
          a.sideCap.euclideanArea_pos.le
      rw [integral_const, measureReal_restrict_apply_univ,
        Measure.real, a.volume_coreCarrier,
        ENNReal.toReal_ofReal hnonneg]
      simp
```

TypeThreeAssembly.euclidean_frontier

Classification: main

Source: TypeThreeAssembly.lean:2937-2949

Declaration SHA-256: 5e03c2ded7f93ebce1b77460290c9d90f4e5be41990287351d94b69c624d60ea

Mathematical statement. The frontier of the Euclidean image of the three-arc carrier equals the union of the Euclidean images of its left arc, right arc, upper cap arc and bottom segment.

The coordinate homeomorphism transports the exact frontier decomposition into the metric used for Hausdorff length.

```
theorem euclidean_frontier :
  frontier (planeEuclideanHomeomorph '' a.carrier) =
    (planeEuclideanHomeomorph '' a.leftArcTrace) ∪
    ((planeEuclideanHomeomorph '' a.rightArcTrace) ∪
     (planeEuclideanHomeomorph ''
```

```

      OneSidedCircularCap.arcTrace a.outerCap) U
      (planeEuclideanHomeomorph '' a.bottomSegmentCarrier))) := by
    rw [← planeEuclideanHomeomorph.image_frontier, a.frontier_carrier]
    simp only [boundaryTrace, image_union]

/-- The component perimeter is the canonical weighted  $H^1$  integral over the
complete topological frontier. This includes the exposed bottom segment and
is valid at its zero-length semicircular transition. -/

```

TypeThreeAssembly.exact_euclidean_bottomSegmentCarrier_hausdorffMeasure

Classification: main

Source: TypeThreeAssembly.lean:1101-1129

Declaration SHA-256: 56949577898a4e9f1638a1a8bab2c43a8263ea9e6cf0976fce47fbff56b3fe00

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the bottom segment is $2b$.

For positive width apply the horizontal-segment theorem; for zero width the singleton has zero measure.

```

theorem exact_euclidean_bottomSegmentCarrier_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' a.bottomSegmentCarrier) =
    ENNReal.ofReal a.bottomChord := by
  rcases eq_or_ne a.h (1 / 2) with hh | hh
  · have hw : a.sideHalfWidth = 0 :=
    a.sideHalfWidth_eq_zero_iff.mpr hh
  have hc : a.bottomChord = 0 :=
    a.bottomChord_eq_zero_iff.mpr hh
  have hcarrier : a.bottomSegmentCarrier = {(0, -1)} := by
    ext p
    simp only [bottomSegmentCarrier, mem_inter_iff, mem_ofPred_eq,
      hw, abs_nonpos_iff, mem_singleton_iff]
  constructor
  · rintro ⟨hy, hx⟩
    exact Prod.ext hx hy
  · intro hp
    rw [hp]
    exact ⟨rfl, rfl⟩
  rw [hcarrier, image_singleton, hc, ENNReal.ofReal_zero]
  let _ : NullSingletonClass (μH[1] : Measure EuclideanPlane) :=
    Measure.nullSingletonClass_hausdorff EuclideanPlane (by norm_num)
  exact measure_singleton _
  · rw [← a.bottomHorizontalSegment_carrier hh,
    exact_euclidean_segmentCarrier_hausdorffMeasure]
    rfl

/-- The four primitive boundary joins. The two bottom points coincide at the
semicircular transition. -/

```

TypeThreeAssembly.frontier_carrier

Classification: main

Source: TypeThreeAssembly.lean:2848-2854

Declaration SHA-256: 0ce2b73ef2aba31ce763d170404d962fb774d3c494bf5afe10757388adbfbdb30

Mathematical statement. The complete topological frontier of the three-arc carrier is exactly its declared boundary trace.

Combine the two inclusions, so the perimeter calculation starts from the actual frontier rather than a postulated sum.

```

theorem frontier_carrier :
  frontier a.carrier = a.boundaryTrace :=
  Set.Subset.antisymm a.frontier_carrier_subset_boundaryTrace
    a.boundaryTrace_subset_frontier_carrier

```

```

/-- The Euclidean realization of the bottom segment is measurable, including
the singleton occurring at the semicircular transition. -/

```

TypeThreeAssembly.integrableOn_carrier

Classification: main

Source: TypeThreeAssembly.lean:3000-3011

Declaration SHA-256: f50bce5009ef96b4081db29960a7e72186fb76edbfedf9fa6fd538bc59ab8c22

Mathematical statement. The strip density is integrable over the actual three-arc carrier.

Its finite-area rectangle and disk pieces have bounded density; finite unions preserve the required integrability.

```

theorem integrableOn_carrier :
  IntegrableOn (StripDensity lam) a.carrier := by
  rw [carrier]
  have hcore : IntegrableOn (StripDensity lam) a.coreCarrier := by
    apply stripDensity_integrableOn_of_volume_ne_top
    rw [a.volume_coreCarrier]
    exact ENNReal.ofReal_ne_top
  exact hcore.union (cap_integrableOn lam a.outerCap)

/-- The density is integrable against the canonical frontier measure. This is
the finite-perimeter obligation for treating the coordinate carrier as a
general admissible competitor. -/

```

TypeThreeAssembly.integrable_frontierMeasure

Classification: main

Source: TypeThreeAssembly.lean:3012-3042

Declaration SHA-256: 242285e9e93302048576853114a4bc7f6ca16cdc917f22c33a2fe025def880a8

Mathematical statement. The strip density is integrable against the canonical frontier measure of the three-arc carrier.

Its frontier decomposes into finite-length pieces, each with bounded density, so the weighted perimeter integral is a genuine finite integral.

```

theorem integrable_frontierMeasure :
  Integrable (StripDensity lam) (FrontierMeasure a.carrier) := by
  have hfrontier :
    (μH[1] : Measure EuclideanPlane)
    (frontier (planeEuclideanHomeomorph '' a.carrier)) * T := by
  rw [a.euclidean_frontier]
  apply measure_union_ne_top
  · rw [a.exact_euclidean_leftArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top
  apply measure_union_ne_top
  · rw [a.exact_euclidean_rightArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top
  apply measure_union_ne_top
  · rw [a.exact_euclidean_capTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top
  · rw [a.exact_euclidean_bottomSegmentCarrier_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top
  unfold FrontierMeasure
  rw [integrable_map_measure
    (measurable_stripDensity lam).astronglyMeasurable
    planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable]
  change IntegrableOn (euclideanStripDensity lam)
    (frontier (planeEuclideanHomeomorph '' a.carrier))
    (μH[1] : Measure EuclideanPlane)
  exact euclideanStripDensity_integrableOn_of_measure_ne_top lam hfrontier

```

```

/-- Geometric realization contract for the coordinate carrier. Both weighted
measure formulas follow from `carrier_eq`; neither is independently selectable
data. -/

```

TypeThreeAssembly.pairwiseOverlap_h1_null

Classification: main

Source: TypeThreeAssembly.lean:1258-1273

Declaration SHA-256: 545b8d3e4922237126e7364ee9bb07b444cf8cada72b02e8afca89501d2f79b3

Mathematical statement. The Euclidean image of the union of those pairwise intersections has zero $\mathcal{H}^1$ -measure.

It is contained in the Euclidean image of a finite junction set, so overlapping endpoints do not affect perimeter addition.

```

theorem pairwiseOverlap_h1_null :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' pairwiseOverlap
    a.leftArcTrace a.rightArcTrace
    (OneSidedCircularCap.arcTrace a.outerCap)
    a.bottomSegmentCarrier) = 0 := by
let _ := Measure.nullSingletonClass_hausdorff EuclideanPlane
  (by norm_num : (0 : ℝ) < 1)
have hjoin :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' a.boundaryJoinSet) = 0 :=
  (a.boundaryJoinSet_finite.image planeEuclideanHomeomorph).measure_zero
  μH[1]
exact measure_mono_null
  (image_mono a.pairwiseOverlap_subset_boundaryJoinSet) hjoin

```

TypeThreeAssembly.weightedArea_formula

Classification: main

Source: TypeThreeAssembly.lean:1584-1588

Declaration SHA-256: 53616c2c25615a95257aa4773c87d4c69464dc905650e6ed5b53696dc5f08570

Mathematical statement. The actual weighted area of the assembly equals its scalar area

$$[T + (q + 2)\delta]/h^2.$$

Compose the integral-to-modeled identity with the algebraic modeled-to-scalar identity.

```

theorem weightedArea_formula :
  a.WeightedArea = a.scalarWeightedArea := by
  rw [a.weightedArea_eq_modeledWeightedArea,
    a.modeledWeightedArea_formula]

```

TypeThreeAssembly.weightedPerimeter_formula

Classification: main

Source: TypeThreeAssembly.lean:2994-2999

Declaration SHA-256: c6f51109b06f9f61cbbcb63e9d6e1d950e5a79c8decdf76b0e2fc3f366f8eca

Mathematical statement. The actual complete-frontier weighted perimeter equals

$$2(T + \delta)/h.$$

Compose the geometric integral-to-modeled identity with the scalar algebraic identity, without assuming a perimeter formula.

```

theorem weightedPerimeter_formula :
  a.WeightedPerimeter = a.scalarWeightedPerimeter := by
  rw [a.weightedPerimeter_eq_modeledWeightedPerimeter,
    a.modeledWeightedPerimeter_formula]

/-- The density is integrable over the complete coordinate carrier. -/

```

WeightedArea

Classification: main

Source: CMVGeometry.lean:55-60

Declaration SHA-256: 959b5ddd7363861d4d0d79c211781b3b19e09df50087472690ae0581725b8c6d

Mathematical statement. For any planar set F and real λ , define the weighted area as the Lebesgue integral

$$\int_F f_\lambda dx dy.$$

Integrability for the explicit carriers is proved separately, so the integral represents their finite geometric area rather than merely a formal total integral.

```

def WeightedArea (lam : ℝ) (carrier : Set PlanePoint) : ℝ :=
  ∫ p in carrier, StripDensity lam p

/-- Canonical Euclidean one-dimensional Hausdorff measure on the complete
topological frontier of a planar carrier, mapped back to the coordinate plane.
A general competitor cannot choose perimeter data independently of its region. -/

```

WeightedPerimeter

Classification: main

Source: CMVGeometry.lean:67-73

Declaration SHA-256: 8d1f1a60a6d4c5ce03ccda31c1bc34ec4f837b6ce55275e4b0c06fe04e566dba

Mathematical statement. For real λ and a measure μ on the coordinate plane, define the weighted perimeter functional

$$\int f_\lambda d\mu.$$

For geometric regions the measure is always their canonical frontier measure. Auxiliary parametrized measures are related to that measure by the frontier bridge before the final theorem.

```

def WeightedPerimeter (lam : ℝ)
  (boundaryMeasure : Measure PlanePoint) : ℝ :=
  ∫ p, StripDensity lam p ∂boundaryMeasure

/-- General finite-perimeter input to minimization. Its perimeter measure is
canonically the one-dimensional Hausdorff measure restricted to the complete
frontier; finiteness is an invariant, not independently supplied data. -/

```

exact_euclidean_capTrace_hausdorffMeasure

Classification: main

Source: FrontierPerimeter.lean:1081-1088

Declaration SHA-256: 8a0602f46d010a0d38a019d5e1052cfe4c5d69fc3db5988eaac8c5dbcc5c5e1c

Mathematical statement. For every cap, the Euclidean $\mathcal{H}^1$ -measure of its selected arc trace equals its scalar arc length.

Combine the exact parameter-image identity and the exact Hausdorff length of the realized parametrization.

```

theorem exact_euclidean_capTrace_hausdorffMeasure
  (c : OneSidedCircularCap) :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' c.arcTrace) =
  ENNReal.ofReal c.arcLength := by
rw [← realizedCapParam_image_1cc]
exact exact_euclidean_capParam_hausdorffMeasure c

```

fourArc_euclidean_frontier

Classification: main

Source: FrontierPerimeter.lean:1338-1348

Declaration SHA-256: 748ff0ec3e12377aa952eabaeafb59718df5bc069c8651bdcd45a9c687a1d41d

Mathematical statement. The frontier of a four-arc carrier in its Euclidean realization is exactly the union of the Euclidean images of its two core arcs and two cap arcs.

A homeomorphism transports the proved coordinate frontier decomposition without adding hidden chord components.

```

theorem fourArc_euclidean_frontier (a : FourArcAssembly) :
  frontier (planeEuclideanHomeomorph '' a.carrier) =
  (planeEuclideanHomeomorph '' StripCore.leftArcTrace a.core) ∪
  ((planeEuclideanHomeomorph '' StripCore.rightArcTrace a.core) ∪
  (planeEuclideanHomeomorph ''
    OneSidedCircularCap.arcTrace a.upperCap) ∪
  (planeEuclideanHomeomorph ''
    OneSidedCircularCap.arcTrace a.lowerCap))) := by
rw [← planeEuclideanHomeomorph.image_frontier, a.frontier_carrier]
simp only [FourArcAssembly.boundaryTrace, image_union]

```

fourArc_frontier_weightedPerimeter_eq

Classification: main

Source: FrontierPerimeter.lean:1794-1844

Declaration SHA-256: e95ee1864c7bc7d9bc3a3ec2659625a510c6cefccdafef0cd2efab0e2d93e1067

Mathematical statement. For every real λ and four-arc assembly, its auxiliary parametrized weighted perimeter equals the density integral against the canonical complete-frontier measure of its carrier.

The exact frontier decomposition, exact Euclidean arc lengths, branchwise density integrals and null overlaps identify the actual perimeter. This theorem removes the possibility of freely assigning perimeter data.

```

theorem fourArc_frontier_weightedPerimeter_eq
  (lam : ℝ) (assembly : FourArcAssembly) :
  assembly.weightedPerimeter lam =
  WeightedPerimeter lam (FrontierMeasure assembly.carrier) := by
rcases fourArc_pairwise assembly with
  ⟨hAB, hAC, hAD, hBC, hBD, hCD⟩
have hAint := euclideanStripDensity_integrableOn_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph ''
    StripCore.leftArcTrace assembly.core) (by
    rw [exact_euclidean_leftArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)
have hBint := euclideanStripDensity_integrableOn_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph ''
    StripCore.rightArcTrace assembly.core) (by
    rw [exact_euclidean_rightArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)
have hCint := euclideanStripDensity_integrableOn_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph ''
    OneSidedCircularCap.arcTrace assembly.upperCap) (by
    rw [exact_euclidean_capTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)

```

```

have hDint := euclideanStripDensity_integrableOn_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph '')
    OneSidedCircularCap.arcTrace assembly.lowerCap) (by
      rw [exact_euclidean_capTrace_hausdorffMeasure]
      exact ENNReal.ofReal_ne_top)
rw [FourArcAssembly.weightedPerimeter_formula]
symm
unfold WeightedPerimeter FrontierMeasure
rw [integral_map
  planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable
  (measurable_stripDensity lam).aestronglyMeasurable]
change (∫ z in frontier (planeEuclideanHomeomorph '' assembly.carrier),
  euclideanStripDensity lam z
  ∂(μH[1] : Measure EuclideanPlane)) =
  assembly.core.boundaryArcLength +
  lam * (assembly.upperCap.arcLength + assembly.lowerCap.arcLength)
rw [fourArc_euclidean_frontier]
rw [integral_four_union hAB hAC hAD hBC hBD hCD
  (measurableSet_euclidean_rightArcTrace assembly.core)
  (measurableSet_euclidean_capTrace assembly.upperCap)
  (measurableSet_euclidean_capTrace assembly.lowerCap)
  hAint hBint hCint hDint]
rw [left_integral, right_integral,
  upper_cap_integral lam assembly.upperCap rfl rfl,
  lower_cap_integral lam assembly.lowerCap rfl rfl,
  core_param_length, StripCore.boundaryArcLength]
ring

/-- The explicit replacement boundary cost equals the canonical weighted  $\mathcal{H}^1$ 
integral on the complete topological frontier of its coordinate carrier. -/

```

fourArc_pairwiseOverlap_h1_null

Classification: main

Source: FrontierPerimeter.lean:1269-1279

Declaration SHA-256: 01eb58d468da4075ed5a4b3c4808bf8deddca98be0f05ef720c74c5a64fd9f4f

Mathematical statement. For every four-arc assembly, the Euclidean image of the union of pairwise arc-trace intersections has zero $\mathcal{H}^1$ -measure.

It is contained in the null finite junction set; this establishes the needed almost-everywhere disjointness.

```

theorem fourArc_pairwiseOverlap_h1_null (a : FourArcAssembly) :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' pairwiseOverlap
      (StripCore.leftArcTrace a.core)
      (StripCore.rightArcTrace a.core)
      (OneSidedCircularCap.arcTrace a.upperCap)
      (OneSidedCircularCap.arcTrace a.lowerCap)) = 0 := by
  exact measure_mono_null
    (image_mono (fourArc_pairwiseOverlap_subset_joinSet a))
    (hausdorffMeasure_one_euclideanJoinSet a.core.chord)

```

volume_upper_circular_segment

Classification: main

Source: CMVGeometry.lean:1820-1944

Declaration SHA-256: d23f6efcc432aaa58959c64bfa3f6f589362c33546093657b1717ec5dbfb1b34

Mathematical statement. For real m, c ,

$$R > 0,$$

and $0 \leq \theta \leq \pi$, the Lebesgue volume of

$$\{(x, y) : (x - m)^2 + (y - c)^2 \leq R^2, y \geq c + R \cos \theta\}$$

is

$$R^2(\theta - \sin \theta \cos \theta),$$

embedded in the nonnegative extended reals.

Fubini integrates horizontal disk sections; the translated semicircle integral proves the formula including the major-cap branch.

```

lemma volume_upper_circular_segment (mx cy r θ : ℝ) (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  volume {p : ℝ × ℝ |
    (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
    cy + r * cos θ ≤ p.2} =
    ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) := by
  let S : Set (ℝ × ℝ) :=
    {p | p.1 ∈ Icc (cy + r * cos θ) (cy + r) ∧
      p.2 ∈ Icc
        (mx - √(r ^ 2 - (p.1 - cy) ^ 2))
        (mx + √(r ^ 2 - (p.1 - cy) ^ 2))}
  have hS : MeasurableSet S := by
    exact measurableSet_region_between_cc
    (f := fun y : ℝ ↦ mx - √(r ^ 2 - (y - cy) ^ 2))
    (g := fun y : ℝ ↦ mx + √(r ^ 2 - (y - cy) ^ 2))
    (s := Icc (cy + r * cos θ) (cy + r))
    (by fun_prop) (by fun_prop) measurableSet_Icc
  have hpre :
    Prod.swap ⁻¹' S =
    {p : ℝ × ℝ |
      (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
      cy + r * cos θ ≤ p.2} := by
  ext p
  constructor
  · intro hp
    rcases hp with ⟨hy, hx⟩
    change p.2 ∈ Icc (cy + r * cos θ) (cy + r) at hy
    change p.1 ∈ Icc
      (mx - √(r ^ 2 - (p.2 - cy) ^ 2))
      (mx + √(r ^ 2 - (p.2 - cy) ^ 2)) at hx
    have hcosmul : -r ≤ r * cos θ := by
      simp using mul_le_mul_of_nonneg_left (neg_one_le_cos θ) hr.le
    have hymin : cy - r ≤ p.2 := by
      linarith [hy.1]
    have hprod :
      0 ≤ (r - (p.2 - cy)) * (r + (p.2 - cy)) :=
      mul_nonneg (by linarith [hy.2]) (by linarith)
    have hD : 0 ≤ r ^ 2 - (p.2 - cy) ^ 2 := by
      nlinarith
    have hsqrt :
      (√(r ^ 2 - (p.2 - cy) ^ 2)) ^ 2 =
      r ^ 2 - (p.2 - cy) ^ 2 :=
      Real.sq_sqrt hD
    have hx1 :
      -(√(r ^ 2 - (p.2 - cy) ^ 2)) ≤ p.1 - mx := by
      linarith [hx.1]
    have hxu :
      p.1 - mx ≤ √(r ^ 2 - (p.2 - cy) ^ 2) := by
      linarith [hx.2]
    have hprod :
      0 ≤ (√(r ^ 2 - (p.2 - cy) ^ 2) - (p.1 - mx)) *
      (√(r ^ 2 - (p.2 - cy) ^ 2) + (p.1 - mx)) :=
      mul_nonneg (by linarith [hxu]) (by linarith [hx1])
    constructor
    · nlinarith
    · exact hy.1
  · rintro ⟨hcircle, hbase⟩
    have hyUpper : p.2 ≤ cy + r := by
      nlinarith [sq_nonneg (p.1 - mx), sq_nonneg (p.2 - cy + r)]

```

```

have hD :  $0 \leq r^2 - (p.2 - cy)^2$  := by
  nlinarith [sq_nonneg (p.1 - mx)]
have hsqrt :
  ( $\sqrt{r^2 - (p.2 - cy)^2}$ )^2 =
   $r^2 - (p.2 - cy)^2$  :=
  Real.sq_sqrt hD
have hsqrt0 :  $0 \leq \sqrt{r^2 - (p.2 - cy)^2}$  :=
  Real.sqrt_nonneg _
have hdist :
   $(p.1 - mx)^2 \leq (\sqrt{r^2 - (p.2 - cy)^2})^2$  := by
  nlinarith
change p.2 ∈ Icc (cy + r * cos θ) (cy + r) ^
p.1 ∈ Icc
  (mx -  $\sqrt{r^2 - (p.2 - cy)^2}$ )
  (mx +  $\sqrt{r^2 - (p.2 - cy)^2}$ )
constructor
· exact ⟨hbase, hyUpper⟩
· constructor <|> nlinarith
have hsec (y : ℝ) :
  volume (Prod.mk y⁻¹' S) =
  (Icc (cy + r * cos θ) (cy + r)).indicator
  (fun y ↦ ENNReal.ofReal
    (2 *  $\sqrt{r^2 - (y - cy)^2}$ )) y := by
by_cases hy : y ∈ Icc (cy + r * cos θ) (cy + r)
· simp only [Set.indicator_of_mem hy]
  have heq :
    Prod.mk y⁻¹' S =
    Icc (mx -  $\sqrt{r^2 - (y - cy)^2}$ )
    (mx +  $\sqrt{r^2 - (y - cy)^2}$ ) := by
  ext x
  change (y ∈ Icc (cy + r * cos θ) (cy + r) ^
    x ∈ Icc (mx -  $\sqrt{r^2 - (y - cy)^2}$ )
    (mx +  $\sqrt{r^2 - (y - cy)^2}$ )) ↔
  x ∈ Icc (mx -  $\sqrt{r^2 - (y - cy)^2}$ )
  (mx +  $\sqrt{r^2 - (y - cy)^2}$ )
  simp only [hy, true_and]
  rw [heq, Real.volume_Icc]
  congr 1
  ring
· simp only [Set.indicator_of_notMem hy]
  have heq : Prod.mk y⁻¹' S = ∅ := by
  ext x
  change (y ∈ Icc (cy + r * cos θ) (cy + r) ^
    x ∈ Icc (mx -  $\sqrt{r^2 - (y - cy)^2}$ )
    (mx +  $\sqrt{r^2 - (y - cy)^2}$ )) ↔ False
  constructor
  · intro h
    exact (hy h.1).elim
  · intro h
    exact h.elim
  rw [heq, measure_empty]
calc
volume {p : ℝ × ℝ |
  (p.1 - mx)^2 + (p.2 - cy)^2 ≤ r^2 ^
  cy + r * cos θ ≤ p.2} =
  volume (Prod.swap⁻¹' S) := by rw [hpre]
_ = volume S := by
  exact Measure.measurePreserving_swap.measure_preimage hS.nullMeasurableSet
_ = ∫ y in Icc (cy + r * cos θ) (cy + r),
  ENNReal.ofReal (2 *  $\sqrt{r^2 - (y - cy)^2}$ ) := by
  rw [Measure.volume_eq_prod, Measure.prod_apply hS]
  simp_rw [hsec]
  rw [lintegral_indicator measurableSet_Icc]
_ = ENNReal.ofReal (r^2 * (θ - sin θ * cos θ)) :=
  lintegral_semicircle_translated hr hθ0 hθπ

```

AdmissibleCompetitor

Classification: supplement

Source: CMVGeometry.lean:3952-3960

Declaration SHA-256: 52c3c4597213ca5cc7d4e12b21ad852ff148bcb8179b3f52a9c03b7d3665311b

Mathematical statement. The competitor data type has four alternatives: empty, a general integrable region datum, a four-arc assembly, or a replacement assembly.

This finite sum is lower-level infrastructure for representing the four-arc candidate. It makes no minimization assertion.

```
inductive AdmissibleCompetitor (lam : ℝ) where
  | empty
  | general (region : FinitePerimeterRegion lam)
  | fourArc (a : FourArcAssembly)
  | replacement (a : ReplacementAssembly)

namespace AdmissibleCompetitor

/-- Coordinate carrier of an admissible regular competitor. -/
```

AdmissibleCompetitor.WeightedArea

Classification: supplement

Source: CMVGeometry.lean:3968-3975

Declaration SHA-256: d58b51a28690771d1b404e9048905bb6eae43f08f6798b27166b1234f3251f5e

Mathematical statement. The area accessor returns zero for the empty alternative and the corresponding weighted-area integral for every nonempty alternative.

For four-arc candidates this reduces to the integral over the assembled carrier.

```
def WeightedArea {lam : ℝ} : AdmissibleCompetitor lam → ℝ
  | .empty => 0
  | .general region => region.weightedArea
  | .fourArc a => a.weightedArea lam
  | .replacement a => a.weightedArea lam

/-- Weighted perimeter is canonically determined by the complete topological
frontier of the competitor's coordinate carrier. -/
```

AdmissibleCompetitor.WeightedPerimeter

Classification: supplement

Source: CMVGeometry.lean:3976-3980

Declaration SHA-256: 17b5c5914d38bc0479d502c48cde9715c303aa7070585ade87ba56d307321437

Mathematical statement. For a competitor with carrier F , its weighted perimeter is

$$\int_{\partial F} f_{\lambda} d\mathcal{H}^1$$

using the canonical Euclidean frontier measure.

The data type cannot choose an independent perimeter measure; all alternatives share the same geometric definition.

```
def WeightedPerimeter {lam : ℝ} (r : AdmissibleCompetitor lam) : ℝ :=
  _root_.WeightedPerimeter lam (FrontierMeasure r.carrier)

/-- The constructor-level weighted area is always the canonical density
integral over the constructor's coordinate carrier. -/
```

AdmissibleCompetitor.carrier

Classification: supplement

Source: CMVGeometry.lean:3961-3967

Declaration SHA-256: b57d4c4c33776a05ffbbbc22f1b363a7f356aa7cd089d7c2b3335ead1bb65644

Mathematical statement. The carrier accessor returns the empty set, the general region carrier, the four-arc carrier, or the replacement carrier according to the alternative.

Each alternative is an actual subset of the coordinate plane.

```
def carrier {lam : ℝ} : AdmissibleCompetitor lam → Set PlanePoint
| .empty => ∅
| .general region => region.carrier
| .fourArc a => a.carrier
| .replacement a => a.carrier

/-- Weighted area on the regular constructor model. -/
```

CMVElementary.D_abs

Classification: supplement

Source: CMVElementary.lean:474-478

Declaration SHA-256: 5db30368ee603248cbd1e4396a9f85e0f017b44b1517e201076738ac74691b68

Mathematical statement. For every real λ, z ,

$$D_\lambda(|z|) = D_\lambda(z).$$

Split according to the sign of the argument and use evenness in the negative case.

```
theorem D_abs (lam z : ℝ) : D lam |z| = D lam z := by
  rcases le_or_gt 0 z with hz | hz
  · rw [abs_of_nonneg hz]
  · rw [abs_of_neg hz, D_even]
```

CMVElementary.DerivativePilot.lower_third_signed_arguments

Classification: supplement

Source: CMVElementary.lean:240-243

Declaration SHA-256: 25037fe9d96454915fdfae797c1768e12148be94cb81c1f09cd91ca928805616

Mathematical statement. If $0 < u < 1/3$, then

$$-1 < 2u - 1 < 0$$

and $0 < u < 1$.

These linear inequalities put the two arguments in the negative and positive derivative-sign intervals.

```
theorem lower_third_signed_arguments {u : ℝ} (hu : u ∈ Ioo 0 (1 / 3)) :
  2 * u - 1 ∈ Ioo (-1) 0 ∧ u ∈ Ioo 0 1 := by
  constructor <|> constructor <|> linarith [hu.1, hu.2]
```

CMElementary.DerivativePilot.radical_denominators

Classification: supplement

Source: CMElementary.lean:177-186

Declaration SHA-256: 11268e29fea0051e68c749ca17d7b30526b0cc0f472498c62a0360246108b38e

Mathematical statement. If $\lambda > 1$ and $-1 < z < 1$, then $1 - z^2, \lambda^2 - z^2,$

$$\sqrt{1 - z^2},$$

and

$$\lambda\sqrt{\lambda^2 - z^2}$$

are strictly positive.

These inequalities justify the reciprocal and square-root differentiation steps only in the open interval.

```
theorem radical_denominators {lam z : ℝ} (hlam : 1 < lam) (hz : z ∈ Ioo (-1) 1) :
  0 < 1 - z ^ 2 ^ 0 < lam ^ 2 - z ^ 2 ^
  0 < Real.sqrt (1 - z ^ 2) ^
  0 < lam * Real.sqrt (lam ^ 2 - z ^ 2) := by
  have hl : 0 < lam := by linarith
  have hi : 0 < 1 - z ^ 2 := by nlinarith [hz.1, hz.2]
  have ho : 0 < lam ^ 2 - z ^ 2 := by nlinarith
  exact ⟨hi, ho, Real.sqrt_pos.2 hi, mul_pos hl (Real.sqrt_pos.2 ho)⟩

/-- The exact derivative, valid for negative, zero, and positive interior arguments. -/
```

CMElementary.GeometryBridge.abs_le_sqrt_iff

Classification: supplement

Source: CMElementaryGeometry.lean:140-150

Declaration SHA-256: 2c06f5120bf21413fd7a76c0dfd930961207a98f7083bc089c888386262367b5

Mathematical statement. For real x, s with $s \geq 0$,

$$|x| \leq \sqrt{s}$$

if and only if

$$x^2 \leq s.$$

Both sides can be compared after squaring nonnegative quantities; use

$$(\sqrt{s})^2 = s.$$

```
theorem abs_le_sqrt_iff (x s : ℝ) (hs : 0 ≤ s) :
  |x| ≤ Real.sqrt s ↔ x ^ 2 ≤ s := by
  have heq := Real.sq_sqrt hs
  constructor
  · intro hx
    have := (sq_le_sq₀ (abs_nonneg x) (Real.sqrt_nonneg s)).2 hx
    simpa [sq_abs, heq] using this
  · intro hx
    apply (sq_le_sq₀ (abs_nonneg x) (Real.sqrt_nonneg s)).1
    simpa [sq_abs, heq] using hx
```

CMElementary.GeometryBridge.cap_bounded

Classification: supplement

Source: CMElementaryGeometry.lean:407-409

Declaration SHA-256: 5f789f5ecfe21a6d9d280eafb5b4f8cb77643861d589df8496f30b5949a88aea

Mathematical statement. Every admissible one-sided circular cap carrier is bounded.

Its coordinate carrier is a subset of the disk with its specified center and positive radius.

```
theorem cap_bounded (c : OneSidedCircularCap) : Bornology.IsBounded c.carrier :=  
  (disk_bounded c.center.1 c.center.2 c.radius).subset (fun _ hp => hp.1)
```

CMElementary.GeometryBridge.core_bounded

Classification: supplement

Source: CMElementaryGeometry.lean:644-660

Declaration SHA-256: f823fe31b5fbe1402665006d839f3f12aa85b1c8db11bbfdf640c49c7cbfe890

Mathematical statement. Every strip-core carrier is bounded.

Its rectangle is bounded in both coordinates, and each side cap is contained in a bounded disk; a finite union remains bounded.

```
theorem core_bounded (c : StripCore) : Bornology.IsBounded c.carrier := by  
  have hr : Bornology.IsBounded c.rectangleCarrier := by  
    apply isBounded_iff_forall_norm_le.mpr  
    refine (|c.chord|/2+1, ?_)  
    intro p hp  
    have hx : |p.1| ≤ c.chord/2 := abs_le.mpr (by linarith [hp.1], hp.2.1)  
    have hy : |p.2| ≤ 1 := hp.2.2  
    rw [Prod.norm_def, Real.norm_eq_abs, Real.norm_eq_abs, max_le_iff]  
    constructor <=> linarith [le_abs_self c.chord, abs_nonneg c.chord]  
  have hl : Bornology.IsBounded c.leftCapCarrier :=  
    (disk_bounded c.leftCenterX 0 c.radius).subset (fun p hp => by  
      simpa only [Set.mem_ofPred_eq, sub_zero] using hp.1)  
  have hh : Bornology.IsBounded c.rightCapCarrier :=  
    (disk_bounded c.rightCenterX 0 c.radius).subset (fun p hp => by  
      simpa only [Set.mem_ofPred_eq, sub_zero] using hp.1)  
  exact (hr.union hl).union hh
```

CMElementary.GeometryBridge.disk_bounded

Classification: supplement

Source: CMElementaryGeometry.lean:390-406

Declaration SHA-256: e6037bd412d0410bbc35975c588a201fc2cbeccbcaafb12028d4c97e43ab9925

Mathematical statement. For any real center (c_x, c_y) and real R , the set

$$\{(x, y) : (x - c_x)^2 + (y - c_y)^2 \leq R^2\}$$

is bounded.

Each coordinate displacement is bounded by $|R|$, so both coordinates have a common finite bound. No positivity of the radius symbol is needed.

```
theorem disk_bounded (cx cy R : ℝ) :  
  Bornology.IsBounded {p : PlanePoint | (p.1-cx)^2 + (p.2-cy)^2 ≤ R^2} := by  
  apply isBounded_iff_forall_norm_le.mpr  
  refine (|cx| + |cy| + |R|, ?_)  
  intro p hp
```

```

change (p.1-cx)^2 + (p.2-cy)^2 ≤ R^2 at hp
have hx : |p.1-cx| ≤ |R| := sq_le_sq.mp (by nlinarith [sq_nonneg (p.2-cy)])
have hy : |p.2-cy| ≤ |R| := sq_le_sq.mp (by nlinarith [sq_nonneg (p.1-cx)])
rw [Prod.norm_def, Real.norm_eq_abs, Real.norm_eq_abs, max_le_iff]
constructor
· have := abs_add_le (p.1-cx) cx
  rw [sub_add_cancel] at this
  linarith [abs_nonneg cy]
· have := abs_add_le (p.2-cy) cy
  rw [sub_add_cancel] at this
  linarith [abs_nonneg cx]

```

CMElementary.GeometryBridge.horizontal_section

Classification: supplement

Source: CMElementaryGeometry.lean:151-175

Declaration SHA-256: e06f6eef4be5d4e887790cde04018ec9bd793a5f96e506896654d18400a3ceff

Mathematical statement. For $b, s \geq 0$ and real x ,

$$|x| \leq b + \sqrt{s}$$

is equivalent to the disjunction

$$-b \leq x \leq b,$$

or

$$(x + b)^2 \leq s$$

and

$$x \leq -b,$$

or

$$(x - b)^2 \leq s$$

and

$$x \geq b.$$

Split a horizontal interval at the rectangle endpoints. The outer portions are exactly the left and right disk-section inequalities.

```

theorem horizontal_section (x b s : ℝ) (hb : 0 ≤ b) (hs : 0 ≤ s) :
  |x| ≤ b + Real.sqrt s ↔
    (-b ≤ x ∧ x ≤ b) ∨
    ((x + b) ^ 2 ≤ s ∧ x ≤ -b) ∨
    ((x - b) ^ 2 ≤ s ∧ b ≤ x) := by
  have hn := Real.sqrt_nonneg s
  constructor
  · intro hx
    obtain ⟨hlo, hhi⟩ := abs_le.mp hx
    by_cases hl : x < -b
    · right; left
      refine ⟨(abs_le_sqrt_iff (x + b) s hs).1 ?_, hl.le⟩
      exact abs_le.mpr ⟨by linarith, by linarith⟩
    by_cases hr : b < x
    · right; right
      refine ⟨(abs_le_sqrt_iff (x - b) s hs).1 ?_, hr.le⟩
      exact abs_le.mpr ⟨by linarith, by linarith⟩
    · exact Or.inl ⟨le_of_not_gt hl, le_of_not_gt hr⟩
  · rintro (⟨hlo, hhi⟩ | ⟨hd, hl⟩ | ⟨hd, hr⟩)
    · exact abs_le.mpr ⟨by linarith, by linarith⟩
    · obtain ⟨hlo, hhi⟩ := abs_le.mp ((abs_le_sqrt_iff (x + b) s hs).2 hd)
      exact abs_le.mpr ⟨by linarith, by linarith⟩

```

```
· obtain ⟨hlo, hhi⟩ := abs_le.mp ((abs_le_sqrt_iff (x - b) s hs).2 hd)
  exact abs_le.mpr ⟨by linarith, by linarith⟩
```

CMElementary.GeometryBridge.offset_section

Classification: supplement

Source: CMElementaryGeometry.lean:515-538

Declaration SHA-256: 0f3ecd491b72066b337a40630ff772b2174b32379dec94670d56b28857d29ea3

Mathematical statement. Let real x, b, c, s satisfy $s \geq 0$,

$$b \leq c,$$

$c \geq 0$, and

$$c \leq b + \sqrt{s}.$$

Then

$$|x| \leq b + \sqrt{s}$$

is equivalent to

$$-c \leq x \leq c,$$

or

$$(x + b)^2 \leq s$$

with

$$x \leq -c,$$

or

$$(x - b)^2 \leq s$$

with

$$x \geq c.$$

The cut points may differ from the disk centers. Splitting the interval there proves the rectangle-and-caps decomposition used for the four-arc core.

```
theorem offset_section (x b c s : ℝ) (hs : 0 ≤ s) (hbc : b ≤ c)
  (hc : 0 ≤ c) (hcs : c ≤ b + Real.sqrt s) :
  |x| ≤ b + Real.sqrt s ↔
  (-c ≤ x ∧ x ≤ c) ∨
  ((x+b)^2 ≤ s ∧ x ≤ -c) ∨ ((x-b)^2 ≤ s ∧ c ≤ x) := by
  constructor
  · intro hx
    obtain ⟨hlo, hhi⟩ := abs_le.mp hx
    by_cases hl : x < -c
    · right; left
      refine ⟨(abs_le_sqrt_iff (x+b) s hs).mp ?_, hl.le⟩
      exact abs_le.mpr ⟨by linarith, by linarith [Real.sqrt_nonneg s]⟩
    by_cases hr : c < x
    · right; right
      refine ⟨(abs_le_sqrt_iff (x-b) s hs).mp ?_, hr.le⟩
      exact abs_le.mpr ⟨by linarith [Real.sqrt_nonneg s], by linarith⟩
    · exact Or.inl ⟨le_of_not_gt hl, le_of_not_gt hr⟩
  · rintro (⟨hlo,hhi⟩ | ⟨hd,hl⟩ | ⟨hd,hr⟩)
    · exact abs_le.mpr ⟨by linarith, by linarith⟩
    · obtain ⟨hlo,hhi⟩ := abs_le.mp ((abs_le_sqrt_iff (x+b) s hs).mpr hd)
      exact abs_le.mpr ⟨by linarith, by linarith⟩
```

```
· obtain ⟨hlo,hhi⟩ := abs_le.mp ((abs_le_sqrt_iff (x-b) s hs).mpr hd)
  exact abs_le.mpr ⟨by linarith, by linarith⟩
```

CMElementary.GeometryBridge.strict_chord_not_arc

Classification: supplement

Source: CMElementaryGeometry.lean:428-441

Declaration SHA-256: b0312da63f9829854c31e4257ff6ffb230e77daca0b49854c001774049378eed

Mathematical statement. For a one-sided circular cap of chord length $L > 0$ with chord midpoint (m, b) , if

$$p_y = b$$

and

$$|p_x - m| < L/2,$$

then p is not on its selected circular arc trace.

The circle equation on the chord line forces

$$|p_x - m| = L/2,$$

using

$$R \sin \theta = L/2,$$

contradicting strictness.

```
theorem strict_chord_not_arc (c : OneSidedCircularCap) {p : PlanePoint}
  (hy : p.2 = c.baseY) (hx : |p.1-c.midpointX| < c.chord/2) : p ∉ c.arcTrace := by
  intro harc
  have hc : (p.1-c.midpointX)^2 + (c.radius * Real.cos c.theta)^2 = c.radius^2 := by
    cases hs : c.side <|>
    simp [OneSidedCircularCap.arcTrace, OneSidedCircularCap.radiusSquaredAt,
      OneSidedCircularCap.center, hs, hy] using harc.1
  have ht := congrArg (fun z : ℝ => c.radius^2*z) (Real.sin_sq_add_cos_sq c.theta)
  have hr := congrArg (fun z : ℝ => z^2) c.radius_mul_sin
  have heq : |p.1-c.midpointX| = c.chord/2 := by
    rw [sq_eq_sq₀ (abs_nonneg _) (by linarith [c.chord_pos]), sq_abs]
    nlinarith
  exact (ne_of_lt hx) heq
```

CMElementary.GeometryBridge.three_bounded

Classification: supplement

Source: CMElementaryGeometry.lean:410-421

Declaration SHA-256: 8dd2208ef6c92ad151b2a77cf2af0f518496c61601318bbba67d9e75d7051721

Mathematical statement. Every admissible three-arc assembly carrier is bounded.

The carrier is a finite union of a bounded rectangle, two subsets of disks, and a bounded upper cap.

```
theorem three_bounded {lam : ℝ} (a : TypeThreeAssembly lam) : Bornology.IsBounded a.carrier := by
  have hr : Bornology.IsBounded a.rectangleCarrier := by
    apply isBounded_iff_forall_norm_le.mpr
    refine (|a.sideHalfWidth| + 1, ?_)
  intro p hp
  have hx : |p.1| ≤ a.sideHalfWidth := abs_le.mpr hp.1
  have hy : |p.2| ≤ 1 := abs_le.mpr hp.2
  rw [Prod.norm_def, Real.norm_eq_abs, Real.norm_eq_abs, max_le_iff]
  constructor <|> linarith [le_abs_self a.sideHalfWidth, abs_nonneg a.sideHalfWidth]
```

```
exact ((hr.union ((disk_bounded _ _).subset (fun _ hp => hp.1))).union
  ((disk_bounded _ _).subset (fun _ hp => hp.1))).union (cap_bounded a.outerCap)
```

CMElementary.GeometryBridge.three_top_circle

Classification: supplement

Source: CMElementaryGeometry.lean:442-449

Declaration SHA-256: 2c42afd2f1041c6eb7aaaecf9fbcab707e24f86eb7b6bf6aaa7c27e8dcdadc9c

Mathematical statement. For every admissible three-arc assembly with radius R and inner angle β ,

$$(R\sin\beta)^2 + (2 - R)^2 = R^2.$$

The incidence

$$R\cos\beta = 2 - R$$

and the Pythagorean trigonometric identity give the top-chord circle equation.

```
theorem three_top_circle {lam : ℝ} (a : TypeThreeAssembly lam) :
  (a.radius * Real.sin a.innerAngle)^2 + (2-a.radius)^2 = a.radius^2 := by
  have hc : a.radius * Real.cos a.innerAngle = 2-a.radius := by
    rw [a.cos_innerAngle, TypeThreeAssembly.shapeParameter]
    nlinarith [a.radius_mul_h]
  have ht := congrArg (fun z : ℝ => a.radius^2*z) (Real.sin_sq_add_cos_sq a.innerAngle)
  nlinarith
```

CMElementary.three_arc_eventuallyEq

Classification: supplement

Source: CMElementary.lean:645-656

Declaration SHA-256: 46dc3f0498de249294a8597ff6c3dc5893312352af2c59075b4bc7135e14271b

Mathematical statement. For $\lambda > 1$ and $0 < t < 1$, A_3 and P_3 agree in a neighborhood of t with the frozen three-arc scalar area and perimeter.

Restrict the neighborhood to $(0, 1)$, where both pointwise identifications hold. Neighborhood equality, not equality at one point, permits derivative transport.

```
theorem three_arc_eventuallyEq {lam t : ℝ} (hlam : 1 < lam) (ht : t ∈ Ioo 0 1) :
  A3 lam =f[ℳ t] LeanSuffixAnalytic.typeThreeArea lam ^
  P3 lam =f[ℳ t] LeanSuffixAnalytic.typeThreePerimeter lam := by
  have hid := scalar_identification_claims hlam
  have hn : ∀ s in ℳ t, s ∈ Ioo (0 : ℝ) 1 := Ioo_mem_nhds ht.1 ht.2
  constructor
  · filter_upwards [hn] with s hs
    exact (hid.area_three s ⟨hs.1, hs.2.le⟩).symm
  · filter_upwards [hn] with s hs
    exact (hid.perimeter_three s ⟨hs.1, hs.2.le⟩).symm

/-- Transport of the frozen variational theorem stays strictly inside `(0,1)`. -/
```

CapSide

Classification: supplement

Source: CMVGeometry.lean:92-99

Declaration SHA-256: 6863ffc025769eb8169e9582a3d673fe180025902f766c2557257f6937cc7a7e

Mathematical statement. The cap orientation has two alternatives: upper or lower relative to a horizontal chord.

The orientation controls the disk center and the half-plane inequality in the cap carrier.

```

inductive CapSide where
| upper
| lower
deriving DecidableEq

/-- A branch-correct circular segment on a horizontal chord. Its carrier is
`disk ∩ half-plane`; this single definition includes minor, semicircular, and
major caps. -/

```

EuclideanPlane

Classification: supplement

Source: CMVGeometry.lean:34-37

Declaration SHA-256: dfa2db26da624a2b5333b886cff8c9cd3151218ea8aa3e3cb39c78359b32470e

Mathematical statement. The Euclidean realization is

$$\mathbb{R} \times \mathbb{R}$$

equipped with the ℓ^2 norm.

This metric, rather than the maximum product metric, determines the Hausdorff measure used for perimeter.

```

abbrev EuclideanPlane := WithLp 2 (ℝ × ℝ)

/-- The coordinate-preserving homeomorphism from the existing coordinate plane
to its Euclidean L2 realization. -/

```

FinitePerimeterRegion

Classification: supplement

Source: CMVGeometry.lean:74-82

Declaration SHA-256: 2efb7176d17b946809f4cd07b78237ddd03cd4197c405c38d08e97342106a1a0

Mathematical statement. A region datum consists of a measurable carrier F , integrability of f_λ over F , and integrability of f_λ against its canonical frontier measure.

This is a complete-frontier input record, not the general bounded-variation definition of a set of finite perimeter.

```

structure FinitePerimeterRegion (lam : ℝ) where
  carrier : Set PlanePoint
  measurable_carrier : MeasurableSet carrier
  finite_weighted_area : IntegrableOn (StripDensity lam) carrier
  finite_weighted_perimeter :
    Integrable (StripDensity lam) (FrontierMeasure carrier)

namespace FinitePerimeterRegion

```

FinitePerimeterRegion.weightedArea

Classification: supplement

Source: CMVGeometry.lean:83-85

Declaration SHA-256: 3892310b68bd795c4ddff8f1d276f58be0efcb96c1e8c4fad96ec7d73deea20a

Mathematical statement. For a region datum with carrier F , its area is $A_\lambda(F)$.

The datum supplies integrability; this accessor uses the actual Lebesgue integral and does not store a freely assignable area.

```

def weightedArea {lam : ℝ} (r : FinitePerimeterRegion lam) : ℝ :=
  WeightedArea lam r.carrier

```

FourArcAssembly

Classification: supplement

Source: CMVGeometry.lean:2491-2498

Declaration SHA-256: 1ac9185bddaf1e6a0074fb39d1e29417262cc431ccce861d3c65ec2e5216fdb1

Mathematical statement. A four-arc assembly consists of a strip core and an exterior half-angle

$$0 < \alpha < \pi/2.$$

Its upper and lower caps share the core chord. A common radius is a further consequence of the incidence condition, not a premise of this general constructor.

```
structure FourArcAssembly where
  core : StripCore
  outerAngle : ℝ
  outerAngle_pos : 0 < outerAngle
  outerAngle_lt_pi_div_two : outerAngle < π / 2

namespace FourArcAssembly
```

FourArcAssembly.boundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:2525-2529

Declaration SHA-256: a07e8d5b98aab97a737c50d9ee9eae871142e1adb2b58a3132be8c4b6a093cce

Mathematical statement. The auxiliary boundary measure is the sum of the left and right core arc measures and the two exterior arc measures.

It has no chord terms. Its weighted integral is later equated with the canonical complete-frontier perimeter.

```
def boundaryMeasure (a : FourArcAssembly) : Measure PlanePoint :=
  leftCoreBoundaryMeasure a.core + rightCoreBoundaryMeasure a.core +
  capBoundaryMeasure a.upperCap + capBoundaryMeasure a.lowerCap

/-- The explicit four-arc boundary measure gives every endpoint zero mass. -/
```

FourArcAssembly.boundaryTrace

Classification: supplement

Source: CMVGeometry.lean:2625-2630

Declaration SHA-256: d8944f364e38eaa5c7b77ce182d417bec91ac425efc229739d886be00ad658c6

Mathematical statement. The declared boundary trace is the union of the two exposed core arc traces and the two exterior cap arc traces.

This coordinate set has no chord interiors and is subsequently identified with the entire frontier.

```
def boundaryTrace (a : FourArcAssembly) : Set PlanePoint :=
  StripCore.leftArcTrace a.core ∪
  (StripCore.rightArcTrace a.core ∪
   (OneSidedCircularCap.arcTrace a.upperCap ∪
    OneSidedCircularCap.arcTrace a.lowerCap))
```

FourArcAssembly.boundaryTrace_subset_frontier_carrier

Classification: supplement

Source: CMVGeometry.lean:3244-3251

Declaration SHA-256: 160385ee99b171abc63b7cc7cb5ade687af8d5249c1ab26a5150be49b8c4654b

Mathematical statement. Every point of the declared four-arc trace is a frontier point of the assembly carrier.

The component trace arguments exhibit proximity to both the carrier and its exterior, including the junction points.

```
theorem boundaryTrace_subset_frontier_carrier (a : FourArcAssembly) :  
  a.boundaryTrace  $\subseteq$  frontier a.carrier := by  
  rintro p (hleft | hright | hupper | hlower)  
  · exact leftArc_mem_frontier a hleft  
  · exact rightArc_mem_frontier a hright  
  · exact upperArc_mem_frontier a hupper  
  · exact lowerArc_mem_frontier a hlower
```

FourArcAssembly.carrier

Classification: supplement

Source: CMVGeometry.lean:2519-2524

Declaration SHA-256: a55616796db22264c4fe11d48e20ab173ecb661c3bdced26752617ea90d1f816

Mathematical statement. The assembled carrier is the union of the core and both exterior cap carriers.

This is a genuine closed geometric union; attaching chord interiors are later eliminated from its frontier.

```
def carrier (a : FourArcAssembly) : Set PlanePoint :=  
  a.core.carrier  $\cup$  a.upperCap.carrier  $\cup$  a.lowerCap.carrier  
  
/-- Explicit arc-length pushforward measure of the complete regular boundary.  
The attachment chords are omitted because they are internal; endpoint atoms  
are null for every summand. -/
```

FourArcAssembly.frontier_carrier_subset_boundaryTrace

Classification: supplement

Source: CMVGeometry.lean:2795-2819

Declaration SHA-256: 0d72e7b21e3528564863313aa2366a2e4806e01c00a4772872911d424abf3cc5

Mathematical statement. Every frontier point of a four-arc assembly belongs to the declared four-arc trace.

The component frontier analysis proves internal chord points are swallowed by the attached region rather than remaining exposed.

```
theorem frontier_carrier_subset_boundaryTrace (a : FourArcAssembly) :  
  frontier a.carrier  $\subseteq$  a.boundaryTrace := by  
  intro p hp  
  have htop := frontier_union_subset  
    (a.core.carrier  $\cup$  a.upperCap.carrier) a.lowerCap.carrier hp  
  rcases htop with hfirst | hlower  
  · have hmiddle := frontier_union_subset a.core.carrier a.upperCap.carrier hfirst.1  
    rcases hmiddle with hcore | hupper  
    · rw [a.core.frontier_carrier] at hcore  
      rcases hcore.1 with hleft | hright | hupperChord | hlowerChord  
      · exact Or.inl hleft  
      · exact Or.inr (Or.inl hright)  
      · exact chord_to_boundaryTrace a (Or.inl hupperChord.1) hupperChord.2 hp  
      · exact chord_to_boundaryTrace a (Or.inr hlowerChord.1) hlowerChord.2 hp  
    · rw [a.upperCap.frontier_carrier] at hupper  
      rcases hupper.2 with harc | hchord
```

```

    · exact Or.inr (Or.inr (Or.inl harc))
    · exact chord_to_boundaryTrace a (Or.inl hchord.1)
      (by simp [upperCap] using hchord.2) hp
  · rw [a.lowerCap.frontier_carrier] at hlower
    rcases hlower.2 with harc | hchord
    · exact Or.inr (Or.inr (Or.inr harc))
    · exact chord_to_boundaryTrace a (Or.inr hchord.1)
      (by simp [lowerCap] using hchord.2) hp

```

FourArcAssembly.isClosed_carrier

Classification: supplement

Source: CMVGeometry.lean:2820-2823

Declaration SHA-256: 7c8404a543b7f83e631b837fdeb1bbef479b4e4161ccfe69e2da8d64a80c0a

Mathematical statement. The four-arc assembly carrier is closed.

Its three component regions are closed, and there are only finitely many components.

```

theorem isClosed_carrier (a : FourArcAssembly) : IsClosed a.carrier :=
  (a.core.isClosed_carrier.union a.upperCap.isClosed_carrier).union
  a.lowerCap.isClosed_carrier

```

FourArcAssembly.lowerCap

Classification: supplement

Source: CMVGeometry.lean:2509-2518

Declaration SHA-256: a60a9393b363872f08f12dbab41380b51afefa746b9504c5841f28d389091bfe

Mathematical statement. The lower cap has the same chord and half-angle, midpoint $(0, -1)$, and lower orientation.

It is the horizontal-axis reflection of the upper cap and lies below the lower interface.

```

def lowerCap (a : FourArcAssembly) : OneSidedCircularCap where
  chord := a.core.chord
  theta := a.outerAngle
  midpointX := 0
  baseY := -1
  side := .lower
  chord_pos := a.core.chord_pos
  theta_pos := a.outerAngle_pos
  theta_lt_pi := lt_trans a.outerAngle_lt_pi_div_two (by linarith [Real.pi_pos])

```

FourArcAssembly.measurableSet_carrier

Classification: supplement

Source: CMVGeometry.lean:2549-2553

Declaration SHA-256: bed9e29097b5f894c003225893e3653e7d99d34980093208388840d2a9198ac1

Mathematical statement. Every four-arc assembly carrier is measurable.

It is a union of a measurable strip core and two measurable caps.

```

theorem measurableSet_carrier (a : FourArcAssembly) : MeasurableSet a.carrier :=
  (a.core.measurableSet_carrier.union a.upperCap.measurableSet_carrier).union
  a.lowerCap.measurableSet_carrier

/-- The ambient area integral equals the geometric component formula. -/

```

FourArcAssembly.outer_radius_eq_core_radius

Classification: supplement

Source: CMVGeometry.lean:3260-3280

Declaration SHA-256: e5fdf5b4f19a967ab6a1b262eade841eb117d850d615f26e5d93bdfd93af65de

Mathematical statement. If an assembly satisfies

$$Lk = 2\sin\alpha,$$

its upper-cap radius equals the core radius.

Use

$$R_{\text{cap}} = L/(2\sin\alpha)$$

and

$$R_{\text{core}} = 1/k,$$

with positive sine and curvature.

```
theorem outer_radius_eq_core_radius (a : FourArcAssembly)
  (hincidence : a.core.chord * a.core.curvature =
    2 * sin a.outerAngle) :
  a.upperCap.radius = a.core.radius := by
  simp only [OneSidedCircularCap.radius, upperCap, StripCore.radius]
  have hchord : a.core.chord =
    2 * sin a.outerAngle / a.core.curvature :=
    (eq_div_iff (ne_of_gt a.core.curvature_pos)).2 hincidence
  calc
    a.core.chord / (2 * sin a.outerAngle) =
      (2 * sin a.outerAngle / a.core.curvature) /
        (2 * sin a.outerAngle) := by rw [hchord]
  _ = 1 / a.core.curvature := by
    have hs := sin_pos_of_pos_of_lt_pi a.outerAngle_pos
      (lt_trans a.outerAngle_lt_pi_div_two (by linarith [Real.pi_pos]))
    field_simp [ne_of_gt hs, ne_of_gt a.core.curvature_pos]

end FourArcAssembly

/-- The exact cap-substitution assembly. It retains the central strip region,
attaches one upper cap, and exposes the lower interface chord. -/
```

FourArcAssembly.upperCap

Classification: supplement

Source: CMVGeometry.lean:2499-2508

Declaration SHA-256: 3f084f0dbec6bee8992e556fe2ca304aae5eb8981815636bf15454a3b8e613f9

Mathematical statement. The upper cap has chord L equal to the core chord, half-angle α , midpoint $(0, 1)$, and upper orientation.

The assembly angle bound is strictly below

$$\pi/2,$$

hence admissible for the general cap constructor.

```
def upperCap (a : FourArcAssembly) : OneSidedCircularCap where
  chord := a.core.chord
  theta := a.outerAngle
  midpointX := 0
  baseY := 1
  side := .upper
```

```
chord_pos := a.core.chord_pos
theta_pos := a.outerAngle_pos
theta_lt_pi := lt_trans a.outerAngle_lt_pi_div_two (by linarith [Real.pi_pos])
```

FourArcAssembly.weightedArea

Classification: supplement

Source: CMVGeometry.lean:2541-2545

Declaration SHA-256: ad43970e4ac79b935baed41701092f4d901053e35bb88583dcbd5a9003db13a3

Mathematical statement. For any real λ , the assembly weighted area is the actual integral A_λ over its carrier.

This definition contains no scalar area prescription.

```
def weightedArea (lam : ℝ) (a : FourArcAssembly) : ℝ :=
  WeightedArea lam a.carrier

/-- Weighted perimeter is the density integral against the explicit regular
boundary measure. -/
```

FourArcAssembly.weightedPerimeter

Classification: supplement

Source: CMVGeometry.lean:2546-2548

Declaration SHA-256: df8ef218db7076c4681d4506265ae824cfa4b1a2ec872c320aeb4f283b25493b

Mathematical statement. The auxiliary assembly weighted perimeter is the density integral against its four-arc parametrized boundary measure.

The frontier module proves this equals the canonical perimeter; the auxiliary definition alone is not the final geometric bridge.

```
def weightedPerimeter (lam : ℝ) (a : FourArcAssembly) : ℝ :=
  WeightedPerimeter lam a.boundaryMeasure
```

FourArcCandidate

Classification: supplement

Source: FourArcCandidate.lean:23-35

Declaration SHA-256: 24dc0ff22e0918bdb2c7f16fab0da52bc08326d88e5cb8e1d2e8a8f7ec2bee62

Mathematical statement. A four-arc candidate at density parameter λ consists of curvature $0 < h \leq 1$ and exterior half-angle

$$0 < \alpha < \pi/2.$$

The constructor does not itself require

$$\lambda > 1$$

or the Snell incidence. Those two conditions are stated separately and proved for the explicit candidate.

```
structure FourArcCandidate (lam : ℝ) where
  h : ℝ
  alpha : ℝ
  h_pos : 0 < h
  h_le_one : h ≤ 1
  alpha_pos : 0 < alpha
  alpha_lt_pi_div_two : alpha < π / 2

namespace FourArcCandidate
```

```
variable {lam : ℝ} (candidate : FourArcCandidate lam)

/-- Radius shared by the four source arcs. -/
```

FourArcCandidate.SatisfiesCMVTypeIVHypotheses

Classification: supplement

Source: FourArcCandidate.lean:84-89

Declaration SHA-256: c09b9c7180b6131040333c0b3c2623f3c89af7f54fe8ab0b9f6a41be873ca0dc

Mathematical statement. For a candidate, this assertion consists of

$$\lambda > 1$$

and

$$\lambda \cos \alpha = h.$$

These are the density contrast and incidence conditions needed for the physical four-arc family; the geometric bridge proves them for

$$\alpha = \arccos(h/\lambda).$$

```
structure SatisfiesCMVTypeIVHypotheses : Prop where
  density_jump : 1 < lam
  snell_incidence : lam * cos candidate.alpha = candidate.h

/-- General minimization semantics: comparison is against every admissible
regular competitor, not merely another four-arc candidate. -/
```

FourArcCandidate.WeightedArea

Classification: supplement

Source: FourArcCandidate.lean:76-78

Declaration SHA-256: 8256dd516e32d853aaed6a85d09fdb0b62411bd0bf67f1df7ffd123e47baed99

Mathematical statement. The candidate weighted area is the common competitor weighted-area accessor applied to its region.

It reduces to the actual Lebesgue density integral over its assembled carrier.

```
def WeightedArea : ℝ := candidate.region.WeightedArea

/-- Candidate weighted perimeter, computed from its four boundary arcs. -/
```

FourArcCandidate.WeightedPerimeter

Classification: supplement

Source: FourArcCandidate.lean:79-83

Declaration SHA-256: d649676f2fd717389a6e9a17a385fe854bc399e17ecfdad24bd8a7bcc9a36521

Mathematical statement. The candidate weighted perimeter is the common competitor perimeter accessor applied to its region.

It is the density integral on the canonical complete Euclidean frontier, not a freely stored scalar.

```
def WeightedPerimeter : ℝ := candidate.region.WeightedPerimeter

/-- Source-specific hypotheses not already enforced by the geometric
```

```

constructor. The equation is CMV (27), equivalently
`alpha = arccos (h / lam)` on the recorded angle domain. -/

```

FourArcCandidate.assembly

Classification: supplement

Source: FourArcCandidate.lean:66-72

Declaration SHA-256: ca67fddb2d167afab63544438387902bb645349ebfde9d83c7a2719a2277c941

Mathematical statement. Construct the four-arc assembly from the candidate strip core and exterior half-angle α .

Both exterior caps have the candidate chord and half-angle, giving congruent pieces above and below the strip.

```

def assembly : FourArcAssembly where
  core := candidate.stripCore
  outerAngle := candidate.alpha
  outerAngle_pos := candidate.alpha_pos
  outerAngle_lt_pi_div_two := candidate.alpha_lt_pi_div_two

/-- The candidate as an admissible regular coordinate region. -/

```

FourArcCandidate.beta

Classification: supplement

Source: FourArcCandidate.lean:39-41

Declaration SHA-256: f3520601e601026585d5024a59c3bee3be514d265897b65e66919a9d3ea7fd1e

Mathematical statement. Define the complementary angle

$$\beta = \pi/2 - \arcsin h.$$

Then

$$\cos \beta = h$$

on the candidate curvature interval; it is used in the candidate area formula.

```

def beta : ℝ := π / 2 - arcsin candidate.h

/-- Common horizontal chord length of the two exterior caps. -/

```

FourArcCandidate.capArea

Classification: supplement

Source: FourArcCandidate.lean:45-49

Declaration SHA-256: eb6c25ab8d5e1e4d2ce589b98fc3af2d04e58bd682ad273d1b07834426ee23d0

Mathematical statement. Define the area of one exterior cap as

$$(\alpha - \sin \alpha \cos \alpha) / h^2.$$

This is the ordinary circular-segment formula at radius

$$1/h,$$

later matched to the cap constructor's normalized area.

```

def capArea : ℝ :=
  (candidate.alpha - sin candidate.alpha * cos candidate.alpha) /
  candidate.h ^ 2

```

```
/-- Scale-free cap area passed to Morgan's arc function. -/
```

FourArcCandidate.capArea_eq_chord_sq_mul_area

Classification: supplement

Source: FourArcCandidate.lean:104-109

Declaration SHA-256: 0e9545ae1662d69d55500dc9264e42c4c5b03b78571f6473a85822aca53c6073

Mathematical statement. For every candidate, its cap area equals

$$L^2 a(\alpha).$$

Substitute the candidate chord and cancel the nonzero sine in the normalized segment area.

```
theorem capArea_eq_chord_sq_mul_area :
  candidate.capArea = candidate.capChord ^ 2 * area candidate.alpha := by
  rw [capArea, capChord, area]
  field_simp [ne_of_gt candidate.h_pos, ne_of_gt candidate.sin_alpha_pos]
  ring
```

FourArcCandidate.capChord

Classification: supplement

Source: FourArcCandidate.lean:42-44

Declaration SHA-256: c09aa141e8bbabbe7279bce2473dd47aa931f875442fe1fc79d759685536fb1e

Mathematical statement. Define the exterior chord length

$$L = 2\sin\alpha/h.$$

It is strictly positive and gives radius

$$1/h$$

for a cap of half-angle α .

```
def capChord : ℝ := 2 * sin candidate.alpha / candidate.h

/-- Euclidean area of either original exterior cap. -/
```

FourArcCandidate.lowerCap_euclideanArea

Classification: supplement

Source: FourArcCandidate.lean:159-166

Declaration SHA-256: 14a860cf99daaf759c3fa7838aed86edfb2879de1a5a3f6f7d84f9996c7da155

Mathematical statement. The constructed lower cap's scalar Euclidean area equals the same candidate cap area.

Reflection changes orientation and center but not the chord-angle area formula.

```
theorem lowerCap_euclideanArea :
  candidate.assembly.lowerCap.euclideanArea = candidate.capArea := by
  rw [OneSidedCircularCap.euclideanArea,
    candidate.capArea_eq_chord_sq_mul_area]
  rfl

/-- The endpoint-incidence definition forces the exterior and strip radii to
agree. -/
```

FourArcCandidate.region

Classification: supplement

Source: FourArcCandidate.lean:73-75

Declaration SHA-256: 8ab9f84c197a1a6dbe90236469e9ff5cc2696f04e42133db44cdf34546558b75

Mathematical statement. Represent the candidate as the four-arc alternative of the common competitor data type.

This is a carrier representation only; it does not assert minimality or invoke a final exclusion theorem.

```
def region : AdmissibleCompetitor lam := .fourArc candidate.assembly

/-- Candidate weighted area, computed from its region constructor. -/
```

FourArcCandidate.side_caps_area_formula

Classification: supplement

Source: FourArcCandidate.lean:351-363

Declaration SHA-256: a639bd8c9eef40f2c4f87fdb62b671cf7db0ce41013e7d923721f6f472bc8cf1

Mathematical statement. For every candidate, the sum of the two side-cap areas is

$$8a(\operatorname{arcsinh}) = 2[\operatorname{arcsinh} - h\sqrt{1 - h^2}] / h^2.$$

Substitute the normalized area function and the principal sine and cosine identities, valid also at $h = 1$.

```
theorem side_caps_area_formula :
  8 * area candidate.stripCore.sideAngle =
    2 * (arcsin candidate.h -
      candidate.h * √(1 - candidate.h ^ 2)) / candidate.h ^ 2 := by
change 8 * area (arcsin candidate.h) =
  2 * (arcsin candidate.h -
    candidate.h * √(1 - candidate.h ^ 2)) / candidate.h ^ 2
rw [area, Real.sin_arcsin (by linarith [candidate.h_pos])
  candidate.h_le_one, Real.cos_arcsin]
field_simp [ne_of_gt candidate.h_pos]
ring

/-- Candidate area as the sum of the actual core and cap constructors. -/
```

FourArcCandidate.sin_alpha_pos

Classification: supplement

Source: FourArcCandidate.lean:95-98

Declaration SHA-256: b92e14d3743fcaa1d6c447d96c6334ba3f5f129152aa26ff47b97d6fd8de2e28

Mathematical statement. For every candidate,

$$\sin \alpha > 0.$$

The exterior half-angle lies strictly between zero and

$$\pi/2.$$

```
theorem sin_alpha_pos : 0 < sin candidate.alpha :=
  sin_pos_of_pos_of_lt_pi candidate.alpha_pos
  (lt_trans candidate.alpha_lt_pi_div_two (by linarith [Real.pi_pos]))
```

FourArcCandidate.stripCore

Classification: supplement

Source: FourArcCandidate.lean:53-65

Declaration SHA-256: 66aff9e1eb934bbbeeb8dd09a1a68a4c23aef3ee09735d3ba9bc12eadb4664ee

Mathematical statement. Construct the strip core with chord

$$L = 2\sin\alpha/h$$

and curvature h .

The candidate bounds prove positive chord and admissible core curvature, including $h = 1$.

```
def stripCore : StripCore where
  chord := candidate.capChord
  curvature := candidate.h
  chord_pos := by
    exact div_pos (mul_pos (by norm_num)
      (sin_pos_of_pos_of_lt_pi candidate.alpha_pos
        (lt_trans candidate.alpha_lt_pi_div_two
          (by linarith [Real.pi_pos])))) candidate.h_pos
  curvature_pos := candidate.h_pos
  curvature_le_one := candidate.h_le_one

/-- Candidate assembled from the retained strip core and reflected upper/lower
caps. -/
```

FourArcCandidate.upperCap_euclideanArea

Classification: supplement

Source: FourArcCandidate.lean:153-158

Declaration SHA-256: 63eb34459de3e09e8c932883edd9446c50efc2cc1a148ef60c5c77cbd56d7496

Mathematical statement. The constructed upper cap's scalar Euclidean area equals the candidate cap area.

Its chord and angle are exactly those in the normalized area identity.

```
theorem upperCap_euclideanArea :
  candidate.assembly.upperCap.euclideanArea = candidate.capArea := by
  rw [OneSidedCircularCap.euclideanArea,
    candidate.capArea_eq_chord_sq_mul_area]
  rfl
```

FourArcCandidate.weightedArea_components

Classification: supplement

Source: FourArcCandidate.lean:364-373

Declaration SHA-256: 654554fba4e2f4847eae4f853550061afa29cf158da02de52109cfffaced702e

Mathematical statement. For every candidate, its actual weighted area equals its strip-core Euclidean area plus twice λ times its cap area.

The assembly integral formula and the equality of upper and lower cap areas give the component sum.

```
theorem weightedArea_components :
  candidate.WeightedArea = candidate.stripCore.euclideanArea +
    2 * lam * candidate.capArea := by
  change candidate.assembly.weightedArea lam =
    candidate.stripCore.euclideanArea + 2 * lam * candidate.capArea
  rw [FourArcAssembly.weightedArea_formula, assembly_core,
    candidate.upperCap_euclideanArea, candidate.lowerCap_euclideanArea]
```

```
ring
```

```
/-- Candidate perimeter as the sum of its four actual arc constructors. -/
```

FourArcCandidate.weightedPerimeter_components

Classification: supplement

Source: FourArcCandidate.lean:374-390

Declaration SHA-256: d02953cec6a51d470b1b940fa2318bbd5c86354d0e420d179322c86356c9434a

Mathematical statement. For every candidate, its canonical weighted perimeter equals its strip-core exposed-arc length plus

$$2\lambda L\ell(\alpha).$$

First bridge the auxiliary assembly perimeter to the canonical frontier perimeter, then use the two congruent cap-arc contributions.

```
theorem weightedPerimeter_components :
  candidate.WeightedPerimeter = candidate.stripCore.boundaryArcLength +
    2 * lam * candidate.capChord * ell candidate.alpha := by
  rw [FourArcCandidate.WeightedPerimeter, region,
    AdmissibleCompetitor.WeightedPerimeter]
  change _root_.WeightedPerimeter lam
  (FrontierMeasure candidate.assembly.carrier) =
    candidate.stripCore.boundaryArcLength +
      2 * lam * candidate.capChord * ell candidate.alpha
  rw [← fourArc_frontier_weightedPerimeter_eq,
    FourArcAssembly.weightedPerimeter_formula]
  simp only [StripCore.boundaryArcLength,
    OneSidedCircularCap.arcLength, FourArcAssembly.upperCap,
    FourArcAssembly.lowerCap, assembly, stripCore]
  ring

/-- CMV equation (27), derived from the boundary pieces. -/
```

HorizontalSegment

Classification: supplement

Source: CMVGeometry.lean:392-399

Declaration SHA-256: e7bebc91d2e392d80b21e5e8b5487591a5477fe10be16627e5b6180dd120506

Mathematical statement. A horizontal segment is specified by a positive length L and midpoint (m, b) .

This primitive requires positive length. The three-arc bottom segment is defined separately so that it may have zero length.

```
structure HorizontalSegment where
  chord : ℝ
  midpointX : ℝ
  baseY : ℝ
  chord_pos : 0 < chord

namespace HorizontalSegment
```

HorizontalSegment.carrier

Classification: supplement

Source: CMVGeometry.lean:406-408

Declaration SHA-256: 094b96b3006204081e6d336250f56e63f6b337cc71255210b9bae8ae3b894041

Mathematical statement. The horizontal segment carrier is

$$\{(x, b): |x - m| \leq L/2\}.$$

It is a genuine closed set including endpoints, with length proved from Euclidean Hausdorff measure.

```
def carrier (s : HorizontalSegment) : Set PlanePoint :=
  {p | p.2 = s.baseY ∧ |p.1 - s.midpointX| ≤ s.chord / 2}
```

HorizontalSegment.euclideanLength

Classification: supplement

Source: CMVGeometry.lean:409-415

Declaration SHA-256: c965c7e2ff7089171e9318c88e3733eb9629da0051af6a9aad082bf1ad5a5aef

Mathematical statement. The scalar Euclidean length of the segment is its given positive chord length L .

A later theorem identifies this value with the Hausdorff measure of its carrier.

```
def euclideanLength (s : HorizontalSegment) : ℝ := s.chord

end HorizontalSegment

/-- The unchanged central part of a type-(iv) candidate. It is the rectangle
of height two and width `chord`, together with the two radius-`1/h` side
segments. -/
```

HorizontalSegment.leftEndpoint

Classification: supplement

Source: CMVGeometry.lean:400-402

Declaration SHA-256: d64f9e0ea48cfbbc1b06bed874068659f0011957eca2b8226c738430113a0615

Mathematical statement. For segment midpoint (m, b) and length L , define the left endpoint $(m - L/2, b)$.

This coordinate is used to identify the segment image with the affine segment in Euclidean space.

```
def leftEndpoint (s : HorizontalSegment) : PlanePoint :=
  (s.midpointX - s.chord / 2, s.baseY)
```

HorizontalSegment.rightEndpoint

Classification: supplement

Source: CMVGeometry.lean:403-405

Declaration SHA-256: a14791633639531ab0cb8801c8e0e16383dd7712b7d5730aa7f0d9178c83e5da

Mathematical statement. For segment midpoint (m, b) and length L , define the right endpoint $(m + L/2, b)$.

The Euclidean distance between this point and the left endpoint is exactly L .

```
def rightEndpoint (s : HorizontalSegment) : PlanePoint :=
  (s.midpointX + s.chord / 2, s.baseY)
```

LeanSuffixAnalytic.hasDerivAt_typeThreeAngle

Classification: supplement

Source: LeanSuffixAnalytic.lean:154-205

Declaration SHA-256: 6ec918819525fbc766cbbf40969985e480000f0bcbcf23d536b033e9e80b6380

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, the three-arc angle combination has derivative

$$2[1/\sqrt{1-q(h)^2} - \lambda/\sqrt{\lambda^2 - q(h)^2}].$$

Differentiate the inverse trigonometric functions and include the affine factor two. Both radical denominators are positive.

```

theorem hasDerivAt_typeThreeAngle {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeAngle lam)
    (2 * (1 / sqrt (1 - typeThreeShape h ^ 2) -
      lam / sqrt (lam ^ 2 - typeThreeShape h ^ 2))) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hshape := hasDerivAt_typeThreeShape h
  have hq_low : -1 < typeThreeShape h := by
    unfold typeThreeShape
    linarith
  have hq_high : typeThreeShape h < 1 := by
    unfold typeThreeShape
    linarith
  have hu_arg : 0 < 1 - typeThreeShape h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - typeThreeShape h ^ 2 := by nlinarith
  have hd_ne : sqrt (lam ^ 2 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  have hratio_low : -1 < typeThreeShape h / lam :=
    (lt_div_iff₀ hlam_pos).2 (by linarith)
  have hratio_high : typeThreeShape h / lam < 1 :=
    (div_lt_one hlam_pos).2 (by linarith)
  have hratio_ne_neg_one : typeThreeShape h / lam ≠ -1 :=
    ne_of_gt hratio_low
  have hratio_ne_one : typeThreeShape h / lam ≠ 1 :=
    ne_of_lt hratio_high
  have hq_ne_neg_one : typeThreeShape h ≠ -1 := ne_of_gt hq_low
  have hq_ne_one : typeThreeShape h ≠ 1 := ne_of_lt hq_high
  have hsqrt_scale :
    sqrt (1 - (typeThreeShape h / lam) ^ 2) =
      sqrt (lam ^ 2 - typeThreeShape h ^ 2) / lam := by
  have hratio_identity :
    1 - (typeThreeShape h / lam) ^ 2 =
      (lam ^ 2 - typeThreeShape h ^ 2) / lam ^ 2 := by
    field_simp [hlam_ne]
  rw [hratio_identity, Real.sqrt_div hd_arg.le, Real.sqrt_sq_eq_abs,
    abs_of_pos hlam_pos]
  unfold typeThreeAngle
  have harccos :=
    (Real.hasDerivAt_arccos (x := typeThreeShape h / lam)
      hratio_ne_neg_one hratio_ne_one).comp h (hshape.div_const lam)
  have harsin :=
    (Real.hasDerivAt_arcsin (x := typeThreeShape h)
      hq_ne_neg_one hq_ne_one).comp h hshape
  have hraw := (harccos.const_mul lam).add harsin
  simpa only [Function.comp_apply, id_eq] using! hraw.congr_deriv (by
    rw [hsqrt_scale]
    field_simp [hlam_ne, hd_ne]
    ring)

/-- Exact derivative of the regular type-(iii) radial displacement on its
natural open domain. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreeDelta

Classification: supplement

Source: LeanSuffixAnalytic.lean:206-240

Declaration SHA-256: 6a259e36542b424c2657b96b59d66f9d38d054281073fe5c590c3110a33522c8

Mathematical statement. For $\lambda > 1$ and $0 < h < 1$, δ_3 has derivative

$$2q(h)[1/\sqrt{1-q(h)^2} - 1/(\lambda\sqrt{\lambda^2 - q(h)^2})].$$

The derivative of each normalized radical is legitimate in the open parameter interval; no endpoint derivative is asserted.

```

theorem hasDerivAt_typeThreeDelta {lam h : ℝ}
  (hlam : 1 < lam) (hh : 0 < h) (hh1 : h < 1) :
  HasDerivAt (typeThreeDelta lam)
    (2 * typeThreeShape h *
      (1 / sqrt (1 - typeThreeShape h ^ 2) -
        1 / (lam * sqrt (lam ^ 2 - typeThreeShape h ^ 2)))) h := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) hlam
  have hlam_ne : lam ≠ 0 := ne_of_gt hlam_pos
  have hshape := hasDerivAt_typeThreeShape h
  have hq_low : -1 < typeThreeShape h := by
    unfold typeThreeShape
    linarith
  have hq_high : typeThreeShape h < 1 := by
    unfold typeThreeShape
    linarith
  have hu_arg : 0 < 1 - typeThreeShape h ^ 2 := by nlinarith
  have hd_arg : 0 < lam ^ 2 - typeThreeShape h ^ 2 := by nlinarith
  have hu_ne : sqrt (1 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hu_arg)
  have hd_ne : sqrt (lam ^ 2 - typeThreeShape h ^ 2) ≠ 0 :=
    ne_of_gt (Real.sqrt_pos.2 hd_arg)
  unfold typeThreeDelta
  have houter :=
    (((hasDerivAt_const h (lam ^ 2)).sub (hshape.pow 2)).sqrt
      (ne_of_gt hd_arg)).div_const lam)
  have hinner :=
    (((hasDerivAt_const h 1).sub (hshape.pow 2)).sqrt (ne_of_gt hu_arg))
  have hraw := houter.sub hinner
  simpa only [id_eq] using! hraw.congr_deriv (by
    norm_num [id_eq, Pi.sub_apply, Pi.pow_apply]
    field_simp [hlam_ne, hu_ne, hd_ne]
    ring)

/-- Exact derivative of the regular type-(iv) angle contribution on its
natural open domain. -/

```

LeanSuffixAnalytic.hasDerivAt_typeThreeShape

Classification: supplement

Source: LeanSuffixAnalytic.lean:146-153

Declaration SHA-256: fc5b59d714869fde00b69f1950c83bd7a32c47524e01ad8d04455ff346c39d58

Mathematical statement. For every real h , the function

$$q(h) = 2h - 1$$

has derivative two.

This is the affine chain-rule factor in the three-arc angle and radical derivatives.

```

theorem hasDerivAt_typeThreeShape (h : ℝ) :
  HasDerivAt typeThreeShape 2 h := by
  unfold typeThreeShape
  simpa only [id_eq, mul_one] using
    (((hasDerivAt_id h).const_mul (2 : ℝ)).sub_const 1)

/-- Exact derivative of the regular type-(iii) angle contribution on its
natural open domain. -/

```

LeanSuffixAnalytic.typeFourAngle

Classification: supplement

Source: LeanSuffixAnalytic.lean:53-56

Declaration SHA-256: 94e6a9f5b491cb7ca4802d89fba1fa7ccb864bbe777d6e41d89b1d6bf892bdd0

Mathematical statement. Define the four-arc angle combination

$$\lambda \arccos(h/\lambda) + \arcsin h.$$

This is exactly the article's $B(h)$.

```
def typeFourAngle (lam h : ℝ) : ℝ :=
  lam * arccos (h / lam) + arcsin h

/-- The radial displacement in the exact regular type-(iv) formulas. -/
```

LeanSuffixAnalytic.typeFourArea

Classification: supplement

Source: LeanSuffixAnalytic.lean:61-64

Declaration SHA-256: 775f7921b66716563dd5e0e4c35562d880f5cef2a71b40cef616cd10eb9488d2

Mathematical statement. Define the normalized four-arc scalar area

$$2[B(h) + h\delta_4(h)]/h^2.$$

The scalar-identification theorem equates it to $A_4(h)$ on the stated domain.

```
def typeFourArea (lam h : ℝ) : ℝ :=
  2 * (typeFourAngle lam h + h * typeFourDelta lam h) / h ^ 2

/-- Exact regular type-(iv) weighted perimeter used by the suffix comparison. -/
```

LeanSuffixAnalytic.typeFourDelta

Classification: supplement

Source: LeanSuffixAnalytic.lean:57-60

Declaration SHA-256: a4c8b30971f59f8f04626dbddf49af92c98dbfa9126bf3766c0be8b6472cbfb3

Mathematical statement. Define

$$\delta_4(h) = \sqrt{\lambda^2 - h^2} / \lambda - \sqrt{1 - h^2}.$$

For positive density this equals $D(h)$, including at the curvature endpoint one.

```
def typeFourDelta (lam h : ℝ) : ℝ :=
  sqrt (lam ^ 2 - h ^ 2) / lam - sqrt (1 - h ^ 2)

/-- Exact regular type-(iv) weighted area used by the suffix comparison. -/
```

LeanSuffixAnalytic.typeFourPerimeter

Classification: supplement

Source: LeanSuffixAnalytic.lean:65-70

Declaration SHA-256: ad0bebbd79c17351c0f12ce6a7fdae3e930dba33de0077415a6116df1d1e6e25

Mathematical statement. Define the four-arc scalar perimeter

$$4B(h)/h.$$

This already agrees definitionally with the article's perimeter expression.

```
def typeFourPerimeter (lam h : ℝ) : ℝ :=
  4 * typeFourAngle lam h / h

/-- The checker numerator for the derivative of the regular type-(iii) area.
It is kept as one expression in the shared height argument so interval
evaluation does not decorrelate its repeated occurrences. -/
```

LeanSuffixAnalytic.typeThreeAngle

Classification: supplement

Source: LeanSuffixAnalytic.lean:35-38

Declaration SHA-256: fb7617b8b745792c6f019f3cbcf3c26d5e54f1fc7dd47bb69cec919c712d110a

Mathematical statement. Define the angle combination

$$\lambda \arccos(q(h)/\lambda) + \arcsin(q(h)).$$

It is the article's $B(q(h))$, with the density argument explicitly retained.

```
def typeThreeAngle (lam h : ℝ) : ℝ :=
  lam * arccos (typeThreeShape h / lam) + arcsin (typeThreeShape h)

/-- The radial displacement in the exact type-(iii) formulas. -/
```

LeanSuffixAnalytic.typeThreeArea

Classification: supplement

Source: LeanSuffixAnalytic.lean:44-48

Declaration SHA-256: 63d13d5879252e16ac6f53a2d4449b4f511e353d6016e98158793484dcfaa8f7

Mathematical statement. Define the normalized three-arc scalar area

$$[B(q(h)) + \pi/2 + (q(h) + 2)\delta_3(h)]/h^2.$$

Once the positive-density radical identity is proved, this is precisely $A_3(h)$; total definitions alone do not impose a geometric domain.

```
def typeThreeArea (lam h : ℝ) : ℝ :=
  (typeThreeAngle lam h + π / 2 +
   (typeThreeShape h + 2) * typeThreeDelta lam h) / h ^ 2

/-- Exact regular type-(iii) weighted perimeter used by the suffix comparison. -/
```

LeanSuffixAnalytic.typeThreeDelta

Classification: supplement

Source: LeanSuffixAnalytic.lean:39-43

Declaration SHA-256: f96722ed089f4e5911c6013a0aa601eb4080090036491702cd0fa78e3e31113f

Mathematical statement. Define

$$\delta_3(h) = \sqrt{\lambda^2 - q(h)^2} / \lambda - \sqrt{1 - q(h)^2}.$$

For positive density, radical normalization identifies this with $D(q(h))$; it is the width correction in the three-arc scalar formulas.

```
def typeThreeDelta (lam h : ℝ) : ℝ :=
  sqrt (lam ^ 2 - typeThreeShape h ^ 2) / lam -
  sqrt (1 - typeThreeShape h ^ 2)

/-- Exact regular type-(iii) weighted area used by the suffix comparison. -/
```

LeanSuffixAnalytic.typeThreePerimeter

Classification: supplement

Source: LeanSuffixAnalytic.lean:49-52

Declaration SHA-256: 22e35e36f9c0930c9f9233f0004c686c48f6f282298a578def5a6e6bba11f34e

Mathematical statement. Define the normalized three-arc scalar perimeter

$$2[B(q(h)) + \pi/2 + \delta_3(h)]/h.$$

The identification with $P_3(h)$ uses the same radical normalization and includes the bottom-segment term.

```
def typeThreePerimeter (lam h : ℝ) : ℝ :=
  2 * (typeThreeAngle lam h + π / 2 + typeThreeDelta lam h) / h

/-- The angle contribution in the exact regular type-(iv) formulas. -/
```

LeanSuffixAnalytic.typeThreeShape

Classification: supplement

Source: LeanSuffixAnalytic.lean:32-34

Declaration SHA-256: 9ef91461acc972465b98decfe31d601788f1354eb0ffbb8d094be6f5c46ca593

Mathematical statement. Define the three-arc shape parameter

$$q(h) = 2h - 1$$

for real h .

This affine parameter is the cosine of the inner angle in the geometric assembly.

```
def typeThreeShape (h : ℝ) : ℝ := 2 * h - 1

/-- The angle contribution in the exact type-(iii) formulas. -/
```

OneSidedCircularCap

Classification: supplement

Source: CMVGeometry.lean:100-112

Declaration SHA-256: a9dd2ba7ee653be367b6a2072de1384c3362b0214117e3aecceae763bc6f8403

Mathematical statement. A cap is specified by chord length $L > 0$, half-angle $0 < \theta < \pi$, chord midpoint (m, b) , and an upper or lower orientation.

The full angle interval includes minor, semicircular and major disk segments; its carrier and radius are defined from these data.

```
structure OneSidedCircularCap where
  chord : ℝ
  theta : ℝ
  midpointX : ℝ
  baseY : ℝ
  side : CapSide
  chord_pos : 0 < chord
```

```

theta_pos : 0 < theta
theta_lt_pi : theta < π

namespace OneSidedCircularCap

/-- Radius determined by chord length and half-central-angle. -/

```

OneSidedCircularCap.arcLength

Classification: supplement

Source: CMVGeometry.lean:167-170

Declaration SHA-256: 60daa6fa20eb20410aecf4cee5a41e50c48a0eb958e6b55f1bcb20cd85ff25

Mathematical statement. Define the scalar selected-arc length as

$$L\ell(\theta),$$

where

$$\ell(\theta) = \theta/\sin\theta.$$

It equals

$$2R\theta;$$

it excludes the chord and is later identified with actual Euclidean Hausdorff measure.

```

def arcLength (c : OneSidedCircularCap) : ℝ := c.chord * ell c.theta

/-- The branch-correct cap with prescribed positive normalized area. This is
the reusable geometric constructor behind the candidate-specific replacement. -/

```

OneSidedCircularCap.arcLength_pos

Classification: supplement

Source: CMVGeometry.lean:220-222

Declaration SHA-256: 615464c0ac73967d872be72b9e5d931fb0644d2db70b6e9168ad1dd7c81aa184

Mathematical statement. Every admissible cap has strictly positive selected-arc length.

Multiply positive chord length by the positive normalized arc length.

```

theorem arcLength_pos (c : OneSidedCircularCap) : 0 < c.arcLength :=
mul_pos c.chord_pos c.ell_theta_pos

```

OneSidedCircularCap.arcPoint

Classification: supplement

Source: CMVGeometry.lean:124-133

Declaration SHA-256: 2f12b44987266410d1b21333c27225fca467817287b4028245b465856f5540c3

Mathematical statement. The upper angle parametrization is

$$c(t) = (m + R\sin t, b + R(\cos t - \cos\theta));$$

the lower one is

$$c(t) = (m + R\sin t, b - R(\cos t - \cos\theta)).$$

The selected arc is obtained for

$$-\theta \leq t \leq \theta;$$

the definitions themselves are total in the parameter.

```
def arcPoint (c : OneSidedCircularCap) (t : ℝ) : PlanePoint :=
  match c.side with
  | .upper =>
    (c.midpointX + c.radius * sin t,
     c.baseY + c.radius * (cos t - cos c.theta))
  | .lower =>
    (c.midpointX + c.radius * sin t,
     c.baseY - c.radius * (cos t - cos c.theta))

/-- Left endpoint of the oriented chord. -/
```

OneSidedCircularCap.arcPoint_in_closed_side

Classification: supplement

Source: CMVGeometry.lean:240-252

Declaration SHA-256: 4692c99be4beede3a58f1e164acd1cfe21db7a9160da63340de9490acd46c184

Mathematical statement. If

$$|t| \leq \theta,$$

the selected upper arc point has

$$y \geq b,$$

and the selected lower arc point has

$$y \leq b.$$

Cosine decreases on $[0, \pi]$, so

$$\cos t \geq \cos \theta;$$

positive radius fixes the direction.

```
theorem arcPoint_in_closed_side (c : OneSidedCircularCap) {t : ℝ}
  (ht : |t| ≤ c.theta) :
  match c.side with
  | .upper => c.baseY ≤ (c.arcPoint t).2
  | .lower => (c.arcPoint t).2 ≤ c.baseY := by
  have hc : cos c.theta ≤ cos t := by
    rw [Real.cos_abs t]
    exact Real.cos_le_cos_of_nonneg_of_le_pi (abs_nonneg t)
      (le_of_lt c.theta_lt_pi) ht
  have hr := c.radius_pos
  cases h : c.side <|> simp [arcPoint, h] <|> nlinarith

/-- A non-endpoint arc point is strictly on the cap side of the chord. -/
```

OneSidedCircularCap.arcPoint_left

Classification: supplement

Source: CMVGeometry.lean:228-232

Declaration SHA-256: 4e22ae8901c6f895322d9f1591e370b613671c823882adf707a5fac1ae750287

Mathematical statement. The parametrized point at

$$t = -\theta$$

equals the left chord endpoint.

Odd sine and even cosine reduce the coordinates to $(m - L/2, b)$.

```
theorem arcPoint_left (c : OneSidedCircularCap) :
  c.arcPoint (-c.theta) = c.leftEndpoint := by
cases h : c.side <=>
simp [arcPoint, leftEndpoint, h, sin_neg, c.radius_mul_sin] <=> ring
```

OneSidedCircularCap.arcPoint_right

Classification: supplement

Source: CMVGeometry.lean:233-239

Declaration SHA-256: ba639553cc75debbcac47cb7f164b0cdee0e14e9586cdc6d1cf857f7fa27f21b

Mathematical statement. The parametrized point at $t = \theta$ equals the right chord endpoint.

The vertical displacement vanishes and the chord-radius identity gives the horizontal displacement.

```
theorem arcPoint_right (c : OneSidedCircularCap) :
  c.arcPoint c.theta = c.rightEndpoint := by
cases h : c.side <=>
simp [arcPoint, rightEndpoint, h, c.radius_mul_sin]

/-- Every point of the selected arc lies in the declared closed half-plane,
including on the major branch. -/
```

OneSidedCircularCap.arcTrace

Classification: supplement

Source: CMVGeometry.lean:656-661

Declaration SHA-256: 5e3edb81f4f16ef8dbff7c15d5756a176b1f99c3bd5fb86ce703134a0a727a89

Mathematical statement. The selected arc trace is the radius- R circle intersected with the closed half-plane defining the cap.

This is a coordinate set, not an assumed parametrized length; equality with the parametrized image is proved in the frontier module.

```
def arcTrace (c : OneSidedCircularCap) : Set PlanePoint :=
{p | c.radiusSquaredAt p = c.radius ^ 2 ^
  match c.side with
  | .upper => c.baseY ≤ p.2
  | .lower => p.2 ≤ c.baseY}
```

OneSidedCircularCap.center

Classification: supplement

Source: CMVGeometry.lean:117-123

Declaration SHA-256: 58a018a3af71f1eb6c5aebb7ee395b60aab79f7b55a9d384d43694baace0d346

Mathematical statement. For chord midpoint (m, b) , the upper-cap center is $(m, b - R\cos\theta)$, and the lower-cap center is $(m, b + R\cos\theta)$.

The sign of cosine allows the center to cross to the cap side for a major segment.

```
def center (c : OneSidedCircularCap) : PlanePoint :=
  match c.side with
  | .upper => (c.midpointX, c.baseY - c.radius * cos c.theta)
  | .lower => (c.midpointX, c.baseY + c.radius * cos c.theta)

/-- Coordinate realization of the selected arc, with parameter
`-theta ≤ t ≤ theta`. -/
```

OneSidedCircularCap.chordCarrier

Classification: supplement

Source: CMVGeometry.lean:160-163

Declaration SHA-256: 0705736836c9203b120221cb0770c365bada15ab749e72d0791d55f58eba4c1e

Mathematical statement. The closed chord is

$$\{(x, b): |x - m| \leq L/2\}.$$

It includes both endpoints and is proved to be exactly the disk section on the base line.

```
def chordCarrier (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | p.2 = c.baseY ∧ |p.1 - c.midpointX| ≤ c.chord / 2}

/-- Euclidean area computed by the circular-segment constructor. -/
```

OneSidedCircularCap.chordCarrier_subset_carrier

Classification: supplement

Source: CMVGeometry.lean:311-334

Declaration SHA-256: a35dcecd8fd218da748131deaec9b02f5328c1034952ad269b6ba1f5b686d947

Mathematical statement. Every point of the closed chord belongs to the cap carrier.

The squared chord bound and the trigonometric identity imply the disk inequality; the half-plane condition is equality.

```
theorem chordCarrier_subset_carrier (c : OneSidedCircularCap) :
  c.chordCarrier ⊆ c.carrier := by
  intro p hp
  have hxabs : |p.1 - c.midpointX| ≤ c.chord / 2 := hp.2
  have hhalf : 0 < c.chord / 2 := div_pos c.chord_pos (by norm_num)
  have hxsq : (p.1 - c.midpointX) ^ 2 ≤ (c.chord / 2) ^ 2 :=
    sq_le_sq.mpr (by simpa [abs_of_pos hhalf] using hxabs)
  rw [← c.radius_mul_sin] at hxsq
  have htrig := Real.sin_sq_add_cos_sq c.theta
  have htrigScaled := congrArg
    (fun z : ℝ => c.radius ^ 2 * z) htrig
  cases hside : c.side with
  | upper =>
    constructor
    · simp only [radiusSquaredAt, center, hside, hp.1]
      nlinarith
    · simp [hside, hp.1]
  | lower =>
    constructor
    · simp only [radiusSquaredAt, center, hside, hp.1]
      nlinarith
    · simp [hside, hp.1]

/-- The coordinate cap is measurable. -/
```

OneSidedCircularCap.ell_theta_pos

Classification: supplement

Source: CMVGeometry.lean:216-219

Declaration SHA-256: 88c4230a8ce8dbe3ed5e64385c18de0588039e6c242ead1e2df6299d087f0ad8

Mathematical statement. For an admissible cap,

$$\ell(\theta) > 0.$$

The ratio

$$\theta/\sin\theta$$

has positive numerator and denominator.

```
theorem ell_theta_pos (c : OneSidedCircularCap) : 0 < ell c.theta := by
  rw [ell]
  exact div_pos c.theta_pos c.sin_theta_pos
```

OneSidedCircularCap.euclideanArea

Classification: supplement

Source: CMVGeometry.lean:164-166

Declaration SHA-256: 5c476b88f35c9e9f6afc55ed7ed9ef6be1395a7b5a3003452ff8691443c49da

Mathematical statement. Define the scalar cap area as

$$L^2 a(\theta),$$

where

$$a(\theta) = (\theta - \sin\theta\cos\theta)/(4\sin^2\theta).$$

The volume theorem later identifies this scalar with the Lebesgue measure of the coordinate carrier.

```
def euclideanArea (c : OneSidedCircularCap) : ℝ := c.chord ^ 2 * area c.theta
/-- Euclidean length of the selected circular arc. -/
```

OneSidedCircularCap.euclideanArea_pos

Classification: supplement

Source: CMVGeometry.lean:213-215

Declaration SHA-256: 3902ccdc9eb3694014c5a47aad3179ebe09a3fd3cddb066b9c4c1ca933463218

Mathematical statement. Every admissible cap has strictly positive scalar Euclidean area.

The chord square is positive and the normalized segment-area function is positive on $(0, \pi)$.

```
theorem euclideanArea_pos (c : OneSidedCircularCap) : 0 < c.euclideanArea := by
  exact mul_pos (sq_pos_of_pos c.chord_pos) (area_pos c.theta_pos c.theta_lt_pi)
```

OneSidedCircularCap.frontier_carrier

Classification: supplement

Source: CMVGeometry.lean:697-761

Declaration SHA-256: d9cb2c749721b51457faae17a95033cf908527d3fdc652ebedacfbb2292d579d

Mathematical statement. For every cap, its topological frontier equals the union of its selected arc trace and its closed chord.

Interior points satisfy both strict inequalities; failure of either strict inequality on the closed carrier gives the circle or base chord. The reverse inclusion is proved by nearby exterior points.

```
theorem frontier_carrier (c : OneSidedCircularCap) :
  frontier c.carrier = arcTrace c ∪ c.chordCarrier := by
  apply Subset.antisymm
  · intro p hp
    have hpc : p ∈ c.carrier :=
      c.isClosed_carrier.frontier_subset hp
  by_cases heq : c.radiusSquaredAt p = c.radius ^ 2
```

```

· exact Or.inl (heq, hpc.2)
have hlt : c.radiusSquaredAt p < c.radius ^ 2 :=
  lt_of_le_of_ne hpc.1 heq
cases hside : c.side with
| upper =>
  have hsidep : c.baseY ≤ p.2 := by
    simpa [OneSidedCircularCap.carrier, hside] using hpc.2
  by_cases hy : p.2 = c.baseY
  · exact Or.inr
    (c.mem_chordCarrier_of_mem_carrier_of_eq_base hpc hy)
  · exfalso
    have hi : p ∈ c.interiorCarrier := by
      simpa [OneSidedCircularCap.interiorCarrier, hside] using
        (hlt, lt_of_le_of_ne hsidep (Ne.symm hy))
    exact (mem_frontier_iff_notMem_interior hpc).mp hp
      (interiorCarrier_subset_interior c hi)
| lower =>
  have hsidep : p.2 ≤ c.baseY := by
    simpa [OneSidedCircularCap.carrier, hside] using hpc.2
  by_cases hy : p.2 = c.baseY
  · exact Or.inr
    (c.mem_chordCarrier_of_mem_carrier_of_eq_base hpc hy)
  · exfalso
    have hi : p ∈ c.interiorCarrier := by
      simpa [OneSidedCircularCap.interiorCarrier, hside] using
        (hlt, lt_of_le_of_ne hsidep hy)
    exact (mem_frontier_iff_notMem_interior hpc).mp hp
      (interiorCarrier_subset_interior c hi)
· rintro p (hp | hp)
· have hpc : p ∈ c.carrier := (hp.1.le, hp.2)
  rw [mem_frontier_iff_notMem_interior hpc]
  intro hi
  apply circle_eq_not_mem_interior_disk
    c.center.1 c.center.2 c.radius c.radius_pos p
    (by simpa [OneSidedCircularCap.radiusSquaredAt] using hp.1)
  apply interior_mono _ hi
  intro q hq
  exact hq.1
· have hpc := c.chordCarrier_subset_carrier hp
  rw [mem_frontier_iff_notMem_interior hpc]
  intro hi
  cases hside : c.side with
  | upper =>
    apply second_eq_not_mem_interior_ge c.baseY p hp.1
    apply interior_mono _ hi
    intro q hq
    simpa [OneSidedCircularCap.carrier, hside] using hq.2
  | lower =>
    apply second_eq_not_mem_interior_le c.baseY p hp.1
    apply interior_mono _ hi
    intro q hq
    simpa [OneSidedCircularCap.carrier, hside] using hq.2

end OneSidedCircularCap

namespace StripCore

/-- The exact left circular trace of a strip core. -/

```

OneSidedCircularCap.interiorCarrier

Classification: supplement

Source: CMVGeometry.lean:153-159

Declaration SHA-256: d9986ab2dd520dfd41d6ccb99ca9878df085185d6c10c136685b43549b4c4dc1

Mathematical statement. The open cap carrier uses the strict disk inequality and the strict upper or lower half-plane inequality.

This auxiliary open set excludes both the chord and the arc; its inclusion in the topological interior is proved below.

```
def interiorCarrier (c : OneSidedCircularCap) : Set PlanePoint :=
  {p | c.radiusSquaredAt p < c.radius ^ 2 ^
    match c.side with
    | .upper => c.baseY < p.2
    | .lower => p.2 < c.baseY}

/-- The horizontal chord, including both endpoints. -/
```

OneSidedCircularCap.interiorCarrier_subset_interior

Classification: supplement

Source: CMVGeometry.lean:687-696

Declaration SHA-256: e055c7441e3e2230b6bd3c3989fc3043eb631862e9cdc4e7fe3a7963b0242ddc

Mathematical statement. The strict cap carrier is contained in the topological interior of the closed cap carrier.

It is an open subset of the carrier, so it lies in the maximal such open set.

```
theorem interiorCarrier_subset_interior (c : OneSidedCircularCap) :
  c.interiorCarrier ⊆ interior c.carrier := by
  apply interior_maximal
  intro p hp
  exact ⟨hp.1.le, by
    cases hside : c.side <;>
    simp only [OneSidedCircularCap.interiorCarrier, hside] at hp <;>
    simp only [OneSidedCircularCap.carrier, hside] <;> linarith [hp.2]⟩
  exact isOpen_interiorCarrier c
```

OneSidedCircularCap.isClosed_carrier

Classification: supplement

Source: CMVGeometry.lean:364-391

Declaration SHA-256: d757ff8cbf988a97323a1f1c27dc667eab9b6475f2eb7d1c16beffa82e3009a4

Mathematical statement. Every cap carrier is closed.

Both the disk inequality and the orientation-dependent half-plane are closed conditions.

```
theorem isClosed_carrier (c : OneSidedCircularCap) :
  IsClosed c.carrier := by
  have hdist : Continuous (fun p : PlanePoint => c.radiusSquaredAt p) := by
    simp only [radiusSquaredAt]
    fun_prop
  have hdisk : IsClosed {p : PlanePoint |
    c.radiusSquaredAt p ≤ c.radius ^ 2} :=
    isClosed_le hdist continuous_const
  cases hside : c.side with
  | upper =>
    rw [carrier]
    simp only [hside]
    change IsClosed
      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | c.baseY ≤ p.2})
    exact hdisk.inter (isClosed_le continuous_const continuous_snd)
  | lower =>
    rw [carrier]
    simp only [hside]
    change IsClosed
      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | p.2 ≤ c.baseY})
    exact hdisk.inter (isClosed_le continuous_snd continuous_const)
```

```

end OneSidedCircularCap

/-- A horizontal segment. Its metric length is its positive chord parameter,
not an independently asserted scalar. -/

```

OneSidedCircularCap.isOpen_interiorCarrier

Classification: supplement

Source: CMVGeometry.lean:662-686

Declaration SHA-256: ec8c11c26c05148d770698c167cdec56869de41dc90864a365cf2514de87bc1

Mathematical statement. The strict disk and strict half-plane carrier is open.

Its two defining inequalities are strict inequalities of continuous coordinate functions.

```

theorem isOpen_interiorCarrier (c : OneSidedCircularCap) :
  IsOpen c.interiorCarrier := by
  have hdisk : IsOpen {p : PlanePoint |
    c.radiusSquaredAt p < c.radius ^ 2} := by
    apply isOpen_lt
    · change Continuous (fun p : PlanePoint =>
      (p.1 - c.center.1) ^ 2 + (p.2 - c.center.2) ^ 2)
      fun_prop
    · fun_prop
  cases hside : c.side with
  | upper =>
    rw [OneSidedCircularCap.interiorCarrier]
    simp only [hside]
    change IsOpen
      ({p : PlanePoint | c.radiusSquaredAt p < c.radius ^ 2} ∩
       {p : PlanePoint | c.baseY < p.2})
    exact hdisk.inter (isOpen_lt continuous_snd continuous_snd)
  | lower =>
    rw [OneSidedCircularCap.interiorCarrier]
    simp only [hside]
    change IsOpen
      ({p : PlanePoint | c.radiusSquaredAt p < c.radius ^ 2} ∩
       {p : PlanePoint | p.2 < c.baseY})
    exact hdisk.inter (isOpen_lt continuous_snd continuous_snd)

```

OneSidedCircularCap.leftEndpoint

Classification: supplement

Source: CMVGeometry.lean:134-137

Declaration SHA-256: a193f337e9021d7e081f4666c9f784092558b2268ec0e9060f11b3d18dfc3c8d

Mathematical statement. The left endpoint is $(m - L/2, b)$.

It is defined on the chord line and later identified with the negative-angle endpoint.

```

def leftEndpoint (c : OneSidedCircularCap) : PlanePoint :=
  (c.midpointX - c.chord / 2, c.baseY)

/-- Right endpoint of the oriented chord. -/

```

OneSidedCircularCap.measurableSet_carrier

Classification: supplement

Source: CMVGeometry.lean:335-363

Declaration SHA-256: e0f551f525fbf0409babb736205ec484e0a0c2ff7039ce55de1e7a7fc5595a27

Mathematical statement. Every cap carrier is measurable.

It is an intersection of closed coordinate inequality sets, hence Borel measurable.

```
theorem measurableSet_carrier (c : OneSidedCircularCap) :
  MeasurableSet c.carrier := by
  have hdist : Continuous (fun p : PlanePoint => c.radiusSquaredAt p) := by
    simp only [radiusSquaredAt]
    fun_prop
  have hdisk : MeasurableSet {p : PlanePoint |
    c.radiusSquaredAt p ≤ c.radius ^ 2} :=
    (isClosed_le hdist continuous_const).measurableSet
  cases h : c.side with
  | upper =>
    have hhalf : MeasurableSet {p : PlanePoint | c.baseY ≤ p.2} :=
      (isClosed_le continuous_const continuous_snd).measurableSet
    rw [carrier]
    simp only [h]
    change MeasurableSet
      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | c.baseY ≤ p.2})
    exact hdisk.inter hhalf
  | lower =>
    have hhalf : MeasurableSet {p : PlanePoint | p.2 ≤ c.baseY} :=
      (isClosed_le continuous_snd continuous_const).measurableSet
    rw [carrier]
    simp only [h]
    change MeasurableSet
      ({p : PlanePoint | c.radiusSquaredAt p ≤ c.radius ^ 2} ∩
       {p : PlanePoint | p.2 ≤ c.baseY})
    exact hdisk.inter hhalf

/-- The disk/closed-half-plane cap carrier is closed. -/
```

OneSidedCircularCap.mem_chordCarrier_of_mem_carrier_of_eq_base

Classification: supplement

Source: CMVGeometry.lean:291-310

Declaration SHA-256: 424261710f758c71ebe7e57dd13ebd11ffc59cb51b840ff8fc86319890ddae19

Mathematical statement. A cap-carrier point on its base line belongs to its closed chord.

Substitute the base height into the disk inequality and use

$$R\sin\theta = L/2$$

to bound its horizontal displacement.

```
theorem mem_chordCarrier_of_mem_carrier_of_eq_base
  (c : OneSidedCircularCap) {p : PlanePoint}
  (hp : p ∈ c.carrier) (hy : p.2 = c.baseY) :
  p ∈ c.chordCarrier := by
  refine ⟨hy, ?_⟩
  have hdisk := hp.1
  cases hside : c.side <|>
  simp only [radiusSquaredAt, center, hside, hy] at hdisk
  all_goals
  have htrig := Real.sin_sq_add_cos_sq c.theta
  have htrigScaled := congrArg
    (fun z : ℝ => c.radius ^ 2 * z) htrig
  have hsq : (p.1 - c.midpointX) ^ 2 ≤
    (c.radius * sin c.theta) ^ 2 := by
    nlinarith
  rw [c.radius_mul_sin] at hsq
  exact abs_le_of_sq_le_sq hsq
    (div_nonneg c.chord_pos.le (by norm_num))

/-- Every point of the finite chord belongs to the closed cap. -/
```

OneSidedCircularCap.radius

Classification: supplement

Source: CMVGeometry.lean:113-116

Declaration SHA-256: e898cc7066a75a606fdc3b1d1e563939ad569bdb0f66fd289569e58c7f4c44b8

Mathematical statement. For cap data

$$\begin{aligned} L &> 0, \\ 0 < \theta < \pi, \end{aligned}$$

define

$$R = L/(2\sin\theta).$$

This is the circle radius whose chord of half-angle θ has length L .

```
def radius (c : OneSidedCircularCap) : ℝ := c.chord / (2 * sin c.theta)

/-- The center lies below an upper chord and above a lower chord. For a major
cap `cos θ < 0`, so the center crosses to the cap side, as it should. -/
```

OneSidedCircularCap.radiusSquaredAt

Classification: supplement

Source: CMVGeometry.lean:142-145

Declaration SHA-256: a1c0f970b426c7f0e6a013d8d8bbb613fbba3b5c253445dacecdb9b47cb6785a

Mathematical statement. For a point p and cap center c , define

$$(p_x - c_x)^2 + (p_y - c_y)^2.$$

This is the Euclidean squared distance in coordinates, regardless of the coordinate product norm.

```
def radiusSquaredAt (c : OneSidedCircularCap) (p : PlanePoint) : ℝ :=
  (p.1 - c.center.1) ^ 2 + (p.2 - c.center.2) ^ 2

/-- The genuine closed coordinate carrier of a one-sided cap. -/
```

OneSidedCircularCap.radius_mul_sin

Classification: supplement

Source: CMVGeometry.lean:223-227

Declaration SHA-256: 4b800b6287f8bc78c34b40f39fff02ab7d25631ad2fd22f8819095ceda60143e

Mathematical statement. For every admissible cap,

$$R\sin\theta = L/2.$$

Cancel the nonzero sine in the definition of radius; this is the chord-radius identity used throughout the geometry.

```
theorem radius_mul_sin (c : OneSidedCircularCap) :
  c.radius * sin c.theta = c.chord / 2 := by
  rw [radius]
  field_simp [ne_of_gt c.sin_theta_pos]
```

OneSidedCircularCap.radius_pos

Classification: supplement

Source: CMVGeometry.lean:210-212

Declaration SHA-256: 7bcdcf3411946f45cb3c854e76728bf4e1477fc16c0bd8673b5d0f6f904d6adf

Mathematical statement. Every admissible cap has

$$R > 0.$$

Both chord length and

$$2\sin\theta$$

are positive.

```
theorem radius_pos (c : OneSidedCircularCap) : 0 < c.radius := by
  exact div_pos c.chord_pos (mul_pos (by norm_num) c.sin_theta_pos)
```

OneSidedCircularCap.rightEndpoint

Classification: supplement

Source: CMVGeometry.lean:138-141

Declaration SHA-256: e83d4942af5a95dd72bd914bc15f416e046e239fc23caff7a851807f832f26a3

Mathematical statement. The right endpoint is $(m + L/2, b)$.

It is the positive-angle endpoint of the selected arc.

```
def rightEndpoint (c : OneSidedCircularCap) : PlanePoint :=
  (c.midpointX + c.chord / 2, c.baseY)

/-- Squared distance from the cap's circle center. -/
```

OneSidedCircularCap.sin_theta_pos

Classification: supplement

Source: CMVGeometry.lean:207-209

Declaration SHA-256: 8c22795350624813eb7b9aec2fdcda7bfc438d144ebb38b607db3cac77688583

Mathematical statement. For every admissible cap,

$$\sin\theta > 0.$$

The half-angle lies strictly between zero and π , ensuring the radius denominator is nonzero.

```
theorem sin_theta_pos (c : OneSidedCircularCap) : 0 < sin c.theta :=
  sin_pos_of_pos_of_lt_pi c.theta_pos c.theta_lt_pi
```

PlanePoint

Classification: supplement

Source: CMVGeometry.lean:30-33

Declaration SHA-256: b6c33ab7d9e6652b4833188977c0694ee665f31ebd07e572ea7a67ee7a8a0183

Mathematical statement. The coordinate plane is

$$\mathbb{R} \times \mathbb{R}.$$

Coordinates are used for all carrier inequalities and for product Lebesgue measure; their default product norm is not used to define geometric arc length.

```
abbrev PlanePoint := ℝ × ℝ

/-- The Euclidean L2 realization used to form geometric Hausdorff measure.
`PlanePoint` keeps its coordinate topology and all existing carrier definitions. -/
```

ReplacementAssembly

Classification: supplement

Source: CMVGeometry.lean:3281-3288

Declaration SHA-256: e0b73790946058b82b8d6a4bdde47ad7560092e26acf3d58cb0c91dc0cfa445f

Mathematical statement. A replacement assembly consists of a strip core and an upper-cap half-angle $0 < \alpha < \pi$.

This lower-level carrier constructor occurs in the competitor data type within the publication closure. No final comparison for this replacement is asserted or used here.

```
structure ReplacementAssembly where
  core : StripCore
  replacementAngle : ℝ
  replacementAngle_pos : 0 < replacementAngle
  replacementAngle_lt_pi : replacementAngle < π

namespace ReplacementAssembly
```

ReplacementAssembly.carrier

Classification: supplement

Source: CMVGeometry.lean:3305-3309

Declaration SHA-256: 36b80491dc0fecbf358d87c6bfe7194faf4bcad4ac91372874e8af60a30fd23e

Mathematical statement. The replacement carrier is the union of its strip core and upper cap.

Only this set constructor is needed through the common competitor representation; it is not the greatest-root competitor theorem.

```
def carrier (a : ReplacementAssembly) : Set PlanePoint :=
  a.core.carrier ∪ a.upperCap.carrier

/-- Explicit pushforward measure of the two retained side arcs, inserted upper
arc, and genuinely exposed lower chord. -/
```

ReplacementAssembly.upperCap

Classification: supplement

Source: CMVGeometry.lean:3289-3298

Declaration SHA-256: b30d86506333a29d0fea0a0c213d5ab5e404eb8902dd90c6019d57dad3b8f56f

Mathematical statement. For a replacement assembly, form the upper cap with the core chord, midpoint $(0, 1)$, and the chosen half-angle.

The positive chord and full angle interval define an actual disk segment, potentially major.

```
def upperCap (a : ReplacementAssembly) : OneSidedCircularCap where
  chord := a.core.chord
  theta := a.replacementAngle
  midpointX := 0
```

```

baseY := 1
side := .upper
chord_pos := a.core.chord_pos
theta_pos := a.replacementAngle_pos
theta_lt_pi := a.replacementAngle_lt_pi

```

ReplacementAssembly.weightedArea

Classification: supplement

Source: CMVGeometry.lean:3326-3329

Declaration SHA-256: 30517c93164485b2476e45ea547b19187ad53db44a33af2bf70144ca4c93c662

Mathematical statement. The replacement assembly weighted area is the integral A_λ over its carrier.

The definition uses the canonical area functional, not a stored scalar witness.

```

def weightedArea (lam : ℝ) (a : ReplacementAssembly) : ℝ :=
  WeightedArea lam a.carrier

/-- Density integral against the complete explicit replacement boundary. -/

```

StripCore

Classification: supplement

Source: CMVGeometry.lean:416-425

Declaration SHA-256: a32ba501747725a0a1307cf7bd2abbffd387c8bc9d810f24cf348c7c04513aad

Mathematical statement. A strip core is specified by a chord length $L > 0$ and curvature

$$0 < k \leq 1.$$

The radius is

$$R = 1/k;$$

a rectangle and two disk segments define its carrier. The closed curvature endpoint is allowed.

```

structure StripCore where
  chord : ℝ
  curvature : ℝ
  chord_pos : 0 < chord
  curvature_pos : 0 < curvature
  curvature_le_one : curvature ≤ 1

namespace StripCore

/-- Common radius of both strip arcs. -/

```

StripCore.boundaryArcLength

Classification: supplement

Source: CMVGeometry.lean:457-459

Declaration SHA-256: 74e634700740d7651d4ed773b1cb3cb71b11c7eedacd0c71b6a84a0e6ec43665

Mathematical statement. Define the exposed core-arc length

$$4\ell(\beta).$$

Each of the two side arcs has chord two and length

$$2\ell(\beta);$$

the attaching chords are deliberately excluded.

```
def boundaryArcLength (c : StripCore) : ℝ :=  
  4 * ell c.sideAngle
```

StripCore.carrier

Classification: supplement

Source: CMVGeometry.lean:449-452

Declaration SHA-256: 05f714bedd5ca3f09fd1dac168964dc190e4830b8099d994e590458d8167e259

Mathematical statement. The core carrier is the union of its rectangle, left cap and right cap.

All components are coordinate sets; later width and volume identities are consequences of this construction.

```
def carrier (c : StripCore) : Set PlanePoint :=  
  c.rectangleCarrier ∪ c.leftCapCarrier ∪ c.rightCapCarrier  
  
/-- Rectangle plus two side circular segments. -/
```

StripCore.carrier_y_bounds

Classification: supplement

Source: CMVGeometry.lean:482-490

Declaration SHA-256: dfd3510aaac02f0e3230984bb1d2516c15f954b7d416fa7e30c71ad44efa3ab8

Mathematical statement. Every point of a strip-core carrier satisfies $|y| \leq 1$.

Each primitive component includes the closed-strip height condition.

```
theorem carrier_y_bounds (c : StripCore) {p : PlanePoint}  
  (hp : p ∈ c.carrier) : |p.2| ≤ 1 := by  
  rcases hp with (hp | hp) | hp  
  · exact hp.2.2  
  · exact hp.2.2  
  · exact hp.2.2  
  
/-- Every horizontal segment across the core rectangle is contained in the  
retained region. -/
```

StripCore.cos_sideAngle

Classification: supplement

Source: CMVGeometry.lean:478-481

Declaration SHA-256: 5c26f39a6dccf790fede705362af68051955e479d2108518ff90a5d9985148ed

Mathematical statement. For a strip core,

$$\cos\beta = \sqrt{1 - k^2}.$$

Use the nonnegative cosine branch for the principal arcsine.

```
theorem cos_sideAngle (c : StripCore) :  
  cos c.sideAngle = √(1 - c.curvature ^ 2) := by  
  simp [sideAngle, Real.cos_arcsin]
```

StripCore.euclideanArea

Classification: supplement

Source: CMVGeometry.lean:453-456

Declaration SHA-256: d05ac0be663f49bfb5425917ab3d2dcdd3367e8ef25ec467ccc3d1ef4ccf07a7

Mathematical statement. Define the scalar core area

$$2L + 8a(\beta).$$

Each side cap has chord length two and area

$$4a(\beta),$$

so the scalar sum adds the rectangle and both caps.

```
def euclideanArea (c : StripCore) : ℝ :=
  2 * c.chord + 8 * area c.sideAngle

/-- The two side arcs, each of chord length two. -/
```

StripCore.frontier_carrier

Classification: supplement

Source: CMVGeometry.lean:1088-1199

Declaration SHA-256: 34bdf246812f32ba3d6167bf3b3ff5fea490b0816170740d97a7d77e703585bf

Mathematical statement. The core frontier is exactly its left arc, right arc, upper chord and lower chord.

Both inclusions are proved from the coordinate pieces; no artificial rectangle-side boundary remains inside the union.

```
theorem frontier_carrier (c : StripCore) :
  frontier c.carrier =
    leftArcTrace c ∪
    (rightArcTrace c ∪ (upperChordTrace c ∪ lowerChordTrace c)) := by
  apply Subset.antisymm
  · intro p hp
    have hpc : p ∈ c.carrier := (isClosed_carrier c).frontier_subset hp
    have hnint : p ∉ interior c.carrier :=
      (mem_frontier_iff_notMem_interior hpc).mp hp
    rcases hpc with (hrect | hleft) | hright
    · change -c.chord / 2 ≤ p.1 ∧ p.1 ≤ c.chord / 2 ∧
      |p.2| ≤ 1 at hrect
      by_cases htop : p.2 = 1
      · exact Or.inr (Or.inr (Or.inl
        ⟨htop, abs_le.mpr
          (by linarith [hrect.1], hrect.2.1)⟩)))
      by_cases hbottom : p.2 = -1
      · exact Or.inr (Or.inr (Or.inr
        ⟨hbottom, abs_le.mpr
          (by linarith [hrect.1], hrect.2.1)⟩)))
      have hybounds := abs_le.mp hrect.2.2
      have hylt : |p.2| < 1 := abs_lt.mpr
        ⟨lt_of_le_of_ne hybounds.1 (Ne.symm hbottom),
          lt_of_le_of_ne hybounds.2 htop)
      by_cases hxl : p.1 = -c.chord / 2
      · ex falso
        exact hnint (leftInterface_subset_interior c ⟨hxl, hylt⟩)
      by_cases hxr : p.1 = c.chord / 2
      · ex falso
        exact hnint (rightInterface_subset_interior c ⟨hxr, hylt⟩)
      · ex falso
        apply hnint
        exact strictRectangle_subset_interior c
          ⟨lt_of_le_of_ne hrect.1 (Ne.symm hxl),
            lt_of_le_of_ne hrect.2.1 hxr, hylt⟩
```

```

· change
  ((p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2) ^
  p.1 ≤ -c.chord / 2 ^ |p.2| ≤ 1 at hleft
by_cases heq :
  (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2
· exact Or.inl (heq, hleft.2)
have hdisk :
  (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 < c.radius ^ 2 :=
  lt_of_le_of_ne hleft.1 heq
have hylt : |p.2| < 1 := by
  apply lt_of_le_of_ne hleft.2.2
  intro hone
  exact heq (leftCap_circle_eq_of_abs_eq_one c hleft hone)
by_cases hx : p.1 = -c.chord / 2
· exfalso
  exact hnint (leftInterface_subset_interior c (hx, hylt))
· exfalso
  exact hnint (strictLeftCap_subset_interior c
    (hdisk, lt_of_le_of_ne hleft.2.1 hx, hylt))
· change
  ((p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2) ^
  c.chord / 2 ≤ p.1 ^ |p.2| ≤ 1 at hright
by_cases heq :
  (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2
· exact Or.inr (Or.inl (heq, hright.2))
have hdisk :
  (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 < c.radius ^ 2 :=
  lt_of_le_of_ne hright.1 heq
have hylt : |p.2| < 1 := by
  apply lt_of_le_of_ne hright.2.2
  intro hone
  exact heq (rightCap_circle_eq_of_abs_eq_one c hright hone)
by_cases hx : p.1 = c.chord / 2
· exfalso
  exact hnint (rightInterface_subset_interior c (hx, hylt))
· exfalso
  exact hnint (strictRightCap_subset_interior c
    (hdisk, lt_of_le_of_ne hright.2.1 (Ne.symm hx), hylt))
· rintro p (hleft | hright | hupper | hlower)
· have hpc : p ∈ c.carrier :=
  Or.inl (Or.inr (hleft.1.le, hleft.2))
  exact (mem_frontier_iff_notMem_interior hpc).2
  (leftArc_not_mem_interior c hleft)
· have hpc : p ∈ c.carrier :=
  Or.inr (hright.1.le, hright.2)
  exact (mem_frontier_iff_notMem_interior hpc).2
  (rightArc_not_mem_interior c hright)
· have hpc : p ∈ c.carrier := by
  apply Or.inl
  apply Or.inl
  have hxbounds := abs_le.mp hupper.2
  exact (by linarith [hxbounds.1], hxbounds.2,
    by simp [hupper.1])
rw [mem_frontier_iff_notMem_interior hpc]
intro hi
apply second_eq_not_mem_interior_le 1 p hupper.1
exact interior_mono (by
  intro q hq
  exact le_trans (le_abs_self q.2) (c.carrier_y_bounds hq)) hi
· have hpc : p ∈ c.carrier := by
  apply Or.inl
  apply Or.inl
  have hxbounds := abs_le.mp hlower.2
  exact (by linarith [hxbounds.1], hxbounds.2,
    by simp [hlower.1])
rw [mem_frontier_iff_notMem_interior hpc]
intro hi
apply second_eq_not_mem_interior_ge (-1) p hlower.1
exact interior_mono (by
  intro q hq
  have hneg : -q.2 ≤ 1 :=
    le_trans (neg_le_abs q.2) (c.carrier_y_bounds hq))

```

```

      show (-1 : ℝ) ≤ q.2
      linarith) hi

end StripCore
/-- Push one-dimensional Lebesgue measure on a closed interval through a curve. -/

```

StripCore.isClosed_carrier

Classification: supplement

Source: CMVGeometry.lean:805-843

Declaration SHA-256: d28fca8c993d5d2b9aa0b2ebe7f3f0372ebdd951a5cda90d26adf748bface0e4

Mathematical statement. The strip-core carrier is closed.

It is a finite union of closed coordinate-defined primitive regions.

```

theorem isClosed_carrier (c : StripCore) : IsClosed c.carrier := by
  have hx : Continuous (fun p : PlanePoint => p.1) := continuous_fst
  have hy : Continuous (fun p : PlanePoint => p.2) := continuous_snd
  have habsy : Continuous (fun p : PlanePoint => |p.2|) := hy.abs
  have hrect : IsClosed c.rectangleCarrier := by
    rw [StripCore.rectangleCarrier]
    change IsClosed
    ({p : PlanePoint | -c.chord / 2 ≤ p.1} ∩
     {p : PlanePoint | p.1 ≤ c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
  exact (isClosed_le continuous_const hx).inter
    ((isClosed_le hx continuous_const).inter
     (isClosed_le habsy continuous_const))
  have hleft : IsClosed c.leftCapCarrier := by
    rw [StripCore.leftCapCarrier]
    change IsClosed
    ({p : PlanePoint |
      (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
     {p : PlanePoint | p.1 ≤ -c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
  exact (isClosed_le
    (((hx.sub continuous_const).pow 2).add (hy.pow 2))
    continuous_const).inter
    ((isClosed_le hx continuous_const).inter
     (isClosed_le habsy continuous_const))
  have hright : IsClosed c.rightCapCarrier := by
    rw [StripCore.rightCapCarrier]
    change IsClosed
    ({p : PlanePoint |
      (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
     {p : PlanePoint | c.chord / 2 ≤ p.1} ∩
     {p : PlanePoint | |p.2| ≤ 1})
  exact (isClosed_le
    (((hx.sub continuous_const).pow 2).add (hy.pow 2))
    continuous_const).inter
    ((isClosed_le continuous_const hx).inter
     (isClosed_le habsy continuous_const))
  exact (hrect.union hleft).union hright

```

StripCore.leftArcTrace

Classification: supplement

Source: CMVGeometry.lean:762-766

Declaration SHA-256: 702872b40533619867ab566579d52d3f6a44e812910796d96106d1c50acbbfc2

Mathematical statement. The left exposed trace is

$$\{(x, y) : (x - x_L)^2 + y^2 = R^2, x \leq -L/2, |y| \leq 1\}.$$

Replacing the disk inequality by equality selects the boundary arc, with endpoints included.

```
def leftArcTrace (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ^
    p.1 ≤ -c.chord / 2 ^ |p.2| ≤ 1}

/-- The exact right circular trace of a strip core. -/
```

StripCore.leftCapCarrier

Classification: supplement

Source: CMVGeometry.lean:440-443

Declaration SHA-256: 2ca301cf56be587995c60fad814a6021c666b20d0f45e1fa3f2964b23f678cbc

Mathematical statement. The left core cap is

$$\{(x, y): (x - x_L)^2 + y^2 \leq R^2, x \leq -L/2, |y| \leq 1\}.$$

The horizontal cutoff retains the disk segment outside the rectangle on its left side.

```
def leftCapCarrier (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ^
    p.1 ≤ -c.chord / 2 ^ |p.2| ≤ 1}
```

StripCore.leftCenterX

Classification: supplement

Source: CMVGeometry.lean:431-433

Declaration SHA-256: 01fd3b14ca7e8a9d341e71e2fb86b773a9bea051016f7f6499489c3b65bab474

Mathematical statement. Define the left circle center coordinate

$$x_L = -L/2 + R\cos\beta.$$

At heights

$$y = \pm 1,$$

the exposed left arc meets the left chord endpoints.

```
def leftCenterX (c : StripCore) : ℝ :=
  -c.chord / 2 + c.radius * cos c.sideAngle
```

StripCore.lowerChordTrace

Classification: supplement

Source: CMVGeometry.lean:774-776

Declaration SHA-256: faa702d9afdf2eed044fad4474692be94c38bb3a9868779993853eb74445596

Mathematical statement. The lower core chord is

$$\{(x, -1): |x| \leq L/2\}.$$

It is part of the core frontier before a lower cap is attached.

```
def lowerChordTrace (c : StripCore) : Set PlanePoint :=
  {p | p.2 = -1 ^ |p.1| ≤ c.chord / 2}
```

StripCore.measurableSet_carrier

Classification: supplement

Source: CMVGeometry.lean:500-551

Declaration SHA-256: c4a110b00efc328d9afc36bfb523f064a5cb69170025121ec86c0022fc798171

Mathematical statement. The strip-core carrier is measurable.

The rectangle and two disk intersections are measurable, and finite unions preserve measurability.

```
theorem measurableSet_carrier (c : StripCore) : MeasurableSet c.carrier := by
  have hx : Continuous (fun p : PlanePoint => p.1) := continuous_fst
  have hy : Continuous (fun p : PlanePoint => p.2) := continuous_snd
  have hrect : MeasurableSet c.rectangleCarrier := by
    have hxmin : IsClosed {p : PlanePoint | -c.chord / 2 ≤ p.1} :=
      isClosed_le continuous_const hx
    have hxmax : IsClosed {p : PlanePoint | p.1 ≤ c.chord / 2} :=
      isClosed_le hx continuous_const
    have hybound : IsClosed {p : PlanePoint | |p.2| ≤ 1} :=
      isClosed_le hy.abs continuous_const
    rw [rectangleCarrier]
    change MeasurableSet
    ({p : PlanePoint | -c.chord / 2 ≤ p.1} ∩
     {p : PlanePoint | p.1 ≤ c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
    exact (hxmin.inter (hxmax.inter hybound)).measurableSet
  have hleft : MeasurableSet c.leftCapCarrier := by
    have hdisk : IsClosed {p : PlanePoint |
      (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} :=
      isClosed_le (((hx.sub continuous_const).pow 2).add (hy.pow 2))
      continuous_const
    have hxbound : IsClosed {p : PlanePoint | p.1 ≤ -c.chord / 2} :=
      isClosed_le hx continuous_const
    have hybound : IsClosed {p : PlanePoint | |p.2| ≤ 1} :=
      isClosed_le hy.abs continuous_const
    rw [leftCapCarrier]
    change MeasurableSet
    ({p : PlanePoint |
      (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
     {p : PlanePoint | p.1 ≤ -c.chord / 2} ∩
     {p : PlanePoint | |p.2| ≤ 1})
    exact (hdisk.inter (hxbound.inter hybound)).measurableSet
  have hright : MeasurableSet c.rightCapCarrier := by
    have hdisk : IsClosed {p : PlanePoint |
      (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} :=
      isClosed_le (((hx.sub continuous_const).pow 2).add (hy.pow 2))
      continuous_const
    have hxbound : IsClosed {p : PlanePoint | c.chord / 2 ≤ p.1} :=
      isClosed_le continuous_const hx
    have hybound : IsClosed {p : PlanePoint | |p.2| ≤ 1} :=
      isClosed_le hy.abs continuous_const
    rw [rightCapCarrier]
    change MeasurableSet
    ({p : PlanePoint |
      (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2} ∩
     {p : PlanePoint | c.chord / 2 ≤ p.1} ∩
     {p : PlanePoint | |p.2| ≤ 1})
    exact (hdisk.inter (hxbound.inter hybound)).measurableSet
  exact (hrect.union hleft).union hright

end StripCore
```

StripCore.radius

Classification: supplement

Source: CMVGeometry.lean:426-428

Declaration SHA-256: f23d45618cc8f0a402155f6b0b9c9bed86df54007d8b97c4f4027753ebfcd373

Mathematical statement. For a strip core of curvature k , define

$$R = 1/k.$$

The constraint

$$0 < k \leq 1$$

ensures $R \geq 1$, enough to span the strip height.

```
def radius (c : StripCore) : ℝ := 1 / c.curvature

/-- Half-central-angle of either strip arc. -/
```

StripCore.radius_pos

Classification: supplement

Source: CMVGeometry.lean:460-462

Declaration SHA-256: 754f25da7343b067aca1ccf07fea956e09bfd461a16a072435b6b938ea98f462

Mathematical statement. Every strip core has positive radius.

The reciprocal curvature is positive because the constructor requires strictly positive curvature.

```
theorem radius_pos (c : StripCore) : 0 < c.radius :=
  one_div_pos.mpr c.curvature_pos
```

StripCore.rectangleCarrier

Classification: supplement

Source: CMVGeometry.lean:437-439

Declaration SHA-256: 5204ba429abcba2c9a7bfd26b44272d1068d5b57b8a97a88331831661e9408f9

Mathematical statement. The core rectangle is

$$\{(x, y): -L/2 \leq x \leq L/2, |y| \leq 1\}.$$

It spans both attaching chords and has Euclidean area $2L$.

```
def rectangleCarrier (c : StripCore) : Set PlanePoint :=
  {p | -c.chord / 2 ≤ p.1 ∧ p.1 ≤ c.chord / 2 ∧ |p.2| ≤ 1}
```

StripCore.rightArcTrace

Classification: supplement

Source: CMVGeometry.lean:767-770

Declaration SHA-256: 059982ba84229cdd58242cd7f98720be73ad274e207529f914e522d73ba5cbeb

Mathematical statement. The right exposed trace is

$$\{(x, y): (x - x_R)^2 + y^2 = R^2, x \geq L/2, |y| \leq 1\}.$$

The right cutoff chooses the exterior circle arc, not the circle portion hidden inside the core.

```
def rightArcTrace (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ^
    c.chord / 2 ≤ p.1 ^ |p.2| ≤ 1}
```

StripCore.rightCapCarrier

Classification: supplement

Source: CMVGeometry.lean:444-448

Declaration SHA-256: 0f7231f5e97656f30e8e5131e2afc098206a3ff9953e2cd02ae8207ac4d54b12

Mathematical statement. The right core cap is

$$\{(x, y): (x - x_R)^2 + y^2 \leq R^2, x \geq L/2, |y| \leq 1\}.$$

It is the reflected disk segment on the right side of the core rectangle.

```
def rightCapCarrier (c : StripCore) : Set PlanePoint :=
  {p | (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ^
    c.chord / 2 ≤ p.1 ^ |p.2| ≤ 1}

/-- Genuine coordinate carrier of the retained central region. -/
```

StripCore.rightCenterX

Classification: supplement

Source: CMVGeometry.lean:434-436

Declaration SHA-256: ff1acc9e9b3ef170758d23460b0f614f6eb1c034ebd024b460f189e9f303f923

Mathematical statement. Define the right circle center coordinate

$$x_R = L/2 - R \cos \beta.$$

This is the reflection of the left center, ensuring vertical reflection symmetry of the core.

```
def rightCenterX (c : StripCore) : ℝ :=
  c.chord / 2 - c.radius * cos c.sideAngle
```

StripCore.sideAngle

Classification: supplement

Source: CMVGeometry.lean:429-430

Declaration SHA-256: 4f6b6599037149db7ce42f881c45d0eb7f35fbc43f124df7507d6a83a6d6e15b

Mathematical statement. Define the strip-core side half-angle

$$\beta = \arcsin k.$$

It lies in $(0, \pi/2]$, with the closed endpoint corresponding to semicircular side arcs.

```
def sideAngle (c : StripCore) : ℝ := arcsin c.curvature
```

StripCore.sideAngle_le_pi_div_two

Classification: supplement

Source: CMVGeometry.lean:466-469

Declaration SHA-256: 0c0857ab96c23693b4448efce2e70a6069dda6f460cf0118b7e365f0f1074987

Mathematical statement. Every strip core has

$$\beta \leq \pi/2.$$

The arcsine range supplies this closed upper bound, including curvature one.

```
theorem sideAngle_le_pi_div_two (c : StripCore) :
  c.sideAngle ≤ π / 2 := by
  exact Real.arcsin_le_pi_div_two c.curvature
```

StripCore.sideAngle_lt_pi

Classification: supplement

Source: CMVGeometry.lean:470-473

Declaration SHA-256: 4d165924e20daf34cd0cd3a93af12e6637e356347b723990beb9d1a85af59a9e

Mathematical statement. Every strip core has

$$\beta < \pi.$$

Combine the arcsine upper bound with

$$\pi/2 < \pi,$$

allowing use of the full cap-angle infrastructure.

```
theorem sideAngle_lt_pi (c : StripCore) : c.sideAngle < π := by
  have hp := Real.pi_pos
  linarith [c.sideAngle_le_pi_div_two]
```

StripCore.sideAngle_pos

Classification: supplement

Source: CMVGeometry.lean:463-465

Declaration SHA-256: 20886da5ba412dbc0666239218b14d32a1bad68cf7a0f4837b3f6216a46aefbd

Mathematical statement. Every strip core has

$$\beta > 0.$$

Arcsine is positive on positive curvature parameters.

```
theorem sideAngle_pos (c : StripCore) : 0 < c.sideAngle := by
  simpa [sideAngle] using (Real.arcsin_pos.mpr c.curvature_pos)
```

StripCore.sin_sideAngle

Classification: supplement

Source: CMVGeometry.lean:474-477

Declaration SHA-256: e85f9cb3c0c2b22c1ce74fdc60660033205e7a0821e76cfdd7248571352ea184

Mathematical statement. For a strip core,

$$\sin \beta = k.$$

The curvature lies in the inverse-trigonometric validity interval.

```
theorem sin_sideAngle (c : StripCore) : sin c.sideAngle = c.curvature := by
  rw [sideAngle]
  exact Real.sin_arcsin (by linarith [c.curvature_pos]) c.curvature_le_one
```

StripCore.upperChordTrace

Classification: supplement

Source: CMVGeometry.lean:771-773

Declaration SHA-256: 71db3f396f38ac0c4ed32b75273f08b0092359d4c0fdd7bc77976000e7791fb0

Mathematical statement. The upper core chord is

$$\{(x, 1) : |x| \leq L/2\}.$$

It is part of the core frontier before an upper cap is attached.

```
def upperChordTrace (c : StripCore) : Set PlanePoint :=
  {p | p.2 = 1 ∧ |p.1| ≤ c.chord / 2}
```

StripCore.volume_leftCapCarrier

Classification: supplement

Source: CMVGeometry.lean:2076-2138

Declaration SHA-256: ee7dbe308fa5b9feabca0b7d1f31d2c91cdf7e530a43dcc64a52dce7563fb7b8

Mathematical statement. The planar volume of the left cap is

$$4a(\beta),$$

in the nonnegative extended reals.

A coordinate transformation reduces the vertical chord of length two to the circular-segment volume formula.

```
lemma volume_leftCapCarrier (c : StripCore) :
  volume c.leftCapCarrier = ENNReal.ofReal (4 * area c.sideAngle) := by
  let U : Set PlanePoint :=
    {p | (p.1 - 0) ^ 2 + (p.2 - c.leftCenterX) ^ 2 ≤ c.radius ^ 2 ∧
      p.2 ≤ c.leftCenterX - c.radius * cos c.sideAngle}
  have hcos : 0 ≤ cos c.sideAngle := by
    rw [c.cos_sideAngle]
    positivity
  have hpre : Prod.swap ⁻¹' U = c.leftCapCarrier := by
    ext p
    change ((p.2 - 0) ^ 2 + (p.1 - c.leftCenterX) ^ 2 ≤ c.radius ^ 2 ∧
      p.1 ≤ c.leftCenterX - c.radius * cos c.sideAngle) ↔
      ((p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ∧
      p.1 ≤ -c.chord / 2 ∧ |p.2| ≤ 1)
  have hbase : c.leftCenterX - c.radius * cos c.sideAngle = -c.chord / 2 := by
    rw [leftCenterX]
    ring
  rw [hbase]
  constructor
  · rintro ⟨hdisk, hx⟩
    have hdx : p.1 - c.leftCenterX ≤ -(c.radius * cos c.sideAngle) := by
      rw [leftCenterX]
      linarith
    have hdx0 : p.1 - c.leftCenterX ≤ 0 := by
      nlinarith [c.radius_pos]
    have hsquare : (c.radius * cos c.sideAngle) ^ 2 ≤
      (p.1 - c.leftCenterX) ^ 2 := by
      apply sq_le_sq.mpr
      rw [abs_of_nonneg (mul_nonneg c.radius_pos.le hcos), abs_of_nonpos hdx0]
      linarith
    have hy2 : p.2 ^ 2 ≤ 1 := by
      nlinarith [radius_sq_cos_sq' c]
    have hyabs : |p.2| ≤ |(1 : ℝ)| := sq_le_sq.mp (by simpa using hy2)
    exact ⟨by nlinarith, hx, by simpa using hyabs⟩
  · rintro ⟨hdisk, hx, _hy⟩
    exact ⟨by nlinarith, hx⟩
  have hU : MeasurableSet U := by
```

```

change MeasurableSet
  ({p : PlanePoint |
    (p.1 - 0) ^ 2 + (p.2 - c.leftCenterX) ^ 2 ≤ c.radius ^ 2} ∩
   {p : PlanePoint |
    p.2 ≤ c.leftCenterX - c.radius * cos c.sideAngle})
exact (isClosed_le (by fun_prop) continuous_const).measurableSet.inter
      (isClosed_le continuous_snd continuous_const).measurableSet
have hmeasure : volume c.leftCapCarrier = volume U := by
  rw [- hpre]
  exact Measure.measurePreserving_swap.measure_preimage hU.nullMeasurableSet
have hreal : c.radius ^ 2 *
  (c.sideAngle - sin c.sideAngle * cos c.sideAngle) =
  4 * area c.sideAngle := by
  rw [area]
  have hs : sin c.sideAngle = c.curvature := c.sin_sideAngle
  rw [hs]
  rw [radius]
  field_simp [ne_of_gt c.curvature_pos]
calc
  volume c.leftCapCarrier = volume U := hmeasure
  _ = ENNReal.ofReal (c.radius ^ 2 *
    (c.sideAngle - sin c.sideAngle * cos c.sideAngle)) := by
    simp [U] using volume_lower_circular_segment 0 c.leftCenterX
      c.radius c.sideAngle c.radius_pos c.sideAngle_pos.le c.sideAngle_lt_pi.le
  _ = ENNReal.ofReal (4 * area c.sideAngle) := congrArg ENNReal.ofReal hreal

```

StripCore.volume_rectangleCarrier

Classification: supplement

Source: CMVGeometry.lean:2060-2075

Declaration SHA-256: b2c1d8716422a2e0169b6aaca489790c14b0594bc75f1e77b6bbec1349d5110

Mathematical statement. The planar volume of the core rectangle is $2L$, in the nonnegative extended reals.

Its side lengths are L and two, with interval endpoints having zero planar measure.

```

lemma volume_rectangleCarrier (c : StripCore) :
  volume c.rectangleCarrier = ENNReal.ofReal (2 * c.chord) := by
  have hset : c.rectangleCarrier =
    Icc (-c.chord / 2) (c.chord / 2) ×ˢ Icc (-1 : ℝ) 1 := by
    ext p
    simp only [rectangleCarrier, mem_setOf_eq, mem_prod, mem_Icc]
  constructor
  · rintro ⟨hx1, hx2, hy⟩
    exact ⟨⟨hx1, hx2⟩, abs_le.mp hy⟩
  · rintro ⟨⟨hx1, hx2⟩, hy1, hy2⟩
    exact ⟨hx1, hx2, abs_le.mpr ⟨hy1, hy2⟩⟩
  rw [hset, Measure.volume_eq_prod, Measure.prod_prod,
    Real.volume_Icc, Real.volume_Icc,
    show c.chord / 2 - -c.chord / 2 = c.chord by ring]
  norm_num [ENNReal.ofReal_mul, c.chord_pos.le, mul_comm]

```

StripCore.volume_rightCapCarrier

Classification: supplement

Source: CMVGeometry.lean:2139-2201

Declaration SHA-256: 7bc214af5d8d445665e416853a1fab80112ca5fe1116500de0e9b2b6285dfdf7

Mathematical statement. The planar volume of the right cap is

$$4a(\beta),$$

in the nonnegative extended reals.

The reflected coordinate computation gives the same area as the left cap.

```

lemma volume_rightCapCarrier (c : StripCore) :
  volume c.rightCapCarrier = ENNReal.ofReal (4 * area c.sideAngle) := by
let U : Set PlanePoint :=
  {p | (p.1 - 0) ^ 2 + (p.2 - c.rightCenterX) ^ 2 ≤ c.radius ^ 2 ∧
    c.rightCenterX + c.radius * cos c.sideAngle ≤ p.2}
have hcos : 0 ≤ cos c.sideAngle := by
  rw [c.cos_sideAngle]
  positivity
have hpre : Prod.swap ⁻¹' U = c.rightCapCarrier := by
  ext p
  change ((p.2 - 0) ^ 2 + (p.1 - c.rightCenterX) ^ 2 ≤ c.radius ^ 2 ∧
    c.rightCenterX + c.radius * cos c.sideAngle ≤ p.1) ↔
    ((p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 ≤ c.radius ^ 2 ∧
    c.chord / 2 ≤ p.1 ∧ |p.2| ≤ 1)
  have hbase : c.rightCenterX + c.radius * cos c.sideAngle = c.chord / 2 := by
    rw [rightCenterX]
    ring
  rw [hbase]
  constructor
  · rintro ⟨hdisk, hx⟩
    have hdx : c.radius * cos c.sideAngle ≤ p.1 - c.rightCenterX := by
      rw [rightCenterX]
      linarith
    have hdx0 : 0 ≤ p.1 - c.rightCenterX := by
      nlinarith [c.radius_pos]
    have hsquare : (c.radius * cos c.sideAngle) ^ 2 ≤
      (p.1 - c.rightCenterX) ^ 2 := by
      apply sq_le_sq.mpr
      rw [abs_of_nonneg (mul_nonneg c.radius_pos.le hcos), abs_of_nonneg hdx0]
      exact hdx
    have hy2 : p.2 ^ 2 ≤ 1 := by
      nlinarith [radius_sq_cos_sq' c]
    have hyabs : |p.2| ≤ |(1 : ℝ)| := sq_le_sq.mp (by simpa using hy2)
    exact ⟨by nlinarith, hx, by simpa using hyabs⟩
  · rintro ⟨hdisk, hx, _hy⟩
    exact ⟨by nlinarith, hx⟩
have hU : MeasurableSet U := by
  change MeasurableSet
  ({p : PlanePoint |
    (p.1 - 0) ^ 2 + (p.2 - c.rightCenterX) ^ 2 ≤ c.radius ^ 2} ∩
  {p : PlanePoint |
    c.rightCenterX + c.radius * cos c.sideAngle ≤ p.2})
  exact (isClosed_le (by fun_prop) continuous_const).measurableSet.inter
    (isClosed_le continuous_const continuous_snd).measurableSet
have hmeasure : volume c.rightCapCarrier = volume U := by
  rw [hpre]
  exact Measure.measurePreserving_swap.measure_preimage hU.nullMeasurableSet
have hreal : c.radius ^ 2 *
  (c.sideAngle - sin c.sideAngle * cos c.sideAngle) =
  4 * area c.sideAngle := by
  rw [area]
  have hs : sin c.sideAngle = c.curvature := c.sin_sideAngle
  rw [hs]
  rw [radius]
  field_simp [ne_of_gt c.curvature_pos]
calc
  volume c.rightCapCarrier = volume U := hmeasure
  _ = ENNReal.ofReal (c.radius ^ 2 *
    (c.sideAngle - sin c.sideAngle * cos c.sideAngle)) := by
    simpa [U] using volume_upper_circular_segment 0 c.rightCenterX
      c.radius c.sideAngle c.radius_pos c.sideAngle_pos.le c.sideAngle_lt_pi.le
  _ = ENNReal.ofReal (4 * area c.sideAngle) := congrArg ENNReal.ofReal hreal

```

TypeThreeAssembly

Classification: supplement

Source: TypeThreeAssembly.lean:29-35

Declaration SHA-256: ba80fb031079587a301c08eeb69ef793b83ab9e2bcde30a061c829264b7e1959

Mathematical statement. A three-arc assembly consists of real λ, h with $\lambda > 1$ and

$$0 < h < 1.$$

All other geometric data are determined by these parameters. Throughout these entries use

$$\begin{aligned} R &= 1/h, \\ q &= 2h - 1, \\ \beta &= \arccos q, \\ \alpha &= \arccos(q/\lambda), \\ \phi &= \pi - \beta, \end{aligned}$$

and

$$b = R(\sin \alpha - \sin \beta).$$

```
structure TypeThreeAssembly (lam : ℝ) where
  h : ℝ
  density_jump : 1 < lam
  h_pos : 0 < h
  h_lt_one : h < 1

/-- The three geometric branches of the source type-(iii) upper arc. -/
```

TypeThreeAssembly.WeightedArea

Classification: supplement

Source: TypeThreeAssembly.lean:265-268

Declaration SHA-256: 5d054747c494e793da416a19e2d895420eb9e121366a5645e79b7ab53a0c3c7f

Mathematical statement. The assembly weighted area is the Lebesgue weighted-area integral over its actual carrier.

It is distinct by definition from the modeled scalar sum, which is identified by a theorem.

```
def WeightedArea : ℝ := _root_.WeightedArea lam a.carrier

/-- Density integral against the complete topological frontier of the
coordinate carrier. -/
```

TypeThreeAssembly.WeightedPerimeter

Classification: supplement

Source: TypeThreeAssembly.lean:269-272

Declaration SHA-256: ac2df4f2daf804e358c901d1a6b496eaadc35076745e389cd399733fc5dead10

Mathematical statement. The assembly weighted perimeter is the density integral against the canonical complete-frontier measure of its carrier.

Unlike the auxiliary four-arc parametrized perimeter, this definition starts directly with the geometric frontier.

```
def WeightedPerimeter : ℝ :=
  _root_.WeightedPerimeter lam (FrontierMeasure a.carrier)
```

```
-- The horizontal expansion between the two translated side half-caps. -/
```

TypeThreeAssembly.angleTerm

Classification: supplement

Source: TypeThreeAssembly.lean:289-291

Declaration SHA-256: 8089feb5fd033f48e23e1ab4579cce22afbbbfde128400123f3b8b2b2a245332

Mathematical statement. Define the angle combination

$$T = \lambda\alpha + \pi - \beta.$$

Using

$$\pi - \arccos q = \pi/2 + \arcsin q,$$

this equals

$$B(q) + \pi/2.$$

```
def angleTerm : ℝ := lam * a.outerAngle + π - a.innerAngle

-- The horizontal sine displacement between the upper and side arcs. -/
```

TypeThreeAssembly.bottomChord

Classification: supplement

Source: TypeThreeAssembly.lean:136-139

Declaration SHA-256: 08cc63d3aedc396b5efa9b27fd1bac931beb8b7d14cbce558e5fde942be277c2

Mathematical statement. Define the bottom chord length as $2b$.

The construction allows this nonnegative length to vanish at the semicircular parameter.

```
def bottomChord : ℝ := 2 * a.sideHalfWidth

-- Finite branch selection; it does not assert membership in a later
area-fold branch. -/
```

TypeThreeAssembly.bottomChord_nonneg

Classification: supplement

Source: TypeThreeAssembly.lean:352-354

Declaration SHA-256: bbe5a0b68c27b1eab70889a99796728aff04d58c47eda0333c16191c78a8f12a

Mathematical statement. For every assembly, the bottom chord length $2b$ is nonnegative.

This permits a degenerate segment without excluding the disk case.

```
theorem bottomChord_nonneg : 0 ≤ a.bottomChord := by
  exact mul_nonneg (by norm_num) a.sideHalfWidth_nonneg
```

TypeThreeAssembly.bottomChord_pos

Classification: supplement

Source: TypeThreeAssembly.lean:359-362

Declaration SHA-256: 5deb1da8ce95e59329d432d4f1484bad3b915566f05bd8a9e6b37a69ad4de177

Mathematical statement. If $h \neq 1/2$, the bottom chord has strictly positive length.

The positive factor two preserves strict positivity of the side half-width.

```
theorem bottomChord_pos (hh : a.h ≠ 1 / 2) : 0 < a.bottomChord := by
  exact mul_pos (by norm_num) (a.sideHalfWidth_pos hh)
```

TypeThreeAssembly.bottomSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:132-135

Declaration SHA-256: 18e70891a1386b33220d27cc96af45d0ffdfcc88b970fd467c64035c04da589f5

Mathematical statement. Define the bottom segment

$$\{(x, -1) : |x| \leq b\}.$$

It is retained also for $b = 0$, when it is a singleton rather than an empty set.

```
def bottomSegmentCarrier : Set PlanePoint :=
  {p | p.2 = -1} ∩ {p | |p.1| ≤ a.sideHalfWidth}

/-- Bottom-segment length, including the zero-length semicircular transition. -/
```

TypeThreeAssembly.bottomSegmentCarrier_subset_frontier

Classification: supplement

Source: TypeThreeAssembly.lean:1058-1079

Declaration SHA-256: 82d466e4484f4553cdf0fe29df6958b297f06ff314377fa09237775f676763e1

Mathematical statement. The entire bottom segment is contained in the topological frontier of the assembled carrier.

Every segment point belongs to the closed carrier and can be approached from below by points outside it, including in the zero-length case.

```
theorem bottomSegmentCarrier_subset_frontier :
  a.bottomSegmentCarrier ⊆ frontier a.carrier := by
  intro p hp
  rw [mem_frontier_iff_notMem_interior
    (a.bottomSegmentCarrier_subset_carrier hp)]
  intro hinterior
  have hnhds : interior a.carrier ∈ nhds p :=
    isOpen_interior.mem_nhds hinterior
  rcases Metric.mem_nhds_iff.mp hnhds with
    (epsilon, hepsilon, hball)
  have hhalf : 0 < epsilon / 2 := div_pos hepsilon (by norm_num)
  have hnear :
    (p.1, p.2 - epsilon / 2) ∈ Metric.ball p epsilon := by
    rw [Metric.mem_ball, Prod.dist_eq, dist_self, Real.dist_eq,
      show p.2 - epsilon / 2 - p.2 = -(epsilon / 2) by ring,
      abs_neg, abs_of_pos hhalf, max_eq_right hhalf.le]
    linarith
  have hmember :
    (p.1, p.2 - epsilon / 2) ∈ a.carrier :=
    interior_subset (hball hnear)
  exact a.below_bottomSegmentCarrier_not_mem hp hhalf hmember
```

TypeThreeAssembly.boundaryJoinSet

Classification: supplement

Source: TypeThreeAssembly.lean:1130-1135

Declaration SHA-256: 4d107bd731d106706191e0cc66c1eb6e52b0ad83df797a9379d71e46e9ea68c7

Mathematical statement. Define the junction set

$$\{(-R\sin\alpha, 1), (R\sin\alpha, 1), (-b, -1), (b, -1)\}.$$

It records possible intersections of the four frontier pieces, with coincident bottom junctions permitted.

```
def boundaryJoinSet : Set PlanePoint :=
  {(-a.radius * sin a.outerAngle, 1),
   (a.radius * sin a.outerAngle, 1),
   (-a.sideHalfWidth, -1),
   (a.sideHalfWidth, -1)}
```

TypeThreeAssembly.boundaryJoinSet_finite

Classification: supplement

Source: TypeThreeAssembly.lean:1136-1141

Declaration SHA-256: fb8b2d3307f8905b1616f7f8934468dae8dc2d1df098e8fa72f92f5a3759db5d

Mathematical statement. The three-arc junction set is finite.

It consists of at most four explicitly specified points.

```
theorem boundaryJoinSet_finite : a.boundaryJoinSet.Finite := by
  simp [boundaryJoinSet]

/-- Every pairwise overlap of the four primitive frontier traces is a boundary
join. In particular, when `h = 1 / 2`, the two side arcs meet only at the
shared bottom point. -/
```

TypeThreeAssembly.boundaryTrace

Classification: supplement

Source: TypeThreeAssembly.lean:1595-1599

Declaration SHA-256: 77ded3af26dffe034b843162e20202cd8e868c69330a0cb95819f9e3b2a0bbf5

Mathematical statement. Define the boundary trace as the union of the left arc, right arc, upper cap arc, and bottom segment.

This set includes every exposed piece but excludes the upper attaching chord interior.

```
def boundaryTrace : Set PlanePoint :=
  a.leftArcTrace ∪
  (a.rightArcTrace ∪
   (OneSidedCircularCap.arcTrace a.outerCap ∪ a.bottomSegmentCarrier))
```

TypeThreeAssembly.boundaryTrace_subset_frontier_carrier

Classification: supplement

Source: TypeThreeAssembly.lean:2840-2847

Declaration SHA-256: daeda01e39ff76764527644f12850660eec76e1d4031dd8579c24a3b55dda6ba

Mathematical statement. Every point of the declared three-arc boundary trace is a frontier point of the carrier.

Each exposed arc and the bottom segment is shown to have exterior points arbitrarily nearby.

```
theorem boundaryTrace_subset_frontier_carrier :
  a.boundaryTrace  $\subseteq$  frontier a.carrier := by
  rintro p (hleft | hright | hupper | hbottom)
  · exact a.leftArc_mem_frontier hleft
  · exact a.rightArc_mem_frontier hright
  · exact a.upperArc_mem_frontier hupper
  · exact a.bottomSegmentCarrier_subset_frontier hbottom
```

TypeThreeAssembly.carrier

Classification: supplement

Source: TypeThreeAssembly.lean:262-264

Declaration SHA-256: 0849c97e398a59f917657f18205fab9ffd6aa8e524ee751befd78f1c4735a4ff

Mathematical statement. The three-arc carrier is its strip core union its upper cap.

The bottom segment is part of its frontier, not a separate area-bearing component.

```
def carrier : Set PlanePoint := a.coreCarrier  $\cup$  a.outerCap.carrier

/-- Ambient weighted-area integral of the coordinate carrier. -/
```

TypeThreeAssembly.continuous_realizedLeftArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:690-699

Declaration SHA-256: 5aa13a8e0a54ece98427e269f77eeffa81d89485ad9d1f32dde5c0e6f344ef9b4

Mathematical statement. The realized left arc parametrization is continuous.

Continuity follows from its trigonometric coordinates and the coordinate homeomorphism.

```
lemma continuous_realizedLeftArcParam :
  Continuous a.realizedLeftArcParam := by
  rw [show a.realizedLeftArcParam =
    sideArcComplexIsometryMap a.leftCenter  $\circ$ 
    unitCircleArc a.radius by
    funext s
    exact a.realizedLeftArcParam_eq s]
  exact (isometry_sideArcComplexIsometryMap a.leftCenter).continuous.comp
    (continuous_unitCircleArc a.radius)
```

TypeThreeAssembly.continuous_realizedRightArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:700-709

Declaration SHA-256: 3d9c0740b38c1fa009799b8c35785a0cb767211b94d4ca0bc37637a119a58afd

Mathematical statement. The realized right arc parametrization is continuous.

Continuity follows from its trigonometric coordinates and the coordinate homeomorphism.

```
lemma continuous_realizedRightArcParam :
  Continuous a.realizedRightArcParam := by
  rw [show a.realizedRightArcParam =
    sideArcComplexIsometryMap a.rightCenter  $\circ$ 
    unitCircleArc a.radius by
    funext s
    exact a.realizedRightArcParam_eq s]
```

```
exact (isometry_sideArcComplexIsometryMap a.rightCenter).continuous.comp
  (continuous_unitCircleArc a.radius)
```

TypeThreeAssembly.coreCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:99-103

Declaration SHA-256: 5e82c9d38bf8a70ad9c2ae143c293bbf0238ee48148c3dd2d922c95471160878

Mathematical statement. The core carrier is the union of the rectangle and both clipped disk segments.

Its horizontal-section description is later identified with the literal three-arc core inequality.

```
def coreCarrier : Set PlanePoint :=
  a.rectangleCarrier ∪ a.leftSegmentCarrier ∪ a.rightSegmentCarrier

/-- The exact left circular trace of the type-(iii) strip-side boundary. -/
```

TypeThreeAssembly.coreCarrier_y_bounds

Classification: supplement

Source: TypeThreeAssembly.lean:1447-1455

Declaration SHA-256: a0a8c181bd45290d862033f8524f84e76d590a21a370fbbc0214aab783d9df85

Mathematical statement. Every point of the three-arc core satisfies $|y| \leq 1$.

Each component is explicitly clipped to the closed strip.

```
theorem coreCarrier_y_bounds {p : PlanePoint}
  (hp : p ∈ a.coreCarrier) : |p.2| ≤ 1 := by
  rcases hp with (hrect | hleft) | hright
  · exact abs_le.mpr ⟨hrect.2.1, hrect.2.2⟩
  · exact abs_le.mpr hleft.2.2
  · exact abs_le.mpr hright.2.2

/-- Above the source strip, the genuine type-(iii) carrier is exactly its
actual exterior cap; none of the three core pieces can occur there. -/
```

TypeThreeAssembly.cos_innerAngle

Classification: supplement

Source: TypeThreeAssembly.lean:178-183

Declaration SHA-256: e774d0a2578a71402d098d5ac027f2d6d1dbb90425ca111832168cfb9f013412

Mathematical statement. For every assembly,

$$\cos\beta = q.$$

The inverse cosine is evaluated within its valid argument interval.

```
theorem cos_innerAngle : cos a.innerAngle = a.shapeParameter := by
  rw [innerAngle, Real.cos_arccos a.shapeParameter_mem.1.le
    a.shapeParameter_mem.2.le]

/-- The principal-angle definitions recover the Snell-adjusted exterior
argument exactly. -/
```

TypeThreeAssembly.cos_outerAngle

Classification: supplement

Source: TypeThreeAssembly.lean:184-188

Declaration SHA-256: 3eb066bbadef1d5d0722d2c322b7cfe8bce8e1eb62204f2872b1f8f1bbb41a24

Mathematical statement. For every assembly,

$$\cos\alpha = q/\lambda.$$

This realizes the prescribed refraction relation in cosine coordinates.

```
theorem cos_outerAngle : cos a.outerAngle = a.outerArgument := by
  rw [outerAngle, Real.cos_arccos a.outerArgument_mem.1.le
    a.outerArgument_mem.2.le]

/-- CMV's type-(iii) incidence equation, derived rather than assumed. -/
```

TypeThreeAssembly.cos_sideArcAngle

Classification: supplement

Source: TypeThreeAssembly.lean:231-237

Declaration SHA-256: a8f9a8c72dcee4cc4dc59b4da10f2ae6d2741d7865a4b2faca10168fa17cc76a

Mathematical statement. For every assembly,

$$\cos\phi = -q.$$

Cosine changes sign under supplementation, and

$$\cos\beta = q.$$

```
theorem cos_sideArcAngle :
  cos a.sideArcAngle = -a.shapeParameter := by
  rw [sideArcAngle, Real.cos_pi_sub, a.cos_innerAngle]

/-- A full lower cap with the same radius and angle as the two translated
half-caps forming the strip-side pieces. Its area and arc length are exactly
the sums of those two congruent halves. -/
```

TypeThreeAssembly.exact_euclidean_leftArcParam_hausdorffMeasure

Classification: supplement

Source: TypeThreeAssembly.lean:742-778

Declaration SHA-256: c6dc496bb93e612df391bdcd13229ada6ddb03a05f7a4a85318498972631a8fc

Mathematical statement. The Euclidean Hausdorff measure of the realized left parameter image on $[-R\phi, 0]$ is

$$R\phi.$$

The parameter span is positive and less than a full circumference. Apply the general shifted-circle length theorem through the center isometry.

```
theorem exact_euclidean_leftArcParam_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
  (a.realizedLeftArcParam ''
    Icc (-a.radius * a.sideArcAngle) 0) =
  ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  have hfun : a.realizedLeftArcParam =
    sideArcComplexIsometryMap a.leftCenter ∘
    unitCircleArc a.radius := by
  funext s
  exact a.realizedLeftArcParam_eq s
```

```

have huv : -a.radius * a.sideArcAngle < 0 := by
  nlinarith [a.radius_pos, a.sideArcAngle_pos]
have hspan :
  0 - (-a.radius * a.sideArcAngle) < 2 * π * a.radius := by
  have hangle : a.sideArcAngle < 2 * π := by
    linarith [a.sideArcAngle_lt_pi, Real.pi_pos]
  have hmul :=
    mul_lt_mul_of_pos_left hangle a.radius_pos
  nlinarith
calc
_ = (μH[1] : Measure EuclideanPlane)
  (sideArcComplexIsometryMap a.leftCenter ''
    (unitCircleArc a.radius ''
      Icc (-a.radius * a.sideArcAngle) 0)) := by
  rw [hfun]
  congr 1
  simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
_ = (μH[1] : Measure ℂ)
  (unitCircleArc a.radius ''
    Icc (-a.radius * a.sideArcAngle) 0) :=
  (isometry_sideArcComplexIsometryMap a.leftCenter).hausdorffMeasure_image
  (Or.inl (by norm_num)) _
_ = ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  simp using
    (hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi
      a.radius_pos huv hspan)

```

TypeThreeAssembly.exact_euclidean_leftArcTrace_hausdorffMeasure

Classification: supplement

Source: TypeThreeAssembly.lean:817-824

Declaration SHA-256: 9b484f3710815971495ad59b2a292d470a94ac2f3cad3edb96b8870979733b3

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the left three-arc trace is

$$R\phi.$$

Use the exact realized parameter-image equality and its Hausdorff measure formula.

```

theorem exact_euclidean_leftArcTrace_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' a.leftArcTrace) =
    ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  rw [← a.realizedLeftArcParam_image_Icc]
  exact a.exact_euclidean_leftArcParam_hausdorffMeasure

/-- Exact Euclidean  $\mathcal{H}^1$  mass of the right strip-side trace. -/

```

TypeThreeAssembly.exact_euclidean_rightArcParam_hausdorffMeasure

Classification: supplement

Source: TypeThreeAssembly.lean:779-816

Declaration SHA-256: f1f573c6c931308699d4611129bc237b3c89fe21d3a82fbb640a2bc4f452f054

Mathematical statement. The Euclidean Hausdorff measure of the realized right parameter image on $[0, R\phi]$ is

$$R\phi.$$

The parameter span is positive and less than a full circumference. Apply the general shifted-circle length theorem through the center isometry.

```

theorem exact_euclidean_rightArcParam_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
    (a.realizedRightArcParam ''
      Icc 0 (a.radius * a.sideArcAngle)) =

```

```

    ENNReal.ofReal (a.radius * a.sideArcAngle) := by
have hfun : a.realizedRightArcParam =
  sideArcComplexIsometryMap a.rightCenter ◦
    unitCircleArc a.radius := by
  funext s
  exact a.realizedRightArcParam_eq s
have huv : 0 < a.radius * a.sideArcAngle :=
  mul_pos a.radius_pos a.sideArcAngle_pos
have hspan :
  a.radius * a.sideArcAngle - 0 < 2 * π * a.radius := by
  have hangle : a.sideArcAngle < 2 * π := by
    linarith [a.sideArcAngle_lt_pi, Real.pi_pos]
  have hmul :=
    mul_lt_mul_of_pos_left hangle a.radius_pos
  nlinarith
calc
_ = (μH[1] : Measure EuclideanPlane)
  (sideArcComplexIsometryMap a.rightCenter ''
    (unitCircleArc a.radius ''
      Icc 0 (a.radius * a.sideArcAngle))) := by
  rw [hfun]
  congr 1
  simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
_ = (μH[1] : Measure ℂ)
  (unitCircleArc a.radius ''
    Icc 0 (a.radius * a.sideArcAngle)) :=
  (isometry_sideArcComplexIsometryMap a.rightCenter).hausdorffMeasure_image
  (Or.inl (by norm_num)) _
_ = ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  simp using
    (hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi
      a.radius_pos huv hspan)

/-- Exact Euclidean  $H^1$  mass of the left strip-side trace. -/

```

TypeThreeAssembly.exact_euclidean_rightArcTrace_hausdorffMeasure

Classification: supplement

Source: TypeThreeAssembly.lean:825-832

Declaration SHA-256: 82deac992e3b3149a73b5a9867d35c880f6af8171bd8434cc5a883788eaac184

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the right three-arc trace is

$$R\phi.$$

Use the exact realized parameter-image equality and its Hausdorff measure formula.

```

theorem exact_euclidean_rightArcTrace_hausdorffMeasure :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' a.rightArcTrace) =
  ENNReal.ofReal (a.radius * a.sideArcAngle) := by
  rw [← a.realizedRightArcParam_image_Icc]
  exact a.exact_euclidean_rightArcParam_hausdorffMeasure

/-- The two translated strip-side half-caps have the area of `sideCap`. -/

```

TypeThreeAssembly.frontier_carrier_subset_boundaryTrace

Classification: supplement

Source: TypeThreeAssembly.lean:2443-2458

Declaration SHA-256: e4a9f2900fa53b9204f0a28ca553872a35240978918b3cf32712160073f759cb

Mathematical statement. Every frontier point of the three-arc carrier belongs to its declared boundary trace.

The coordinate proof removes interior seams and rules out boundary pieces not among the two side arcs, exterior arc or bottom segment.

```
theorem frontier_carrier_subset_boundaryTrace :
  frontier a.carrier ⊆ a.boundaryTrace := by
  intro p hp
  have htop := frontier_union_subset a.coreCarrier a.outerCap.carrier hp
  rcases htop with hcore | hcap
  · rcases a.frontier_coreCarrier_subset hcore.1 with
    hleft | hright | hchord | hbottom
    · exact Or.inl hleft
    · exact Or.inr (Or.inl hright)
    · exact a.outerChord_to_boundaryTrace hchord hp
    · exact Or.inr (Or.inr (Or.inr hbottom))
  · rw [a.outerCap.frontier_carrier] at hcap
    rcases hcap.2 with harc | hchord
    · exact Or.inr (Or.inr (Or.inl harc))
    · exact a.outerChord_to_boundaryTrace hchord hp
```

TypeThreeAssembly.innerAngle

Classification: supplement

Source: TypeThreeAssembly.lean:53-55

Declaration SHA-256: bb9acabc2bc9f5427d7041305c401b4e2793720759dd063e3153448db0793dd9

Mathematical statement. Define

$$\beta = \arccos q.$$

This angle locates the inner circle's top intersection; the exposed side-arc angle is its supplement.

```
def innerAngle : ℝ := arccos a.shapeParameter

/-- Snell-adjusted arccosine argument for the exterior angle `a₃`. -/
```

TypeThreeAssembly.innerAngle_lt_pi

Classification: supplement

Source: TypeThreeAssembly.lean:168-170

Declaration SHA-256: 31385b2dc48409bb1698d05b2b8cedcea6151f9ce4bedc38534228b77f838b5c

Mathematical statement. For every assembly,

$$\beta < \pi.$$

The shape parameter is strictly above minus one.

```
theorem innerAngle_lt_pi : a.innerAngle < π :=
  Real.arccos_lt_pi.mpr a.shapeParameter_mem.1
```

TypeThreeAssembly.innerAngle_pos

Classification: supplement

Source: TypeThreeAssembly.lean:165-167

Declaration SHA-256: e78599fffc8793999ca239c142df39a815df5b9220a19bd2bd882d169b85658b5

Mathematical statement. For every assembly,

$$\beta > 0.$$

The shape parameter is strictly below one.

```
theorem innerAngle_pos : 0 < a.innerAngle :=
  Real.arccos_pos.mpr a.shapeParameter_mem.2
```

TypeThreeAssembly.innerAngle_semicircular

Classification: supplement

Source: TypeThreeAssembly.lean:370-373

Declaration SHA-256: 136b95db3cfbe6ffba49a9d377637e6fd44e92cfd37653b368ff453e7d5af642

Mathematical statement. If $h = 1/2$, the inner angle is

$$\beta = \pi/2.$$

The shape parameter vanishes, so its inverse cosine is the right angle.

```
theorem innerAngle_semicircular (hh : a.h = 1 / 2) :
  a.innerAngle = π / 2 := by
  simp [innerAngle, shapeParameter, hh]
```

TypeThreeAssembly.innerSin_pos

Classification: supplement

Source: TypeThreeAssembly.lean:216-218

Declaration SHA-256: 88694b992a38f1dff3c67f32d082f4eb742c98f01874e62941887bfa3c13f04d

Mathematical statement. For every assembly,

$$\sin\beta > 0.$$

The interior angle has the same strict principal range.

```
theorem innerSin_pos : 0 < sin a.innerAngle :=
  sin_pos_of_pos_of_lt_pi a.innerAngle_pos a.innerAngle_lt_pi
```

TypeThreeAssembly.isClosed_bottomSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1589-1594

Declaration SHA-256: 4502cfa8cbb9cfeeb9921da9cbccfe0b4a2a7c969d9473aede4beb6ee9ada554

Mathematical statement. The bottom segment is closed, including when its length is zero.

It is the intersection of a horizontal line and a closed absolute-value sublevel set.

```
theorem isClosed_bottomSegmentCarrier : IsClosed a.bottomSegmentCarrier := by
  unfold bottomSegmentCarrier
  exact (isClosed_eq continuous_snd continuous_const).inter
    (isClosed_le continuous_fst.abs continuous_const)

/-- The exact complete trace of the type-(iii) boundary. -/
```

TypeThreeAssembly.isClosed_carrier

Classification: supplement

Source: TypeThreeAssembly.lean:1307-1309

Declaration SHA-256: 86d49c14fed38ff9d123ec7002dc853d7ab13d112817308f4d1ef5ac7d463980

Mathematical statement. The entire three-arc carrier is closed.

Take the union of its closed core and closed outer cap.

```
theorem isClosed_carrier : IsClosed a.carrier := by
  exact a.isClosed_coreCarrier.union a.outerCap.isClosed_carrier
```

TypeThreeAssembly.isClosed_coreCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1303-1306

Declaration SHA-256: c084bdc57b0172b2f03efa14200c62e0c7ef9b7e0265bac3ea60cec5c75fc390

Mathematical statement. The assembled strip core is closed.

It is a finite union of the closed rectangle and clipped disks.

```
theorem isClosed_coreCarrier : IsClosed a.coreCarrier := by
  exact (a.isClosed_rectangleCarrier.union a.isClosed_leftSegmentCarrier).union
    a.isClosed_rightSegmentCarrier
```

TypeThreeAssembly.isClosed_leftSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1289-1295

Declaration SHA-256: 9a06aa24438ad23f7c7f01c4c87ea6fc01e48b3ecec4d358799e726bea34abb1

Mathematical statement. The left clipped disk carrier is closed.

It is an intersection of a closed disk with closed half-planes.

```
theorem isClosed_leftSegmentCarrier : IsClosed a.leftSegmentCarrier := by
  unfold leftSegmentCarrier
  exact (isClosed_disk a.leftCenter a.radius).inter
    ((isClosed_le continuous_fst continuous_const).inter
      ((isClosed_le continuous_const continuous_snd).inter
        (isClosed_le continuous_snd continuous_const)))
```

TypeThreeAssembly.isClosed_rectangleCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1282-1288

Declaration SHA-256: a040fbd7853bc5d56529ed0d45def53a17db2084e9e6f891db5067b23f32a560

Mathematical statement. The core rectangle is closed.

Each of its four coordinate inequalities defines a closed half-plane.

```
theorem isClosed_rectangleCarrier : IsClosed a.rectangleCarrier := by
  unfold rectangleCarrier
  exact ((isClosed_le continuous_const continuous_fst).inter
    (isClosed_le continuous_fst continuous_const)).inter
    ((isClosed_le continuous_const continuous_snd).inter
      (isClosed_le continuous_snd continuous_const))
```

TypeThreeAssembly.isClosed_rightSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1296-1302

Declaration SHA-256: 97ec89af687cf23451a62fdd3fb4816f90150533b2e6c5aca3fc7cb1fe8e300f

Mathematical statement. The right clipped disk carrier is closed.

The right cutoff and both strip height bounds are closed conditions.

```
theorem isClosed_rightSegmentCarrier : IsClosed a.rightSegmentCarrier := by
  unfold rightSegmentCarrier
  exact (isClosed_disk a.rightCenter a.radius).inter
    ((isClosed_le continuous_const continuous_fst).inter
      ((isClosed_le continuous_const continuous_snd).inter
        (isClosed_le continuous_snd continuous_const)))
```

TypeThreeAssembly.isometry_sideArcComplexIsometryMap

Classification: supplement

Source: TypeThreeAssembly.lean:653-664

Declaration SHA-256: 5f9f98564eebaed09c3cd958295e743ceabd92f2b665793225e0fb64c4812ba4

Mathematical statement. For every center c , the side-arc complex map is an isometry.

A coordinate interchange, sign change and translation preserve the Euclidean metric.

```
theorem isometry_sideArcComplexIsometryMap (center : PlanePoint) :
  Isometry (sideArcComplexIsometryMap center) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [WithLp.prod_dist_eq_add (by norm_num)]
  simp only [sideArcComplexIsometryMap]
  norm_num [Real.dist_eq, sq_abs]
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im]
  congr 1
  ring

/-- Euclidean realization of the left signed-arclength parameter. -/
```

TypeThreeAssembly.leftArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:120-126

Declaration SHA-256: 4f29c060ae66edf060cbefb1375e17d2584dc8c159343c7bce4a1ff009d7632c

Mathematical statement. Define the left arclength parametrization

$$s \mapsto (-b + R\sin(s/R), R - 1 - R\cos(s/R)).$$

The selected parameter interval is $[-R\phi, 0]$, ending at the bottom tangency point.

```
def leftArcParam (s : ℝ) : PlanePoint :=
  (a.leftCenter.1 + a.radius * sin (s / a.radius),
   a.leftCenter.2 - a.radius * cos (s / a.radius))

/-- Signed-arclength parameter for the right strip-side half-circle. The
parameter runs from the bottom point at zero to the upper junction at
`radius * sideArcAngle`. -/
```

TypeThreeAssembly.leftArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:446-546

Declaration SHA-256: a53d3bd58a5cf0ca117ca4045079faea921dc67e709030e1bc93b3bc04963479

Mathematical statement. The left arc parametrization maps $[-R\phi, 0]$ exactly onto the left coordinate arc trace.

The circle and strip constraints give one inclusion; inverse trigonometric reconstruction on the declared arc gives the reverse inclusion.

```
theorem leftArcParam_image_Icc :
  a.leftArcParam ''
    Icc (-a.radius * a.sideArcAngle) 0 = a.leftArcTrace := by
  ext p
  constructor
  · rintro ⟨s, hs, rfl⟩
    have hu :
      s / a.radius ∈ Icc (-a.sideArcAngle) 0 := by
    constructor
    · apply (le_div_iff₀ a.radius_pos).2
      nlinarith [hs.1, a.radius_pos]
    · exact div_nonpos_of_nonpos_of_nonneg hs.2 a.radius_pos.le
  have hsin_nonpos : sin (s / a.radius) ≤ 0 :=
    Real.sin_nonpos_of_nonpos_of_neg_pi_le hu.2
    (by linarith [hu.1, a.sideArcAngle_lt_pi])
  have habs : |s / a.radius| ≤ a.sideArcAngle := by
    rw [abs_le]
    exact ⟨hu.1, hu.2.trans a.sideArcAngle_pos.le⟩
  have hcos :
    cos a.sideArcAngle ≤ cos (s / a.radius) := by
    simp only [Real.cos_abs] using
      (Real.cos_le_cos_of_nonneg_of_le_pi
        (abs_nonneg (s / a.radius)) a.sideArcAngle_lt_pi.le habs)
  have hcircle := Real.sin_sq_add_cos_sq (s / a.radius)
  change
    ((a.leftCenter.1 + a.radius * sin (s / a.radius)) -
      a.leftCenter.1) ^ 2 +
      ((a.leftCenter.2 - a.radius * cos (s / a.radius)) -
        a.leftCenter.2) ^ 2 = a.radius ^ 2 ∧
    a.leftCenter.1 + a.radius * sin (s / a.radius) ≤
      a.leftCenter.1 ∧
    (-1 : ℝ) ≤
      a.leftCenter.2 - a.radius * cos (s / a.radius) ∧
    a.leftCenter.2 - a.radius * cos (s / a.radius) ≤ 1
  refine (by nlinarith [sq_nonneg a.radius], ?_, ?_, ?_)
  · nlinarith [mul_nonpos_of_nonneg_of_nonpos a.radius_pos.le hsin_nonpos]
  · change
      (-1 : ℝ) ≤
        (a.radius - 1) - a.radius * cos (s / a.radius)
    nlinarith [mul_le_mul_of_nonneg_left
      (Real.cos_le_one (s / a.radius)) a.radius_pos.le]
  · change
      (a.radius - 1) - a.radius * cos (s / a.radius) ≤ 1
    nlinarith [mul_le_mul_of_nonneg_left hcos a.radius_pos.le,
      a.radius_mul_cos_sideArcAngle]
  · intro hp
    change
      (p.1 - a.leftCenter.1) ^ 2 +
        (p.2 - a.leftCenter.2) ^ 2 = a.radius ^ 2 ∧
      p.1 ≤ a.leftCenter.1 ∧
      (-1 : ℝ) ≤ p.2 ∧ p.2 ≤ 1 at hp
  let z : ℂ :=
    ⟨a.leftCenter.2 - p.2, p.1 - a.leftCenter.1⟩
  have hnorm_sq : ||z|| ^ 2 = a.radius ^ 2 := by
    rw [Complex.sq_norm, Complex.normSq_apply]
    simp only [z]
    nlinarith [hp.1]
  have hnorm : ||z|| = a.radius := by
```

```

nlinarith [norm_nonneg z, a.radius_pos]
have hsin :
  a.radius * sin z.arg = p.1 - a.leftCenter.1 := by
  simpa [hnorm, z] using Complex.norm_mul_sin_arg z
have hcos :
  a.radius * cos z.arg = a.leftCenter.2 - p.2 := by
  simpa [hnorm, z] using Complex.norm_mul_cos_arg z
have hcos_le : cos a.sideArcAngle ≤ cos z.arg := by
have hmul :
  a.radius * cos a.sideArcAngle ≤ a.radius * cos z.arg := by
  rw [a.radius_mul_cos_sideArcAngle, hcos]
  change a.radius - 2 ≤ (a.radius - 1) - p.2
  linarith [hp.2.2.2]
nlinarith [hmul, a.radius_pos]
have harg : |z.arg| ≤ a.sideArcAngle := by
  by_contra h
  have hlt : a.sideArcAngle < |z.arg| := lt_of_not_ge h
  have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
  a.sideArcAngle_pos.le (Complex.abs_arg_le_pi z) hlt
  rw [Real.cos_abs] at hc
  linarith
have harg_nonpos : z.arg ≤ 0 := by
  by_contra h
  have harg_pos : 0 < z.arg := lt_of_not_ge h
  have harg_lt_pi : z.arg < π :=
    lt_of_le_of_lt (le_trans (le_abs_self z.arg) harg)
    a.sideArcAngle_lt_pi
  have hsin_pos := Real.sin_pos_of_pos_of_lt_pi harg_pos harg_lt_pi
  nlinarith [a.radius_pos, hp.2.1]
refine (a.radius * z.arg, ?_, ?_)
· constructor
· nlinarith [a.radius_pos, (abs_le.mp harg).1]
· exact mul_nonpos_of_nonneg_of_nonpos a.radius_pos.le harg_nonpos
· have hangle : a.radius * z.arg / a.radius = z.arg := by
  field_simp [a.radius_pos.ne']
  apply Prod.ext
  · simp only [leftArcParam, hangle]
    linarith
  · simp only [leftArcParam, hangle]
    linarith

```

```

/-- The right signed-arclength parameter fills exactly the right strip-side
trace, on every geometric branch (including the semicircular transition). -/

```

TypeThreeAssembly.leftArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:104-110

Declaration SHA-256: cfb74d8a54830ee4e695b37318a8d8c8a086063cc569f3e8d4fa167da3d5d0b9

Mathematical statement. The left trace consists of points on the circle centered at $(-b, R - 1)$ with radius R , satisfying

$$x \leq -b$$

and

$$-1 \leq y \leq 1.$$

The circle equality, rather than disk inequality, picks out the exposed left arc.

```

def leftArcTrace : Set PlanePoint :=
  {p | (p.1 - a.leftCenter.1) ^ 2 + (p.2 - a.leftCenter.2) ^ 2 =
    a.radius ^ 2 ^
    p.1 ≤ a.leftCenter.1 ^
    (-1 : ℝ) ≤ p.2 ^ p.2 ≤ 1}

/-- The exact right circular trace of the type-(iii) strip-side boundary. -/

```

TypeThreeAssembly.leftCenter

Classification: supplement

Source: TypeThreeAssembly.lean:70-72

Declaration SHA-256: ae634544d50a625f2da0ab83dddef536c2004aef8a002e19ec3ff792534bdf2

Mathematical statement. Define the left circle center $(-b, R - 1)$.

Its lowest point has height minus one, providing tangency to the bottom interface.

```
def leftCenter : PlanePoint := (-a.sideHalfWidth, a.radius - 1)

/-- Source circle center for the right strip arc. -/
```

TypeThreeAssembly.leftSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:85-91

Declaration SHA-256: 7a6b1fb5a536fc1614b926138e279c79369045c2b292e00c145e1a0e699dbced

Mathematical statement. Define the radius- R disk centered at $(-b, R - 1)$, clipped by

$$x \leq -b$$

and

$$-1 \leq y \leq 1.$$

This is the left part of the strip core outside its central rectangle.

```
def leftSegmentCarrier : Set PlanePoint :=
  {p | (p.1 - a.leftCenter.1) ^ 2 + (p.2 - a.leftCenter.2) ^ 2 ≤
    a.radius ^ 2} ∩
  ({p | p.1 ≤ a.leftCenter.1} ∩
    ({p | (-1 : ℝ) ≤ p.2} ∩ {p | p.2 ≤ 1}))

/-- Closed right disk segment of `K₃`. -/
```

TypeThreeAssembly.leftUpperJunction

Classification: supplement

Source: TypeThreeAssembly.lean:946-957

Declaration SHA-256: a660a79d351317f739116ce079a065c0c8bcd856fae9b73442ad8d1339dd7ec

Mathematical statement. The point $(-b - R\sin\beta, 1)$ equals the outer cap's left endpoint.

The defining width

$$b = R(\sin\alpha - \sin\beta)$$

makes the core and cap half-chords agree.

```
theorem leftUpperJunction :
  (a.leftCenter.1 - a.radius * sin a.innerAngle, 1) =
    a.outerCap.leftEndpoint := by
  rw [a.outerCap_leftEndpoint]
  change
    (-a.sideHalfWidth - a.radius * sin a.innerAngle, 1) =
      (-a.radius * sin a.outerAngle, 1)
  rw [sideHalfWidth]
  ring_nf
```

```
-- The right strip arc and exterior cap use the same upper junction
coordinate. -/
```

TypeThreeAssembly.measurableSet_carrier

Classification: supplement

Source: TypeThreeAssembly.lean:1310-1316

Declaration SHA-256: 25696aa660c5c71e8ce3e6a179a069d25633bc99ff664ad8e93ea6e319cb501f

Mathematical statement. The three-arc carrier is measurable.

Closed subsets of the coordinate plane are Borel measurable.

```
theorem measurableSet_carrier : MeasurableSet a.carrier :=
  a.isClosed_carrier.measurableSet

/-! ## Carrier-derived weighted area -/

/-- The left half of the standard side cap. Translating it horizontally gives
the actual left strip-side segment. -/
```

TypeThreeAssembly.measurableSet_euclidean_bottomSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:2855-2860

Declaration SHA-256: ac873340c9031353c9d26e2404b771c8a0ff1b638ea75b1a24ed627ed5bc802d

Mathematical statement. The Euclidean image of the bottom segment is measurable.

It is a compact affine segment, including the singleton case.

```
theorem measurableSet_euclidean_bottomSegmentCarrier :
  MeasurableSet
    (planeEuclideanHomeomorph '' a.bottomSegmentCarrier) :=
  (planeEuclideanHomeomorph.isClosedMap _
    a.isClosed_bottomSegmentCarrier).measurableSet
```

TypeThreeAssembly.measurableSet_euclidean_leftArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:732-736

Declaration SHA-256: 791dc1e2fe6e2d0cb10c51a3bd4f4018c5ee53a25eeef24b22bc0d4e6f0f1192

Mathematical statement. The Euclidean image of the left three-arc trace is measurable.

The exact parameter image is compact, hence Borel measurable.

```
theorem measurableSet_euclidean_leftArcTrace :
  MeasurableSet (planeEuclideanHomeomorph '' a.leftArcTrace) := by
  rw [← a.realizedLeftArcParam_image_Icc]
  exact (isCompact_Icc.image a.continuous_realizedLeftArcParam).measurableSet
```

TypeThreeAssembly.measurableSet_euclidean_rightArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:737-741

Declaration SHA-256: 64197e0a340a1643ad8bc1525fe961f5b8792a0e7cb9321824fa2436e9d02bb4

Mathematical statement. The Euclidean image of the right three-arc trace is measurable.

The exact parameter image is compact, hence Borel measurable.

```
theorem measurableSet_euclidean_rightArcTrace :
  MeasurableSet (planeEuclideanHomeomorph '' a.rightArcTrace) := by
  rw [← a.realizedRightArcParam_image_Icc]
  exact (isCompact_Icc.image a.continuous_realizedRightArcParam).measurableSet
```

TypeThreeAssembly.modeledWeightedArea

Classification: supplement

Source: TypeThreeAssembly.lean:278-284

Declaration SHA-256: 35499fdca0a3bbbed11909d6dec0bbb1c83123d6ea5cb0599717ee289f567e87e

Mathematical statement. Define the modeled weighted area as $4b$ plus the auxiliary side-cap area plus λ times the outer-cap area.

This scalar decomposition is proved to agree with the actual carrier integral; it is not an extra hypothesis.

```
def modeledWeightedArea : ℝ :=
  a.rectangleArea + a.sideCap.euclideanArea +
  lam * a.outerCap.euclideanArea

/-- Component perimeter of the modeled type-(iii) profile. The side-cap arc
is the sum of the two translated side arcs; `bottomChord` also covers the
zero-length transition at `h = 1 / 2`. -/
```

TypeThreeAssembly.modeledWeightedArea_formula

Classification: supplement

Source: TypeThreeAssembly.lean:885-895

Declaration SHA-256: b0e70e8aeafad1e18dbaff1d3d6364f544c6042f48ce97bebd3fa5c6cc8e74e

Mathematical statement. The modeled weighted area equals

$$[T + (q + 2)\delta]/h^2.$$

Expand the rectangle and circular-segment areas and simplify the incidence and sine identities; this step is algebraic, before the actual integral bridge.

```
theorem modeledWeightedArea_formula :
  a.modeledWeightedArea = a.scalarWeightedArea := by
  rw [modeledWeightedArea, rectangleArea, a.sideCap_euclideanArea,
  a.outerCap_euclideanArea, scalarWeightedArea, angleTerm, sineGap,
  sideHalfWidth, a.sin_sideArcAngle, a.cos_sideArcAngle, a.cos_outerAngle,
  outerArgument, sideArcAngle, radius, shapeParameter]
  field_simp [ne_of_gt a.h_pos,
  ne_of_gt (lt_trans (by norm_num : (0 : ℝ) < 1) a.density_jump)]
  ring

/-- CMV equation (26), including the degenerate bottom-chord branch. -/
```

TypeThreeAssembly.modeledWeightedPerimeter

Classification: supplement

Source: TypeThreeAssembly.lean:285-288

Declaration SHA-256: 76198be6eb5b8790a4af34449bf29c4b963314e1e4bd268b4d3c9cdb2e69ea0d

Mathematical statement. Define the modeled perimeter as the auxiliary side-cap arc length plus λ times the outer-cap arc length plus $2b$.

The last term is the density-one bottom segment, and the side-cap arc length equals the total length of the two strip arcs.

```
def modeledWeightedPerimeter : ℝ :=
  a.sideCap.arcLength + lam * a.outerCap.arcLength + a.bottomChord

/-- The angle term `lambda * a_3 + pi - b_3` in CMV equation (26). -/
```

TypeThreeAssembly.modeledWeightedPerimeter_formula

Classification: supplement

Source: TypeThreeAssembly.lean:896-903

Declaration SHA-256: bb4c05a2ca2fd9d5b63be1b6ad16da716cc43497abacad228054765e6eaa2e39e

Mathematical statement. The modeled perimeter equals

$$2(T + \delta)/h.$$

Sum the two strip arcs, weighted exterior arc and bottom segment using

$$R = 1/h.$$

```
theorem modeledWeightedPerimeter_formula :
  a.modeledWeightedPerimeter = a.scalarWeightedPerimeter := by
  rw [modeledWeightedPerimeter, a.sideCap_arcLength,
    a.outerCap_arcLength, scalarWeightedPerimeter, angleTerm, sineGap,
    bottomChord, sideHalfWidth, sideArcAngle, radius]
  field_simp [ne_of_gt a.h_pos]
  ring
```

TypeThreeAssembly.one_lt_radius

Classification: supplement

Source: TypeThreeAssembly.lean:148-152

Declaration SHA-256: 4e892eef63bc8252d5a243e2b5495e748b55fa956d516b3510c9e5289ec264d9

Mathematical statement. Every admissible three-arc assembly has

$$R > 1.$$

The strict upper curvature bound makes the reciprocal strictly greater than one, ensuring a regular three-arc core.

```
theorem one_lt_radius : 1 < a.radius := by
  rw [radius]
  apply (lt_div_iff0 a.h_pos).2
  nlinarith [a.h_lt_one]
```

TypeThreeAssembly.outerAngle

Classification: supplement

Source: TypeThreeAssembly.lean:59-63

Declaration SHA-256: 0e2ff2070394fdda5b4ec09824f47ed543dabdfef101c076ad378e0c92be2ff47

Mathematical statement. Define

$$\alpha = \arccos(q/\lambda).$$

The full principal angle interval $(0, \pi)$ permits both minor and major exterior caps.

```
def outerAngle : ℝ := arccos a.outerArgument
```

```
-- Half-central-angle of either strip-side arc. This is the complementary
branch to `innerAngle`, so it remains valid on both sides of `h = 1 / 2`. -/
```

TypeThreeAssembly.outerAngle_lt_pi

Classification: supplement

Source: TypeThreeAssembly.lean:174-177

Declaration SHA-256: ce135b6a3dc191fb33efd487aab9da31365b8ec038647a624b62a436ba911d34

Mathematical statement. For every assembly,

$$\alpha < \pi.$$

The outer argument is strictly above minus one, excluding a fully closed circular traversal with coincident endpoints.

```
theorem outerAngle_lt_pi : a.outerAngle < π :=
  Real.arccos_lt_pi.mpr a.outerArgument_mem.1

/-- The principal-angle definitions recover the source strip angle exactly. -/
```

TypeThreeAssembly.outerAngle_pos

Classification: supplement

Source: TypeThreeAssembly.lean:171-173

Declaration SHA-256: 760f9b90fd549c278aa760178ec027d7c643f8b5bd6d4bf942c86f6a65bb0e28

Mathematical statement. For every assembly,

$$\alpha > 0.$$

The outer argument is strictly below one, preventing a zero-angle cap.

```
theorem outerAngle_pos : 0 < a.outerAngle :=
  Real.arccos_pos.mpr a.outerArgument_mem.2
```

TypeThreeAssembly.outerAngle_semicircular

Classification: supplement

Source: TypeThreeAssembly.lean:374-377

Declaration SHA-256: 9ff26a1092e7089058ecf98f24fecce78972031b475870c0e4d30009e831f8ee

Mathematical statement. If $h = 1/2$, the outer half-angle is

$$\alpha = \pi/2.$$

The outer argument also vanishes for the positive density parameter.

```
theorem outerAngle_semicircular (hh : a.h = 1 / 2) :
  a.outerAngle = π / 2 := by
  simp [outerAngle, outerArgument, shapeParameter, hh]
```

TypeThreeAssembly.outerArgument

Classification: supplement

Source: TypeThreeAssembly.lean:56-58

Declaration SHA-256: d6cfc025aa1368c77ccefa6b050dbf8bfa113c35569b4434625ec2d735b91f41

Mathematical statement. Define the outer inverse-cosine argument

$$q/\lambda.$$

The density contrast rescales the inner cosine through the incidence relation.

```
def outerArgument : ℝ := a.shapeParameter / lam

/-- The exterior half-central-angle `a₃`. -/
```

TypeThreeAssembly.outerArgument_mem

Classification: supplement

Source: TypeThreeAssembly.lean:156-164

Declaration SHA-256: fc2a8b1032448c01ece41eff0bcd5d748f70ee556d5721867bb36b59d56ce63a

Mathematical statement. For every assembly,

$$-1 < q/\lambda < 1.$$

The positive density contrast exceeds one, so rescaling preserves strict containment in the inverse-cosine domain.

```
theorem outerArgument_mem : -1 < a.outerArgument ∧ a.outerArgument < 1 := by
  have hlam_pos : 0 < lam := lt_trans (by norm_num) a.density_jump
  have hx := a.shapeParameter_mem
  constructor
  · apply (lt_div_iff₀ hlam_pos).2
    nlinarith [a.density_jump, hx.1]
  · apply (div_lt_iff₀ hlam_pos).2
    nlinarith [a.density_jump, hx.2]
```

TypeThreeAssembly.outerCap

Classification: supplement

Source: TypeThreeAssembly.lean:251-261

Declaration SHA-256: 5482055613f2ec0454ddd9c71d722bea04494543bc659e447732870c662eb86f

Mathematical statement. Define the upper cap with chord

$$2R\sin\alpha,$$

half-angle α , midpoint $(0, 1)$, and upper orientation.

Its radius is proved to be R , and its actual carrier completes the competitor above the strip.

```
def outerCap : OneSidedCircularCap where
  chord := 2 * a.radius * sin a.outerAngle
  theta := a.outerAngle
  midpointX := 0
  baseY := 1
  side := .upper
  chord_pos := mul_pos (mul_pos (by norm_num) a.radius_pos) a.outerSin_pos
  theta_pos := a.outerAngle_pos
  theta_lt_pi := a.outerAngle_lt_pi

/-- Genuine closed coordinate carrier `E₃ = K₃ ∪ upper_cap`. -/
```

TypeThreeAssembly.outerCap_arcLength

Classification: supplement

Source: TypeThreeAssembly.lean:875-884

Declaration SHA-256: e4c6a09e492a4c9395262e1c277f2cd0b185e72ae34042a7f14138b12c6b4c19

Mathematical statement. The exterior arc length is

$$2R\alpha.$$

The cap-radius identity converts normalized chord length into the usual radius times central angle.

```
theorem outerCap_arclength :
  a.outerCap.arclength = 2 * a.radius * a.outerAngle := by
  rw [OneSidedCircularCap.arclength, ell]
  change
    (2 * a.radius * sin a.outerAngle) *
      (a.outerAngle / sin a.outerAngle) =
    2 * a.radius * a.outerAngle
  field_simp [ne_of_gt a.outerSin_pos]

/-- CMV equation (26), derived from the branch-complete component geometry. -/
```

TypeThreeAssembly.outerCap_center

Classification: supplement

Source: TypeThreeAssembly.lean:929-933

Declaration SHA-256: 1057a7a30ad6e7558c1c0e586fc0147d20f1fc01e6c3e508acda901e55191c4c

Mathematical statement. The outer-cap center equals the declared upper center $(0, 1 - R\cos\alpha)$.

Expand the cap constructor and use its radius identity.

```
theorem outerCap_center : a.outerCap.center = a.upperCenter := by
  change (0, 1 - a.outerCap.radius * cos a.outerAngle) =
    (0, 1 - a.radius * cos a.outerAngle)
  rw [a.outerCap_radius]
```

TypeThreeAssembly.outerCap_euclideanArea

Classification: supplement

Source: TypeThreeAssembly.lean:851-864

Declaration SHA-256: 3c7709918bd2e08d7c7ea431b339d946f2eb6c3c9d1138f4304b6afe64a153c2

Mathematical statement. The outer-cap area is

$$R^2(\alpha - \sin\alpha\cos\alpha).$$

This formula is valid for every admissible exterior half-angle, including a major cap.

```
theorem outerCap_euclideanArea :
  a.outerCap.euclideanArea =
    a.radius ^ 2 *
      (a.outerAngle - sin a.outerAngle * cos a.outerAngle) := by
  rw [OneSidedCircularCap.euclideanArea, area]
  change
    (2 * a.radius * sin a.outerAngle) ^ 2 *
      ((a.outerAngle - sin a.outerAngle * cos a.outerAngle) /
        (4 * sin a.outerAngle ^ 2)) =
    a.radius ^ 2 *
      (a.outerAngle - sin a.outerAngle * cos a.outerAngle)
  field_simp [ne_of_gt a.outerSin_pos]
  ring
```

TypeThreeAssembly.outerCap_leftEndpoint

Classification: supplement

Source: TypeThreeAssembly.lean:934-938

Declaration SHA-256: 369e532ae6bbe742ee13471a1a8cc638a185e8c74c4f29cdde432417763b0fdb

Mathematical statement. The upper cap's left endpoint is $(-R\sin\alpha, 1)$.

Its chord is

$$2R\sin\alpha,$$

centered on the vertical axis at the top interface.

```
theorem outerCap_leftEndpoint :  
  a.outerCap.leftEndpoint = (-a.radius * sin a.outerAngle, 1) := by  
  simp [OneSidedCircularCap.leftEndpoint, outerCap]  
  ring
```

TypeThreeAssembly.outerCap_radius

Classification: supplement

Source: TypeThreeAssembly.lean:419-422

Declaration SHA-256: f107696210fe89f5a746d54584303c70da482f05587bc229f2d37aaa9494b57f

Mathematical statement. The constructed outer cap has radius R .

Its chord is

$$2R\sin\alpha;$$

cancellation in the cap-radius definition proves the identity.

```
theorem outerCap_radius : a.outerCap.radius = a.radius := by  
  change (2 * a.radius * sin a.outerAngle) / (2 * sin a.outerAngle) = a.radius  
  field_simp [ne_of_gt a.outerSin_pos]
```

TypeThreeAssembly.outerCap_rightEndpoint

Classification: supplement

Source: TypeThreeAssembly.lean:939-945

Declaration SHA-256: 0bf0b00b58f0396d9b9b8bc9e08617bcd76cf3f0b76937e23b52dec1f9b5fe12

Mathematical statement. The upper cap's right endpoint is $(R\sin\alpha, 1)$.

The positive half-chord gives the right endpoint coordinate.

```
theorem outerCap_rightEndpoint :  
  a.outerCap.rightEndpoint = (a.radius * sin a.outerAngle, 1) := by  
  simp [OneSidedCircularCap.rightEndpoint, outerCap]  
  ring  
  
/-- The left strip arc and exterior cap use the same upper junction  
coordinate. -/
```

TypeThreeAssembly.outerSin_pos

Classification: supplement

Source: TypeThreeAssembly.lean:213-215

Declaration SHA-256: feedd827e6203dedcb21f4f474b9fea36ae3ea708b33b644b944093c5f958959

Mathematical statement. For every assembly,

$$\sin\alpha > 0.$$

The exterior half-angle lies strictly between zero and π .

```
theorem outerSin_pos : 0 < sin a.outerAngle :=  
  sin_pos_of_pos_of_lt_pi a.outerAngle_pos a.outerAngle_lt_pi
```

TypeThreeAssembly.pairwiseOverlap_subset_boundaryJoinSet

Classification: supplement

Source: TypeThreeAssembly.lean:1142-1257

Declaration SHA-256: f8736f4dd5a67fc4149e15b1ed3e3f21030a49e12643cd517b41158566d4664f

Mathematical statement. The union of pairwise intersections among the left arc, right arc, exterior arc and bottom segment is contained in the junction set.

Coordinate height and circle constraints force all common points to the stated endpoints, including the degenerate bottom case.

```
theorem pairwiseOverlap_subset_boundaryJoinSet :
  pairwiseOverlap a.leftArcTrace a.rightArcTrace
    (OneSidedCircularCap.arcTrace a.outerCap)
    a.bottomSegmentCarrier  $\subseteq$  a.boundaryJoinSet := by
  intro p hp
  simp only [pairwiseOverlap, mem_union, mem_inter_iff] at hp
  rcases hp with hLR | hLU | hLB | hRU | hRB | hUB
  · by_cases hw : a.sideHalfWidth = 0
    · have hx : p.1 = 0 := by
        have hl := hLR.1.2.1
        have hr := hLR.2.2.1
        simp only [leftCenter, rightCenter, hw, neg_zero] at hl hr
        linarith
      have hcircle := hLR.1.1
      rw [hx] at hcircle
      simp only [leftCenter, hw, neg_zero, sub_zero] at hcircle
      have hfactor :
        (p.2 + 1) * (p.2 - (2 * a.radius - 1)) = 0 := by
          nlinarith
      rcases mul_eq_zero.mp hfactor with hy | hy
      · have hy' : p.2 = -1 := by linarith
        have hpEq : p = (0, -1) := Prod.ext hx hy'
        simp [boundaryJoinSet, hpEq, hw]
      · have hy' : p.2 = 2 * a.radius - 1 := by linarith
        have hyUpper := hLR.1.2.2.2
        exfalso
        nlinarith [a.one_lt_radius]
    · have hwpos : 0 < a.sideHalfWidth :=
        lt_of_le_of_ne a.sideHalfWidth_nonneg (Ne.symm hw)
      have hl := hLR.1.2.1
      have hr := hLR.2.2.1
      simp only [leftCenter, rightCenter] at hl hr
      exfalso
      linarith
    · have hyCore : p.2  $\leq$  1 := hLU.1.2.2.2
      have hyCap : (1 :  $\mathbb{R}$ )  $\leq$  p.2 := by
        simp [outerCap] using hLU.2.2
      have hy : p.2 = 1 := le_antisymm hyCore hyCap
      have hdx : p.1 - a.leftCenter.1  $\leq$  0 := by
        linarith [hLU.1.2.1]
      have hsq :
        (p.1 - a.leftCenter.1) ^ 2 =
          (a.radius * sin a.innerAngle) ^ 2 := by
        have hpCircle := hLU.1.1
        rw [hy] at hpCircle
        simp only [leftCenter] at hpCircle
        rw [leftCenter]
        nlinarith [a.side_circle_at_top]
      have habs := (sq_eq_sq_iff_abs_eq_abs
        (p.1 - a.leftCenter.1)
        (a.radius * sin a.innerAngle)).mp hsq
      rw [abs_of_nonpos hdx,
        abs_of_pos (mul_pos a.radius_pos a.innerSin_pos)] at habs
      have hdxEq :
        p.1 - a.leftCenter.1 =
          -(a.radius * sin a.innerAngle) := by linarith
      have hx : p.1 = -(a.radius * sin a.outerAngle) := by
```

```

    simp only [leftCenter] at hdxEq
    rw [sideHalfWidth] at hdxEq
    linarith
  have hpEq : p = (-a.radius * sin a.outerAngle, 1) := by
    apply Prod.ext
    · simpa using hx
    · simpa using hy
  simp [boundaryJoinSet, hpEq]
  · have hy : p.2 = -1 := hLB.2.1
    have hbound : |p.1| ≤ a.sideHalfWidth := hLB.2.2
    have hxLower := (abs_le.mp hbound).1
    have hxUpper := hLB.1.2.1
    simp only [leftCenter] at hxUpper
    have hx : p.1 = -a.sideHalfWidth := by linarith
    have hpEq : p = (-a.sideHalfWidth, -1) := Prod.ext hx hy
    simp [boundaryJoinSet, hpEq]
  · have hyCore : p.2 ≤ 1 := hRU.1.2.2.2
    have hyCap : (1 : ℝ) ≤ p.2 := by
      simpa [outerCap] using hRU.2.2
    have hy : p.2 = 1 := le_antisymm hyCore hyCap
    have hdx : 0 ≤ p.1 - a.rightCenter.1 := by
      linarith [hRU.1.2.1]
    have hsq :
      (p.1 - a.rightCenter.1) ^ 2 =
      (a.radius * sin a.innerAngle) ^ 2 := by
      have hpCircle := hRU.1.1
      rw [hy] at hpCircle
      simp only [rightCenter] at hpCircle
      rw [rightCenter]
      nlinarith [a.side_circle_at_top]
    have habs := (sq_eq_sq_iff_abs_eq_abs
      (p.1 - a.rightCenter.1)
      (a.radius * sin a.innerAngle)).mp hsq
    rw [abs_of_nonneg hdx,
      abs_of_pos (mul_pos a.radius_pos a.innerSin_pos)] at habs
    have hdxEq :
      p.1 - a.rightCenter.1 =
      a.radius * sin a.innerAngle := habs
    have hx : p.1 = a.radius * sin a.outerAngle := by
      simp only [rightCenter] at hdxEq
      rw [sideHalfWidth] at hdxEq
      linarith
    have hpEq : p = (a.radius * sin a.outerAngle, 1) := Prod.ext hx hy
    simp [boundaryJoinSet, hpEq]
  · have hy : p.2 = -1 := hRB.2.1
    have hbound : |p.1| ≤ a.sideHalfWidth := hRB.2.2
    have hxUpper := (abs_le.mp hbound).2
    have hxLower := hRB.1.2.1
    simp only [rightCenter] at hxLower
    have hx : p.1 = a.sideHalfWidth := by linarith
    have hpEq : p = (a.sideHalfWidth, -1) := Prod.ext hx hy
    simp [boundaryJoinSet, hpEq]
  · have hyCap : (1 : ℝ) ≤ p.2 := by
      simpa [outerCap] using hUB.1.2
    have hyBottom : p.2 = -1 := hUB.2.1
    exfalse
    linarith

```

-- Pairwise primitive overlaps carry no Euclidean H^1 mass. -/

TypeThreeAssembly.radius

Classification: supplement

Source: TypeThreeAssembly.lean:47-49

Declaration SHA-256: 44bdb144ee33a0fe634670e914d151b3f9a50f2b2f7db8da7c3fad23bdfbf70e

Mathematical statement. Define

$$R = 1/h.$$

The admissible curvature is strictly between zero and one, so the radius is greater than one.

```
def radius : ℝ := 1 / a.h

/-- Source scalar `2h - 1`, equal to `cos b₃`. -/
```

TypeThreeAssembly.radius_mul_h

Classification: supplement

Source: TypeThreeAssembly.lean:197-200

Declaration SHA-256: e90d4095fef5e1e089527a179a53736b5dbf462b6149abecb5375e96b7380078

Mathematical statement. For every assembly,

$$Rh = 1.$$

Cancel the positive curvature in the reciprocal radius.

```
theorem radius_mul_h : a.radius * a.h = 1 := by
  rw [radius]
  field_simp [ne_of_gt a.h_pos]
```

TypeThreeAssembly.radius_pos

Classification: supplement

Source: TypeThreeAssembly.lean:145-147

Declaration SHA-256: a610fc617d9a92f1154e4b9627bb16a0d909464945f9e22ccb900c975acb2c7d

Mathematical statement. Every admissible three-arc assembly has

$$R > 0.$$

Its curvature is positive, so its reciprocal is positive.

```
theorem radius_pos : 0 < a.radius := by
  exact one_div_pos.mpr a.h_pos
```

TypeThreeAssembly.realizedLeftArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:665-668

Declaration SHA-256: 51fbf9517aa10415de53dc106db635c5f70b9c39860ba75778e696cdebf2d6c3

Mathematical statement. Define the realized left arc parametrization by applying the Euclidean coordinate homeomorphism to its coordinate parametrization.

This makes the Euclidean metric explicit without altering the point set.

```
noncomputable def realizedLeftArcParam (s : ℝ) : EuclideanPlane :=
  planeEuclideanHomeomorph (a.leftArcParam s)

/-- Euclidean realization of the right signed-arclength parameter. -/
```

TypeThreeAssembly.realizedLeftArcParam_eq

Classification: supplement

Source: TypeThreeAssembly.lean:672-680

Declaration SHA-256: 538a7925bd9a6977d5e74933bd5f951885abb82dc534e3d102db01bf6aa01a1e

Mathematical statement. At every real s , the realized left parametrization equals the side-arc complex isometry centered at its left center applied to $\gamma_R(s)$.

Expand the sine and cosine coordinates of the standard complex circle.

```
lemma realizedLeftArcParam_eq (s : ℝ) :
  a.realizedLeftArcParam s =
    sideArcComplexIsometryMap a.leftCenter
      (unitCircleArc a.radius s) := by
  rw [show a.realizedLeftArcParam s =
    WithLP.toLP 2 (a.leftArcParam s) by rfl]
  simp [leftArcParam, sideArcComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]
```

TypeThreeAssembly.realizedLeftArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:710-720

Declaration SHA-256: 06e6662a39cd836487840418c7224db9c1b8779c37aa295aab4e1109503a76c7

Mathematical statement. The realized left parameter image on $[-R\phi, 0]$ equals the Euclidean image of the left coordinate trace.

Transport the coordinate parameter-image theorem through the homeomorphism.

```
theorem realizedLeftArcParam_image_Icc :
  a.realizedLeftArcParam ''
    Icc (-a.radius * a.sideArcAngle) 0 =
    planeEuclideanHomeomorph '' a.leftArcTrace := by
  calc
    _ = planeEuclideanHomeomorph ''
      (a.leftArcParam '' Icc (-a.radius * a.sideArcAngle) 0) := by
      rw [Set.image_image]
      rfl
    _ = _ := by rw [a.leftArcParam_image_Icc]
```

TypeThreeAssembly.realizedRightArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:669-671

Declaration SHA-256: 8a963395af5bf096a9c5748471f90364728d8fe511c4e2cc0591bd2068d9e016

Mathematical statement. Define the realized right arc parametrization by applying the Euclidean coordinate homeomorphism to its coordinate parametrization.

This makes the Euclidean metric explicit without altering the point set.

```
noncomputable def realizedRightArcParam (s : ℝ) : EuclideanPlane :=
  planeEuclideanHomeomorph (a.rightArcParam s)
```

TypeThreeAssembly.realizedRightArcParam_eq

Classification: supplement

Source: TypeThreeAssembly.lean:681-689

Declaration SHA-256: 3339fa9296e8c62289cd021c3334658bb86b73401d06cffbaa790d7423ffd93e

Mathematical statement. At every real s , the realized right parametrization equals the side-arc complex isometry centered at its right center applied to $\gamma_R(s)$.

Expand the sine and cosine coordinates of the standard complex circle.

```
lemma realizedRightArcParam_eq (s : ℝ) :
  a.realizedRightArcParam s =
    sideArcComplexIsometryMap a.rightCenter
      (unitCircleArc a.radius s) := by
  rw [show a.realizedRightArcParam s =
    WithLp.toLp 2 (a.rightArcParam s) by rfl]
  simp [rightArcParam, sideArcComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]
```

TypeThreeAssembly.realizedRightArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:721-731

Declaration SHA-256: 9469dc298ecad88323016b9907111e322a864f0831a991b637ca11824881af16

Mathematical statement. The realized right parameter image on $[0, R\phi]$ equals the Euclidean image of the right coordinate trace.

Transport the coordinate parameter-image theorem through the homeomorphism.

```
theorem realizedRightArcParam_image_Icc :
  a.realizedRightArcParam ''
    Icc 0 (a.radius * a.sideArcAngle) =
    planeEuclideanHomeomorph '' a.rightArcTrace := by
  calc
  _ = planeEuclideanHomeomorph ''
    (a.rightArcParam '' Icc 0 (a.radius * a.sideArcAngle)) := by
    rw [Set.image_image]
    rfl
  _ = _ := by rw [a.rightArcParam_image_Icc]
```

TypeThreeAssembly.rectangleArea

Classification: supplement

Source: TypeThreeAssembly.lean:273-277

Declaration SHA-256: 3314187b7b48ab958a7d8415bcf10afdb2dd85317d354dfd049f5344c2569e90

Mathematical statement. Define the scalar rectangle area as $4b$.

The height is two and the width is $2b$; the formula includes zero width.

```
def rectangleArea : ℝ := 4 * a.sideHalfWidth

/-- Component area of the modeled type-(iii) profile. The side-cap term is
the sum of the two congruent translated half-caps; the outer cap has density
`lam`. -/
```

TypeThreeAssembly.rectangleCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:80-84

Declaration SHA-256: d451775e4018b5bae444c6a8ed6911c5adcc23db4a0efc68ab71c655f2933477

Mathematical statement. Define the rectangle

$$\{(x, y) : -b \leq x \leq b, -1 \leq y \leq 1\}.$$

The rectangle may have zero width when $h = 1/2$; the overall carrier is still a disk there.

```
def rectangleCarrier : Set PlanePoint :=
  ({p | -a.sideHalfWidth ≤ p.1} ∩ {p | p.1 ≤ a.sideHalfWidth}) ∩
  ({p | (-1 : ℝ) ≤ p.2} ∩ {p | p.2 ≤ 1})

/-- Closed left disk segment of `K₃`. -/
```

TypeThreeAssembly.rightArcParam

Classification: supplement

Source: TypeThreeAssembly.lean:127-131

Declaration SHA-256: 74515bd4fc474dfa2c52a14662b8cc34aa0f40ef9d8185819e05d1de94ccf588

Mathematical statement. Define the right arclength parametrization

$$s \mapsto (b + R\sin(s/R), R - 1 - R\cos(s/R)).$$

The selected parameter interval is $[0, R\phi]$, starting at the bottom tangency point.

```
def rightArcParam (s : ℝ) : PlanePoint :=
  (a.rightCenter.1 + a.radius * sin (s / a.radius),
   a.rightCenter.2 - a.radius * cos (s / a.radius))

/-- The possibly degenerate bottom segment. -/
```

TypeThreeAssembly.rightArcParam_image_Icc

Classification: supplement

Source: TypeThreeAssembly.lean:547-648

Declaration SHA-256: d47f88fb5eadc69e4f4bc7ce3bc201dc34d60ad570f34d3d29c4d0184ef0f8f1

Mathematical statement. The right arc parametrization maps $[0, R\phi]$ exactly onto the right coordinate arc trace.

The circle and strip constraints give one inclusion; inverse trigonometric reconstruction on the declared arc gives the reverse inclusion.

```
theorem rightArcParam_image_Icc :
  a.rightArcParam ''
    Icc 0 (a.radius * a.sideArcAngle) = a.rightArcTrace := by
  ext p
  constructor
  · rintro ⟨s, hs, rfl⟩
    have hu :
      s / a.radius ∈ Icc 0 a.sideArcAngle := by
      constructor
      · exact div_nonneg hs.1 a.radius_pos.le
      · apply (div_le_iff₀ a.radius_pos).2
        nlinarith [hs.2, a.radius_pos]
    have hsin_nonneg : 0 ≤ sin (s / a.radius) :=
      Real.sin_nonneg_of_nonneg_of_le_pi hu.1
      (hu.2.trans a.sideArcAngle_lt_pi.le)
```

```

have habs : |s / a.radius| ≤ a.sideArcAngle := by
  rw [abs_le]
  exact ⟨by linarith [hu.1, a.sideArcAngle_pos], hu.2⟩
have hcos :
  cos a.sideArcAngle ≤ cos (s / a.radius) := by
  simpa only [Real.cos_abs] using
    (Real.cos_le_cos_of_nonneg_of_le_pi
      (abs_nonneg (s / a.radius)) a.sideArcAngle_lt_pi.le habs)
have hcircle := Real.sin_sq_add_cos_sq (s / a.radius)
change
  ((a.rightCenter.1 + a.radius * sin (s / a.radius)) -
    a.rightCenter.1) ^ 2 +
    ((a.rightCenter.2 - a.radius * cos (s / a.radius)) -
      a.rightCenter.2) ^ 2 = a.radius ^ 2 ^
  a.rightCenter.1 ≤
    a.rightCenter.1 + a.radius * sin (s / a.radius) ^
  (-1 : ℝ) ≤
    a.rightCenter.2 - a.radius * cos (s / a.radius) ^
  a.rightCenter.2 - a.radius * cos (s / a.radius) ≤ 1
refine ⟨by nlinarith [sq_nonneg a.radius], ?_, ?_, ?_⟩
· nlinarith [mul_nonneg a.radius_pos.le hsin_nonneg]
· change
  (-1 : ℝ) ≤
    (a.radius - 1) - a.radius * cos (s / a.radius)
  nlinarith [mul_le_mul_of_nonneg_left
    (Real.cos_le_one (s / a.radius)) a.radius_pos.le]
· change
  (a.radius - 1) - a.radius * cos (s / a.radius) ≤ 1
  nlinarith [mul_le_mul_of_nonneg_left hcos a.radius_pos.le,
    a.radius_mul_cos_sideArcAngle]
· intro hp
  change
    (p.1 - a.rightCenter.1) ^ 2 +
      (p.2 - a.rightCenter.2) ^ 2 = a.radius ^ 2 ^
    a.rightCenter.1 ≤ p.1 ^
    (-1 : ℝ) ≤ p.2 ^ p.2 ≤ 1 at hp
let z : ℂ :=
  ⟨a.rightCenter.2 - p.2, p.1 - a.rightCenter.1⟩
have hnorm_sq : ||z|| ^ 2 = a.radius ^ 2 := by
  rw [Complex.sq_norm, Complex.normSq_apply]
  simp only [z]
  nlinarith [hp.1]
have hnorm : ||z|| = a.radius := by
  nlinarith [norm_nonneg z, a.radius_pos]
have hsin :
  a.radius * sin z.arg = p.1 - a.rightCenter.1 := by
  simpa [hnorm, z] using Complex.norm_mul_sin_arg z
have hcos :
  a.radius * cos z.arg = a.rightCenter.2 - p.2 := by
  simpa [hnorm, z] using Complex.norm_mul_cos_arg z
have hcos_le : cos a.sideArcAngle ≤ cos z.arg := by
  have hmul :
    a.radius * cos a.sideArcAngle ≤ a.radius * cos z.arg := by
    rw [a.radius_mul_cos_sideArcAngle, hcos]
    change a.radius - 2 ≤ (a.radius - 1) - p.2
    linarith [hp.2.2.2]
    nlinarith [hmul, a.radius_pos]
have harg : |z.arg| ≤ a.sideArcAngle := by
  by_contra h
  have hlt : a.sideArcAngle < |z.arg| := lt_of_not_ge h
  have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
    a.sideArcAngle_pos.le (Complex.abs_arg_le_pi z) hlt
  rw [Real.cos_abs] at hc
  linarith
have harg_nonneg : 0 ≤ z.arg := by
  by_contra h
  have harg_neg : z.arg < 0 := lt_of_not_ge h
  have harg_gt_neg_pi : -π < z.arg :=
    lt_of_lt_of_le (neg_lt_neg a.sideArcAngle_lt_pi)
      (abs_le.mp harg).1
  have hsin_neg :=
    Real.sin_neg_of_neg_of_neg_pi_lt harg_neg harg_gt_neg_pi

```

```

    nlinarith [a.radius_pos, hp.2.1]
  refine (a.radius * z.arg, ?_, ?_)
  · constructor
  · exact mul_nonneg a.radius_pos.le harg_nonneg
  · nlinarith [a.radius_pos, (abs_le.mp harg).2]
  · have hangle : a.radius * z.arg / a.radius = z.arg := by
      field_simp [a.radius_pos.ne']
    apply Prod.ext
  · simp only [rightArcParam, hangle]
    linarith
  · simp only [rightArcParam, hangle]
    linarith

```

```

/-- Euclidean isometry that places the standard signed-arclength circle at a
strip-side center, with the orientation used by both side parameters. -/

```

TypeThreeAssembly.rightArcTrace

Classification: supplement

Source: TypeThreeAssembly.lean:111-119

Declaration SHA-256: 828cebab0c56e67a2a3cfaae9e912a8a76f497287dcea6b589e1a22a40edd616

Mathematical statement. The right trace consists of points on the circle centered at $(b, R - 1)$ with radius R , satisfying

$$x \geq b$$

and

$$-1 \leq y \leq 1.$$

It includes the top and bottom endpoints of the exposed right arc.

```

def rightArcTrace : Set PlanePoint :=
  {p | (p.1 - a.rightCenter.1) ^ 2 + (p.2 - a.rightCenter.2) ^ 2 =
    a.radius ^ 2 ^
    a.rightCenter.1 ≤ p.1 ^
    (-1 : ℝ) ≤ p.2 ^ p.2 ≤ 1}

/-- Signed-arclength parameter for the left strip-side half-circle. The
parameter runs from the upper junction at -radius * sideArcAngle to the
bottom point at zero. -/

```

TypeThreeAssembly.rightCenter

Classification: supplement

Source: TypeThreeAssembly.lean:73-75

Declaration SHA-256: c1d50740a1dfb76a9c424f19f6e78589f8a8ab930ab1727da5282aecd42c3f25

Mathematical statement. Define the right circle center $(b, R - 1)$.

It is the vertical-axis reflection of the left center.

```

def rightCenter : PlanePoint := (a.sideHalfWidth, a.radius - 1)

/-- Source circle center for the exterior upper arc. -/

```

TypeThreeAssembly.rightSegmentCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:92-98

Declaration SHA-256: 889596cedc4d7d957005581540478e3a458f9f3d1f37c9af6b1a713eee05b078

Mathematical statement. Define the radius- R disk centered at $(b, R - 1)$, clipped by

$$x \geq b$$

and

$$-1 \leq y \leq 1.$$

The right clipped disk is the reflection of the left one.

```
def rightSegmentCarrier : Set PlanePoint :=
  {p | (p.1 - a.rightCenter.1) ^ 2 + (p.2 - a.rightCenter.2) ^ 2 ≤
    a.radius ^ 2} ∩
  ({p | a.rightCenter.1 ≤ p.1} ∩
    ({p | (-1 : ℝ) ≤ p.2} ∩ {p | p.2 ≤ 1}))

/-- The part of the type-(iii) carrier inside the closed strip. -/
```

TypeThreeAssembly.rightUpperJunction

Classification: supplement

Source: TypeThreeAssembly.lean:958-968

Declaration SHA-256: ae6c6a14cc5b76c0f3aa60155a1fb16c629e4d67ae173c7593aa69c135bbff39

Mathematical statement. The point $(b + R \sin \beta, 1)$ equals the outer cap's right endpoint.

The same width identity identifies the right attaching junction exactly.

```
theorem rightUpperJunction :
  (a.rightCenter.1 + a.radius * sin a.innerAngle, 1) =
    a.outerCap.rightEndpoint := by
  rw [a.outerCap_rightEndpoint]
  change
    (a.sideHalfWidth + a.radius * sin a.innerAngle, 1) =
      (a.radius * sin a.outerAngle, 1)
  rw [sideHalfWidth]
  ring_nf

/-- The core and upper cap overlap exactly along the upper cap's chord. -/
```

TypeThreeAssembly.scalarWeightedArea

Classification: supplement

Source: TypeThreeAssembly.lean:295-299

Declaration SHA-256: 195fdb382725116907cc721bab689c5c6f6f34b356a442f900ba649d89b62b9c

Mathematical statement. Define the scalar area

$$[T + (q + 2)\delta]/h^2.$$

After the angle and sine identifications, this is exactly the article's $A_3(h)$.

```
def scalarWeightedArea : ℝ :=
  (a.angleTerm + (a.shapeParameter + 2) * a.sineGap) / a.h ^ 2
```

```
/-- Scalar type-(iii) weighted-perimeter formula used by the suffix
certificate. -/
```

TypeThreeAssembly.scalarWeightedPerimeter

Classification: supplement

Source: TypeThreeAssembly.lean:300-302

Declaration SHA-256: fcbac6602a2b133a8e714c37e1ef51bfb698ab0ae754dcd7eca0cb6e9c49204d

Mathematical statement. Define the scalar perimeter

$$2(T + \delta)/h.$$

This is the article's $P_3(h)$, including the exposed bottom segment.

```
def scalarWeightedPerimeter : ℝ :=
  2 * (a.angleTerm + a.sineGap) / a.h
```

TypeThreeAssembly.shapeParameter

Classification: supplement

Source: TypeThreeAssembly.lean:50-52

Declaration SHA-256: 29190b0ec0676db3ee8da889fc7b1e078bce75188f6b4271945e3fee045f2920

Mathematical statement. Define

$$q = 2h - 1.$$

This affine shape parameter belongs to $(-1, 1)$ on the assembly domain.

```
def shapeParameter : ℝ := 2 * a.h - 1

/-- The source angle `b₃`, represented by its principal arccosine. -/
```

TypeThreeAssembly.shapeParameter_mem

Classification: supplement

Source: TypeThreeAssembly.lean:153-155

Declaration SHA-256: 018fe2d07fb99b9420ec19a75327dc616f0119968b1c22fb83265de47b1261df

Mathematical statement. For every assembly,

$$-1 < q < 1.$$

This follows directly from $0 < h < 1$ and

$$q = 2h - 1.$$

```
theorem shapeParameter_mem : -1 < a.shapeParameter ∧ a.shapeParameter < 1 := by
  constructor <=> simp only [shapeParameter] <=> linarith [a.h_pos, a.h_lt_one]
```

TypeThreeAssembly.sideArcAngle

Classification: supplement

Source: TypeThreeAssembly.lean:64-65

Declaration SHA-256: c68849312591ff361810874588f817eab8327b61ad9b815244ad1ab61a55597a

Mathematical statement. Define

$$\phi = \pi - \beta.$$

Each strip arc turns through ϕ , while the auxiliary symmetric side cap has central angle 2ϕ .

```
def sideArcAngle : ℝ := π - a.innerAngle
/-- Half the bottom-segment length. -/
```

TypeThreeAssembly.sideArcAngle_lt_pi

Classification: supplement

Source: TypeThreeAssembly.lean:223-226

Declaration SHA-256: 27c4b01d9c8710ab675792367c9a16f3b7952720c452f471c97b46d487e08c08

Mathematical statement. For every assembly,

$$\phi < \pi.$$

The inner angle is strictly positive.

```
theorem sideArcAngle_lt_pi : a.sideArcAngle < π := by
  rw [sideArcAngle]
  linarith [a.innerAngle_pos]
```

TypeThreeAssembly.sideArcAngle_pos

Classification: supplement

Source: TypeThreeAssembly.lean:219-222

Declaration SHA-256: fb3e4cdd6ff568be2285a238c6293705a5d18ec17ca0000cf9fc5fd3be6f53e2

Mathematical statement. For every assembly,

$$\phi > 0.$$

It is the supplement of an angle strictly below π .

```
theorem sideArcAngle_pos : 0 < a.sideArcAngle := by
  rw [sideArcAngle]
  linarith [a.innerAngle_lt_pi]
```

TypeThreeAssembly.sideArcComplexIsometryMap

Classification: supplement

Source: TypeThreeAssembly.lean:649-652

Declaration SHA-256: 2d2b0ae6442b1aeb7779e0e2c761b53071a3b1948418645b546ba443dcc3b322

Mathematical statement. For any planar center c , define the complex-to-Euclidean map

$$z \mapsto (c_x + \operatorname{Im} z, c_y - \operatorname{Re} z).$$

It carries the standard circle to a circle centered at c , with the orientation used for the inner arcs.

```
noncomputable def sideArcComplexIsometryMap
  (center : PlanePoint) (z : ℂ) : EuclideanPlane :=
  WithLp.toLp 2 (center.1 + z.im, center.2 - z.re)
```

TypeThreeAssembly.sideCap

Classification: supplement

Source: TypeThreeAssembly.lean:238-250

Declaration SHA-256: 62a5fb2c6003ebc71ae1cea156d819929dd2061bdfd35d03ac33c3bf896e8fd0

Mathematical statement. Define the auxiliary lower cap with chord

$$2R\sin\beta,$$

half-angle ϕ , midpoint $(0, 1)$, and lower orientation.

Its area equals the sum of the two translated side pieces. It is a calculation device, not an additional exterior cap in the competitor.

```
def sideCap : OneSidedCircularCap where
  chord := 2 * a.radius * sin a.innerAngle
  theta := a.sideArcAngle
  midpointX := 0
  baseY := 1
  side := .lower
  chord_pos := mul_pos (mul_pos (by norm_num) a.radius_pos) a.innerSin_pos
  theta_pos := a.sideArcAngle_pos
  theta_lt_pi := a.sideArcAngle_lt_pi

/-- The upper one-sided cap. `OneSidedCircularCap` deliberately permits a
half-central-angle above  $\pi / 2$ , so this definition covers the major branch.
-/
```

TypeThreeAssembly.sideCap_arcLength

Classification: supplement

Source: TypeThreeAssembly.lean:865-874

Declaration SHA-256: 0735084a84605f457abd29f126bfc3b41f13fe2e361355b34befa71a89121ae9

Mathematical statement. The auxiliary side-cap arc length is

$$2R\phi.$$

This equals the sum of the two strip arc lengths, each

$$R\phi.$$

```
theorem sideCap_arcLength :
  a.sideCap.arcLength = 2 * a.radius * a.sideArcAngle := by
  rw [OneSidedCircularCap.arcLength, ell]
  change
    (2 * a.radius * sin a.innerAngle) *
    (a.sideArcAngle / sin a.sideArcAngle) =
    2 * a.radius * a.sideArcAngle
  rw [a.sin_sideArcAngle]
  field_simp [ne_of_gt a.innerSin_pos]
```

TypeThreeAssembly.sideCap_center

Classification: supplement

Source: TypeThreeAssembly.lean:430-438

Declaration SHA-256: 5c32cc16475c4c5293c8fcbce19421c65fe608e9916172122a7cb5ff16d6b389

Mathematical statement. The auxiliary side cap has center $(0, R - 1)$.

For its lower orientation the center height is

$$1 + R\cos\phi = 1 - Rq = R - 1,$$

using

$$Rh = 1.$$

```
theorem sideCap_center :
  a.sideCap.center = (0, a.radius - 1) := by
  change
    (0, 1 + a.sideCap.radius * cos a.sideArcAngle) =
      (0, a.radius - 1)
  rw [a.sideCap.radius, a.cos_sideArcAngle, radius, shapeParameter]
  field_simp [ne_of_gt a.h_pos]
  ring
```

TypeThreeAssembly.sideCap_euclideanArea

Classification: supplement

Source: TypeThreeAssembly.lean:833-850

Declaration SHA-256: b6b94a03ab653ee46e011e7fa757125fdbdfe26f1473cd6911d9100fe92583e4

Mathematical statement. The auxiliary side-cap area is

$$R^2(\phi - \sin\phi\cos\phi).$$

Substitute its chord and radius into the normalized circular-segment formula.

```
theorem sideCap_euclideanArea :
  a.sideCap.euclideanArea =
    a.radius ^ 2 *
      (a.sideArcAngle -
        sin a.sideArcAngle * cos a.sideArcAngle) := by
  rw [OneSidedCircularCap.euclideanArea, area]
  change
    (2 * a.radius * sin a.innerAngle) ^ 2 *
      ((a.sideArcAngle -
        sin a.sideArcAngle * cos a.sideArcAngle) /
        (4 * sin a.sideArcAngle ^ 2)) =
    a.radius ^ 2 *
      (a.sideArcAngle -
        sin a.sideArcAngle * cos a.sideArcAngle)
  rw [a.sin_sideArcAngle]
  field_simp [ne_of_gt a.innerSin_pos]
  ring
```

TypeThreeAssembly.sideCap_radius

Classification: supplement

Source: TypeThreeAssembly.lean:423-429

Declaration SHA-256: 860b391d38da56536b3934963761766a28238619240605477033823fc900d1af

Mathematical statement. The auxiliary side cap has radius R .

Its chord uses

$$\sin\beta = \sin\phi,$$

which cancels the sine of its cap half-angle.

```
theorem sideCap_radius : a.sideCap.radius = a.radius := by
  change
    (2 * a.radius * sin a.innerAngle) /
      (2 * sin a.sideArcAngle) = a.radius
```

```
rw [a.sin_sideArcAngle]
field_simp [ne_of_gt a.innerSin_pos]
```

TypeThreeAssembly.sideHalfWidth

Classification: supplement

Source: TypeThreeAssembly.lean:66-69

Declaration SHA-256: c1a6374b950f6fdf60b58062fd5753e6cafd953a8fbac46cb6164191b66af1b3

Mathematical statement. Define

$$b = R(\sin\alpha - \sin\beta).$$

It is the half-length of the exposed bottom segment and equals

$$D(q)/h.$$

```
def sideHalfWidth : ℝ :=
  a.radius * (sin a.outerAngle - sin a.innerAngle)

/-- Source circle center for the left strip arc. -/
```

TypeThreeAssembly.sideHalfWidth_eq_zero_iff

Classification: supplement

Source: TypeThreeAssembly.lean:344-351

Declaration SHA-256: aa31042682af8c9fc65e3a19a4a9ce7dc71ed7e2ad3e807732dcfcc2da35644f

Mathematical statement. For an admissible assembly,

$$b = 0$$

if and only if $h = 1/2$.

At one half both angles are

$$\pi/2;$$

away from one half the gap is strictly positive.

```
theorem sideHalfWidth_eq_zero_iff : a.sideHalfWidth = 0 ↔ a.h = 1 / 2 := by
  constructor
  · intro hzero
    by_contra hh
    linarith [a.sideHalfWidth_pos hh]
  · intro hh
    simp [sideHalfWidth, innerAngle, outerAngle, outerArgument, shapeParameter, hh]
```

TypeThreeAssembly.sideHalfWidth_nonneg

Classification: supplement

Source: TypeThreeAssembly.lean:338-340

Declaration SHA-256: be70cb71b25ed2f43754461915097af9183cc7aac3504655dda98d8a67b605ec

Mathematical statement. For every assembly, $b \geq 0$.

It is the positive radius times a nonnegative sine gap.

```
theorem sideHalfWidth_nonneg : 0 ≤ a.sideHalfWidth := by
  exact mul_nonneg a.radius_pos.le (sub_nonneg.mpr a.sin_inner_le_sin_outer)
```

TypeThreeAssembly.sideHalfWidth_pos

Classification: supplement

Source: TypeThreeAssembly.lean:341-343

Declaration SHA-256: a769284707d6bf25198d747cd5846bae22f2f394cc9d09e601cfcab854579d7a

Mathematical statement. If $h \neq 1/2$, then

$$b > 0.$$

Use strict positivity of the sine gap away from the zero shape parameter.

```
theorem sideHalfWidth_pos (hh : a.h ≠ 1 / 2) : 0 < a.sideHalfWidth := by
  exact mul_pos a.radius_pos (sub_pos.mpr (a.sin_inner_lt_sin_outer hh))
```

TypeThreeAssembly.sin_inner_le_sin_outer

Classification: supplement

Source: TypeThreeAssembly.lean:303-311

Declaration SHA-256: def468198882d033570072298a83ae1d41fc003f0ecb9d0b4ee3a6b94e6a1d10

Mathematical statement. For every assembly,

$$\sin \beta \leq \sin \alpha.$$

Compare

$$\sqrt{1 - q^2}$$

and

$$\sqrt{1 - q^2/\lambda^2},$$

using $\lambda > 1$.

```
theorem sin_inner_le_sin_outer : sin a.innerAngle ≤ sin a.outerAngle := by
  rw [innerAngle, outerAngle, Real.sin_arccos, Real.sin_arccos]
  apply Real.sqrt_le_sqrt
  have hlam_pos : 0 < lam := lt_trans (by norm_num) a.density_jump
  have habs : |a.outerArgument| ≤ |a.shapeParameter| := by
    rw [outerArgument, abs_div, abs_of_pos hlam_pos]
    exact div_le_self (abs_nonneg _) (le_of_lt a.density_jump)
  exact sub_le_sub_left (sq_le_sq.mpr habs) 1
```

TypeThreeAssembly.sin_inner_lt_sin_outer

Classification: supplement

Source: TypeThreeAssembly.lean:312-337

Declaration SHA-256: b3a6348544f17baf0cf9a8d51663f5b8b6187359294117163fc5d2afba78dd58

Mathematical statement. If an assembly has

$$h \neq 1/2,$$

then

$$\sin \beta < \sin \alpha.$$

Then $q \neq 0$, so rescaling its square by

$$\lambda^2 > 1$$

gives a strict radical comparison.

```
theorem sin_inner_lt_sin_outer (hh : a.h ≠ 1 / 2) :
  sin a.innerAngle < sin a.outerAngle := by
  rw [innerAngle, outerAngle, Real.sin_arccos, Real.sin_arccos]
  have hlam_pos : 0 < lam := lt_trans (by norm_num) a.density_jump
  have hx : a.shapeParameter ≠ 0 := by
    intro hzero
    apply hh
    simp only [shapeParameter] at hzero
  linarith
  have hx_sq : 0 < a.shapeParameter ^ 2 := sq_pos_of_ne_zero hx
  have hlam_sq : 1 < lam ^ 2 := by nlinarith [a.density_jump]
  have harg_sq : a.outerArgument ^ 2 < a.shapeParameter ^ 2 := by
    rw [outerArgument, div_pow]
    apply (div_lt_iff0 (sq_pos_of_pos hlam_pos)).2
    nlinarith
  have habs : |a.shapeParameter| < 1 := abs_lt.mpr a.shapeParameter_mem
  have hshape_sq : a.shapeParameter ^ 2 ≤ 1 := by
    have habs_le : |a.shapeParameter| ≤ |(1 : ℝ)| := by
      simp using le_of_lt habs
    have hsq : a.shapeParameter ^ 2 ≤ (1 : ℝ) ^ 2 :=
      sq_le_sq.mpr habs_le
    simp using hsq
  have hleft : 0 ≤ 1 - a.shapeParameter ^ 2 := by
    linarith
  exact Real.sqrt_lt_sqrt hleft (by nlinarith)
```

TypeThreeAssembly.sin_sideArcAngle

Classification: supplement

Source: TypeThreeAssembly.lean:227-230

Declaration SHA-256: 41165bd4a88985079154632e1304282b8868bb98cc6b1d97bb3ea19faaebe0dd

Mathematical statement. For every assembly,

$$\sin\phi = \sin\beta.$$

Sine is unchanged under supplementation.

```
theorem sin_sideArcAngle :
  sin a.sideArcAngle = sin a.innerAngle := by
  rw [sideArcAngle, Real.sin_pi_sub]
```

TypeThreeAssembly.sineGap

Classification: supplement

Source: TypeThreeAssembly.lean:292-294

Declaration SHA-256: e9d7a7074e27fe28a23aa0db9c4cf9b0ef4e5f2181f1cfab6098f7cc2e240ecb

Mathematical statement. Define the sine gap

$$\delta = \sin\alpha - \sin\beta.$$

The coordinate bridge identifies it with $D(q)$, and the side half-width is

$$R\delta.$$

```
def sineGap : ℝ := sin a.outerAngle - sin a.innerAngle

/-- Scalar type-(iii) weighted-area formula used by the suffix certificate. -/
```

TypeThreeAssembly.upperCenter

Classification: supplement

Source: TypeThreeAssembly.lean:76-79

Declaration SHA-256: 6dabb1e2d2dfee1af758b047dfb6728a97191f4c64742b75a35f102c81c1c3c1

Mathematical statement. Define the upper circle center $(0, 1 - R\cos\alpha)$.

Its intersections with the top interface have half-chord

$$R\sin\alpha.$$

```
def upperCenter : PlanePoint :=
  (0, 1 - a.radius * cos a.outerAngle)

/-- Closed central rectangle of `K₃`. -/
```

TypeThreeAssembly.volume_coreCarrier

Classification: supplement

Source: TypeThreeAssembly.lean:1481-1529

Declaration SHA-256: e422839d8a82e578aa8619bf8a6d240c5dc840fd4d4e1880ecc09a0307558380

Mathematical statement. The planar volume of the core equals the rectangle area plus the auxiliary side-cap area, embedded in the nonnegative extended reals.

Decompose the two side pieces by translation and reflection, use circular-segment volumes, and remove only planar-null overlaps.

```
theorem volume_coreCarrier :
  volume a.coreCarrier =
    ENNReal.ofReal (a.rectangleArea + a.sideCap.euclideanArea) := by
  have hrectLeft :
    AEDisjoint volume a.rectangleCarrier a.leftSegmentCarrier := by
    refine measure_mono_null ?_ (volume_verticalline (-a.sideHalfWidth))
    rintro p ⟨hrect, hleft⟩
    exact le_antisymm hleft.2.1 hrect.1.1
  have hfirstRight :
    AEDisjoint volume
      (a.rectangleCarrier ∪ a.leftSegmentCarrier)
      a.rightSegmentCarrier := by
    refine measure_mono_null ?_ (volume_verticalline a.sideHalfWidth)
    rintro p ⟨hfirst, hright⟩
    rcases hfirst with hrect | hleft
    · exact le_antisymm hrect.1.2 hright.2.1
    · have hxl : p.1 ≤ -a.sideHalfWidth := hleft.2.1
      have hxr : a.sideHalfWidth ≤ p.1 := hright.2.1
      exact le_antisymm (by linarith [a.sideHalfWidth_nonneg]) hxr
  have hrectangle : 0 ≤ a.rectangleArea := by
    exact mul_nonneg (by norm_num) a.sideHalfWidth_nonneg
  calc
    volume a.coreCarrier =
      volume a.rectangleCarrier + volume a.leftSegmentCarrier +
        volume a.rightSegmentCarrier := by
  rw [coreCarrier,
    measure_union₀
      a.isClosed_rightSegmentCarrier.measurableSet.nullMeasurableSet
      hfirstRight,
    measure_union₀
      a.isClosed_leftSegmentCarrier.measurableSet.nullMeasurableSet
      hrectLeft]
  _ = ENNReal.ofReal a.rectangleArea +
    (volume a.sideCapLeftHalf + volume a.sideCapRightHalf) := by
  rw [a.volume_rectangleCarrier, a.volume_leftSegmentCarrier,
    a.volume_rightSegmentCarrier, add_assoc]
  _ = ENNReal.ofReal a.rectangleArea + volume a.sideCap.carrier := by
```

```

    rw [a.volume_sideCap_partition]
  _ = ENNReal.ofReal a.rectangleArea +
    ENNReal.ofReal a.sideCap.euclideanArea := by
    rw [a.sideCap.volume_carrier]
  _ = ENNReal.ofReal
    (a.rectangleArea + a.sideCap.euclideanArea) := by
    exact
    (ENNReal.ofReal_add hrectangle
      a.sideCap.euclideanArea_pos.le).symm

/-- The actual density integral over the strip core is its Euclidean component
area. -/

```

TypeThreeAssembly.weightedArea_eq_modeledWeightedArea

Classification: supplement

Source: TypeThreeAssembly.lean:1562-1583

Declaration SHA-256: 5f7913f0171c133683a735330062b186a060d0e518987c5488549de7bfd06347

Mathematical statement. The actual weighted-area integral of the three-arc carrier equals the modeled rectangle-and-cap sum.

The core and upper cap overlap only on the top interface, an area-null set; apply the core and weighted cap integral formulas.

```

theorem weightedArea_eq_modeledWeightedArea :
  a.WeightedArea = a.modeledWeightedArea := by
  have hcore : IntegrableOn (StripDensity lam) a.coreCarrier := by
    apply stripDensity_integrableOn_of_volume_ne_top
    rw [a.volume_coreCarrier]
    exact ENNReal.ofReal_ne_top
  have hcap := cap_integrableOn lam a.outerCap
  unfold TypeThreeAssembly.WeightedArea _root_.WeightedArea
  rw [carrier,
    setIntegral_union₀ a.carrier_core_outer_aedisjoint
    a.outerCap.measurableSet_carrier.nullMeasurableSet hcore hcap]
  rw [show (∫ p in a.coreCarrier, StripDensity lam p) =
    a.rectangleArea + a.sideCap.euclideanArea by
    simp [a.carrier_core_outer_aedisjoint] using a.core_weightedArea_formula,
    show (∫ p in a.outerCap.carrier, StripDensity lam p) =
    lam * a.outerCap.euclideanArea by
    simp [a.carrier_core_outer_aedisjoint] using
    cap_upper_weightedArea lam a.outerCap rfl rfl,
    modeledWeightedArea]

/-- CMV equation (26) is the exact canonical weighted area of the coordinate
carrier, not only a sum assigned to its modeled components. -/

```

TypeThreeAssembly.weightedPerimeter_eq_modeledWeightedPerimeter

Classification: supplement

Source: TypeThreeAssembly.lean:2950-2993

Declaration SHA-256: 3dd334c8f3cfff9146f23a5d90e6da2520151c567d49c8604446b90aa7a9f4d2b

Mathematical statement. The actual complete-frontier weighted perimeter equals the modeled sum of the two strip arc lengths, weighted exterior arc length and bottom chord length.

Use the exact Euclidean frontier, null pairwise overlaps, exact Hausdorff lengths, and the density-one bottom interface. The whole segment remains counted.

```

theorem weightedPerimeter_eq_modeledWeightedPerimeter :
  a.WeightedPerimeter = a.modeledWeightedPerimeter := by
  rcases a.boundaryTrace_pairwise with

```

```

(hAB, hAC, hAD, hBC, hBD, hCD)
have hAint := euclideanStripDensity_integrable0n_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph '' a.leftArcTrace) (by
    rw [a.exact_euclidean_leftArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)
have hBint := euclideanStripDensity_integrable0n_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph '' a.rightArcTrace) (by
    rw [a.exact_euclidean_rightArcTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)
have hCint := euclideanStripDensity_integrable0n_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph ''
    OneSidedCircularCap.arcTrace a.outerCap) (by
    rw [exact_euclidean_capTrace_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)
have hDint := euclideanStripDensity_integrable0n_of_measure_ne_top
  lam (s := planeEuclideanHomeomorph '' a.bottomSegmentCarrier) (by
    rw [a.exact_euclidean_bottomSegmentCarrier_hausdorffMeasure]
    exact ENNReal.ofReal_ne_top)
unfold TypeThreeAssembly.WeightedPerimeter _root_.WeightedPerimeter
FrontierMeasure
rw [integral_map
  planeEuclideanHomeomorph.symm.continuous.measurable.aemeasurable
  (measurable_stripDensity lam).aestronglyMeasurable]
change (∫ z in frontier (planeEuclideanHomeomorph '' a.carrier),
  euclideanStripDensity lam z
  ∂(μH[1] : Measure EuclideanPlane)) =
  a.modeledWeightedPerimeter
rw [a.euclidean_frontier]
rw [integral_four_union hAB hAC hAD hBC hBD hCD
  a.measurableSet_euclidean_rightArcTrace
  (measurableSet_euclidean_capTrace a.outerCap)
  a.measurableSet_euclidean_bottomSegmentCarrier
  hAint hBint hCint hDint]
rw [a.leftArc_integral, a.rightArc_integral,
  upper_cap_integral lam a.outerCap rfl rfl,
  a.bottomSegment_integral, modeledWeightedPerimeter,
  a.sideCap_arcLength]
ring

/-- CMV equation (26) is the exact canonical weighted perimeter of the
coordinate carrier, not only a sum assigned to its modeled components. -/

```

area

Classification: supplement

Source: MorgansArcFunction.lean:92-93

Declaration SHA-256: 8c44e8dff416b2208afe91aad54bf1678ad6f2299c75592fe1fed4f6cbf55dc2

Mathematical statement. For real θ , define

$$a(\theta) = (\theta - \sin\theta\cos\theta)/(4\sin^2\theta).$$

For $0 < \theta < \pi$, this is the area of the circular segment with unit chord and half-central-angle θ , including major segments.

```
noncomputable def area (θ : ℝ) : ℝ := (θ - sin θ * cos θ) / (4 * sin θ ^ 2)
```

area_pos

Classification: supplement

Source: MorgansArcFunction.lean:148-157

Declaration SHA-256: 0af41b50988d0bace92fc2e638571302802d8538245a2c254bedb9f164463ede

Mathematical statement. For $0 < \theta < \pi$, the normalized segment area satisfies

$$a(\theta) > 0.$$

The numerator is strictly positive on this interval and the squared sine denominator is positive; scaling by a positive chord square gives positive cap area.

```
lemma area_pos {θ : ℝ} (hθ : 0 < θ) (hπ : θ < π) : 0 < area θ := by
  have hs : 0 < sin θ := sin_pos_of_pos_of_lt_pi hθ hπ
  have hnum : 0 < θ - sin θ * cos θ := by
    nlinarith [sin_lt hθ, neg_one_le_cos θ, cos_le_one θ, sq_nonneg (sin θ)]
  rw [area]
  positivity

/-- On `(0, 1]` the area is bounded above by `θ`. The constants come from
`sin θ > θ - θ³/6`: the numerator is `< 2θ³/3` and `< 4 sin²θ > 25θ²/9`, so
`area θ < 6θ/25`. -/
```

capBoundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:1249-1251

Declaration SHA-256: 5dbd12f9bc57860ad2ce92a638514e0e8d15006685aca7388e2fa2e1689504cd

Mathematical statement. For a cap, define the pushforward of Lebesgue measure on $[-R\theta, R\theta]$ by its arclength parametrization.

This is an auxiliary arc measure, not the perimeter of the entire closed cap including its chord.

```
def capBoundaryMeasure (c : OneSidedCircularCap) : Measure PlanePoint :=
  intervalPushforward (capParam c) (capStart c) (capEnd c)
```

capBoundaryMeasure_integrable

Classification: supplement

Source: CMVGeometry.lean:2260-2267

Declaration SHA-256: 0065afd47de261467dd963dc3f68bd9fc5e9eab2c424cffffb1d00f2e7c2919a7

Mathematical statement. For any real λ and cap, the strip density is integrable against its parametrized arc measure.

The continuous arc parametrization and finite arclength interval satisfy the general pushforward-integrability lemma.

```
lemma capBoundaryMeasure_integrable (lam : ℝ) (c : OneSidedCircularCap) :
  Integrable (StripDensity lam) (capBoundaryMeasure c) := by
  rw [capBoundaryMeasure]
  exact stripDensity_integrable_intervalPushforward lam (by
    apply Continuous.measurable
    unfold capParam OneSidedCircularCap.arcPoint
    split <;> fun_prop)
```

capComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:390-397

Declaration SHA-256: 3ed78deaf1597a91c6218ca8fe10519ba693355e7329e0301f17fcce226f5a43

Mathematical statement. For a cap with midpoint (m, b) , radius R , and half-angle θ , map $z \in \mathbb{C}$ to $(m + \text{Im}z, b - R\cos\theta + \text{Re}z)$ for an upper cap, and to $(m + \text{Im}z, b + R\cos\theta - \text{Re}z)$ for a lower cap.

These maps take the standard circular parametrization into the Euclidean coordinate realization of the cap.

```
noncomputable def capComplexIsometryMap (c : OneSidedCircularCap) (z : ℂ) :
  EuclideanPlane :=
  match c.side with
```

```
| .upper => WithLp.toLp 2
  (c.midpointX + z.im, c.baseY - c.radius * Real.cos c.theta + z.re)
| .lower => WithLp.toLp 2
  (c.midpointX + z.im, c.baseY + c.radius * Real.cos c.theta - z.re)
```

capEnd

Classification: supplement

Source: CMVGeometry.lean:1246-1248

Declaration SHA-256: a140ca726638df619aef507f8019eb9f7ca9503d3231f8bfa8f0a64feb377161

Mathematical statement. The final arclength coordinate of a cap is

$$R\theta.$$

The span of the parameter interval is

$$2R\theta,$$

the exact selected-arc length.

```
def capEnd (c : OneSidedCircularCap) : ℝ := c.radius * c.theta

/-- Explicit regular boundary measure of a one-sided circular arc. -/
```

capParam

Classification: supplement

Source: CMVGeometry.lean:1241-1243

Declaration SHA-256: d80d6cd5283176fe8b29770f039d1135a8892539144575a4bc1d9962ad427168

Mathematical statement. For a cap with radius R , define its arclength-coordinate parametrization by

$$s \mapsto c(s/R),$$

where $c(t)$ is its angle parametrization.

The selected parameter interval runs from $-R\theta$ to $R\theta$; the speed is one in the Euclidean realization.

```
def capParam (c : OneSidedCircularCap) (s : ℝ) : PlanePoint :=
  c.arcPoint (s / c.radius)
```

capParam_image_Icc_eq_arcTrace

Classification: supplement

Source: FrontierPerimeter.lean:947-1070

Declaration SHA-256: f412e9294ab946fde4ef494447fa75b0b57b0600c2bd61007639df1bb35c0e9e

Mathematical statement. For every cap, the image of its coordinate arclength parametrization on $[-R\theta, R\theta]$ is exactly its selected arc trace.

The forward inclusion follows from the circle and half-plane identities. Inverse trigonometric reconstruction gives a parameter for every trace point, including major-arc points.

```
theorem capParam_image_Icc_eq_arcTrace (c : OneSidedCircularCap) :
  capParam c '' Icc (capStart c) (capEnd c) = c.arcTrace := by
  apply Set.Subset.antisymm
  · rintro p ⟨s, hs, rfl⟩
    have ht : s / c.radius ∈ Icc (-c.theta) c.theta := by
      have hr := c.radius_pos
      constructor
```

```

· apply (le_div_iff₀ hr).2
  simp [capStart, mul_comm] using hs.1
· apply (div_le_iff₀ hr).2
  simp [capEnd, mul_comm] using hs.2
have habs : |s / c.radius| ≤ c.theta := (abs_le).2 ht
constructor
· cases hside : c.side <=>
  simp only [capParam, OneSidedCircularCap.radiusSquaredAt,
    OneSidedCircularCap.center, OneSidedCircularCap.arcPoint, hside]
  all_goals
    nlinarith [Real.sin_sq_add_cos_sq (s / c.radius)]
· exact c.arcPoint_in_closed_side habs
intro p hp
cases hside : c.side with
| upper =>
  let z : ℂ := ⟨p.2 - (c.baseY - c.radius * Real.cos c.theta),
    p.1 - c.midpointX⟩
  have hcircle :
    (p.1 - c.midpointX) ^ 2 +
    (p.2 - (c.baseY - c.radius * Real.cos c.theta)) ^ 2 =
    c.radius ^ 2 := by
    simp [OneSidedCircularCap.arcTrace,
      OneSidedCircularCap.radiusSquaredAt,
      OneSidedCircularCap.center, hside] using hp.1
  have hnorm_sq : ||z|| ^ 2 = c.radius ^ 2 := by
    rw [Complex.sq_norm, Complex.normSq_apply]
    simp only [z]
    nlinarith
  have hnorm : ||z|| = c.radius := by
    nlinarith [norm_nonneg z, c.radius_pos]
  have hsin :
    c.radius * Real.sin z.arg = p.1 - c.midpointX := by
    simp [hnorm, z] using Complex.norm_mul_sin_arg z
  have hcos :
    c.radius * Real.cos z.arg =
    p.2 - (c.baseY - c.radius * Real.cos c.theta) := by
    simp [hnorm, z] using Complex.norm_mul_cos_arg z
  have hsidep : c.baseY ≤ p.2 := by
    simp [OneSidedCircularCap.arcTrace, hside] using hp.2
  have hcos_le : Real.cos c.theta ≤ Real.cos z.arg := by
    nlinarith [c.radius_pos]
  have harg : |z.arg| ≤ c.theta := by
    by_contra h
    have hlt : c.theta < |z.arg| := lt_of_not_ge h
    have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
      (le_of_lt c.theta_pos) (Complex.abs_arg_le_pi z) hlt
    rw [Real.cos_abs] at hc
    linarith
  let s := c.radius * z.arg
  have hs : s ∈ Icc (capStart c) (capEnd c) := by
    rcases (abs_le.mp harg) with ⟨hl, hu⟩
    dsimp [s]
    constructor <=> simp only [capStart, capEnd] <=>
      nlinarith [c.radius_pos]
  refine ⟨s, hs, ?_⟩
  have hangle : s / c.radius = z.arg := by
    dsimp [s]
    field_simp [ne_of_gt c.radius_pos]
  apply Prod.ext
  · simp only [capParam, OneSidedCircularCap.arcPoint, hside,
    hangle]
    linarith
  · simp only [capParam, OneSidedCircularCap.arcPoint, hside,
    hangle]
    linarith
| lower =>
  let z : ℂ := ⟨(c.baseY + c.radius * Real.cos c.theta) - p.2,
    p.1 - c.midpointX⟩
  have hcircle :
    (p.1 - c.midpointX) ^ 2 +
    (p.2 - (c.baseY + c.radius * Real.cos c.theta)) ^ 2 =
    c.radius ^ 2 := by

```

```

    simp [OneSidedCircularCap.arcTrace,
      OneSidedCircularCap.radiusSquaredAt,
      OneSidedCircularCap.center, hside] using hp.1
have hnorm_sq : ||z|| ^ 2 = c.radius ^ 2 := by
  rw [Complex.sq_norm, Complex.normSq_apply]
  simp only [z]
  nlinarith
have hnorm : ||z|| = c.radius := by
  nlinarith [norm_nonneg z, c.radius_pos]
have hsin :
  c.radius * Real.sin z.arg = p.1 - c.midpointX := by
  simp [hnorm, z] using Complex.norm_mul_sin_arg z
have hcos :
  c.radius * Real.cos z.arg =
    (c.baseY + c.radius * Real.cos c.theta) - p.2 := by
  simp [hnorm, z] using Complex.norm_mul_cos_arg z
have hsidep : p.2 ≤ c.baseY := by
  simp [OneSidedCircularCap.arcTrace, hside] using hp.2
have hcos_le : Real.cos c.theta ≤ Real.cos z.arg := by
  nlinarith [c.radius_pos]
have harg : |z.arg| ≤ c.theta := by
  by_contra h
  have hlt : c.theta < |z.arg| := lt_of_not_ge h
  have hc := Real.cos_lt_cos_of_nonneg_of_le_pi
    (le_of_lt c.theta_pos) (Complex.abs_arg_le_pi z) hlt
  rw [Real.cos_abs] at hc
  linarith
let s := c.radius * z.arg
have hs : s ∈ Icc (capStart c) (capEnd c) := by
  rcases (abs_le.mp harg) with ⟨hl, hu⟩
  dsimp [s]
  constructor <=> simp only [capStart, capEnd] <=>
    nlinarith [c.radius_pos]
refine ⟨s, hs, ?_⟩
have hangle : s / c.radius = z.arg := by
  dsimp [s]
  field_simp [ne_of_gt c.radius_pos]
apply Prod.ext
· simp only [capParam, OneSidedCircularCap.arcPoint, hside,
  hangle]
  linarith
· simp only [capParam, OneSidedCircularCap.arcPoint, hside,
  hangle]
  linarith

```

capPartitionPoint

Classification: supplement

Source: FrontierPerimeter.lean:154-156

Declaration SHA-256: 49ac6ecbe67b9040843eeee672d7b0f08055016781814c3ce0fa4e7d31ff805a

Mathematical statement. Define

$$t_i = -R\theta + i(2R\theta/N)$$

for real R, θ and natural numbers N, i .

For positive subdivision count these are uniform arclength subdivision points; the definition is total also at zero count.

```

def capPartitionPoint (r theta : ℝ) (N i : ℕ) : ℝ :=
  -r * theta + (i : ℝ) * (2 * r * theta / (N : ℝ))

```

capPartitionPoint_dist

Classification: supplement

Source: FrontierPerimeter.lean:185-207

Declaration SHA-256: 6b90645e53e16e3855618b966b04af3ee0f15439043926ba1133c583e65b5a00

Mathematical statement. For $R > 0$,

$$0 < \theta < \pi,$$

natural $N > 0$, and any natural i , the distance between consecutive circle partition points is

$$2R\sin(\theta/N).$$

Their angular separation is

$$2\theta/N.$$

The half-angle sine is nonnegative, so the absolute value in the chord-distance formula simplifies.

```
lemma capPartitionPoint_dist {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) {N i : ℕ} (hN : 0 < N) :
  dist (unitCircleArc r (capPartitionPoint r theta N i))
    (unitCircleArc r (capPartitionPoint r theta N (i + 1))) =
    2 * r * Real.sin (theta / (N : ℝ)) := by
  rw [unitCircleArc_dist hr]
  have hNreal : 0 < (N : ℝ) := by positivity
  have harg :
    ((capPartitionPoint r theta N i - capPartitionPoint r theta N (i + 1)) / r) / 2 =
      -(theta / (N : ℝ)) := by
    unfold capPartitionPoint
    push_cast
    field_simp [hr.ne', hNreal.ne']
    ring
  have hangle_pos : 0 < theta / (N : ℝ) := div_pos htheta.1 hNreal
  have hN_one : (1 : ℝ) ≤ (N : ℝ) := by exact_mod_cast hN
  have hangle_lt : theta / (N : ℝ) < Real.pi := by
    apply (div_lt_iff₀ hNreal).2
    nlinarith [htheta.2, Real.pi_pos]
  rw [harg, Real.sin_neg, abs_mul, abs_of_pos (by norm_num : (0 : ℝ) < 2),
    abs_neg, abs_of_pos (Real.sin_pos_of_pos_of_lt_pi hangle_pos hangle_lt)]
  ring
```

capPartitionPoint_end

Classification: supplement

Source: FrontierPerimeter.lean:161-166

Declaration SHA-256: b5d0e986af4c921461bd13ce89490a41fdc02991eaadda7d26f70db465413519

Mathematical statement. For $N > 0$, the partition point with index N is

$$R\theta.$$

Cancellation of the nonzero subdivision count gives the right endpoint exactly.

```
lemma capPartitionPoint_end {r theta : ℝ} {N : ℕ} (hN : 0 < N) :
  capPartitionPoint r theta N N = r * theta := by
  unfold capPartitionPoint
  field_simp [Nat.ne_of_gt hN]
  ring
```

capPartitionPoint_mem

Classification: supplement

Source: FrontierPerimeter.lean:175-184

Declaration SHA-256: 8f26d09621c071ac8a6a1935a21253dfb29cb269e0ec21dc71e59a40f0529e9f

Mathematical statement. For $R > 0$,

$$\theta > 0,$$

natural $N > 0$, and

$$i \leq N,$$

the point t_i lies in $[-R\theta, R\theta]$.

Monotonicity bounds every subdivision point by the computed first and last points.

```
lemma capPartitionPoint_mem {r theta : ℝ} (hr : 0 < r) (htheta : 0 < theta)
  {N i : ℕ} (hN : 0 < N) (hi : i ≤ N) :
  capPartitionPoint r theta N i ∈ Icc (-r * theta) (r * theta) := by
  have hm := monotone_capPartitionPoint hr htheta hN
  constructor
  · rw [- capPartitionPoint_zero r theta N]
    exact hm (Nat.zero_le i)
  · rw [- capPartitionPoint_end hN]
    exact hm hi
```

capPartitionPoint_zero

Classification: supplement

Source: FrontierPerimeter.lean:157-160

Declaration SHA-256: 1714782661300a21ee2d4243c7e4fbe79e1e97ce545ea187699c13d709db2db9

Mathematical statement. For all real R, θ and natural N , the zeroth partition point is

$$-R\theta.$$

The zero-index term vanishes independently of the division convention.

```
lemma capPartitionPoint_zero (r theta : ℝ) (N : ℕ) :
  capPartitionPoint r theta N 0 = -r * theta := by
  simp [capPartitionPoint]
```

capStart

Classification: supplement

Source: CMVGeometry.lean:1244-1245

Declaration SHA-256: 594eb6e2bbf3cbc7a2397b6af7a93506027168fa75ecf7f70b7012b2fc6e208a

Mathematical statement. The initial arclength coordinate of a cap is

$$-R\theta.$$

Together with the final coordinate it describes the full selected arc, including both endpoints.

```
def capStart (c : OneSidedCircularCap) : ℝ := -c.radius * c.theta
```

cap_arclength_eq_two_radius_theta

Classification: supplement

Source: FrontierPerimeter.lean:451-456

Declaration SHA-256: edad6e24948a1499ce1a66a5efdc84d9c8f7899f5c52d6551716312e95163fb3

Mathematical statement. For any cap, its scalar arc length is

$$2R\theta.$$

Substitute

$$L\theta/\sin\theta$$

and the radius definition, using positive sine to cancel the denominator.

```

lemma cap_arclength_eq_two_radius_theta (c : OneSidedCircularCap) :
  c.arclength = 2 * c.radius * c.theta := by
  rw [OneSidedCircularCap.arclength, OneSidedCircularCap.radius, ell]
  field_simp [ne_of_gt c.sin_theta_pos]

/-- Exact Euclidean H1 mass for every existing `OneSidedCircularCap`, including major arcs. -/

```

cap_integrableOn

Classification: supplement

Source: CMVGeometry.lean:2303-2308

Declaration SHA-256: bbc4a1a9af0d45bee6aa1c1152d956e0e945a869f3550ca289ef7022556292ef

Mathematical statement. For every real λ and admissible cap, f_λ is integrable over its carrier.

The computed finite cap volume implies integrability of the bounded density.

```

lemma cap_integrableOn (lam : ℝ) (c : OneSidedCircularCap) :
  IntegrableOn (StripDensity lam) c.carrier := by
  apply stripDensity_integrableOn_of_volume_ne_top
  rw [c.volume_carrier]
  exact ENNReal.ofReal_ne_top

```

cap_lower_external_density

Classification: supplement

Source: CMVGeometry.lean:1341-1352

Declaration SHA-256: baaf83c56acf8edc18268269454b4946af9c7098831da004f1da856c3f80d9b2

Mathematical statement. For any real λ , a lower cap based on

$$y = -1$$

has density λ at every parametrized arc point with

$$-R\theta < s < R\theta.$$

Strictly interior arc parameters lie below the lower interface, including for a major cap.

```

theorem cap_lower_external_density (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .lower) (hbase : c.baseY = -1)
  {s : ℝ} (hs : s ∈ Ioo (capStart c) (capEnd c)) :
  StripDensity lam (capParam c s) = lam := by
  have hopen := c.arcPoint_in_open_side (cap_angle_mem_open hs)
  have hy : (capParam c s).2 < -1 := by
    simpa [capParam, hside, hbase] using hopen
  rw [StripDensity, if_neg]

```

```

apply not_le.mpr
exact (show 1 < -(capParam c s).2 by linarith).trans_le (neg_le_abs _)

/-- Upper external cap: weighted integral equals `lam` times arc length. -/

```

cap_lower_weightedArea

Classification: supplement

Source: CMVGeometry.lean:2360-2389

Declaration SHA-256: af4bcab3c0cdacd24a9ba22f365ac164c96fd2be59faf0a944ca162de8e4ada2

Mathematical statement. For real λ and a lower cap based on

$$y = -1,$$

its weighted area equals λ times its Euclidean area.

The lower exterior has constant density λ ; the base chord has zero planar measure.

```

lemma cap_lower_weightedArea (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .lower) (hbase : c.baseY = -1) :
  WeightedArea lam c.carrier = lam * c.euclideanArea := by
  have hne : ∀ᵐ p : PlanePoint ∂volume, p.2 ≠ -1 := by
    rw [ae_iff]
    simpa only [not_not] using volume_horizontalline (-1)
  rw [WeightedArea]
  calc
    (f p in c.carrier, StripDensity lam p) =
      f _p in c.carrier, lam := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem c.measurableSet_carrier,
          ae_restrict_of_ae hne] with p hp hpne
        have hple : p.2 ≤ (-1 : ℝ) := by
          simpa [OneSidedCircularCap.carrier, hside, hbase] using hp.2
        have hplt : p.2 < (-1 : ℝ) := lt_of_le_of_ne hple hpne
        rw [StripDensity, if_neg]
        apply not_le.mpr
        exact (show 1 < -p.2 by linarith).trans_le (neg_le_abs p.2)
    _ = lam * c.euclideanArea := by
      rw [integral_const, measureReal_restrict_apply_univ]
      have hvolume : volume.real c.carrier = c.euclideanArea := by
        rw [Measure.real, c.volume_carrier,
          ENNReal.toReal_ofReal c.euclideanArea_pos.le]
      rw [hvolume]
      ring

/-- The coordinate four-arc assembly: two side arcs from the strip core and two
congruent reflected exterior caps. -/

```

cap_lower_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1373-1394

Declaration SHA-256: 5575ee03ce1e095004c86962659c2bcf1dde6c0b1ad90b439220173f4e11e7fc

Mathematical statement. For a lower cap based on

$$y = -1$$

and any real λ , the density integral against its parametrized arc measure equals λ times its arc length.

The open arc lies in the lower exterior; its endpoint exceptions have zero parameter measure.

```

theorem cap_lower_weightedIntegral (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .lower) (hbase : c.baseY = -1) :
  ∫ p, StripDensity lam p ∂capBoundaryMeasure c = lam * c.arcLength := by
rw [capBoundaryMeasure]
rw [integral_intervalPushforward (capParam_measurable c) (measurable_stripDensity lam)]
rw [integral_Icc_eq_integral_Ioo]
calc
  (∫ s in Ioo (capStart c) (capEnd c), StripDensity lam (capParam c s)) =
    ∫ _s in Ioo (capStart c) (capEnd c), lam := by
      apply integral_congr_ae
      filter_upwards [ae_restrict_mem measurableSet_Ioo] with s hs
      exact cap_lower_external_density lam c hside hbase hs
_ = lam * (capEnd c - capStart c) := by
  have hlen : 0 ≤ capEnd c - capStart c := by
    dsimp [capEnd, capStart]
    nlinarith [c.radius_pos, c.theta_pos]
  simp [max_eq_left hlen, mul_comm]
_ = lam * c.arcLength := by rw [cap_arcLength_eq c]

/-! ## Horizontal segments -/

/-- Unit-speed left-to-right segment parametrization. -/

```

cap_partition_polygon_lower

Classification: supplement

Source: FrontierPerimeter.lean:208-268

Declaration SHA-256: 4dc7fcd936b6c56a4643f00c8d67c38ce24d6351f625a8873960fc070d61a446

Mathematical statement. For $R > 0$,

$$0 < \theta < \pi,$$

and natural $N > 0$,

$$N2R\sin(\theta/N) \leq \mathcal{H}^1(\gamma_R([-R\theta, R\theta])).$$

Split into injective half-open subarcs. Their measures add, and each subarc measure bounds its endpoint chord distance; null endpoints do not affect the total.

```

lemma cap_partition_polygon_lower {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) (N : ℕ) (hN : 0 < N) :
  ENNReal.ofReal ((N : ℝ) * (2 * r * Real.sin (theta / (N : ℝ)))) ≤
  (μH[1] : Measure C)
  (unitCircleArc r '' Icc (-r * theta) (r * theta)) := by
let p : ℕ → ℝ := capPartitionPoint r theta N
let A : ℕ → Set ℝ := fun i => Ico (p i) (p (i + 1))
let S : Set ℝ := Icc (-r * theta) (r * theta)
let f : ℝ → C := unitCircleArc r
let μ : Measure C := μH[1]
have hpmmono : Monotone p := monotone_capPartitionPoint hr htheta.1 hN
have hinj : Set.InjOn f S := unitCircleArc_injOn hr htheta
have hpiece_sub (i : ℕ) (hi : i < N) : Icc (p i) (p (i + 1)) ⊆ S := by
  intro x hx
  exact ((capPartitionPoint_mem hr htheta.1 hN hi.le).1.trans hx.1,
    hx.2.trans (capPartitionPoint_mem hr htheta.1 hN (Nat.succ_le_iff.mpr hi)).2)
have hpair : Set.Pairwise (↑(Finset.range N))
  (Function.onFun Disjoint fun i => f '' A i) := by
  intro i hi j hj hij
  change Disjoint (f '' A i) (f '' A j)
  have hsrc : Disjoint (A i) (A j) := hpmmono.pairwise_disjoint_on_Ico_succ hij
  rw [Set.disjoint_left]
  rintro y ⟨x, hxi, rfl⟩ ⟨z, hzj, hzx⟩
  have hiN : i < N := Finset.mem_range.mp hi
  have hjN : j < N := Finset.mem_range.mp hj
  have hxS : x ∈ S := hpiece_sub i hiN ⟨hxi.1, hxi.2.le⟩
  have hzS : z ∈ S := hpiece_sub j hjN ⟨hzj.1, hzj.2.le⟩

```

```

have hxz : x = z := hnj hxS hzS hzx.symm
exact (Set.disjoint_left.mp hsrc) hxi (hxz ▸ hzj)
have hmeas (i : N) (hi : i ∈ Finset.range N) : MeasurableSet (f '' A i) := by
  have hiN : i < N := Finset.mem_range.mp hi
  rw [show f '' A i = f '' Icc (p i) (p (i + 1)) \ {f (p (i + 1))} by
    exact image_Ico_eq_image_Icc_diff_endpoint hnj (hpmmono (Nat.le_succ i))
    (hpiece_sub i hiN)]
  exact (isCompact_Icc.image (continuous_unitCircleArc r)).measurableSet.diff
    (measurableSet_singleton _)
have hunion_sub : (U i ∈ Finset.range N, f '' A i) ⊆ f '' S := by
  intro y hy
  rcases mem_iUnion.mp hy with ⟨i, hy⟩
  rcases mem_iUnion.mp hy with ⟨hi, hy⟩
  rcases hy with ⟨x, hxA, rfl⟩
  exact ⟨x, hpiece_sub i (Finset.mem_range.mp hi) ⟨hxA.1, hxA.2.le⟩, rfl⟩
calc
  ENNReal.ofReal ((N : ℝ) * (2 * r * Real.sin (theta / (N : ℝ)))) =
    ∑ i ∈ Finset.range N,
      ENNReal.ofReal (2 * r * Real.sin (theta / (N : ℝ))) := by
        rw [ENNReal.ofReal_mul (by positivity), ENNReal.ofReal_natCast]
        simp [Finset.sum_const, nsmul_eq_mul]
  _ ≤ ∑ i ∈ Finset.range N, μ (f '' A i) := by
    apply Finset.sum_le_sum
    intro i hi
    have hiN : i < N := Finset.mem_range.mp hi
    rw [hausdorffMeasure_image_Ico_eq_Icc hnj (hpmmono (Nat.le_succ i))
      (hpiece_sub i hiN)]
    simp [f, p, μ, capPartitionPoint_dist hr htheta hN] using
      ofReal_dist_le_hausdorffMeasure_image_Icc (continuous_unitCircleArc r)
      (hpmmono (Nat.le_succ i))
  _ = μ (U i ∈ Finset.range N, f '' A i) :=
    (measure_biUnion_finset hpair hmeas).symm
  _ ≤ μ (f '' S) := measure_mono hunion_sub

```

cap_upper_external_density

Classification: supplement

Source: CMVGeometry.lean:1329-1340

Declaration SHA-256: 0d4d779bde08a636e8536f594b78dfa9705a31925f9d2e9248d0d62fdf5aa11a

Mathematical statement. For any real λ , an upper cap based on

$$y = 1$$

has density λ at every parametrized arc point with

$$-R\theta < s < R\theta.$$

Strictly interior arc parameters lie above the interface; the density-one endpoints are excluded from this pointwise assertion.

```

theorem cap_upper_external_density (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .upper) (hbase : c.baseY = 1)
  {s : ℝ} (hs : s ∈ Ioo (capStart c) (capEnd c)) :
  StripDensity lam (capParam c s) = lam := by
  have hopen := c.arcPoint_in_open_side (cap_angle_mem_open hs)
  have hy : 1 < (capParam c s).2 := by
    simp [capParam, hside, hbase] using hopen
  rw [StripDensity, if_neg]
  exact not_le.mpr (hy.trans_le (le_abs_self _))

/-- The non-endpoint part of a lower cap based on `y = -1` is in the external
(density `lam`) branch. -/

```

cap_upper_weightedArea

Classification: supplement

Source: CMVGeometry.lean:2334-2359

Declaration SHA-256: 2f2165b544886739ec40b854c29666e67f06c7a423bd046c04eb224a5c6ee0ce

Mathematical statement. For real λ and an upper cap based on

$$y = 1,$$

its weighted area equals λ times its Euclidean area.

Off the area-null chord line, every point is in the exterior density branch; the carrier-volume theorem evaluates the resulting constant integral.

```
lemma cap_upper_weightedArea (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .upper) (hbase : c.baseY = 1) :
  WeightedArea lam c.carrier = lam * c.euclideanArea := by
  have hne : ∀ᵐ p : PlanePoint ∂volume, p.2 ≠ 1 := by
    rw [ae_iff]
    simpa only [not_not] using volume_horizontalline 1
  rw [WeightedArea]
  calc
    (∫ p in c.carrier, StripDensity lam p) =
      ∫ _p in c.carrier, lam := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem c.measurableSet_carrier,
          ae_restrict_of_ae hne] with p hp hpne
        have hpge : (1 : ℝ) ≤ p.2 := by
          simpa [OneSidedCircularCap.carrier, hside, hbase] using hp.2
        have hpgt : (1 : ℝ) < p.2 := lt_of_le_of_ne hpge (Ne.symm hpne)
        rw [StripDensity, if_neg]
        exact not_le.mpr (hpgt.trans_le (le_abs_self p.2))
    _ = lam * c.euclideanArea := by
      rw [integral_const, measureReal_restrict_apply_univ]
      have hvolume : volume.real c.carrier = c.euclideanArea := by
        rw [Measure.real, c.volume_carrier,
          ENNReal.toReal_ofReal c.euclideanArea_pos.le]
      rw [hvolume]
      ring
```

cap_upper_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1353-1372

Declaration SHA-256: 45644ff66dcb2be9e3cb6c626527848bf909f532df9be34ffa1977e24d25b8f4

Mathematical statement. For an upper cap based on

$$y = 1$$

and any real λ , the density integral against its parametrized arc measure equals λ times its arc length.

The density is constant except at the two parameter endpoints, a Lebesgue-null set, so the endpoint density convention does not change the integral.

```
theorem cap_upper_weightedIntegral (lam : ℝ) (c : OneSidedCircularCap)
  (hside : c.side = .upper) (hbase : c.baseY = 1) :
  ∫ p, StripDensity lam p ∂capBoundaryMeasure c = lam * c.arcLength := by
  rw [capBoundaryMeasure]
  rw [integral_intervalPushforward (capParam_measurable c) (measurable_stripDensity lam)]
  rw [integral_Icc_eq_integral_Ioo]
  calc
    (∫ s in Ioo (capStart c) (capEnd c), StripDensity lam (capParam c s)) =
      ∫ _s in Ioo (capStart c) (capEnd c), lam := by
```

```

    apply integral_congr_ae
    filter_upwards [ae_restrict_mem measurableSet_Ioo] with s hs
    exact cap_upper_external_density lam c hside hbase hs
  _ = lam * (capEnd c - capStart c) := by
  have hlen : 0 ≤ capEnd c - capStart c := by
    dsimp [capEnd, capStart]
    nlinarith [c.radius_pos, c.theta_pos]
  simp [max_eq_left hlen, mul_comm]
  _ = lam * c.arclength := by rw [cap_arclength_eq c]

/-- Lower external cap: weighted integral equals `lam` times arc length. -/

```

continuous_realizedCapParam

Classification: supplement

Source: FrontierPerimeter.lean:442-450

Declaration SHA-256: 439602ca8a442d23dec8f858626a27ca71e9723ce90b04b56ee6e94cd89590f1

Mathematical statement. Every realized cap parametrization is continuous.

It is the composition of the continuous circular parametrization and a Euclidean isometry.

```

lemma continuous_realizedCapParam (c : OneSidedCircularCap) :
  Continuous (realizedCapParam c) := by
  rw [show realizedCapParam c = capComplexIsometryMap c ∘
    unitCircleArc c.radius by
    funext s
    exact realizedCapParam_eq c s]
  exact (isometry_capComplexIsometryMap c).continuous.comp
    (continuous_unitCircleArc c.radius)

```

continuous_realizedLeftCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:727-735

Declaration SHA-256: 4dc1fbf3a43e8117f791c26e28bd5a2a2466716abc2324d51a2522841ccc46bf

Mathematical statement. The realized left core parametrization is continuous.

It is a composition of continuous trigonometric functions and the coordinate homeomorphism.

```

lemma continuous_realizedLeftCoreParam (c : StripCore) :
  Continuous (realizedLeftCoreParam c) := by
  rw [show realizedLeftCoreParam c = leftCoreComplexIsometryMap c ∘
    unitCircleArc c.radius by
    funext s
    exact realizedLeftCoreParam_eq c s]
  exact (isometry_leftCoreComplexIsometryMap c).continuous.comp
    (continuous_unitCircleArc c.radius)

```

continuous_realizedRightCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:736-744

Declaration SHA-256: db06971d2e0a6e54b3ce1b2cb1ffe296cb5ed7ac07495118f10c8f5f82bfe132

Mathematical statement. The realized right core parametrization is continuous.

It is a composition of continuous trigonometric functions and the coordinate homeomorphism.

```

lemma continuous_realizedRightCoreParam (c : StripCore) :
  Continuous (realizedRightCoreParam c) := by

```

```
rw [show realizedRightCoreParam c = rightCoreComplexIsometryMap c °
    unitCircleArc c.radius by
  funext s
  exact realizedRightCoreParam_eq c s]
exact (isometry_rightCoreComplexIsometryMap c).continuous.comp
      (continuous_unitCircleArc c.radius)
```

continuous_unitCircleArc

Classification: supplement

Source: FrontierPerimeter.lean:14-17

Declaration SHA-256: 343bc4472a64687ca829ebbf897c060cb6c7c3cad3716259e96fb84cf091470c

Mathematical statement. For every real R , the function

$$s \mapsto \gamma_R(s)$$

is continuous.

Complex exponential and real scalar operations are continuous, including the total convention at zero radius.

```
lemma continuous_unitCircleArc (r : ℝ) : Continuous (unitCircleArc r) := by
  unfold unitCircleArc
  fun_prop
```

coreArcEnd

Classification: supplement

Source: CMVGeometry.lean:1474-1476

Declaration SHA-256: 7ee06e3535f2fd49529132649f006dc8125adff906e2bb1c8e325a0ca4b835a4

Mathematical statement. For a strip core with radius R and side angle β , define the final arc parameter

$$R\beta.$$

Each side arc has parameter span $2R\beta$.

```
def coreArcEnd (c : StripCore) : ℝ := c.radius * c.sideAngle

/-- Unit-speed parametrization of the left core arc. -/
```

coreArcStart

Classification: supplement

Source: CMVGeometry.lean:1472-1473

Declaration SHA-256: 4c395d23f09d7608737696a9d4043f61933ddaffbb62953e4470c1085d950b4e

Mathematical statement. For a strip core with radius R and side angle β , define the initial arc parameter

$$-R\beta.$$

The same interval is used for each side arc.

```
def coreArcStart (c : StripCore) : ℝ := -c.radius * c.sideAngle
```

core_integrableOn

Classification: supplement

Source: CMVGeometry.lean:2309-2314

Declaration SHA-256: ebecbc727d05931a9afed5e77aa787703ada65169aa70e3f372bec6e04bddcac

Mathematical statement. For every real λ and strip core, f_λ is integrable over its carrier.

Use the finite core volume and the bounded-density lemma.

```
lemma core_integrableOn (lam : ℝ) (c : StripCore) :  
  IntegrableOn (StripDensity lam) c.carrier := by  
  apply stripDensity_integrableOn_of_volume_ne_top  
  rw [c.volume_carrier]  
  exact ENNReal.ofReal_ne_top
```

ell

Classification: supplement

Source: MorgansArcFunction.lean:89-91

Declaration SHA-256: 9adb32c6bf9324acec1e791c0ec46f0e17df835ffee7b3549cab07bdc3a760c9

Mathematical statement. For real θ , define

$$\ell(\theta) = \theta / \sin \theta.$$

For $0 < \theta < \pi$, this is the circular arc length normalized by chord length; the definition is total outside that interval as well.

```
noncomputable def ell (θ : ℝ) : ℝ := θ / sin θ  
  
/-- Area enclosed by a unit-chord circular segment with half-central-angle `θ`. -/
```

euclideanJoinSet

Classification: supplement

Source: FrontierPerimeter.lean:1257-1259

Declaration SHA-256: 96d51d86f6c32a7f1c31ecb0290be17bec48e5d48a1aa38fdfbae4457025d389

Mathematical statement. The Euclidean junction set is the coordinate homeomorphism image of the planar junction set.

It has identical point coordinates and is used with Euclidean Hausdorff measure.

```
def euclideanJoinSet (q : ℝ) : Set EuclideanPlane :=  
  planeEuclideanHomeomorph '' joinSet q
```

euclideanJoinSet_finite

Classification: supplement

Source: FrontierPerimeter.lean:1260-1262

Declaration SHA-256: 9539ffd4336468eab518f7e951c9dc4db7ee2d6bd08f0fa98bbda9dcf0817f38

Mathematical statement. The Euclidean junction set is finite for every real chord parameter.

Images of finite sets are finite.

```
theorem euclideanJoinSet_finite (q : ℝ) : (euclideanJoinSet q).Finite :=  
  (joinSet_finite q).image planeEuclideanHomeomorph
```

euclideanPairwiseOverlap

Classification: supplement

Source: FrontierPerimeter.lean:1591-1597

Declaration SHA-256: c1559aa1737fb404d99a3376b70fbd4f0ff514e71a947a7d30b01a1ce4c1177e

Mathematical statement. For four subsets A, B, C, D of any type, define the union of their six pairwise intersections.

Despite its name, this is a generic set expression; it is instantiated in the Euclidean plane for measure arguments.

```
def euclideanPairwiseOverlap {α : Type*}
  (A B C D : Set α) : Set α :=
  (A ∩ B) ∪ ((A ∩ C) ∪ ((A ∩ D) ∪
    ((B ∩ C) ∪ ((B ∩ D) ∪ (C ∩ D)))))

/-- A null union of all pairwise overlaps makes the four sets pairwise
almost disjoint. -/
```

euclideanStripDensity

Classification: supplement

Source: FrontierPerimeter.lean:1389-1391

Declaration SHA-256: 3d074997fd7c2f4c47fc25c7e8a9a39259d6fd4628b103b9b3e4da77e1679848

Mathematical statement. Define the Euclidean density as the coordinate density composed with the inverse coordinate homeomorphism.

It has the same values at the same coordinates, with Euclidean measure semantics.

```
def euclideanStripDensity (lam : ℝ) (z : EuclideanPlane) : ℝ :=
  StripDensity lam (planeEuclideanHomeomorph.symm z)
```

euclideanStripDensity_integrableOn_of_measure_ne_top

Classification: supplement

Source: FrontierPerimeter.lean:1404-1411

Declaration SHA-256: 509b7e5c9b45ab61b5e958fce825f92bda813640616ddf5c7591ac420818c5d2

Mathematical statement. For any real λ and Euclidean set S with

$$\mathcal{H}^1(S) < \infty,$$

the Euclidean strip density is integrable over S against $\mathcal{H}^1$.

Measurability and a global finite bound on the piecewise constant density give integrability on finite measure sets.

```
lemma euclideanStripDensity_integrableOn_of_measure_ne_top
  (lam : ℝ) {s : Set EuclideanPlane}
  (hs : (μH[1] : Measure EuclideanPlane) s ≠ ∞) :
  IntegrableOn (euclideanStripDensity lam) s (μH[1] : Measure EuclideanPlane) := by
  apply Measure.integrableOn_of_bounded (M := 1 + |lam|) hs
  · exact (measurable_euclideanStripDensity lam).aestronglyMeasurable
  · exact Filter.Eventually.of_forall (euclideanStripDensity_bound lam)
```

exact_euclidean_capParam_hausdorffMeasure

Classification: supplement

Source: FrontierPerimeter.lean:457-483

Declaration SHA-256: ce3a2c8ff8b380f7da5859563d117de2848052560e457ef5f88e25aa88808f43

Mathematical statement. For every cap, the Euclidean Hausdorff measure of its realized parameter image on $[-R\theta, R\theta]$ equals its scalar arc length.

An isometry carries the standard arc to this image, so the exact circular Hausdorff-length theorem applies to every admissible half-angle.

```
theorem exact_euclidean_capParam_hausdorffMeasure (c : OneSidedCircularCap) :
  (μH[1] : Measure EuclideanPlane)
    (realizedCapParam c '' Icc (capStart c) (capEnd c)) =
    ENNReal.ofReal c.arcLength := by
  have hfun : realizedCapParam c = capComplexIsometryMap c ∘ unitCircleArc c.radius := by
    funext s
    exact realizedCapParam_eq c s
  calc
    (μH[1] : Measure EuclideanPlane)
      (realizedCapParam c '' Icc (capStart c) (capEnd c)) =
      (μH[1] : Measure EuclideanPlane)
        (capComplexIsometryMap c ''
          (unitCircleArc c.radius '' Icc (capStart c) (capEnd c))) := by
        rw [hfun]
        congr 1
        simp only [Function.comp_apply] using (Set.image_image _ _ _).symm
  _ = (μH[1] : Measure C)
    (unitCircleArc c.radius '' Icc (capStart c) (capEnd c)) :=
    (isometry_capComplexIsometryMap c).hausdorffMeasure_image
    (Or.inl (by norm_num)) _
  _ = ENNReal.ofReal (2 * c.radius * c.theta) := by
    simp [capStart, capEnd] using
      hausdorffMeasure_unitCircleArc_image_eq c.radius_pos
      ⟨c.theta_pos, c.theta_lt_pi⟩
  _ = ENNReal.ofReal c.arcLength := by rw [cap_arcLength_eq_two_radius_theta]
```

exact_euclidean_leftArcTrace_hausdorffMeasure

Classification: supplement

Source: FrontierPerimeter.lean:821-827

Declaration SHA-256: f6b756b038d2dd53ff9e214034534494d6216a6823376bb73c389a754a1ad0d9

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the left exposed strip-core arc is

$$2R\beta.$$

Use the realized trace-image equality and exact parametrized arc measure.

```
theorem exact_euclidean_leftArcTrace_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
    (planeEuclideanHomeomorph '' c.leftArcTrace) =
    ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  rw [← realizedLeftCoreParam_image_Icc]
  exact exact_euclidean_leftCoreParam_hausdorffMeasure c
```

exact_euclidean_leftCoreParam_hausdorffMeasure

Classification: supplement

Source: FrontierPerimeter.lean:745-772

Declaration SHA-256: e0211c9e8b0941bc0bbeb3bd977afe28d78688025726aac98a4d738bf7f180e9

Mathematical statement. The Euclidean Hausdorff measure of the realized left core parameter image on $[-R\beta, R\beta]$ is

$$2R\beta.$$

The side angle lies in $(0, \pi/2]$, so the standard circular arc theorem applies after an isometry. The source expresses the result as final minus initial arclength parameter.

```
theorem exact_euclidean_leftCoreParam_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
  (realizedLeftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) =
  ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  have hfun : realizedLeftCoreParam c =
    leftCoreComplexIsometryMap c ∘ unitCircleArc c.radius := by
    funext s
    exact realizedLeftCoreParam_eq c s
  calc
  _ = (μH[1] : Measure EuclideanPlane)
    (leftCoreComplexIsometryMap c ''
      (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c))) := by
    rw [hfun]
    congr 1
    simpa only [Function.comp_apply] using (Set.image_image _ _ _).symm
  _ = (μH[1] : Measure ℂ)
    (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c)) :=
    (isometry_leftCoreComplexIsometryMap c).hausdorffMeasure_image
      (Or.inl (by norm_num)) _
  _ = ENNReal.ofReal (2 * c.radius * c.sideAngle) := by
    simpa [coreArcStart, coreArcEnd] using
      hausdorffMeasure_unitCircleArc_image_eq c.radius_pos
        ⟨c.sideAngle_pos, c.sideAngle_lt_pi⟩
  _ = ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
    congr 1
    simp only [coreArcStart, coreArcEnd]
    ring
```

exact_euclidean_rightArcTrace_hausdorffMeasure

Classification: supplement

Source: FrontierPerimeter.lean:828-834

Declaration SHA-256: 47e43c0cab0cb015904a2fe672eeec0720a25537d058b25dd2b767b05bcf1d39

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of the right exposed strip-core arc is

$$2R\beta.$$

Use the realized trace-image equality and exact parametrized arc measure.

```
theorem exact_euclidean_rightArcTrace_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' c.rightArcTrace) =
  ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  rw [← realizedRightCoreParam_image_Icc]
  exact exact_euclidean_rightCoreParam_hausdorffMeasure c
```

exact_euclidean_rightCoreParam_hausdorffMeasure

Classification: supplement

Source: FrontierPerimeter.lean:773-800

Declaration SHA-256: 5b9d17b9e5921b7fba027be4ebfa3845e68c3c43a92a67ce9d4854bf1b244db0

Mathematical statement. The Euclidean Hausdorff measure of the realized right core parameter image on $[-R\beta, R\beta]$ is

$$2R\beta.$$

The side angle lies in $(0, \pi/2]$, so the standard circular arc theorem applies after an isometry. The source expresses the result as final minus initial arclength parameter.

```
theorem exact_euclidean_rightCoreParam_hausdorffMeasure (c : StripCore) :
  (μH[1] : Measure EuclideanPlane)
  (realizedRightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) =
  ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
  have hfun : realizedRightCoreParam c =
    rightCoreComplexIsometryMap c ∘ unitCircleArc c.radius := by
    funext s
    exact realizedRightCoreParam_eq c s
  calc
    _ = (μH[1] : Measure EuclideanPlane)
      (rightCoreComplexIsometryMap c ''
        (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c))) := by
      rw [hfun]
      congr 1
      simpa only [Function.comp_apply] using (Set.image_image _ _ _).symm
    _ = (μH[1] : Measure ℂ)
      (unitCircleArc c.radius '' Icc (coreArcStart c) (coreArcEnd c)) :=
      (isometry_rightCoreComplexIsometryMap c).hausdorffMeasure_image
        (Or.inl (by norm_num)) _
    _ = ENNReal.ofReal (2 * c.radius * c.sideAngle) := by
      simpa [coreArcStart, coreArcEnd] using
        hausdorffMeasure_unitCircleArc_image_eq c.radius_pos
          ⟨c.sideAngle_pos, c.sideAngle_lt_pi⟩
    _ = ENNReal.ofReal (coreArcEnd c - coreArcStart c) := by
      congr 1
      simp only [coreArcStart, coreArcEnd]
      ring
```

exact_euclidean_segmentCarrier_hausdorffMeasure

Classification: supplement

Source: FrontierPerimeter.lean:937-946

Declaration SHA-256: f2e6ec79942c7df8c258fab2922138cb782506dbaa0184c81a94b12e6e9b9896

Mathematical statement. The Euclidean $\mathcal{H}^1$ -measure of a horizontal segment carrier equals its scalar length L .

Identify it with the affine segment and use the endpoint-distance formula for the measure of a segment.

```
theorem exact_euclidean_segmentCarrier_hausdorffMeasure
  (seg : HorizontalSegment) :
  (μH[1] : Measure EuclideanPlane)
  (planeEuclideanHomeomorph '' seg.carrier) =
  ENNReal.ofReal seg.euclideanLength := by
  rw [← realizedSegmentParam_image_carrier,
    realizedSegmentParam_image_affineSegment,
    hausdorffMeasure_affineSegment,
    segmentEndpoint_edist]
```

fourArc_pairwiseOverlap_subset_joinSet

Classification: supplement

Source: FrontierPerimeter.lean:1142-1195

Declaration SHA-256: eea91f9cc72f6aabc972107df3b4a4acd7547fe5fa392689f59302f202356a4

Mathematical statement. For every four-arc assembly, the union of pairwise intersections of its four arc traces is contained in the junction set for its core chord.

The trace inequalities and circle equations restrict any shared point to an attaching endpoint.

```
theorem fourArc_pairwiseOverlap_subset_joinSet (a : FourArcAssembly) :
  pairwiseOverlap
    (StripCore.leftArcTrace a.core)
    (StripCore.rightArcTrace a.core)
    (OneSidedCircularCap.arcTrace a.upperCap)
    (OneSidedCircularCap.arcTrace a.lowerCap)  $\subseteq$ 
    joinSet a.core.chord := by
  intro p hp
  simp only [pairwiseOverlap, mem_union, mem_inter_iff] at hp
  rcases hp with hLR | hLU | hLL | hRU | hRL | hUL
  · exact False.elim (by
    have h1 := hLR.1.2.1
    have hr := hLR.2.2.1
    linarith [a.core.chord_pos])
  · have hyCore : p.2  $\leq$  1 := (le_abs_self p.2).trans hLU.1.2.2
    have hyCap : (1 :  $\mathbb{R}$ )  $\leq$  p.2 := by
      simpa [FourArcAssembly.upperCap] using hLU.2.2
    have hy : p.2 = 1 := le_antisymm hyCore hyCap
    have hxBound := four_upper_arc_x_bound a hLU.2 hy
    have hx := eq_left_of_bounds hLU.1.2.1 hxBound
    have hpEq : p = (-a.core.chord / 2, 1) := Prod.ext hx hy
    simp [joinSet, hpEq]
  · have hyCore : -1  $\leq$  p.2 := by
      linarith [(neg_le_abs p.2).trans hLL.1.2.2]
    have hyCap : p.2  $\leq$  (-1 :  $\mathbb{R}$ ) := by
      simpa [FourArcAssembly.lowerCap] using hLL.2.2
    have hy : p.2 = -1 := le_antisymm hyCap hyCore
    have hxBound := four_lower_arc_x_bound a hLL.2 hy
    have hx := eq_left_of_bounds hLL.1.2.1 hxBound
    have hpEq : p = (-a.core.chord / 2, -1) := Prod.ext hx hy
    simp [joinSet, hpEq]
  · have hyCore : p.2  $\leq$  1 := (le_abs_self p.2).trans hRU.1.2.2
    have hyCap : (1 :  $\mathbb{R}$ )  $\leq$  p.2 := by
      simpa [FourArcAssembly.upperCap] using hRU.2.2
    have hy : p.2 = 1 := le_antisymm hyCore hyCap
    have hxBound := four_upper_arc_x_bound a hRU.2 hy
    have hx := eq_right_of_bounds hRU.1.2.1 hxBound
    have hpEq : p = (a.core.chord / 2, 1) := Prod.ext hx hy
    simp [joinSet, hpEq]
  · have hyCore : -1  $\leq$  p.2 := by
      linarith [(neg_le_abs p.2).trans hRL.1.2.2]
    have hyCap : p.2  $\leq$  (-1 :  $\mathbb{R}$ ) := by
      simpa [FourArcAssembly.lowerCap] using hRL.2.2
    have hy : p.2 = -1 := le_antisymm hyCap hyCore
    have hxBound := four_lower_arc_x_bound a hRL.2 hy
    have hx := eq_right_of_bounds hRL.1.2.1 hxBound
    have hpEq : p = (a.core.chord / 2, -1) := Prod.ext hx hy
    simp [joinSet, hpEq]
  · have hyUpper : (1 :  $\mathbb{R}$ )  $\leq$  p.2 := by
      simpa [FourArcAssembly.upperCap] using hUL.1.2
    have hyLower : p.2  $\leq$  (-1 :  $\mathbb{R}$ ) := by
      simpa [FourArcAssembly.lowerCap] using hUL.2.2
    linarith
```

hausdorffMeasure_image_Ico_eq_Icc

Classification: supplement

Source: FrontierPerimeter.lean:142-153

Declaration SHA-256: 1c850c9b7ca6e169808ebfb87cac4902c44102258f4c3ba0c9b6433987f2a5de

Mathematical statement. Let X be an extended metric space with Borel measurable structure. If f is injective on S ,

$$u \leq v,$$

and

$$[u, v] \subseteq S,$$

then

$$\mathcal{H}^1(f([u, v])) = \mathcal{H}^1(f([u, v])).$$

The images differ by at most the endpoint singleton, whose one-dimensional Hausdorff measure is zero.

```
lemma hausdorffMeasure_image_Ico_eq_Icc
  {E : Type*} [EMetricSpace E] [MeasurableSpace E] [BorelSpace E]
  {f : ℝ → E} {S : Set ℝ} (hf : Set.InjOn f S)
  {u v : ℝ} (huv : u ≤ v) (hsub : Icc u v ⊆ S) :
  (μH[1] : Measure E) (f '' Ico u v) =
    (μH[1] : Measure E) (f '' Icc u v) := by
  rw [image_Ico_eq_image_Icc_diff_endpoint hf huv hsub]
  letI : NullSingletonClass (μH[1] : Measure E) :=
    Measure.nullSingletonClass_hausdorff E (by norm_num)
  exact measure_sdiff_null (measure_singleton (f v))

/-- Equally spaced partition points on the signed-arclength interval. -/
```

hausdorffMeasure_one_euclideanJoinSet

Classification: supplement

Source: FrontierPerimeter.lean:1263-1268

Declaration SHA-256: 7ebe8fae03ad531268ff769afa248cd1067ee674eba1a7f8bc77f36d3ab49ebc

Mathematical statement. Every Euclidean junction set has zero one-dimensional Hausdorff measure.

Finite sets have zero Hausdorff measure in positive dimension.

```
theorem hausdorffMeasure_one_euclideanJoinSet (q : ℝ) :
  (μH[1] : Measure EuclideanPlane) (euclideanJoinSet q) = 0 := by
  let _ := Measure.nullSingletonClass_hausdorff EuclideanPlane
  (by norm_num : (0 : ℝ) < 1)
  exact (euclideanJoinSet_finite q).measure_zero μH[1]
```

hausdorffMeasure_unitCircleArc_image_eq

Classification: supplement

Source: FrontierPerimeter.lean:294-311

Declaration SHA-256: f4746d8c9b2c6ef1416bccca1f814590cdef08c393e10b7189ff167eb0402ab3

Mathematical statement. For $R > 0$ and $0 < \theta < \pi$,

$$\mathcal{H}^1(\gamma_R([-R\theta, R\theta])) = 2R\theta.$$

The Lipschitz upper bound and the limiting inscribed-polygon lower bound agree, proving actual Hausdorff length.

```

theorem hausdorffMeasure_unitCircleArc_image_eq {r theta : ℝ} (hr : 0 < r)
  (htheta : theta ∈ Ioo 0 Real.pi) :
  (μH[1] : Measure ℂ)
    (unitCircleArc r '' Icc (-r * theta) (r * theta)) =
    ENNReal.ofReal (2 * r * theta) := by
  apply le_antisymm
  · calc
    (μH[1] : Measure ℂ)
      (unitCircleArc r '' Icc (-r * theta) (r * theta)) ≤
      ENNReal.ofReal (r * theta - (-r * theta)) :=
      hausdorffMeasure_unitCircleArc_image_le hr
    _ = ENNReal.ofReal (2 * r * theta) := by congr 1 <=> ring
  · apply le_of_tendsto'
    (ENNReal.tendsto_ofReal (tendsto_cap_polygon_length (r := r) htheta.1))
  intro n
  exact cap_partition_polygon_lower hr htheta (n + 1) (Nat.zero_lt_succ n)

/-- Rotation of the standard complex circle by the signed-arclength phase `m`. -/

```

hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi

Classification: supplement

Source: FrontierPerimeter.lean:350-389

Declaration SHA-256: a5a68294ffad2a36064599c08079161b28ea03171c66437e00e8f24eba632c26

Mathematical statement. For $R > 0$,

$$u < v,$$

and

$$v - u < 2\pi R,$$

$$\mathcal{H}^1(\gamma_R([u, v])) = v - u.$$

Rotate the interval to one centered at zero and apply the symmetric-arc formula; the span hypothesis prevents multiple traversal.

```

theorem hausdorffMeasure_unitCircleArc_image_eq_of_span_lt_two_pi
  {r u v : ℝ} (hr : 0 < r) (huv : u < v)
  (hspan : v - u < 2 * Real.pi * r) :
  (μH[1] : Measure ℂ) (unitCircleArc r '' Icc u v) =
  ENNReal.ofReal (v - u) := by
  let theta : ℝ := (v - u) / (2 * r)
  let m : ℝ := (u + v) / 2
  have htheta : theta ∈ Ioo 0 Real.pi := by
    dsimp [theta]
    constructor
    · exact div_pos (sub_pos.mpr huv) (mul_pos (by norm_num) hr)
    · apply (div_lt_iff₀ (mul_pos (by norm_num) hr)).2
      nlinarith [Real.pi_pos]
  have hleft : -r * theta + m = u := by
    dsimp [theta, m]
    field_simp [hr.ne']
    ring
  have hright : r * theta + m = v := by
    dsimp [theta, m]
    field_simp [hr.ne']
    ring
  calc
    (μH[1] : Measure ℂ) (unitCircleArc r '' Icc u v) =
    (μH[1] : Measure ℂ)
      (unitCircleArcRotation r m ''
        (unitCircleArc r '' Icc (-r * theta) (r * theta))) := by
      rw [unitCircleArcRotation_image_Icc hr.ne', hleft, hright]
    _ = (μH[1] : Measure ℂ)
      (unitCircleArc r '' Icc (-r * theta) (r * theta)) :=
      (isometry_unitCircleArcRotation r m).hausdorffMeasure_image

```

```

      (Or.inl (by norm_num)) _
    = ENNReal.ofReal (2 * r * theta) :=
      hausdorffMeasure_unitCircleArc_image_eq hr htheta
    = ENNReal.ofReal (v - u) := by
      congr 1
      dsimp [theta]
      field_simp [hr.ne']

```

```

/-- Translation and possible reflection carrying the standard complex circle to a cap. -/

```

hausdorffMeasure_unitCircleArc_image_le

Classification: supplement

Source: FrontierPerimeter.lean:93-104

Declaration SHA-256: 07ee598762aa4a7964c4db549c22833ebf32ee2d325dcc190dd9a72212015319

Mathematical statement. For $R > 0$ and real a, b ,

$$\mathcal{H}^1(\gamma_R([a, b])) \leq \max\{b - a, 0\}.$$

Apply the one-Lipschitz image bound to the interval; if its endpoints are reversed, it is empty. The source writes the right side using the nonnegative-real embedding.

```

lemma hausdorffMeasure_unitCircleArc_image_le {r a b : ℝ} (hr : 0 < r) :
  (μH[1] : Measure ℂ) (unitCircleArc r '' Icc a b) ≤
    ENNReal.ofReal (b - a) := by
  calc
    (μH[1] : Measure ℂ) (unitCircleArc r '' Icc a b)
      ≤ (μH[1] : Measure ℝ) (Icc a b) := by
        simpa using (lipschitzWith_unitCircleArc hr).hausdorffMeasure_image_le
          (d := (1 : ℝ)) (by norm_num) (Icc a b)
    = ENNReal.ofReal (b - a) := by
      rw [hausdorffMeasure_real, Real.volume_Icc]

/-- Every continuous curve image has  $H^1$  mass at least the distance between its endpoints. -/

```

image_Ico_eq_image_Icc_diff_endpoint

Classification: supplement

Source: FrontierPerimeter.lean:124-141

Declaration SHA-256: c4fb0e64c0405d8f9fa3cb2429d9a9bc01ac95be8aba1e08560d1aeae3f939e9

Mathematical statement. If

$$f: \mathbb{R} \rightarrow X$$

is injective on S ,

$$u \leq v,$$

and

$$[u, v] \subseteq S,$$

then

$$f([u, v]) = f([u, v]) \setminus \{f(v)\}.$$

Injectivity ensures that deleting the endpoint parameter deletes exactly its image point.

```

lemma image_Ico_eq_image_Icc_diff_endpoint
  {α : Type*} {f : ℝ → α} {S : Set ℝ} (hf : Set.InjOn f S)
  {u v : ℝ} (huv : u ≤ v) (hsub : Icc u v ⊆ S) :

```

```

f '' Ico u v = f '' Icc u v \ {f v} := by
ext y
constructor
· rintro (x, hx, rfl)
  refine (⟨x, ⟨hx.1, hx.2.le⟩, rfl⟩, ?_)
  simp only [mem_singleton_iff]
  intro hfv
  have hxv := hf (hsub ⟨hx.1, hx.2.le⟩) (hsub ⟨huv, le_rfl⟩) hfv
  exact hx.2.ne hxv
· rintro (⟨x, hx, rfl⟩, hne)
  refine (x, ⟨hx.1, lt_of_le_of_ne hx.2 ?_⟩, rfl)
  intro hxv
  subst x
  exact hne (mem_singleton (f v))

```

integral_four_union

Classification: supplement

Source: FrontierPerimeter.lean:1562-1590

Declaration SHA-256: 8b3fbc02eae74d84636263a1b0bedcffb91c58bcc3410ac92807b51d54cc9ea4

Mathematical statement. Let A, B, C, D be Euclidean sets whose six pairwise intersections are

$$\mathcal{H}^1$$

-null, with B, C, D measurable. If a real function is integrable on each of the four sets against

$$\mathcal{H}^1,$$

its integral over

$$A \cup (B \cup (C \cup D))$$

equals the sum of the four integrals.

Repeated two-set integral additivity applies under almost-everywhere disjointness. The statement does not require a separately supplied measurability hypothesis on A .

```

theorem integral_four_union
  {f : EuclideanPlane → ℝ} {A B C D : Set EuclideanPlane}
  (hAB : AEDisjoint (μH[1] : Measure EuclideanPlane) A B)
  (hAC : AEDisjoint (μH[1] : Measure EuclideanPlane) A C)
  (hAD : AEDisjoint (μH[1] : Measure EuclideanPlane) A D)
  (hBC : AEDisjoint (μH[1] : Measure EuclideanPlane) B C)
  (hBD : AEDisjoint (μH[1] : Measure EuclideanPlane) B D)
  (hCD : AEDisjoint (μH[1] : Measure EuclideanPlane) C D)
  (hB : MeasurableSet B) (hC : MeasurableSet C) (hD : MeasurableSet D)
  (hfA : IntegrableOn f A (μH[1] : Measure EuclideanPlane))
  (hfB : IntegrableOn f B (μH[1] : Measure EuclideanPlane))
  (hfC : IntegrableOn f C (μH[1] : Measure EuclideanPlane))
  (hfD : IntegrableOn f D (μH[1] : Measure EuclideanPlane)) :
  (∫ z in A ∪ (B ∪ (C ∪ D)), f z ∂(μH[1] : Measure EuclideanPlane)) =
    (∫ z in A, f z ∂(μH[1] : Measure EuclideanPlane)) +
    ((∫ z in B, f z ∂(μH[1] : Measure EuclideanPlane)) +
     ((∫ z in C, f z ∂(μH[1] : Measure EuclideanPlane)) +
      ∫ z in D, f z ∂(μH[1] : Measure EuclideanPlane))) := by
  rw [setIntegral_union₀
    (hAB.union_right (hAC.union_right hAD))
    (hB.union (hC.union hD)).nullMeasurableSet
    hfA (hfB.union (hfC.union hfD)),
    setIntegral_union₀
    (hBC.union_right hBD)
    (hC.union hD).nullMeasurableSet
    hfB (hfC.union hfD),
    setIntegral_union₀ hCD hD.nullMeasurableSet hfC hfD]
  /-- The union of all six pairwise overlaps among four Euclidean sets. -/

```

integral_semicircle_scaled

Classification: supplement

Source: CMVGeometry.lean:1759-1791

Declaration SHA-256: fe66bbeed922271b7cf0e4043f58e6b4f6651310c0d02aa776383ac42b18240d

Mathematical statement. For $R > 0$ and $0 \leq \theta \leq \pi$,

$$\int_{R\cos\theta}^R 2\sqrt{R^2 - y^2} dy = R^2(\theta - \sin\theta\cos\theta).$$

Rescale the normalized circular-segment integral by the positive radius and include both horizontal half-widths.

```
lemma integral_semicircle_scaled {r θ : ℝ} (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  (∫ y in r * cos θ..r, 2 * √(r ^ 2 - y ^ 2)) =
    r ^ 2 * (θ - sin θ * cos θ) := by
  have hcomp := intervalIntegral.integral_comp_mul_deriv
  (a := cos θ) (b := 1) (f := fun x : ℝ ↦ r * x) (f' := fun _ ↦ r)
  (g := fun y : ℝ ↦ 2 * √(r ^ 2 - y ^ 2))
  (fun x _ ↦ by simp using (hasDerivAt_id x).const_mul r)
  continuousOn_const (by fun_prop)
  simp only [Function.comp_apply, mul_one] at hcomp
  rw [← hcomp]
  calc
    (∫ x in cos θ..1, 2 * √(r ^ 2 - (r * x) ^ 2) * r) =
      ∫ x in cos θ..1, (2 * r ^ 2) * √(1 - x ^ 2) := by
        apply intervalIntegral.integral_congr
        intro x hx
        dsimp
        have hx' : x ∈ Icc (-1 : ℝ) 1 := by
          have hcos : -1 ≤ cos θ := neg_one_le_cos θ
          have hcos' : cos θ ≤ 1 := cos_le_one θ
          have : x ∈ Icc (cos θ) 1 := by
            simp [uIcc, hcos'] using hx
          exact ⟨hcos.trans this.1, this.2⟩
        have hnonneg : 0 ≤ 1 - x ^ 2 := by nlinarith [hx'.1, hx'.2]
        rw [show r ^ 2 - (r * x) ^ 2 = r ^ 2 * (1 - x ^ 2) by ring,
          Real.sqrt_mul (sq_nonneg r), Real.sqrt_sq hr.le]
        ring
    _ = (2 * r ^ 2) * (∫ x in cos θ..1, √(1 - x ^ 2)) := by
      rw [intervalIntegral.integral_const_mul]
    _ = r ^ 2 * (θ - sin θ * cos θ) := by
      rw [integral_sqrt_one_sub_sq_from_cos hθ0 hθπ]
      ring
```

integral_semicircle_translated

Classification: supplement

Source: CMVGeometry.lean:1792-1800

Declaration SHA-256: 06721e5dc911d180205234ff69b820e369a8d82b66f142e17c0a1377b9b27f8d

Mathematical statement. For real a ,

$$R > 0,$$

and $0 \leq \theta \leq \pi$,

$$\int_{a + R\cos\theta}^{a + R} 2\sqrt{R^2 - (y - a)^2} dy = R^2(\theta - \sin\theta\cos\theta).$$

Translation of the vertical integration coordinate reduces the integral to the scaled segment formula.

```
lemma integral_semicircle_translated {a r θ : ℝ} (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
```

```

(f y in a + r * cos θ..a + r, 2 * √(r ^ 2 - (y - a) ^ 2)) =
  r ^ 2 * (θ - sin θ * cos θ) := by
rw [← intervalIntegral.integral_comp_add_left
  (f := fun y : ℝ → 2 * √(r ^ 2 - (y - a) ^ 2)) a]
simp only [add_sub_cancel_left]
exact integral_semicircle_scaled hr h00 h0π

```

integral_sqrt_one_sub_sq_from_cos

Classification: supplement

Source: CMVGeometry.lean:1729-1758

Declaration SHA-256: e4a969c6c1b67c6f267db8fb9582c7cd96174fa24489265c840c646c3085a52a

Mathematical statement. For $0 \leq \theta \leq \pi$,

$$\int_{\cos \theta}^1 \sqrt{1-x^2} \, dx = (\theta - \sin \theta \cos \theta)/2.$$

The inverse-trigonometric antiderivative includes the whole interval of cap angles, not just acute angles.

```

lemma integral_sqrt_one_sub_sq_from_cos {θ : ℝ} (h00 : 0 ≤ θ) (h0π : θ ≤ π) :
  (f x in cos θ..1, √(1 - x ^ 2)) =
    (θ - sin θ * cos θ) / 2 := by
have hcomp := intervalIntegral.integral_comp_mul_deriv
  (a := θ) (b := 0) (f := cos) (f' := fun x ↦ -sin x)
  (g := fun x : ℝ ↦ √(1 - x ^ 2))
  (fun x _ ↦ Real.hasDerivAt_cos x)
  (continuous_sin.neg.continuousOn) (by fun_prop)
simp only [Function.comp_apply, cos_zero] at hcomp
rw [← hcomp]
calc
  (f x in 0..θ, √(1 - cos x ^ 2) * -sin x) =
    f x in 0..θ, -(sin x ^ 2) := by
      apply intervalIntegral.integral_congr
      intro x hx
      dsimp
      have hx' : x ∈ Icc 0 θ := by
        simpa [uIcc, h00] using hx
      rw [← Real.sin_eq_sqrt_one_sub_cos_sq]
      · ring
      · exact hx'.1
      · exact hx'.2.trans h0π
_ = f x in 0..θ, sin x ^ 2 := by
  rw [intervalIntegral.integral_symm]
  simp
_ = (θ - sin θ * cos θ) / 2 := by
  rw [integral_sin_sq]
  simp
  ring

```

intervalPushforward

Classification: supplement

Source: CMVGeometry.lean:1200-1202

Declaration SHA-256: 366d70f2307ddf912f8fe8519c486284fdac dab2c9bb76f371f9a65dd8ee807d

Mathematical statement. For a curve

$$\gamma: \mathbb{R} \rightarrow \mathbb{R}^2$$

and real endpoints a, b , define the pushforward of Lebesgue measure restricted to $[a, b]$ by γ .

This gives the auxiliary parametrized boundary measures. A separate Euclidean frontier theorem ensures their integrated lengths agree with the canonical perimeter.

```
def intervalPushforward (curve :  $\mathbb{R} \rightarrow \text{PlanePoint}$ ) (a b :  $\mathbb{R}$ ) : Measure PlanePoint :=
  Measure.map curve (volume.restrict (Icc a b))
```

isometry_capComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:398-408

Declaration SHA-256: 5d4136606591a26f6abf24f8ba286ddba7154e77e9f96c81673196c1854009b0

Mathematical statement. For every cap, its complex-to-Euclidean coordinate map is an isometry.

Coordinate interchange, sign changes and translation preserve the sum of squared coordinate differences.

```
lemma isometry_capComplexIsometryMap (c : OneSidedCircularCap) :
  Isometry (capComplexIsometryMap c) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [WithLp.prod_dist_eq_add (by norm_num)]
  simp only [capComplexIsometryMap]
  split <;>
  norm_num [Real.dist_eq, sq_abs] <;>
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im] <;>
  congr 1 <;> ring
```

isometry_leftCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:685-695

Declaration SHA-256: a61c4ab4c222e88c43376bf46386ccf7697c506b7a23f81458ff102288a9e14a

Mathematical statement. The complex-to-Euclidean map for the left strip-core arc is an isometry.

Translation and a possible coordinate reflection preserve Euclidean squared distances.

```
lemma isometry_leftCoreComplexIsometryMap (c : StripCore) :
  Isometry (leftCoreComplexIsometryMap c) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [WithLp.prod_dist_eq_add (by norm_num)]
  simp only [leftCoreComplexIsometryMap]
  norm_num [Real.dist_eq, sq_abs]
  rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im]
  congr 1
  ring
```

isometry_rightCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:696-704

Declaration SHA-256: 2857926ac4225e60a51ede217224fd6a63879d202e0be14411ea5c13b2fc6bf

Mathematical statement. The complex-to-Euclidean map for the right strip-core arc is an isometry.

Translation and a possible coordinate reflection preserve Euclidean squared distances.

```
lemma isometry_rightCoreComplexIsometryMap (c : StripCore) :
  Isometry (rightCoreComplexIsometryMap c) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [WithLp.prod_dist_eq_add (by norm_num)]
  simp only [rightCoreComplexIsometryMap]
```

```
norm_num [Real.dist_eq, sq_abs]
rw [← Real.sqrt_eq_rpow, Complex.dist_eq_re_im]
```

isometry_unitCircleArcRotation

Classification: supplement

Source: FrontierPerimeter.lean:315-322

Declaration SHA-256: 0390b164e94132c53e908a7960b31f68dc786e7272a9a9f6482fa79cade88362

Mathematical statement. For all real R, m , multiplication by

$$\exp(im/R)$$

is an isometry of the complex plane.

Its modulus is one, so it preserves all Euclidean distances and Hausdorff measures.

```
lemma isometry_unitCircleArcRotation (r m : ℝ) :
  Isometry (unitCircleArcRotation r m) := by
  apply Isometry.of_dist_eq
  intro z w
  rw [Complex.dist_eq, Complex.dist_eq]
  simp only [unitCircleArcRotation, ← sub_mul, norm_mul, Complex.norm_exp]
  norm_num
```

joinSet

Classification: supplement

Source: FrontierPerimeter.lean:1115-1117

Declaration SHA-256: 613dcfda543c5255d80175237d7db38b3386ce55bc3215682252569ce26c42d5

Mathematical statement. For real L , define the set

$$\{(-L/2, 1), (L/2, 1), (-L/2, -1), (L/2, -1)\}.$$

These are the possible four-arc junctions at the two interfaces; repeated points are harmless.

```
def joinSet (q : ℝ) : Set PlanePoint :=
  {(-q / 2, 1), (q / 2, 1), (-q / 2, -1), (q / 2, -1)}
```

joinSet_finite

Classification: supplement

Source: FrontierPerimeter.lean:1253-1256

Declaration SHA-256: c95df1795c640672a88890bc658373f829154b7fa3a2361c967c14e17f22b551

Mathematical statement. For every real L , the junction set is finite.

It is explicitly a set of at most four points.

```
theorem joinSet_finite (q : ℝ) : (joinSet q).Finite := by
  simp [joinSet]

/-- The join set realized in the metric where Euclidean  $H^1$  is formed. -/
```

leftCoreBoundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:1487-1490

Declaration SHA-256: e243bfd8c440c79be3e701f3e1ba7f0b4af716d3ad6d1c3de2eb88023b1a30c8

Mathematical statement. The left core boundary measure is the pushforward of Lebesgue measure on $[-R\beta, R\beta]$ by the left core parametrization.

It represents only the exposed left arc, not the upper or lower attaching chords.

```
def leftCoreBoundaryMeasure (c : StripCore) : Measure PlanePoint :=
  intervalPushforward (leftCoreParam c) (coreArcStart c) (coreArcEnd c)

/-- Explicit regular boundary measure of the right core arc. -/
```

leftCoreBoundaryMeasure_integrable

Classification: supplement

Source: CMVGeometry.lean:2275-2281

Declaration SHA-256: efdd79d96b8f43a7c478608fb7242070e102fd586af27d2826a566f07542e0a9

Mathematical statement. For any real λ and strip core, the density is integrable against the left parametrized boundary measure.

The left arc is continuously parametrized over a finite interval and the density is bounded.

```
lemma leftCoreBoundaryMeasure_integrable (lam : ℝ) (c : StripCore) :
  Integrable (StripDensity lam) (leftCoreBoundaryMeasure c) := by
  rw [leftCoreBoundaryMeasure]
  exact stripDensity_integrable_intervalPushforward lam (by
    unfold leftCoreParam
    fun_prop)
```

leftCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:677-680

Declaration SHA-256: c0385e604fc62ea1f5b752f17a4538ba2edeedf20e52cbfb6848b3b966b2ab57

Mathematical statement. Define the complex-to-Euclidean map

$$z \mapsto (x_L - \operatorname{Re} z, \operatorname{Im} z)$$

for a strip core.

The horizontal translation and indicated sign place the standard circle at the left core center.

```
noncomputable def leftCoreComplexIsometryMap (c : StripCore) (z : ℂ) :
  EuclideanPlane :=
  WithLp.toLp 2 (c.leftCenterX - z.re, z.im)
```

leftCoreParam

Classification: supplement

Source: CMVGeometry.lean:1477-1481

Declaration SHA-256: 80f940fe52c510e69a283605275f10b49a7ca369836a80a40ece896dbbf606db

Mathematical statement. For a strip core with left center coordinate

$$x_L,$$

define

$$s \mapsto (x_L - R\cos(s/R), R\sin(s/R)).$$

On the declared interval this runs along the exposed left circle arc, with unit Euclidean speed.

```
def leftCoreParam (c : StripCore) (s : ℝ) : PlanePoint :=
  (c.leftCenterX - c.radius * cos (s / c.radius),
   c.radius * sin (s / c.radius))

/-- Unit-speed parametrization of the right core arc. -/
```

leftCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:517-596

Declaration SHA-256: 01c2af95d70e04d0bac530bc681fda34196e33e841f34e26466b4a3a9c6ecd18

Mathematical statement. For any strip core, the coordinate left arc parametrization maps $[-R\beta, R\beta]$ exactly onto the left exposed strip-core arc.

One inclusion follows by the circle equation and the strip height bound. Conversely, inverse sine reconstructs the parameter from the height; the selected side determines the cosine sign.

```
theorem leftCoreParam_image_Icc (c : StripCore) :
  leftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =
    c.leftArcTrace := by
  ext p
  constructor
  · rintro (s, hs, rfl)
    have hu := core_angle_mem' hs
    have htrig := Real.sin_sq_add_cos_sq (s / c.radius)
    have hcircle :
      ((leftCoreParam c s).1 - c.leftCenterX) ^ 2 +
      (leftCoreParam c s).2 ^ 2 = c.radius ^ 2 := by
      simp only [leftCoreParam, Prod.fst, Prod.snd]
      nlinarith [sq_nonneg c.radius]
    have habs : |s / c.radius| ≤ c.sideAngle := abs_le.mpr hu
    have hcos : Real.cos c.sideAngle ≤ Real.cos (s / c.radius) := by
      simp only [Real.cos_abs] using
        (Real.cos_le_cos_of_nonneg_of_le_pi (abs_nonneg (s / c.radius))
         c.sideAngle_lt_pi.le habs)
    have hx : (leftCoreParam c s).1 ≤ -c.chord / 2 := by
      simp only [leftCoreParam, Prod.fst]
      unfold StripCore.leftCenterX
      nlinarith [mul_le_mul_of_nonneg_left hcos c.radius_pos.le]
    exact ⟨hcircle, hx, coreParam_y_bound' c hs⟩
  · intro hp
    change (p.1 - c.leftCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ^
      p.1 ≤ -c.chord / 2 ^ |p.2| ≤ 1 at hp
    let u : ℝ := Real.arcsin (p.2 / c.radius)
    have hp2 := abs_le.mp hp.2.2
    have hlo : -c.curvature ≤ p.2 / c.radius := by
      apply (le_div_iff₀ c.radius_pos).2
      nlinarith [core_radius_mul_curvature' c]
    have hhi : p.2 / c.radius ≤ c.curvature := by
      apply (div_le_iff₀ c.radius_pos).2
      nlinarith [core_radius_mul_curvature' c]
    have hu : u ∈ Icc (-c.sideAngle) c.sideAngle := by
      constructor
      · simp [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hlo
      · simp [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hhi
    have hratio : p.2 / c.radius ∈ Icc (-1 : ℝ) 1 :=
      ⟨(by linarith [c.curvature_le_one] : (-1 : ℝ) ≤ -c.curvature) |>.trans hlo,
       hhi.trans c.curvature_le_one⟩
```

```

have hsinu : Real.sin u = p.2 / c.radius :=
  Real.sin_arcsin hratio.1 hratio.2
have hy : c.radius * Real.sin u = p.2 := by
  rw [hsinu]
  field_simp [ne_of_gt c.radius_pos]
have hcostheta : 0 ≤ Real.cos c.sideAngle :=
  Real.cos_nonneg_of_mem_Icc
  (by linarith [c.sideAngle_pos, Real.pi_pos],
   c.sideAngle_le_pi_div_two)
have hxu : p.1 - c.leftCenterX ≤ 0 := by
  unfold StripCore.leftCenterX
  nlinarith [mul_nonneg c.radius_pos.le hcostheta]
have hcosu : 0 ≤ Real.cos u :=
  Real.cos_nonneg_of_mem_Icc
  ((neg_le_neg c.sideAngle_le_pi_div_two).trans hu.1,
   hu.2.trans c.sideAngle_le_pi_div_two)
have htrig := Real.sin_sq_add_cos_sq u
have hsquares :
  (p.1 - c.leftCenterX) ^ 2 = (c.radius * Real.cos u) ^ 2 := by
  calc
  _ = c.radius ^ 2 - p.2 ^ 2 := by nlinarith [hp.1]
  _ = (c.radius * Real.cos u) ^ 2 := by rw [← hy]; nlinarith
have hcosprod : 0 ≤ c.radius * Real.cos u :=
  mul_nonneg c.radius_pos.le hcosu
have hx : p.1 = c.leftCenterX - c.radius * Real.cos u := by
  rcases sq_eq_sq_iff_eq_or_eq_neg.mp hsquares with hpos | hneg
  · have hz : p.1 - c.leftCenterX = 0 := by nlinarith
    nlinarith
  · linarith
refine (c.radius * u, ?_, ?_)
· change -c.radius * c.sideAngle ≤ c.radius * u ^
  c.radius * u ≤ c.radius * c.sideAngle
  constructor <=> nlinarith [c.radius_pos, hu.1, hu.2]
· have hdiv : c.radius * u / c.radius = u := by
  field_simp [ne_of_gt c.radius_pos]
  apply Prod.ext
  · simp [leftCoreParam, hdiv, hx]
  · simp [leftCoreParam, hdiv, hy]

```

leftCore_strip_density

Classification: supplement

Source: CMVGeometry.lean:1662-1667

Declaration SHA-256: 3222d64a655ce3af15a638606821b923e89c31ae75558031c08763aeb76b72b6

Mathematical statement. For every strip core and real λ , the density at any left arc parameter in $[-R\beta, R\beta]$ is one.

The entire arc, including the interface endpoints, stays inside the closed strip.

```

theorem leftCore_strip_density (lam : ℝ) (c : StripCore) {s : ℝ}
  (hs : s ∈ Icc (coreArcStart c) (coreArcEnd c)) :
  StripDensity lam (leftCoreParam c s) = 1 := by
  rw [StripDensity, if_pos]
  simp [leftCoreParam] using coreParam_y_bound c hs

```

leftCore_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1675-1694

Declaration SHA-256: e333f475432d30c40456ab56a9f5187cebdb61288c7e07d5eec293804ac77f49

Mathematical statement. For any strip core and real λ , the density integral against the left core boundary measure is

$$2\ell(\beta).$$

Since

$$R \sin \beta = 1,$$

the parameter span $2R\beta$ equals

$$2\beta/\sin\beta;$$

the integrand is one.

```
theorem leftCore_weightedIntegral (lam : ℝ) (c : StripCore) :
  ∫ p, StripDensity lam p ∂leftCoreBoundaryMeasure c = 2 * ell c.sideAngle := by
  rw [leftCoreBoundaryMeasure]
  rw [integral_intervalPushforward (leftCoreParam_measurable c)
    (measurable_stripDensity lam)]
  calc
    (∫ s in Icc (coreArcStart c) (coreArcEnd c),
      StripDensity lam (leftCoreParam c s)) =
      ∫ _s in Icc (coreArcStart c) (coreArcEnd c), 1 := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem measurableSet_Icc] with s hs
        exact leftCore_strip_density lam c hs
    _ = coreArcEnd c - coreArcStart c := by
      have horder : coreArcStart c ≤ coreArcEnd c := by
        dsimp [coreArcStart, coreArcEnd]
        nlinarith [c.radius_pos, c.sideAngle_pos]
      simp [horder]
    _ = 2 * ell c.sideAngle := core_single_arc_length c

/-- Weighted integral of the right core arc equals its Euclidean length. -/
```

lintegral_semicircle_translated

Classification: supplement

Source: CMVGeometry.lean:1801-1819

Declaration SHA-256: 3c8e90cd635d133e832b64fe2731d46db4f68d0a1a341d292df3904cf66e67e8

Mathematical statement. Under the same hypotheses $a \in \mathbb{R}$,

$$R > 0,$$

$$0 \leq \theta \leq \pi,$$

the nonnegative Lebesgue integral of

$$2\sqrt{R^2 - (y - a)^2}$$

over $[a + R\cos\theta, a + R]$ equals the nonnegative-real embedding of

$$R^2(\theta - \sin\theta\cos\theta).$$

The integrand is nonnegative and integrable; this is the extended-nonnegative-real form needed by the volume computation.

```
lemma lintegral_semicircle_translated {a r θ : ℝ} (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  (∫ y in Icc (a + r * cos θ) (a + r),
    ENNReal.ofReal (2 * √(r ^ 2 - (y - a) ^ 2))) =
    ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) := by
  have hcos : cos θ ≤ 1 := cos_le_one θ
  have hab : a + r * cos θ ≤ a + r := by nlinarith
  have hcont : Continuous (fun y : ℝ ↦ 2 * √(r ^ 2 - (y - a) ^ 2)) := by
    fun_prop
  have hint :
    IntegrableOn (fun y : ℝ ↦ 2 * √(r ^ 2 - (y - a) ^ 2))
      (Icc (a + r * cos θ) (a + r)) :=
    hcont.integrableOn_Icc
  rw [← MeasureTheory.ofReal_integral_eq_lintegral_ofReal hint
```

```
(Filter.Eventually.of_forall fun _ => by positivity))
congr 1
rw [integral_Icc_eq_integral_Ioc, ← intervalIntegral.integral_of_le hab]
exact integral_semicircle_translated hr h00 h0π
```

lipschitzWith_unitCircleArc

Classification: supplement

Source: FrontierPerimeter.lean:33-54

Declaration SHA-256: 6ddc026dd017d7b20d15e385a48bc34f5bcbfcad3796885aedaeb1fd06730313

Mathematical statement. For $R > 0$, γ_R is Lipschitz with constant one.

Chord distance is at most arclength separation, giving the upper Hausdorff-measure bound for an arc image.

```
lemma lipschitzWith_unitCircleArc {r : ℝ} (hr : 0 < r) :
  LipschitzWith 1 (unitCircleArc r) := by
  rw [lipschitzWith_iff_dist_le_mul]
  intro s t
  simp only [NNReal.coe_one, one_mul, Complex.dist_eq]
  rw [unitCircleArc_sub_factor hr.ne', norm_mul, norm_mul]
  have hexp : ‖Complex.exp (Complex.I * (t / r : ℝ))‖ = 1 := by
    rw [Complex.norm_exp]
    simp
  rw [hexp, mul_one]
  have hnormr : ‖(r : ℂ)‖ = r := by simp [abs_of_pos hr]
  rw [hnormr]
  calc
    r * ‖Complex.exp (Complex.I * ((s - t) / r : ℝ)) - 1‖
      ≤ r * ‖(s - t) / r‖ := by
        gcongr
        exact Real.norm_exp_I_mul_ofReal_sub_one_le
    _ = ‖s - t‖ := by
      rw [norm_div, Real.norm_eq_abs, Real.norm_eq_abs, abs_of_pos hr]
      field_simp
    _ = dist s t := by simp only [Real.norm_eq_abs, Real.dist_eq]
```

measurableSet_euclidean_capTrace

Classification: supplement

Source: FrontierPerimeter.lean:1104-1109

Declaration SHA-256: 345ff49d99187e89c9b6a7558ffa41e164108dfd32b3dc525b699de014a17b5a

Mathematical statement. The Euclidean image of every cap arc trace is measurable.

It is a continuous image of a compact interval in a Hausdorff space, hence compact and Borel measurable.

```
theorem measurableSet_euclidean_capTrace (c : OneSidedCircularCap) :
  MeasurableSet (planeEuclideanHomeomorph '' c.arcTrace) := by
  rw [← realizedCapParam_image_Icc]
  exact (isCompact_Icc.image (continuous_realizedCapParam c)).measurableSet

/-- Union of all six pairwise intersections of four primitive traces. -/
```

measurableSet_euclidean_leftArcTrace

Classification: supplement

Source: FrontierPerimeter.lean:1089-1093

Declaration SHA-256: a31ed55013e95ae690ffcc5bab6587c2933302f4970b8b673a167e6c53e1b7f8

Mathematical statement. The Euclidean image of the left exposed strip-core arc is measurable.

The realized continuous parametrization has a compact interval domain, so its image is compact and Borel measurable.

```
theorem measurableSet_euclidean_leftArcTrace (c : StripCore) :
  MeasurableSet (planeEuclideanHomeomorph '' c.leftArcTrace) := by
  rw [← realizedLeftCoreParam_image_Icc]
  exact (isCompact_Icc.image (continuous_realizedLeftCoreParam c)).measurableSet
```

measurableSet_euclidean_rightArcTrace

Classification: supplement

Source: FrontierPerimeter.lean:1094-1098

Declaration SHA-256: 771198ca90f5054f065ccd16a46e41b5fce0886173f7b3c654e6b61824be845e

Mathematical statement. The Euclidean image of the right exposed strip-core arc is measurable.

The realized continuous parametrization has a compact interval domain, so its image is compact and Borel measurable.

```
theorem measurableSet_euclidean_rightArcTrace (c : StripCore) :
  MeasurableSet (planeEuclideanHomeomorph '' c.rightArcTrace) := by
  rw [← realizedRightCoreParam_image_Icc]
  exact (isCompact_Icc.image (continuous_realizedRightCoreParam c)).measurableSet
```

measurable_stripDensity

Classification: supplement

Source: CMVGeometry.lean:1234-1240

Declaration SHA-256: b6fc908d8bba478440a5c35f7c75a94366dc691bdf7d2125848e2a8f512284af

Mathematical statement. For every real λ , the strip density is measurable.

The closed-strip condition defines a measurable set, and each branch of the density is constant.

```
theorem measurable_stripDensity (lam : ℝ) : Measurable (StripDensity lam) := by
  unfold StripDensity
  exact Measurable.ite (by measurability) measurable_const measurable_const

/-! ## One-sided circular caps -/

/-- The cap arc, parametrized by signed arclength. -/
```

monotone_capPartitionPoint

Classification: supplement

Source: FrontierPerimeter.lean:167-174

Declaration SHA-256: 62e77199a2eb7f24e5f93307536675c49f368eab2d84be96574f544f2152a857

Mathematical statement. For $R > 0$,

$$\theta > 0,$$

and natural $N > 0$, the sequence

$$i \mapsto t_i$$

is nondecreasing.

Its common increment

$$2R\theta/N$$

is positive.

```

lemma monotone_capPartitionPoint {r theta : ℝ} (hr : 0 < r) (htheta : 0 < theta)
  {N : ℕ} (hN : 0 < N) : Monotone (capPartitionPoint r theta N) := by
  intro i j hij
  unfold capPartitionPoint
  have hden : 0 < (N : ℝ) := by positivity
  have hstep : 0 ≤ 2 * r * theta / (N : ℝ) := (div_pos (by positivity) hden).le
  gcongr

```

ofReal_dist_le_hausdorffMeasure_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:105-123

Declaration SHA-256: 272ec4c298bc0f3582f8dd04d91ee8113b10ec4ec1c37538e792ec5391cc0936

Mathematical statement. Let X be a metric space with its Borel measurable structure and

$$f: \mathbb{R} \rightarrow X$$

continuous. For

$$u \leq v,$$

$$d(f(u), f(v)) \leq \mathcal{H}^1(f([u, v])),$$

interpreted in the nonnegative extended reals.

A connected path image has one-dimensional Hausdorff measure at least the distance between its endpoints.

```

lemma ofReal_dist_le_hausdorffMeasure_image_Icc
  {E : Type*} [MetricSpace E] [MeasurableSpace E] [BorelSpace E]
  {f : ℝ → E} (hf : Continuous f) {u v : ℝ} (huv : u ≤ v) :
  ENNReal.ofReal (dist (f u) (f v)) ≤
    (μH[1] : Measure E) (f '' Icc u v) := by
  let g : E → ℝ := fun z => dist (f u) z
  have hg : LipschitzWith 1 g := LipschitzWith.dist_right (f u)
  have hI : Icc 0 (dist (f u) (f v)) ⊆ g '' (f '' Icc u v) := by
    have hIV := intermediate_value_Icc huv (hg.continuous.comp hf).continuousOn
    simpa only [g, dist_self, Set.image_image, Function.comp_apply] using hIV
  calc
    ENNReal.ofReal (dist (f u) (f v)) =
      (μH[1] : Measure ℝ) (Icc 0 (dist (f u) (f v))) := by
        rw [hausdorffMeasure_real, Real.volume_Icc, sub_zero]
    _ ≤ (μH[1] : Measure ℝ) (g '' (f '' Icc u v)) := measure_mono hI
    _ ≤ (μH[1] : Measure E) (f '' Icc u v) := by
      simpa using hg.hausdorffMeasure_image_le (d := (1 : ℝ)) (by norm_num)
      (f '' Icc u v)

```

pairwiseOverlap

Classification: supplement

Source: FrontierPerimeter.lean:1110-1114

Declaration SHA-256: f2051dcd764bb55c1f4c980a58c56ba3f51d928d6141df3262dbdd919f1b5fbd

Mathematical statement. For four planar sets A, B, C, D , define the union of the six pairwise intersections.

This single set records all possible overlap errors in four-component perimeter additivity.

```

def pairwiseOverlap (A B C D : Set PlanePoint) : Set PlanePoint :=
  (A ∩ B) ∪ ((A ∩ C) ∪ ((A ∩ D) ∪
    ((B ∩ C) ∪ ((B ∩ D) ∪ (C ∩ D)))))

/-- The four possible primitive joins in either assembly. -/

```

pairwise_aedisjoint_of_overlap_null

Classification: supplement

Source: FrontierPerimeter.lean:1598-1616

Declaration SHA-256: 4b46ea8bfc070cb38c17e38f03c2486a1eb119281a4706f7a55c5da843e21d49

Mathematical statement. If the union of six pairwise intersections of four Euclidean sets has zero $\mathcal{H}^1$ -measure, then every pair of those sets is almost-everywhere disjoint for that measure.

Each intersection is contained in the null union, so monotonicity of measure proves the six assertions.

```
theorem pairwise_aedisjoint_of_overlap_null
  {A B C D : Set EuclideanPlane}
  (hnull : (μH[1] : Measure EuclideanPlane)
    (euclideanPairwiseOverlap A B C D) = 0) :
  AEDisjoint (μH[1] : Measure EuclideanPlane) A B ∧
  AEDisjoint (μH[1] : Measure EuclideanPlane) A C ∧
  AEDisjoint (μH[1] : Measure EuclideanPlane) A D ∧
  AEDisjoint (μH[1] : Measure EuclideanPlane) B C ∧
  AEDisjoint (μH[1] : Measure EuclideanPlane) B D ∧
  AEDisjoint (μH[1] : Measure EuclideanPlane) C D := by
  refine ⟨measure_mono_null (fun _ hz => Or.inl hz) hnull,
    measure_mono_null (fun _ hz => Or.inr (Or.inl hz)) hnull,
    measure_mono_null (fun _ hz => Or.inr (Or.inr (Or.inl hz))) hnull,
    measure_mono_null (fun _ hz => Or.inr (Or.inr (Or.inr (Or.inl hz)))) hnull,
    measure_mono_null
      (fun _ hz => Or.inr (Or.inr (Or.inr (Or.inr (Or.inl hz)))) hnull,
    measure_mono_null
      (fun _ hz => Or.inr (Or.inr (Or.inr (Or.inr (Or.inr hz)))) hnull)⟩
```

planeEuclideanHomeomorph

Classification: supplement

Source: CMVGeometry.lean:38-42

Declaration SHA-256: 0cae97f48ab4c061d7f08a907381bf52e9560477df33ffb993c23c2cbbf11049

Mathematical statement. The coordinate-preserving map from the product coordinate plane to its Euclidean realization is a homeomorphism.

It changes the metric representation while preserving coordinates, topology, frontiers and measurable sets; the source constructs it from the standard product realization.

```
noncomputable def planeEuclideanHomeomorph : PlanePoint ≈t EuclideanPlane :=
  (WithLp.homeomorphProd 2 ℝ ℝ).symm

/-- CMV's strip density. The closed interfaces belong to the density-one
branch. -/
```

realizedCapParam

Classification: supplement

Source: FrontierPerimeter.lean:430-433

Declaration SHA-256: 95b1661a30d7b5d3f17854c6eabbe1a4da9d944d9ad542e596607c6cb4fd12b0

Mathematical statement. The realized cap parametrization is its coordinate arclength parametrization followed by the coordinate homeomorphism into the Euclidean plane.

This changes the metric representation, not the point coordinates.

```
noncomputable def realizedCapParam (c : OneSidedCircularCap) (s : ℝ) :
  EuclideanPlane :=
```

```
planeEuclideanHomeomorph (capParam c s)
```

realizedCapParam_eq

Classification: supplement

Source: FrontierPerimeter.lean:434-441

Declaration SHA-256: 9c421cce21d93c49e9d419ecc30c8d31de995d391da25f034d29cf3f0bd85599

Mathematical statement. For a cap of radius R , its realized parametrization at s equals its complex isometry applied to $\gamma_R(s)$.

Expand sine and cosine coordinates to identify the two maps exactly.

```
lemma realizedCapParam_eq (c : OneSidedCircularCap) (s : ℝ) :
  realizedCapParam c s = capComplexIsometryMap c (unitCircleArc c.radius s) := by
  rw [show realizedCapParam c s = WithLp.toLp 2 (capParam c s) by rfl]
  cases hside : c.side <|>
  simp [capParam, OneSidedCircularCap.arcPoint, capComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im, hside] <|>
  ring
```

realizedCapParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:1071-1080

Declaration SHA-256: a37b40496e1e5c2ae0b778fe01e7fee7a91557ce9473bf6e0b7755afebbd941b

Mathematical statement. The realized cap parameter image on its closed parameter interval equals the Euclidean image of the cap arc trace.

Transport the preceding set equality through the coordinate homeomorphism.

```
theorem realizedCapParam_image_Icc (c : OneSidedCircularCap) :
  realizedCapParam c '' Icc (capStart c) (capEnd c) =
    planeEuclideanHomeomorph '' c.arcTrace := by
  calc
  _ = planeEuclideanHomeomorph ''
    (capParam c '' Icc (capStart c) (capEnd c)) := by
    rw [Set.image_image]
    rfl
  _ = _ := by rw [capParam_image_Icc_eq_arcTrace]
```

realizedLeftCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:705-708

Declaration SHA-256: 854e6d1ea17e2b2d6576f273610fd43051ac61c0bb7e75c185b3d3aa821f7da4

Mathematical statement. Define the realized left core parametrization by composing the coordinate parametrization with the Euclidean coordinate homeomorphism.

The points are unchanged, while lengths are computed in the Euclidean metric.

```
noncomputable def realizedLeftCoreParam (c : StripCore) (s : ℝ) :
  EuclideanPlane :=
  planeEuclideanHomeomorph (leftCoreParam c s)
```

realizedLeftCoreParam_eq

Classification: supplement

Source: FrontierPerimeter.lean:713-719

Declaration SHA-256: e2c0026193d6fa0dd58c252b75c6df1a108616049c60130059ddb93311c0b2a0

Mathematical statement. The realized left core parametrization at s equals its complex isometry applied to $\gamma_R(s)$.

Expand real and imaginary parts of the complex exponential and match the coordinate formulas.

```
lemma realizedLeftCoreParam_eq (c : StripCore) (s : ℝ) :
  realizedLeftCoreParam c s =
    leftCoreComplexIsometryMap c (unitCircleArc c.radius s) := by
  rw [show realizedLeftCoreParam c s = WithLp.toLp 2 (leftCoreParam c s) by rfl]
  simp [leftCoreParam, leftCoreComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]
```

realizedLeftCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:801-810

Declaration SHA-256: 43b7188db3ec73adee2747ff0dc65e5368084e5e17965fa7fdaa8f3c9301ccd7

Mathematical statement. The realized left core parameter image on its closed interval equals the Euclidean image of the coordinate arc trace.

Apply the homeomorphism to the proved coordinate parameter-image equality.

```
theorem realizedLeftCoreParam_image_Icc (c : StripCore) :
  realizedLeftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =
    planeEuclideanHomeomorph '' c.leftArcTrace := by
  calc
    _ = planeEuclideanHomeomorph ''
      (leftCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) := by
      rw [Set.image_image]
      rfl
    _ = _ := by rw [leftCoreParam_image_Icc]
```

realizedRightCoreParam

Classification: supplement

Source: FrontierPerimeter.lean:709-712

Declaration SHA-256: 4f8de565538cdbe799e2faf0a2d6452b4030fcfacd5b43a7347015b8e46924a9

Mathematical statement. Define the realized right core parametrization by composing the coordinate parametrization with the Euclidean coordinate homeomorphism.

The points are unchanged, while lengths are computed in the Euclidean metric.

```
noncomputable def realizedRightCoreParam (c : StripCore) (s : ℝ) :
  EuclideanPlane :=
  planeEuclideanHomeomorph (rightCoreParam c s)
```

realizedRightCoreParam_eq

Classification: supplement

Source: FrontierPerimeter.lean:720-726

Declaration SHA-256: 4aab5f14bc5465771d4021be2ff0ba59e74b69eeaba0b9a9c834bd0bf72ed625

Mathematical statement. The realized right core parametrization at s equals its complex isometry applied to $\gamma_R(s)$.

Expand real and imaginary parts of the complex exponential and match the coordinate formulas.

```
lemma realizedRightCoreParam_eq (c : StripCore) (s : ℝ) :
  realizedRightCoreParam c s =
    rightCoreComplexIsometryMap c (unitCircleArc c.radius s) := by
  rw [show realizedRightCoreParam c s = WithLp.toLp 2 (rightCoreParam c s) by rfl]
  simp [rightCoreParam, rightCoreComplexIsometryMap,
    unitCircleArc_re, unitCircleArc_im]
```

realizedRightCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:811-820

Declaration SHA-256: e28432904f0d4f1c849ed6ed56e30bdae00185a8e05f01ccceab8ef89d16f29b

Mathematical statement. The realized right core parameter image on its closed interval equals the Euclidean image of the coordinate arc trace.

Apply the homeomorphism to the proved coordinate parameter-image equality.

```
theorem realizedRightCoreParam_image_Icc (c : StripCore) :
  realizedRightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =
    planeEuclideanHomeomorph '' c.rightArcTrace := by
  calc
    _ = planeEuclideanHomeomorph ''
      (rightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c)) := by
      rw [Set.image_image]
      rfl
    _ = _ := by rw [rightCoreParam_image_Icc]
```

realizedSegmentParam

Classification: supplement

Source: FrontierPerimeter.lean:864-867

Declaration SHA-256: 4c1728c18c2df9a1da871e357fce4f6439c6307fe654c71c273fe06b8caaa0fd

Mathematical statement. The realized segment parametrization is its coordinate parametrization followed by the Euclidean coordinate homeomorphism.

It is the same horizontal affine curve with the Euclidean metric used for length.

```
noncomputable def realizedSegmentParam (seg : HorizontalSegment) (s : ℝ) :
  EuclideanPlane :=
  planeEuclideanHomeomorph (segmentParam seg s)
```

realizedSegmentParam_image_affineSegment

Classification: supplement

Source: FrontierPerimeter.lean:873-912

Declaration SHA-256: c3be9a5bc2d447d5c5665690a46411881b4f0933eb5c3cf50deaa41a88946c72

Mathematical statement. The realized parameter image on $[-L/2, L/2]$ equals the affine segment joining the realized left and right endpoints.

Affine interpolation and midpoint displacement describe the same closed horizontal segment.

```
theorem realizedSegmentParam_image_affineSegment (seg : HorizontalSegment) :
  realizedSegmentParam seg '' Icc (segmentStart seg) (segmentEnd seg) =
    affineSegment ℝ
      (planeEuclideanHomeomorph seg.leftEndpoint)
      (planeEuclideanHomeomorph seg.rightEndpoint) := by
  rw [affineSegment]
  ext p
  constructor
  · rintro ⟨s, hs, rfl⟩
    have hs' : -seg.chord / 2 ≤ s ∧ s ≤ seg.chord / 2 := by
      simpa only [segmentStart, segmentEnd, mem_Icc] using hs
    let t : ℝ := (s + seg.chord / 2) / seg.chord
    refine ⟨t, ?_, ?_⟩
    · constructor
      · exact div_nonneg (by linarith) seg.chord_pos.le
      · exact (div_le_one seg.chord_pos).2 (by linarith)
    · rw [AffineMap.lineMap_apply_module, WithLp.ext_iff]
      apply Prod.ext
      · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,
        HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint, t]
        field_simp [ne_of_gt seg.chord_pos]
        ring
      · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,
        HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint]
        ring
    · rintro ⟨t, ht, rfl⟩
      let s : ℝ := -seg.chord / 2 + t * seg.chord
      refine ⟨s, ?_, ?_⟩
      · change -seg.chord / 2 ≤ s ∧ s ≤ seg.chord / 2
        dsimp [s]
        constructor <=> nlinarith [seg.chord_pos, ht.1, ht.2]
      · rw [AffineMap.lineMap_apply_module, WithLp.ext_iff]
        apply Prod.ext
        · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,
          HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint, s]
          ring
        · simp [realizedSegmentParam, planeEuclideanHomeomorph_apply, segmentParam,
          HorizontalSegment.leftEndpoint, HorizontalSegment.rightEndpoint]
          ring
```

realizedSegmentParam_image_carrier

Classification: supplement

Source: FrontierPerimeter.lean:913-922

Declaration SHA-256: fc21a65690cb2d00a27cb6aebde20046f1dffbcb15b7f30d3caf58ac1aefd1725

Mathematical statement. The realized segment parameter image equals the Euclidean image of its coordinate carrier.

Apply the coordinate homeomorphism to the exact coordinate parameter-image equality.

```
theorem realizedSegmentParam_image_carrier (seg : HorizontalSegment) :
  realizedSegmentParam seg '' Icc (segmentStart seg) (segmentEnd seg) =
    planeEuclideanHomeomorph '' seg.carrier := by
  calc
    _ = planeEuclideanHomeomorph ''
```

```

(segmentParam seg '' Icc (segmentStart seg) (segmentEnd seg)) := by
  rw [Set.image_image]
  rfl
_ = _ := by rw [segmentParam_image_Icc]

```

rightCoreBoundaryMeasure

Classification: supplement

Source: CMVGeometry.lean:1491-1493

Declaration SHA-256: 33cbc13fa6fcd71207ac7e5dfb0579966744c8887bb1c9af1b95e6c688f55ea8

Mathematical statement. The right core boundary measure is the pushforward of Lebesgue measure on $[-R\beta, R\beta]$ by the right core parametrization.

The later canonical-frontier comparison relates this parametrized measure to Euclidean arc length.

```

def rightCoreBoundaryMeasure (c : StripCore) : Measure PlanePoint :=
  intervalPushforward (rightCoreParam c) (coreArcStart c) (coreArcEnd c)

```

rightCoreBoundaryMeasure_integrable

Classification: supplement

Source: CMVGeometry.lean:2282-2288

Declaration SHA-256: 63cd5ea28e72c7cf39aef8bfd2e698b16940d7ae74acc20f074d60c55994913c

Mathematical statement. For any real λ and strip core, the density is integrable against the right parametrized boundary measure.

This is the right-arc instance of the finite-interval pushforward result.

```

lemma rightCoreBoundaryMeasure_integrable (lam : ℝ) (c : StripCore) :
  Integrable (StripDensity lam) (rightCoreBoundaryMeasure c) := by
  rw [rightCoreBoundaryMeasure]
  exact stripDensity_integrable_intervalPushforward lam (by
    unfold rightCoreParam
    fun_prop)

```

rightCoreComplexIsometryMap

Classification: supplement

Source: FrontierPerimeter.lean:681-684

Declaration SHA-256: 24206b125ec8e6de86a054126e34b32568c5cb9c365728de42d394ff9cc0ede6

Mathematical statement. Define the complex-to-Euclidean map

$$z \mapsto (x_R + \operatorname{Re} z, \operatorname{Im} z)$$

for a strip core.

The horizontal translation and indicated sign place the standard circle at the right core center.

```

noncomputable def rightCoreComplexIsometryMap (c : StripCore) (z : ℂ) :
  EuclideanPlane :=
  WithLp.toLp 2 (c.rightCenterX + z.re, z.im)

```

rightCoreParam

Classification: supplement

Source: CMVGeometry.lean:1482-1486

Declaration SHA-256: 38ea0d845e704b5d320adc3936a1b6a5fba52e17ce5fb320da7c20200afa9e4d

Mathematical statement. For a strip core with right center coordinate

$$x_R,$$

define

$$s \mapsto (x_R + R\cos(s/R), R\sin(s/R)).$$

The sign of the horizontal cosine selects the exposed right circle arc.

```
def rightCoreParam (c : StripCore) (s : ℝ) : PlanePoint :=
  (c.rightCenterX + c.radius * cos (s / c.radius),
   c.radius * sin (s / c.radius))

/-- Explicit regular boundary measure of the left core arc. -/
```

rightCoreParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:597-676

Declaration SHA-256: 78bb53ade5c270538e82ae914ea314e16857498de0ee3dfc73dde7468e65c773

Mathematical statement. For any strip core, the coordinate right arc parametrization maps $[-R\beta, R\beta]$ exactly onto the right exposed strip-core arc.

One inclusion follows by the circle equation and the strip height bound. Conversely, inverse sine reconstructs the parameter from the height; the selected side determines the cosine sign.

```
theorem rightCoreParam_image_Icc (c : StripCore) :
  rightCoreParam c '' Icc (coreArcStart c) (coreArcEnd c) =
    c.rightArcTrace := by
  ext p
  constructor
  · rintro (s, hs, rfl)
    have hu := core_angle_mem' hs
    have htrig := Real.sin_sq_add_cos_sq (s / c.radius)
    have hcircle :
      ((rightCoreParam c s).1 - c.rightCenterX) ^ 2 +
      (rightCoreParam c s).2 ^ 2 = c.radius ^ 2 := by
      simp only [rightCoreParam, Prod.fst, Prod.snd]
      nlinarith [sq_nonneg c.radius]
    have habs : |s / c.radius| ≤ c.sideAngle := abs_le.mpr hu
    have hcos : Real.cos c.sideAngle ≤ Real.cos (s / c.radius) := by
      simpa only [Real.cos_abs] using
        (Real.cos_le_cos_of_nonneg_of_le_pi (abs_nonneg (s / c.radius))
         c.sideAngle_lt_pi.le habs)
    have hx : c.chord / 2 ≤ (rightCoreParam c s).1 := by
      simp only [rightCoreParam, Prod.fst]
      unfold StripCore.rightCenterX
      nlinarith [mul_le_mul_of_nonneg_left hcos c.radius_pos.le]
    exact ⟨hcircle, hx, coreParam_y_bound' c hs⟩
  · intro hp
    change (p.1 - c.rightCenterX) ^ 2 + p.2 ^ 2 = c.radius ^ 2 ∧
      c.chord / 2 ≤ p.1 ∧ |p.2| ≤ 1 at hp
    let u : ℝ := Real.arcsin (p.2 / c.radius)
    have hp2 := abs_le.mp hp.2.2
    have hlo : -c.curvature ≤ p.2 / c.radius := by
      apply (le_div_iff₀ c.radius_pos).2
      nlinarith [core_radius_mul_curvature' c]
```

```

have hhi : p.2 / c.radius ≤ c.curvature := by
  apply (div_le_iff₀ c.radius_pos).2
  nlinarith [core_radius_mul_curvature' c]
have hu : u ∈ Icc (-c.sideAngle) c.sideAngle := by
  constructor
  · simp [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hlo
  · simp [u, StripCore.sideAngle] using Real.arcsin_le_arcsin hhi
have hratio : p.2 / c.radius ∈ Icc (-1 : ℝ) 1 :=
  ⟨(by linarith [c.curvature_le_one] : (-1 : ℝ) ≤ -c.curvature) |>.trans hlo,
    hhi.trans c.curvature_le_one⟩
have hsinu : Real.sin u = p.2 / c.radius :=
  Real.sin_arcsin hratio.1 hratio.2
have hy : c.radius * Real.sin u = p.2 := by
  rw [hsinu]
  field_simp [ne_of_gt c.radius_pos]
have hcostheta : 0 ≤ Real.cos c.sideAngle :=
  Real.cos_nonneg_of_mem_Icc
  ⟨by linarith [c.sideAngle_pos, Real.pi_pos],
    c.sideAngle_le_pi_div_two⟩
have hxu : 0 ≤ p.1 - c.rightCenterX := by
  unfold StripCore.rightCenterX
  nlinarith [mul_nonneg c.radius_pos.le hcostheta]
have hcosu : 0 ≤ Real.cos u :=
  Real.cos_nonneg_of_mem_Icc
  ⟨(neg_le_neg c.sideAngle_le_pi_div_two).trans hu.1,
    hu.2.trans c.sideAngle_le_pi_div_two⟩
have htrig := Real.sin_sq_add_cos_sq u
have hsquares :
  (p.1 - c.rightCenterX) ^ 2 = (c.radius * Real.cos u) ^ 2 := by
  calc
  _ = c.radius ^ 2 - p.2 ^ 2 := by nlinarith [hp.1]
  _ = (c.radius * Real.cos u) ^ 2 := by rw [← hy]; nlinarith
have hcosp : 0 ≤ c.radius * Real.cos u :=
  mul_nonneg c.radius_pos.le hcosu
have hx : p.1 = c.rightCenterX + c.radius * Real.cos u := by
  rcases sq_eq_sq_iff_eq_or_eq_neg.mp hsquares with hpos | hneg
  · linarith
  · have hz : p.1 - c.rightCenterX = 0 := by nlinarith
    nlinarith
refine (c.radius * u, ?_, ?_)
· change -c.radius * c.sideAngle ≤ c.radius * u ^
  c.radius * u ≤ c.radius * c.sideAngle
  constructor <|> nlinarith [c.radius_pos, hu.1, hu.2]
· have hdiv : c.radius * u / c.radius = u := by
  field_simp [ne_of_gt c.radius_pos]
  apply Prod.ext
  · simp [rightCoreParam, hdiv, hx]
  · simp [rightCoreParam, hdiv, hy]

```

rightCore_strip_density

Classification: supplement

Source: CMVGeometry.lean:1668-1674

Declaration SHA-256: d879e18591ac7ff0f0a228fad33d85d1e290fb3c77820aaf80c672ec9411662

Mathematical statement. For every strip core and real λ , the density at any right arc parameter in $[-R\beta, R\beta]$ is one.

The coordinate height bound is

$$|y| \leq 1,$$

exactly the density-one branch.

```

theorem rightCore_strip_density (lam : ℝ) (c : StripCore) {s : ℝ}
  (hs : s ∈ Icc (coreArcStart c) (coreArcEnd c)) :
  StripDensity lam (rightCoreParam c s) = 1 := by
  rw [StripDensity, if_pos]
  simp [rightCoreParam] using coreParam_y_bound c hs

```

```
/-- Weighted integral of the left core arc equals its Euclidean length. -/
```

rightCore_weightedIntegral

Classification: supplement

Source: CMVGeometry.lean:1695-1714

Declaration SHA-256: 12c65212e5417b38acb8560f789f61c31ede351c3de6efe2b847438f7d6cf718

Mathematical statement. For any strip core and real λ , the density integral against the right core boundary measure is

$$2\ell(\beta).$$

Integrate the constant density one over the arclength interval, using

$$R = 1/\sin\beta.$$

```
theorem rightCore_weightedIntegral (lam : ℝ) (c : StripCore) :
  ∫ p, StripDensity lam p ∂rightCoreBoundaryMeasure c = 2 * ell c.sideAngle := by
  rw [rightCoreBoundaryMeasure]
  rw [integral_intervalPushforward (rightCoreParam_measurable c)
    (measurable_stripDensity lam)]
  calc
    (∫ s in Icc (coreArcStart c) (coreArcEnd c),
      StripDensity lam (rightCoreParam c s)) =
      ∫ _s in Icc (coreArcStart c) (coreArcEnd c), 1 := by
        apply integral_congr_ae
        filter_upwards [ae_restrict_mem measurableSet_Icc] with s hs
        exact rightCore_strip_density lam c hs
    _ = coreArcEnd c - coreArcStart c := by
      have horder : coreArcStart c ≤ coreArcEnd c := by
        dsimp [coreArcStart, coreArcEnd]
        nlinarith [c.radius_pos, c.sideAngle_pos]
      simp [horder]
    _ = 2 * ell c.sideAngle := core_single_arc_length c

/-- The two arc masses add to the declared strip-core boundary-arc length. -/
```

segmentEnd

Classification: supplement

Source: CMVGeometry.lean:1400-1402

Declaration SHA-256: 161abc8655ce59212c511d028e178416823adf64c523c73dae6b889ae03ed1f4

Mathematical statement. The final parameter of a horizontal segment of length $L > 0$ is

$$L/2.$$

The parameter interval has length L and includes both endpoints.

```
def segmentEnd (seg : HorizontalSegment) : ℝ := seg.chord / 2

/-- Explicit regular boundary measure of a horizontal segment. -/
```

segmentEndpoint_edist

Classification: supplement

Source: FrontierPerimeter.lean:923-936

Declaration SHA-256: 667e4fe55a19b902ccd7cc29782bd67f737d862a1bddf625b46565ed0a83a1cd

Mathematical statement. The extended Euclidean distance between the realized endpoints of a horizontal segment equals its positive scalar length L , embedded in the nonnegative extended reals.

The endpoints have equal vertical coordinates and horizontal separation L .

```
lemma segmentEndpoint_edist (seg : HorizontalSegment) :
  edist (planeEuclideanHomeomorph seg.leftEndpoint)
    (planeEuclideanHomeomorph seg.rightEndpoint) =
  ENNReal.ofReal seg.euclideanLength := by
  rw [edist_dist, WithLp.prod_dist_eq_add (by norm_num)]
  simp only [planeEuclideanHomeomorph_apply]
  norm_num [Real.dist_eq, HorizontalSegment.leftEndpoint,
    HorizontalSegment.rightEndpoint, HorizontalSegment.euclideanLength]
  congr 1
  have h : seg.midpointX -
    (seg.midpointX + seg.chord / 2 + seg.chord / 2) = -seg.chord := by ring
  rw [h, neg_sq, ← Real.sqrt_eq_rpow, Real.sqrt_sq_eq_abs,
    abs_of_pos seg.chord_pos]
```

segmentParam

Classification: supplement

Source: CMVGeometry.lean:1395-1397

Declaration SHA-256: ba8ed6bbc5cd19b9c405036bea7c221ae24e5c76ea0f6ef5cc2d62c327ae1dc5

Mathematical statement. For a segment with midpoint (m, b) , define the parametrization

$$s \mapsto (m + s, b).$$

The parameter measures horizontal length directly, between minus and plus half the segment length.

```
def segmentParam (seg : HorizontalSegment) (s : ℝ) : PlanePoint :=
  (seg.midpointX + s, seg.baseY)
```

segmentParam_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:835-863

Declaration SHA-256: 78cfdac8328c8d0308b83fec45a7d72c3a0a830edb6a3f41f9bd2da54db0f76

Mathematical statement. The image of a horizontal segment parametrization on $[-L/2, L/2]$ is exactly its coordinate carrier.

The forward inclusion is the absolute displacement bound; for the reverse choose the horizontal displacement from the midpoint as parameter.

```
theorem segmentParam_image_Icc (seg : HorizontalSegment) :
  segmentParam seg '' Icc (segmentStart seg) (segmentEnd seg) = seg.carrier := by
  ext p
  constructor
  · rintro (s, hs, rfl)
    have hs' : -seg.chord / 2 ≤ s ∧ s ≤ seg.chord / 2 := by
      simpa only [segmentStart, segmentEnd, mem_Icc] using hs
    have habs : |s| ≤ seg.chord / 2 := by
      rw [abs_le]
      constructor <=> linarith [hs'.1]
    change seg.baseY = seg.baseY ^
      |seg.midpointX + s - seg.midpointX| ≤ seg.chord / 2
    rw [add_sub_cancel_left]
    exact ⟨rfl, habs⟩
  · intro hp
    change p.2 = seg.baseY ^ |p.1 - seg.midpointX| ≤ seg.chord / 2 at hp
    refine ⟨p.1 - seg.midpointX, ?_, ?_⟩
    · have hb := abs_le.mp hp.2
      simpa only [segmentStart, segmentEnd, mem_Icc]
        using (show -seg.chord / 2 ≤ p.1 - seg.midpointX ^
```

```

      p.1 - seg.midpointX ≤ seg.chord / 2 by
        constructor <=> linarith [hb.1])
    · apply Prod.ext
    · simp [segmentParam]
    · simpa [segmentParam] using hp.1.symm

@[simp] lemma planeEuclideanHomeomorph_apply (p : PlanePoint) :
  planeEuclideanHomeomorph p = WithLp.toLp 2 p := rfl

```

segmentStart

Classification: supplement

Source: CMVGeometry.lean:1398-1399

Declaration SHA-256: e90adf884523aab9d2e3af449adbc9d655d375051a612c0b4970a212a5dca61f

Mathematical statement. The initial parameter of a horizontal segment of length $L > 0$ is

$$-L/2.$$

This is the signed displacement of the left endpoint from the midpoint.

```
def segmentStart (seg : HorizontalSegment) : ℝ := -seg.chord / 2
```

stripDensity_integrableOn_of_volume_ne_top

Classification: supplement

Source: CMVGeometry.lean:2289-2302

Declaration SHA-256: d26957a576fdcc692079b92ac5054cbbeda2a914af15d77e6d70ad1d418d20fd

Mathematical statement. For real λ , if a planar set F has finite Lebesgue measure, then f_λ is integrable over F .

The piecewise constant density is globally measurable and bounded, so it is integrable against any finite restricted volume measure.

```

lemma stripDensity_integrableOn_of_volume_ne_top (lam : ℝ)
  {s : Set PlanePoint} (hs : volume s ≠ ⊤) :
  IntegrableOn (StripDensity lam) s := by
  apply Measure.integrableOn_of_bounded (M := 1 + |lam|) hs
  · exact (measurable_stripDensity lam).aestronglyMeasurable
  · apply Filter.Eventually.of_forall
    intro p
    change |StripDensity lam p| ≤ 1 + |lam|
    unfold StripDensity
    split_ifs
    · simp only [abs_one]
      nlinarith [abs_nonneg lam]
    · nlinarith [abs_nonneg lam]

```

stripDensity_integrable_intervalPushforward

Classification: supplement

Source: CMVGeometry.lean:2243-2259

Declaration SHA-256: 6315c9471ea2031d23e0cb4552b23f2812fe90938358c371a246f04d7f2d8095

Mathematical statement. For any real λ , measurable curve

$$\gamma: \mathbb{R} \rightarrow \mathbb{R}^2,$$

and real a, b , the strip density is integrable against the pushforward of Lebesgue measure on $[a, b]$.

The density is measurable and bounded by a finite constant depending on λ ; the restricted parameter measure is finite.

```

lemma stripDensity_integrable_intervalPushforward (lam : ℝ)
  {curve : ℝ → PlanePoint} {a b : ℝ} (hcurve : Measurable curve) :
  Integrable (StripDensity lam) (intervalPushforward curve a b) := by
  rw [intervalPushforward, integrable_map_measure
    (measurable_stripDensity lam).astronglyMeasurable hcurve.aemeasurable]
  apply Measure.integrableOn_of_bounded (M := 1 + |lam|)
    measure_Icc_lt_top.ne
  · exact ((measurable_stripDensity lam).comp hcurve).astronglyMeasurable
  · apply Filter.Eventually.of_forall
    intro x
    change |StripDensity lam (curve x)| ≤ 1 + |lam|
    unfold StripDensity
    split_ifs
    · simp only [abs_one]
      nlinarith [abs_nonneg lam]
    · nlinarith [abs_nonneg lam]

```

tendsto_cap_polygon_length

Classification: supplement

Source: FrontierPerimeter.lean:269-293

Declaration SHA-256: 32d7803e3d290677cdac6cd7bd331241d0a1e99888d3a8665e326de2ca241a11

Mathematical statement. For $\theta > 0$ and real R ,

$$(n+1)2R\sin(\theta/(n+1)) \rightarrow 2R\theta$$

as natural $n \rightarrow \infty$.

This is the small-angle sine limit, written with positive subdivision counts to avoid a zero denominator.

```

lemma tendsto_cap_polygon_length {r theta : ℝ} (htheta : 0 < theta) :
  Tendsto
    (fun n : ℕ => ((n + 1 : ℕ) : ℝ) *
      (2 * r * Real.sin (theta / ((n + 1 : ℕ) : ℝ))))
    atTop (λ (2 * r * theta)) := by
  have hden : Tendsto (fun n : ℕ => ((n + 1 : ℕ) : ℝ)) atTop atTop :=
    (tendsto_natCast_atTop_iff (R := ℝ)).2 (tendsto_add_atTop_nat 1)
  have hquot : Tendsto (fun n : ℕ => theta / ((n + 1 : ℕ) : ℝ)) atTop (λ 0) :=
    hden.const_div_atTop theta
  have hsinc : Tendsto (fun n : ℕ => Real.sinc (theta / ((n + 1 : ℕ) : ℝ)))
    atTop (λ 1) := by
    convert (Real.continuous_sinc.tendsto 0).comp hquot using 1 <=>
      simp [Function.comp_def]
  have hscaled : Tendsto
    (fun n : ℕ => (2 * r * theta) * Real.sinc (theta / ((n + 1 : ℕ) : ℝ)))
    atTop (λ (2 * r * theta)) := by
    simpa using hsinc.const_mul (2 * r * theta)
  apply hscaled.congr'
  filter_upwards [] with n
  have hN : (((n + 1 : ℕ) : ℝ)) ≠ 0 := by positivity
  have hq : theta / (((n + 1 : ℕ) : ℝ)) ≠ 0 := div_ne_zero htheta.ne' hN
  rw [Real.sinc_of_ne_zero hq]
  field_simp

/-- Exact H¹ mass of every injective circular arc with half-angle in `(0, π)`. -/

```

unitCircleArc

Classification: supplement

Source: FrontierPerimeter.lean:11-13

Declaration SHA-256: 6865b0b68c07403ee39252efa35feb9b50850a4644c2bf915fb506723c7a07c7

Mathematical statement. Define

$$\gamma_R(s) = \text{Rexp}(is/R)$$

as a complex-valued function of real R, s .

For $R > 0$, this is the radius- R circle parametrized by arclength; the source name does not restrict the radius to one.

```
def unitCircleArc (r s : ℝ) : ℂ :=
  (r : ℂ) * Complex.exp (Complex.I * (s / r : ℝ))
```

unitCircleArcRotation

Classification: supplement

Source: FrontierPerimeter.lean:312-314

Declaration SHA-256: cfdc45400ed0d08a69b42a215854d1f81999a500771f48a7f18476aafe48ef

Mathematical statement. For real R, m , define the complex map

$$z \mapsto z \exp(im/R).$$

This is multiplication by a unit complex number, hence a rotation even with the total division convention.

```
noncomputable def unitCircleArcRotation (r m : ℝ) (z : ℂ) : ℂ :=
  z * Complex.exp (Complex.I * (m / r : ℝ))
```

unitCircleArcRotation_apply

Classification: supplement

Source: FrontierPerimeter.lean:323-331

Declaration SHA-256: dcedc37c9428c0a722c5bf130023e2389a83c7faa4036432847247c3401f6b9b

Mathematical statement. For $R \neq 0$ and real m, s , rotating $\gamma_R(s)$ by

$$\exp(im/R)$$

gives

$$\gamma_R(s + m).$$

The exponential addition law translates the arclength parameter.

```
lemma unitCircleArcRotation_apply {r : ℝ} (hr : r ≠ 0) (m s : ℝ) :
  unitCircleArcRotation r m (unitCircleArc r s) =
    unitCircleArc r (s + m) := by
  unfold unitCircleArcRotation unitCircleArc
  rw [mul_assoc, ← Complex.exp_add]
  congr 2
  push_cast
  field_simp
```

unitCircleArcRotation_image_Icc

Classification: supplement

Source: FrontierPerimeter.lean:332-349

Declaration SHA-256: 6bbfe5bd6af2f20e278e2e30ff5adc9fe7c0761359113d169ea5baae1f20d6a2

Mathematical statement. For $R \neq 0$ and real m, u, v , the rotation sends $\gamma_R([u, v])$ onto

$$\gamma_R([u + m, v + m]).$$

Apply the pointwise parameter-shift identity and translate the interval; the set equality also handles reversed endpoints.

```

lemma unitCircleArcRotation_image_Icc {r : ℝ} (hr : r ≠ 0)
  (m u v : ℝ) :
  unitCircleArcRotation r m '' (unitCircleArc r '' Icc u v) =
    unitCircleArc r '' Icc (u + m) (v + m) := by
  ext z
  constructor
  · rintro ⟨_, ⟨s, hs, rfl⟩, rfl⟩
    refine ⟨s + m, ⟨by linarith [hs.1], by linarith [hs.2]⟩, ?_⟩
    exact (unitCircleArcRotation_apply hr m s).symm
  · rintro ⟨t, ht, rfl⟩
    refine ⟨unitCircleArc r (t - m), ?_, ?_⟩
    · exact ⟨t - m, ⟨by linarith [ht.1], by linarith [ht.2]⟩, rfl⟩
    · rw [unitCircleArcRotation_apply hr]
      congr 1
      ring

/-- Exact H¹ mass of an arbitrary injective signed-arclength interval on a
circle. The strict span bound rules out wrapping through a full revolution. -/

```

unitCircleArc_dist

Classification: supplement

Source: FrontierPerimeter.lean:55-66

Declaration SHA-256: 1c583965a8663dcd3d0483716b4801199d867b2ff4cbe96f04d4cbeb57ff9c08

Mathematical statement. For $R > 0$ and real s, t ,

$$|\gamma_R(s) - \gamma_R(t)| = R|2\sin((s - t)/(2R))|.$$

The exponential difference factorization gives the exact circular chord length.

```

lemma unitCircleArc_dist {r s t : ℝ} (hr : 0 < r) :
  dist (unitCircleArc r s) (unitCircleArc r t) =
    r * |2 * Real.sin ((s - t) / r) / 2| := by
  rw [Complex.dist_eq, unitCircleArc_sub_factor hr.ne', norm_mul, norm_mul]
  have hexp : ||Complex.exp (Complex.I * (t / r : ℝ))|| = 1 := by
    rw [Complex.norm_exp]
    simp
  rw [hexp, mul_one]
  have hnormr : ||(r : ℂ)|| = r := by simp [abs_of_pos hr]
  rw [hnormr, Complex.norm_exp_I_mul_ofReal_sub_one]
  simp only [Real.norm_eq_abs]

```

unitCircleArc_eq

Classification: supplement

Source: FrontierPerimeter.lean:409-415

Declaration SHA-256: 9342d6a8ed8b7dcd1fbce36251dddf6b271aeea6ec0298f5d467e178c8fbf9cec

Mathematical statement. For all real R, s ,

$$\gamma_R(s) = R[\cos(s/R) + i\sin(s/R)].$$

Euler's formula gives a coordinate form useful for the geometric parametrizations.

```

lemma unitCircleArc_eq (r s : ℝ) :
  unitCircleArc r s =
    (r : ℂ) * ((Real.cos (s / r) : ℂ) + (Real.sin (s / r) : ℂ) * Complex.I) := by
  unfold unitCircleArc
  rw [show Complex.I * (s / r : ℝ) = ((s / r : ℝ) : ℂ) * Complex.I by ring]
  rw [Complex.exp_ofReal_mul_I]

```

unitCircleArc_im

Classification: supplement

Source: FrontierPerimeter.lean:423-429

Declaration SHA-256: 0666e8d860656cd1bc24ade42af99acdafd9034497a0ba0f27acf44cddcd77e5

Mathematical statement. For all real R, s ,

$$\text{Im}\gamma_R(s) = R\sin(s/R).$$

Take the imaginary part of the exponential coordinate identity.

```
lemma unitCircleArc_im (r s : ℝ) :  
  (unitCircleArc r s).im = r * Real.sin (s / r) := by  
  rw [unitCircleArc_eq]  
  simp only [Complex.mul_im, Complex.add_im, Complex.ofReal_re, Complex.ofReal_im,  
    Complex.I_re, Complex.I_im]  
  ring
```

unitCircleArc_injOn

Classification: supplement

Source: FrontierPerimeter.lean:67-92

Declaration SHA-256: ceabbb1bd00ed7b3d5a248b0d9d8b4d41f8f1fc0af8194bd56a8092deed637e9

Mathematical statement. For $R > 0$ and $0 < \theta < \pi$, γ_R is injective on $[-R\theta, R\theta]$.

The angular interval has span strictly below 2π , so no distinct parameters differ by a full period.

```
lemma unitCircleArc_injOn {r theta : ℝ} (hr : 0 < r)  
  (htheta : theta ∈ Ioo 0 Real.pi) :  
  Set.InjOn (unitCircleArc r) (Icc (-r * theta) (r * theta)) := by  
  intro s hs t ht hst  
  have hsdiv : s / r ∈ Icc (-theta) theta := by  
    constructor  
    · apply (le_div_iff₀ hr).2  
      nlinarith [hs.1]  
    · apply (div_le_iff₀ hr).2  
      nlinarith [hs.2]  
  have htdiv : t / r ∈ Icc (-theta) theta := by  
    constructor  
    · apply (le_div_iff₀ hr).2  
      nlinarith [ht.1]  
    · apply (div_le_iff₀ hr).2  
      nlinarith [ht.2]  
  rcases hsdiv with ⟨hs1, hsr⟩  
  rcases htdiv with ⟨ht1, htr⟩  
  have hdist : |s / r - t / r| < 2 * Real.pi := by  
    rw [abs_lt]  
    constructor <=> nlinarith [htheta.2]  
  have hcircle : circleMap 0 r (s / r) = circleMap 0 r (t / r) := by  
    simpa [unitCircleArc, circleMap_zero, mul_comm] using hst  
  have hang := eq_of_circleMap_eq hr.ne' hdist hcircle  
  exact (div_left_inj' hr.ne').mp hang
```

unitCircleArc_re

Classification: supplement

Source: FrontierPerimeter.lean:416-422

Declaration SHA-256: 238aa74976048276575d89eb69760ceb968a224b5358a1775e61818ec60df916

Mathematical statement. For all real R, s ,

$$\operatorname{Re} \gamma_R(s) = R \cos(s/R).$$

Take the real part of the exponential coordinate identity.

```
lemma unitCircleArc_re (r s : ℝ) :
  (unitCircleArc r s).re = r * Real.cos (s / r) := by
  rw [unitCircleArc_eq]
  simp only [Complex.mul_re, Complex.add_re, Complex.ofReal_re, Complex.ofReal_im,
    Complex.I_re, Complex.I_im]
  ring
```

unitCircleArc_sub_factor

Classification: supplement

Source: FrontierPerimeter.lean:18-32

Declaration SHA-256: d6c59e7f498e1352e21969a1c8084fa6600038a792db1172f4e5bd3b2836eece

Mathematical statement. For real $R \neq 0$ and real s, t ,

$$\gamma_R(s) - \gamma_R(t) = R \exp(it/R) [\exp(i(s-t)/R) - 1].$$

Factor the common exponential; this form reduces distances along the circle to a single angular difference.

```
lemma unitCircleArc_sub_factor {r s t : ℝ} (_hr : r ≠ 0) :
  unitCircleArc r s - unitCircleArc r t =
    (r : ℂ) * Complex.exp (Complex.I * (t / r : ℝ)) *
      (Complex.exp (Complex.I * ((s - t) / r : ℝ)) - 1) := by
  have he : Complex.exp (Complex.I * (s / r : ℝ)) =
    Complex.exp (Complex.I * (t / r : ℝ)) *
      Complex.exp (Complex.I * ((s - t) / r : ℝ)) := by
  rw [← Complex.exp_add]
  congr 2
  push_cast
  field_simp
  ring
  rw [unitCircleArc, unitCircleArc, he]
  ring
```

upper_cap_integral

Classification: supplement

Source: FrontierPerimeter.lean:1545-1552

Declaration SHA-256: 447beffa9af9c0047e982aa709b3b243852770c1fde70cc3c6676215458bf

Mathematical statement. For real λ and an upper cap based on

$$y = 1,$$

the integral of Euclidean strip density over its actual arc trace against $\mathcal{H}^1$ equals λ times its arc length.

Density is λ away from the two null endpoints; the trace has exactly the proved circular arc length.

```
theorem upper_cap_integral (lam : ℝ)
  (c : OneSidedCircularCap) (hside : c.side = .upper)
  (hbase : c.baseY = 1) :
  (∫ z in planeEuclideanHomeomorph '' c.arcTrace,
    euclideanStripDensity lam z ∂(μH[1] : Measure EuclideanPlane)) =
    lam * c.arcLength :=
  cap_integral lam c (cap_upper_external_density lam c hside hbase)
```

volume_horizontalline

Classification: supplement

Source: CMVGeometry.lean:2029-2036

Declaration SHA-256: ea745db5e4b12e1d05fe0934ca3c81eb24b84ea87dd522e3aa6824c86d1a2dce

Mathematical statement. For every real b , the line

$$\{(x, y) : y = b\}$$

has zero planar Lebesgue measure.

This justifies neglecting attaching-chord overlaps in area sums, but not in length sums.

```
lemma volume_horizontalline (b : ℝ) :
  volume {p : PlanePoint | p.2 = b} = 0 := by
  have hset : {p : PlanePoint | p.2 = b} = Set.univ ×ˢ ({b} : Set ℝ) := by
    ext p
    simp
  rw [hset, Measure.volume_eq_prod, Measure.prod_prod]
  simp
```

volume_lower_circular_segment

Classification: supplement

Source: CMVGeometry.lean:1945-1995

Declaration SHA-256: 2301e25d7ced8d1d9e2c23a9b63e02ba7c6e8f9ea615112d4c41d15aeb777045

Mathematical statement. For real m, c ,

$$R > 0,$$

and $0 \leq \theta \leq \pi$, the Lebesgue volume of

$$\{(x, y) : (x - m)^2 + (y - c)^2 \leq R^2, y \leq c - R \cos \theta\}$$

is

$$R^2(\theta - \sin \theta \cos \theta),$$

embedded in the nonnegative extended reals.

Reflection of the vertical coordinate reduces the lower segment to the upper one without changing volume.

```
lemma volume_lower_circular_segment (mx cy r θ : ℝ) (hr : 0 < r)
  (hθ0 : 0 ≤ θ) (hθπ : θ ≤ π) :
  volume {p : ℝ × ℝ |
    (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
    p.2 ≤ cy - r * cos θ} =
  ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) := by
  let U : Set (ℝ × ℝ) :=
    {p | (p.1 - mx) ^ 2 + (p.2 - (-cy)) ^ 2 ≤ r ^ 2 ∧
    -cy + r * cos θ ≤ p.2}
  have hU : MeasurableSet U := by
    have hcircle : Measurable (fun p : ℝ × ℝ ↦
      (p.1 - mx) ^ 2 + (p.2 - (-cy)) ^ 2) := by fun_prop
    have hside : Measurable (fun p : ℝ × ℝ ↦ p.2) := measurable_snd
    exact (measurableSet_le hcircle measurable_const).inter
      (measurableSet_le measurable_const hside)
  let F : ℝ × ℝ → ℝ × ℝ := Prod.map id (fun y : ℝ ↦ -y)
  have hF :
    MeasurePreserving F (volume : Measure (ℝ × ℝ)) volume := by
    change MeasurePreserving (Prod.map id (fun y : ℝ ↦ -y))
      ((volume : Measure ℝ).prod volume) ((volume : Measure ℝ).prod volume)
    exact (MeasurePreserving.id (volume : Measure ℝ)).prod
```

```

(Measure.measurePreserving_neg (volume : Measure ℝ))
have hpre :
  F-1' U =
    {p : ℝ × ℝ |
      (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
      p.2 ≤ cy - r * cos θ} := by
  ext p
  change ((p.1 - mx) ^ 2 + (-p.2 - -cy) ^ 2 ≤ r ^ 2 ∧
    -cy + r * cos θ ≤ -p.2) ↔
    ((p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
      p.2 ≤ cy - r * cos θ)
  constructor <=> rintro ⟨hcircle, hside⟩
  · constructor
  · nlinarith
  · linarith
  · constructor
  · nlinarith
  · linarith
calc
  volume {p : ℝ × ℝ |
    (p.1 - mx) ^ 2 + (p.2 - cy) ^ 2 ≤ r ^ 2 ∧
    p.2 ≤ cy - r * cos θ} =
    volume (F-1' U) := by rw [hpre]
  _ = volume U := hF.measure_preimage hU.nullMeasurableSet
  _ = ENNReal.ofReal (r ^ 2 * (θ - sin θ * cos θ)) :=
    volume_upper_circular_segment mx (-cy) r θ hr hθ0 hθπ

```

namespace OneSidedCircularCap

volume_verticalLine

Classification: supplement

Source: CMVGeometry.lean:2037-2046

Declaration SHA-256: b32e9d96eca7ded1e8b5354b6d117e61bcda7b34c1fe056587cd14e1871da61b

Mathematical statement. For every real a , the line

$$\{(x, y) : x = a\}$$

has zero planar Lebesgue measure.

Rectangle and disk-piece decompositions can overlap along such lines without changing area additivity.

```

lemma volume_verticalLine (a : ℝ) :
  volume {p : PlanePoint | p.1 = a} = 0 := by
  have hset : {p : PlanePoint | p.1 = a} = ({a} : Set ℝ) ×ˢ Set.univ := by
    ext p
    simp
  rw [hset, Measure.volume_eq_prod, Measure.prod_prod]
  simp
namespace StripCore

```

Generated certificate families

External references

- **cmv2010** — Antonio Cañete, Michele Miranda Jr., and Davide Vittone, *Some isoperimetric problems in planes with density*, Journal of Geometric Analysis 20 (2010), 243–290; §2, Lemma 3.8 (including proof, Step 2), Proposition 3.9, Conjecture 3.12, and Theorem 3.16. Canete2010.pdf

- **leon-pavlak-pulla-shen-note** — Skilyn Leon, Rosa Pavlak, Evelyn Pulla, and Xi Sisi Shen, *An elementary comparison for four-arc regions in a planar strip density*, Supplied undated note; both printed Theorem 1 statements, including (3.2); §§3–6 and Remarks 1–3. CMV_Strip_Density_Problem.pdf